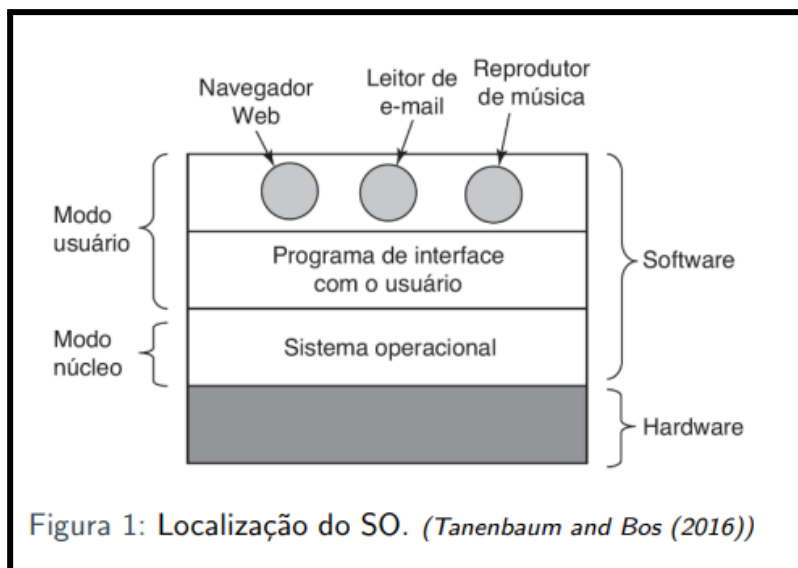


Introdução

O que é SO?

- É uma **camada de software** que faz a **interface entre o hardware, o usuário e os programas usuários.**
- É uma estrutura ampla, complexa, que incorpora aspectos:
 - **baixo nível**, como **drivers de dispositivos** e **gerência de memória física**;
 - **alto nível**, como **programas utilitários** e a **própria interface gráfica.**
- SO segundo Tanenbaum:



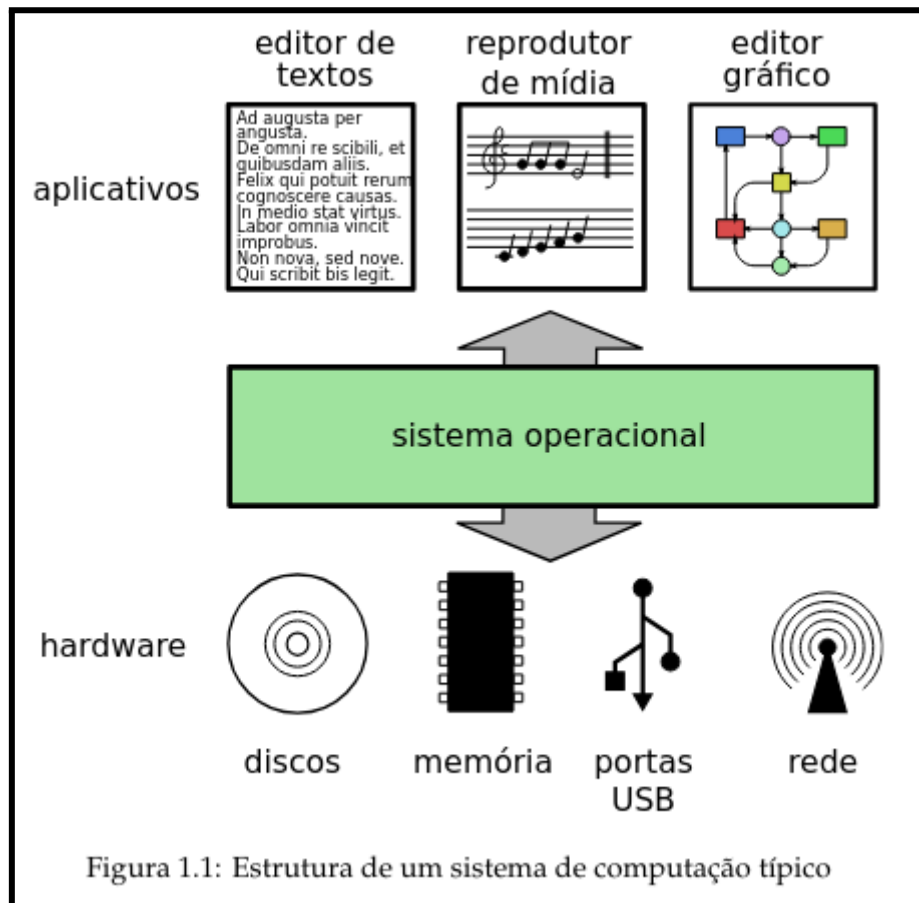
Definição

- É uma **máquina estendida**
 - Oculta os detalhes complicados que precisam ser executados
 - Apresenta ao **usuário uma máquina virtual**, mais fácil de usar
- É um **gerenciador de recurso**
 - Cada programa tem um tempo com o recurso
 - Cada programa tem um espaço no recurso

Objetivos de um SO

O sistema operacional é uma camada de software que opera entre o hardware e os programas aplicativos voltados ao usuário final.

Trata-se de uma estrutura de software ampla, muitas vezes complexa, que incorpora aspectos de baixo nível (como drivers de dispositivos e gerência de memória física) e de alto nível (como programas utilitários e a própria interface gráfica).



A Figura acima ilustra a estrutura geral de um sistema de computação típico. Nela, podemos observar **elementos de hardware, o sistema operacional e alguns programas aplicativos**.

Os **objetivos básicos de um sistema operacional podem ser sintetizados em duas palavras-chave: “abstração” e “gerência”**, cujos principais aspectos são detalhados a seguir.

Abstração de Recursos

- O acesso aos recursos de hardware pode ser uma tarefa complexa, devido às características específicas dos dispositivos e a complexidade de suas interfaces.
- O SO pode prover mecanismos e interfaces abstratas:

- Para o acesso aos dispositivos: **acesso ao disco** → **primitivas (open, read e close)**.
- **Tornar os aplicativos independentes do hardware.** (portabilidade)

Gerência de Recursos

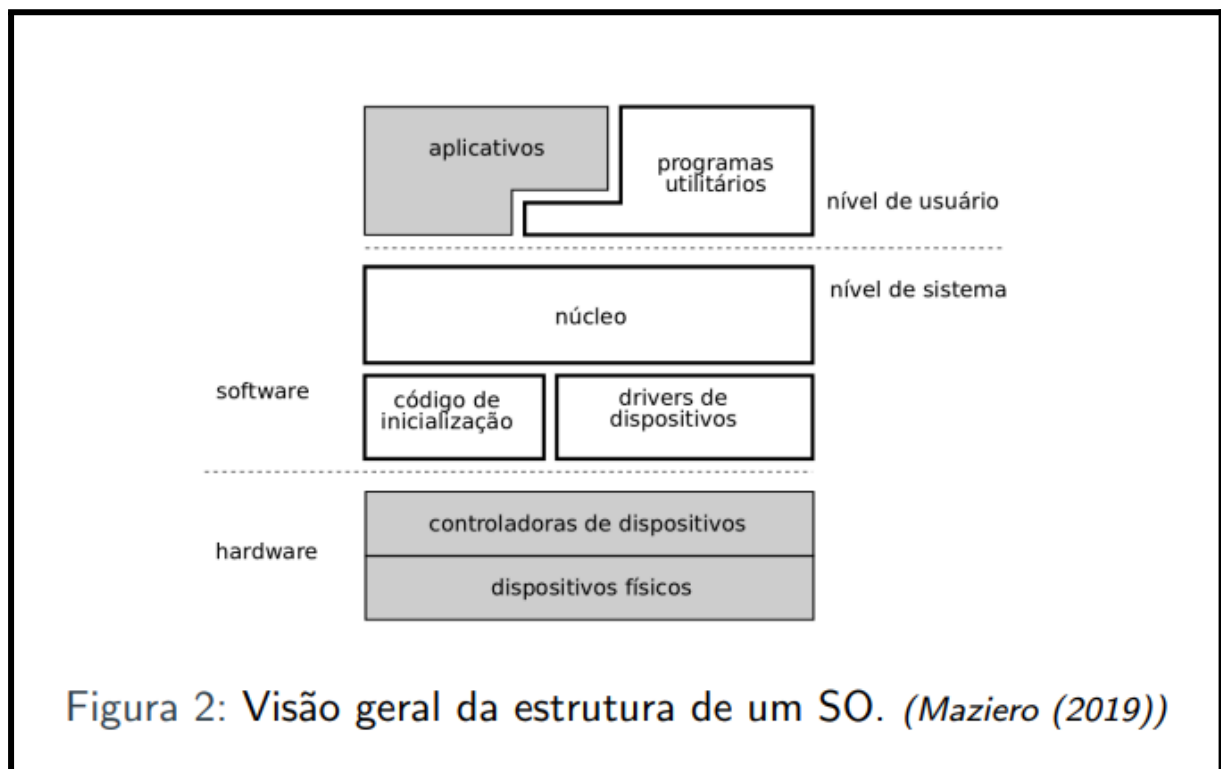
- O SO define políticas para gerenciar o uso dos recursos de hardware pelos aplicativos, e resolver eventuais disputas e conflitos.
- **Recursos:**
 - **Processador**
 - **Memória**
 - **Disco**
 - **Vídeo**
 - **Impressora**
 - etc.
- **Permite que múltiplos programas sejam executados ao mesmo tempo.**
- **Gerencia e protege a memória, os dispositivos de entrada e saída e outros recursos.**
- **Inclui a multiplexação (partilha) de recursos de duas maneiras diferentes:**
 - **No tempo**
 - Nesse modelo, os **diferentes usuários ou processos se revezam no uso do recurso**. Cada um utiliza o recurso com exclusividade, **mas por um intervalo de tempo determinado**.
 - Exemplos: CPU, impressora, teclado
 - **No espaço**
 - Em vez de esperar a vez, **os recursos são divididos fisicamente**. **Cada processo recebe uma fatia ou parte do recurso e todos podem usar suas respectivas partes simultaneamente**.
 - Exemplos: Memória RAM, HD
 - **Ambos (Tempo e Espaço)**
 - Muitos recursos modernos acabam exigindo uma combinação das duas técnicas para máxima eficiência.

- Exemplo: Em uma **placa de rede**, os dados são divididos em **pacotes (espaço)** e esses pacotes de diferentes processos são enviados um após o outro na fila de envio (**tempo**).
- Exemplo: **Monitor**, divisão em **janelas (espaço)** e atualização de quadros (**tempo**).

Funcionalidades

- Gerência do **Processador**
- Gerência de **Memória**
- Gerência de **Dispositivos (Entrada/Saída)**
- Gerência de **Arquivos**
- Gerência de **Proteção (Segurança)**

Estrutura de um SO

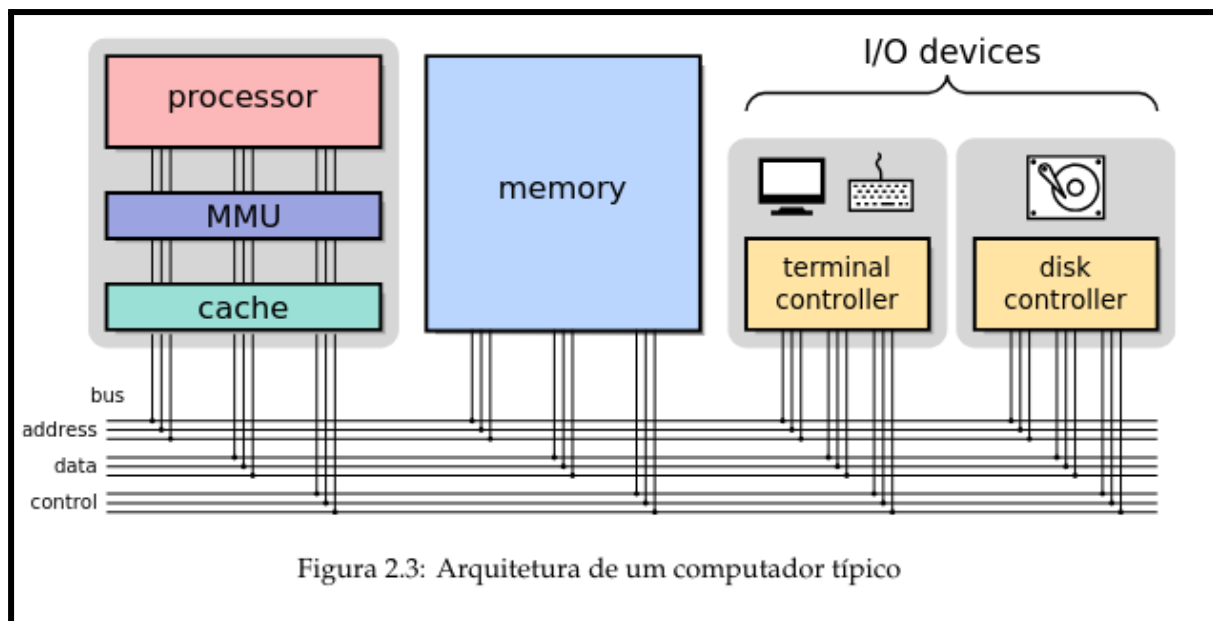


- **Núcleo:** Ou **kernel** é o **coração do SO**, sendo o responsável pela **gerência dos recursos do hardware**, implementa as principais abstrações utilizadas pelos programas.

- **Drivers:** São **módulos de código específicos para acessar os dispositivos físicos**. Por exemplo: **Driver da Placa de Vídeo**.
- **Código de inicialização:** A inicialização do hardware requer uma série de **tarefas complexas**, como **reconhecer e testar os dispositivos**, e **carregar o núcleo do sistema operacional em memória e iniciar sua execução**.
- **Programas utilitários:** São programas que **facilitam o uso do sistema computacional**, fornecendo **funcionalidades complementares ao núcleo**.
- Como:
 - **formatação de discos e mídias**.
 - **configuração de dispositivos**.
 - **manipulação de arquivos** (mover, copiar, apagar).
 - **interpretador de comandos**.
 - **terminal, interface gráfica, gerência de janelas**, etc.
 - **desenvolvimento de aplicações** (tradutores, compiladores, linkers).

Arquitetura do Computador

Um computador típico é constituído de **um ou mais processadores**, responsáveis pela **execução das instruções das aplicações**, **uma ou mais áreas de memórias** que armazenam as aplicações em execução (seus códigos e dados) e **dispositivos periféricos** que permitem o **armazenamento de dados e a comunicação com o mundo exterior**, como **discos, terminais e teclados**.



O centro do sistema de computação é o **processador**. Ele é responsável por continuamente ler instruções e dados da memória ou dos periféricos, processá-los e enviar os resultados de volta à memória ou a outros periféricos. Em sua forma mais simples, um processador convencional é constituído de uma **unidade lógica e aritmética (ULA)**, que realiza os cálculos e operações lógicas, um conjunto de registradores para armazenar dados de trabalho e alguns registradores para funções especiais (contador de programa, ponteiro de pilha, flags de status, etc.).

Todas as transferências de dados entre processador, memória e periféricos são feitas através dos **barramentos**: o barramento de endereços indica a posição de memória (ou o dispositivo) a acessar, o barramento de controle indica a operação a efetuar (leitura ou escrita) e o barramento de dados transporta a informação a ser transferida entre o processador e a memória ou um controlador de dispositivo.

O acesso do processador à memória é mediado por um controlador específico (que pode estar fisicamente dentro do próprio chip do processador): a **Unidade de Gerência de Memória (MMU - Memory Management Unit)**. Ela é responsável por analisar cada endereço de memória acessado pelo processador, validá-lo, efetuar conversões de endereçamento porventura necessárias e executar a operação solicitada pelo processador (leitura ou escrita de uma posição de memória). Uma **memória cache** permite armazenar os dados mais recentemente lidos da memória, para melhorar o desempenho.

Os periféricos do computador (discos, teclado, monitor, etc.) são acessados através de circuitos eletrônicos específicos, denominados **controladores**: a placa de vídeo permite o acesso ao monitor, a placa

ethernet dá acesso à rede, o controlador USB permite acesso ao mouse, teclado e outros dispositivos USB externos. Dentro do computador, cada dispositivo é representado por seu respectivo controlador.

Os controladores podem ser acessados através de portas de entrada/saída endereçáveis: a cada controlador é atribuída uma faixa de endereços de portas de entrada/saída. A Tabela a seguir apresenta alguns endereços portas de entrada/saída para acessar controladores em um PC típico (podem variar):

Dispositivo	Endereços de acesso
temporizador	0040-0043
teclado	0060-006F
porta serial COM1	02F8-02FF
controlador SATA	30BC-30BF
controlador Ethernet	3080-309F
controlador	3000-303F

Tabela 2.1: Endereços de acesso a dispositivos (em hexadecimal).

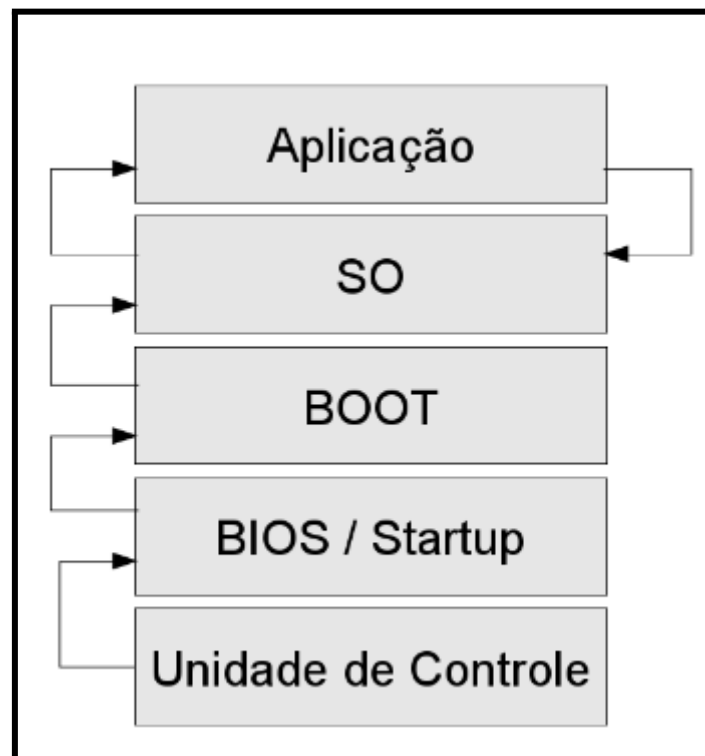
Carregamento e Transferência de Controle

- Sabemos que os programas (processos) têm suas instruções executadas pelo processador.
 - O Sistema Operacional (SO) é responsável por carregar os programas para a memória e gerenciar sua execução.
 - Contudo, o SO também é um programa que precisa ser carregado na memória para entrar em execução.
 - Solução: Um pequeno programa chamado bootstrap ou boot é armazenado no início da memória como ROM.
1. Quando o computador é ligado, quem entra em execução é o Sistema Básico de Entrada/Saída (BIOS), que é um programa pré-gravado em um chip de memória ROM (firmware), sendo responsável pelo suporte básico de acesso ao hardware.
 2. Junto com o BIOS existem na ROM outros dois programas: Setup (configurar parâmetros do BIOS) e o POST (Power On Self Test ou

Auto teste) que **realiza uma sequência de testes e verificações do hardware.**

3. Após as verificações do BIOS, ele **efetua a descompactação para a Memória RAM do bootstrap.**
4. O **contador de programa (PC) da CPU é direcionado para o endereço da primeira instrução do boot executando as instruções deste programa que é responsável por carregar o SO** ou parte dele que for necessária para iniciar o computador.
5. **Ao terminar de carregar o SO, o PC é redirecionado para a primeira instrução do SO na RAM.**
6. **E então o SO está em execução.**

Resumo



1. O **hardware executa o Startup**, que já está em **posição pré-definida da memória ROM.**
2. O **Startup busca o BOOT que deve estar em uma posição pré-definida do disco e transfere o controle para ele.**
 - a. O BIOS lê o setor zero (que contém apenas 512 bytes, denominado Master Boot Record) do HD.

- b. Essa área contém um código que alavanca a inicialização do SO.
3. O **BOOT LOADER** carrega e inicia o kernel e transfere o controle para o SO.
4. O SO busca e transfere o controle para a aplicação.

Questões

1. Como as camadas superiores de software são executadas pelo hardware?

R:

As camadas superiores (como aplicativos) **não falam direto com o hardware.**

O fluxo é assim:

Aplicativo → Sistema Operacional → Hardware

- O programa faz chamadas ao sistema (syscalls)
- O Sistema Operacional traduz isso para instruções de máquina
- A CPU executa essas instruções

Ou seja: **o hardware só executa código de máquina**, e o SO faz a ponte.

2. O hardware consegue distinguir se as instruções são do SO ou se é de uma aplicação qualquer?

R:

Sim!

Isso é feito através de modos de execução da CPU:

- **Modo usuário (user mode)** → programas comuns
- **Modo kernel (kernel mode)** → Sistema Operacional

Algumas instruções são privilegiadas e só podem rodar no modo kernel (ex: acessar dispositivos, memória crítica).

3. Como as instruções de um programa são colocadas na memória, para serem executadas pelo hardware?

R:

Passo a passo:

- Você executa o programa
- O **Sistema Operacional** entra em ação
- Ele usa um **programa chamado loader** (carregador)
- O loader:
 - **Lê o arquivo executável** (ex: .exe, .out)
 - **Coloca na memória RAM**

- **Prepara o processo**
- **Define o ponto de início**

4. Um programa compilado para um tipo de arquitetura pode ser executado em outro tipo de arquitetura?

R:

Normalmente, não.

Exemplo: **Programa compilado para x86 → não roda em ARM**

5. Depois que um programa fonte foi compilado e está pronto para ser executado, quem o coloca na memória?

R:

O Sistema Operacional, usando o loader.

Ele:

- **cria um processo**
- **aloca memória**
- **carrega o código**
- **inicia a execução**

6. Mas quem coloca o S.O. na memória?

R:

Um programa chamado **bootloader**.

Ele é responsável por:

- **carregar o Sistema Operacional da memória secundária (HD/SSD) para a RAM**

7. Mas quem carrega o BOOT na memória?

R:

O firmware da máquina:

- **BIOS ou UEFI**

Eles:

- **ligam o computador**
- **fazem testes (POST)**
- **procuram o bootloader**
- **carregam ele na memória**

8. Mas onde está a BIOS e o Start Up?

R:

A **BIOS/UEFI** fica em uma memória especial da placa-mãe:

- **ROM ou Flash** (não apaga quando desliga)

O **Startup (bootloader)** fica:

- no **HD/SSD** (ex: setor de boot ou partição EFI)

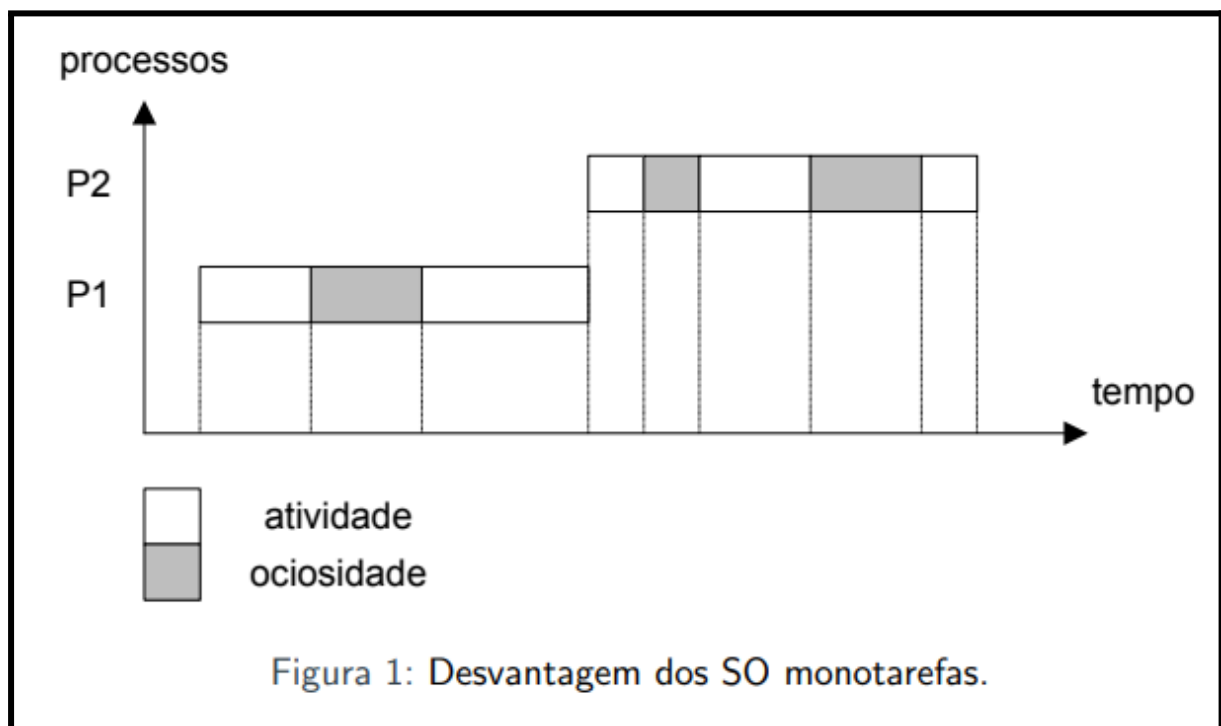
Tipos de SO

Classificação de SO

- Considera características como **tamanho**, **finalidade**, **recursos**, **limitações**, entre outros.
- Um sistema operacional pode se enquadrar em mais de um tipo.

SO Monoprogramados (monotarefas)

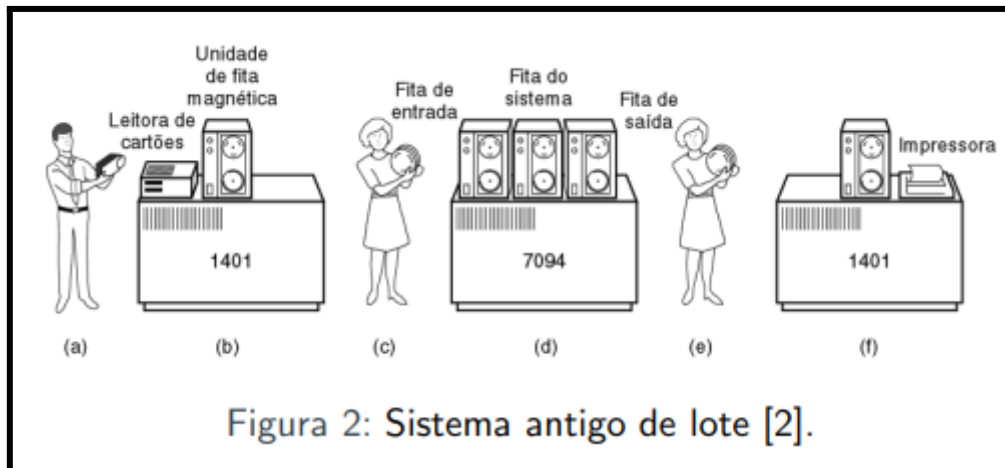
- Um processador, um processo e um usuário.
- **Executa apenas uma tarefa por vez!**
- CPU ociosa durante E/S.
- Simples implementação.
- Exemplo: **MS-DOS** (antes 4.0)



SO em Lotes (batch)

- Programas em fila para execução.
- **Executa tarefas em sequência!**
- Não interação com os usuários.
- Alto grau de uso do sistema.

- Exemplo: OS360



SO Multiprogramados (Multitarefa)

- Um ou mais processadores, processos e usuários.
- Não fica ocioso durante E/S.
- Implementação complexa.
- Conceito de compartilhamento de tempo (time slice)
- **Vários programas na memória, alternando CPU**
- Exemplos: Unix, Linux, Windows NT.

SO Multiusuários

- Suporte a vários usuários.
- Controle de acesso a recursos segundo usuários e permissões.
- **Permitem que diversos usuários acessem os recursos do computador ao mesmo tempo.**
- Exemplos: Unix, Linux, Windows NT

SO Desktop

- Suporte a usuários domésticos e corporativos.
- Ambiente gráfico para interatividade e facilidade de acesso à rede.
- Exemplos: Windows 7, Linux Desktop, MacOS.

SO Servidores

- Gestão de grande quantidade de recursos e provimento de serviços.
- Exemplos: Windows Server, Solaris, Linux.

SO Embarcados

- Voltado para dispositivos com recursos limitados.
- Usados em dispositivos industriais, domésticos, entre outros.
- Exemplos: LynxOS.

SO Portáteis

- Voltado para dispositivos com recursos limitados e portáteis.
- Usados em smartphones e tablets.
- Exemplos: Android, Symbian, iOS.

SO Tempo Real

- Tempo de resposta rígido, ou seja, seu tempo de resposta deve ser previsível no melhor e no pior caso de operação.
- Menos tempo compartilhado e mais prioridade.
- Processos ativados por sensores.
- **Respostas dentro de prazos definidos!**
- Aplicações: usinas, refinarias, tráfego aéreo, entre outros.
- Duas classificações: **soft real-time systems** e **hard real-time systems**.
 - **Soft:** a perda de um prazo é perceptível e degrada o serviço prestado, sem maiores consequências. Exemplos desse tipo de sistema são os softwares de reprodução de mídia: em caso de atrasos, podem ocorrer falhas na música que está sendo tocada.
 - Ou seja, atrasos são tolerados.
 - **Hard:** a perda de um prazo pelo sistema pode perturbar seriamente o sistema físico sob seu controle, com graves consequências humanas, econômicas ou ambientais. Exemplos desse tipo de sistema seriam o controle de funcionamento de uma turbina de avião ou de um freio ABS.
 - Ou seja, não pode atrasar!
- Exemplos: o QNX, RT-Linux e VxWorks.

SO Distribuído

- Neste tipo, o processamento não ocorre em uma única máquina, mas em uma rede de computadores independentes que aparecem para o usuário como um único sistema coerente.
 - **Vários computadores funcionando como um só!**

- **Os recursos de cada computador estão disponíveis a todos na rede, de forma transparente aos usuários.**
- Ao lançar uma aplicação, **o usuário interage com sua interface, mas não sabe onde ela está executando ou armazenando seus arquivos:** o sistema é quem decide, de forma transparente ao usuário.

Multiprogramação

- **Interrupções e Exceções**
- **Buffering**
- **Spooling**

Interrupções de Hardware e Exceções

A comunicação entre o **processador e os dispositivos** se dá através do **acesso às portas de entrada/saída**, que podem ser **lidas e/ou escritas pelo processador**. Esse acesso é feito por iniciativa do processador, quando este **precisa ler ou escrever dados no dispositivo**. Entretanto, muitas vezes um **dispositivo precisa informar o processador rapidamente sobre um evento interno**, como a chegada de um pacote de rede, um clique de mouse ou a conclusão de uma operação de disco. Neste caso, o controlador tem duas alternativas:

- **aguardar até que o processador o consulte**, o que poderá ser demorado caso o processador esteja ocupado com outras tarefas (o que geralmente ocorre);
- **notificar o processador**, enviando a ele uma **requisição de interrupção (IRQ – Interrupt ReQuest)** através do **barramento de controle**.

Ao receber a **requisição de interrupção**, os circuitos do processador suspendem seu fluxo de execução corrente e desviam para um **endereço predefinido**, onde se encontra uma **rotina de tratamento de interrupção (interrupt handler)**. Essa rotina é responsável por tratar a interrupção, ou seja, **executar as ações necessárias para atender o dispositivo que a gerou**. Ao final da rotina de tratamento da interrupção, o processador retoma o código que estava executando quando recebeu a requisição.

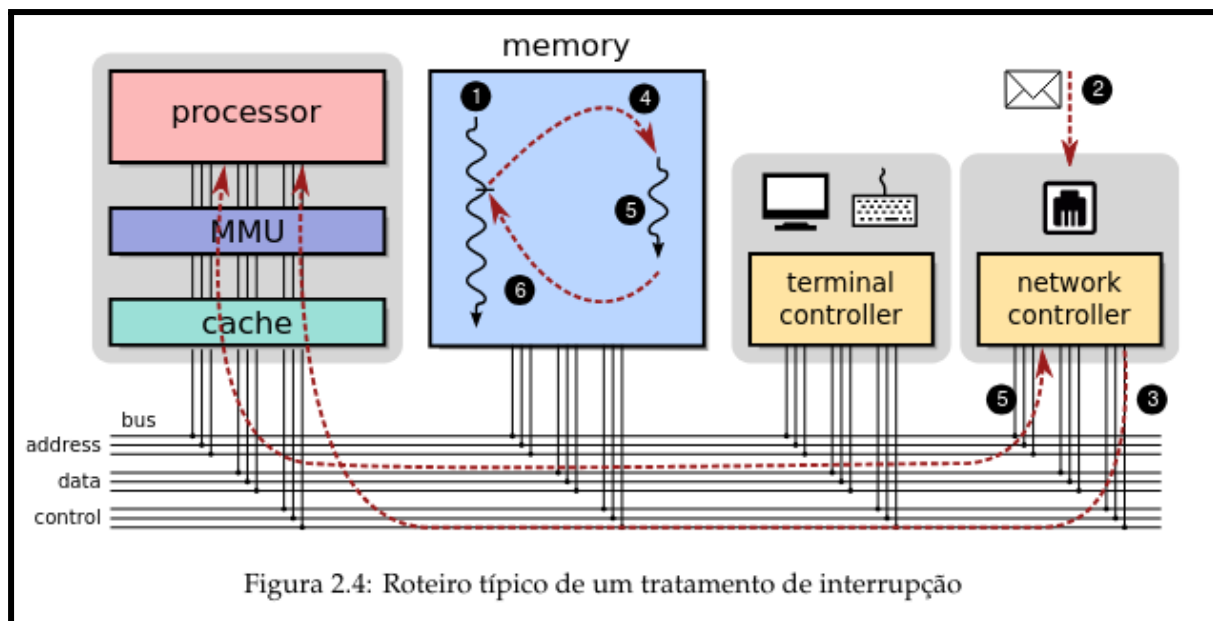
Resumo

- **Interrupções de Hardware:** tratam **eventos assíncronos externos ao processador**.
 - Vêm de dispositivos externos (ex: teclado, disco)
 - **são assíncronas (acontecem a qualquer momento)**
- **Interrupções de Software (Trap):** tratam **eventos síncronos solicitados por aplicação**.
 - geradas intencionalmente pelo programa
 - usadas para **chamar o sistema operacional (syscall)**
 - **são síncronas (dependem da instrução atual)**
- **Exceções:** tratam **eventos síncronos internos ao processador**.
 - causadas por erros durante execução (ex: divisão por zero)
 - **são síncronas (dependem da instrução atual)**

Tratamento de Interrupções de Hardware

Exemplo 1 - A figura a seguir representa os principais passos associados ao tratamento de uma interrupção envolvendo a placa de rede Ethernet, enumerados a seguir:

1. o processador está executando um programa qualquer (em outras palavras, um fluxo de execução);
2. **pacote recebido pela placa Ethernet;**
3. **controlador Ethernet envia uma solicitação de interrupção (IRQ) ao processador;**
4. **o processamento é desviado do programa em execução para a rotina de tratamento da interrupção;**
5. **rotina transfere os dados para a memória;**
6. **a rotina de tratamento da interrupção é finalizada e o processador retorna à execução do programa que havia sido interrompido.**



Essa sequência de ações ocorre a cada requisição de interrupção recebida pelo processador, que geralmente corresponde a um evento ocorrido em um dispositivo: a chegada de um pacote de rede, um click no mouse, uma operação concluída pelo disco, etc. Interrupções não são eventos raros, pelo contrário: centenas ou mesmo milhares de interrupções são recebidas pelo processador por segundo, dependendo da carga e da configuração do sistema (número e tipo dos periféricos). Assim, as rotinas de tratamento de interrupção devem ser curtas e realizar suas tarefas rapidamente, para não prejudicar o desempenho do sistema.

Para distinguir interrupções geradas por dispositivos distintos, cada interrupção é identificada pelo hardware por um número inteiro. Como cada interrupção pode exigir um tipo de tratamento diferente (pois os dispositivos são diferentes), cada IRQ deve disparar sua própria rotina de tratamento de interrupção. A maioria das arquiteturas atuais define uma tabela de endereços de funções denominada **Tabela de Interrupções (IVT - Interrupt Vector Table)**; cada entrada dessa tabela aponta para a rotina de tratamento da interrupção correspondente. Por exemplo, se a entrada 5 da tabela contém o valor 3C20h, então a rotina de tratamento da IRQ 5 iniciará na posição 3C20h da memória RAM. A tabela de interrupções reside em uma posição fixa da memória RAM, definida pelo fabricante do processador, ou tem sua posição indicada pelo conteúdo de um registrador da CPU específico para esse fim.

- Exemplo 2:
 - O relógio (clock) interrompe a CPU em intervalos de tempo fixo.

- Medidos em **ticks**. No Linux é geralmente 10ms.
- A rotina de **interrupção de relógio** (Clock Interrupt Handler) é executada para tratar uma interrupção de relógio.
- Funcionalidades básicas:
 - reiniciar o relógio;
 - atualizar estatísticas de CPU;
 - funções de escalonamento;
 - manipular sinais;
 - atualizar a hora e temporizadores;
 - alarmes (notificar os processos após um período de tempo)

Exceções

A maioria das interrupções recebidas pelo processador têm como origem eventos externos a ele, ocorridos nos dispositivos periféricos e reportados por seus controladores. **Entretanto, alguns eventos gerados pelo próprio processador podem ocasionar o desvio da execução usando o mesmo mecanismo das interrupções:** são as **exceções**. **Ações como instruções ilegais** (inexistentes ou com operandos inválidos), tentativas de divisão por zero ou outros erros de software disparam exceções no processador, que resultam na ativação de uma rotina de tratamento de exceção, usando o mesmo mecanismo das interrupções (e a mesma tabela de endereços de funções). A Tabela a seguir representa parte da tabela de interrupções do processador Intel Pentium.

Tabela 2.2: Tabela de Interrupções do processador Pentium [Patterson and Hennessy, 2005]

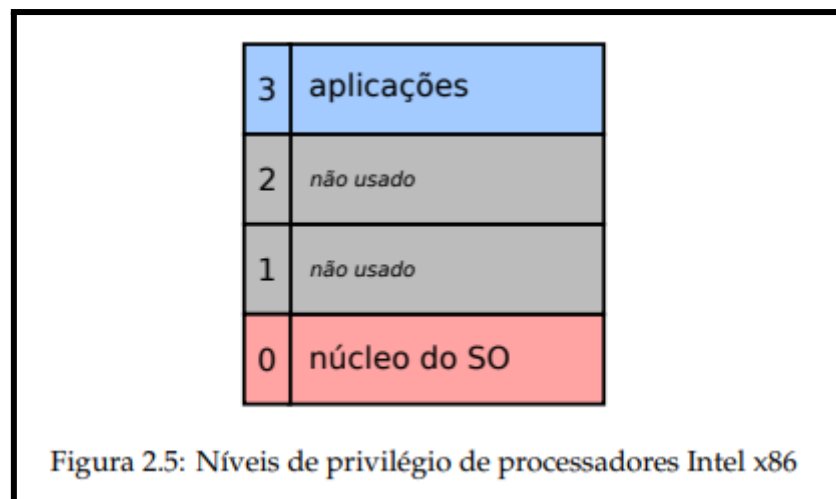
IRQ	Descrição
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	Intel reserved
16	floating point error
17	alignment check
18	machine check
19-31	Intel reserved
32-255	maskable interrupts (devices & exceptions)

Níveis de Privilégio

Núcleo, drivers, utilitários e aplicações são constituídos basicamente de **código de máquina**. Todavia, devem ser diferenciados em sua capacidade de interagir com o hardware: **enquanto o núcleo e os drivers devem ter pleno acesso ao hardware**, para poder configurá-lo e gerenciá-lo, os **aplicativos e utilitários devem ter acesso mais restrito a ele**, para não interferir nas configurações e na gerência, o que poderia desestabilizar o sistema inteiro. Além disso, **aplicações com acesso pleno ao hardware seriam um risco à segurança**, pois poderiam contornar facilmente os mecanismos de controle de acesso aos recursos (tais como arquivos e áreas de memória).

Para **permitir diferenciar os privilégios de execução dos diferentes tipos de software**, os **processadores modernos implementam níveis de**

privilégio de execução. Esses níveis são controlados por **flags especiais na CPU**, que podem ser ajustados em função do tipo de código em execução. Os processadores no padrão Intel x86, por exemplo, dispõem de 4 níveis de privilégio (0 . . . 3, sendo 0 o nível mais privilegiado), embora a maioria dos sistemas operacionais construídos para esses processadores só use os níveis extremos (0 para o núcleo do sistema operacional e 3 para utilitários e aplicações)¹.



Na forma mais simples desse esquema, podemos **considerar dois níveis básicos de privilégio**:

Nível núcleo: também denominado **nível supervisor, sistema, monitor** ou ainda **kernel space**. Para um código executando nesse nível, **todas as funcionalidades do processador estão disponíveis**: todas as **instruções disponíveis podem ser executadas** e **todos os recursos internos (registradores e portas de entrada/saída) e áreas de memória podem ser acessados**. Ao ser ligado durante a inicialização do computador, o processador entra em operação neste nível. **Acesso total ao hardware!**

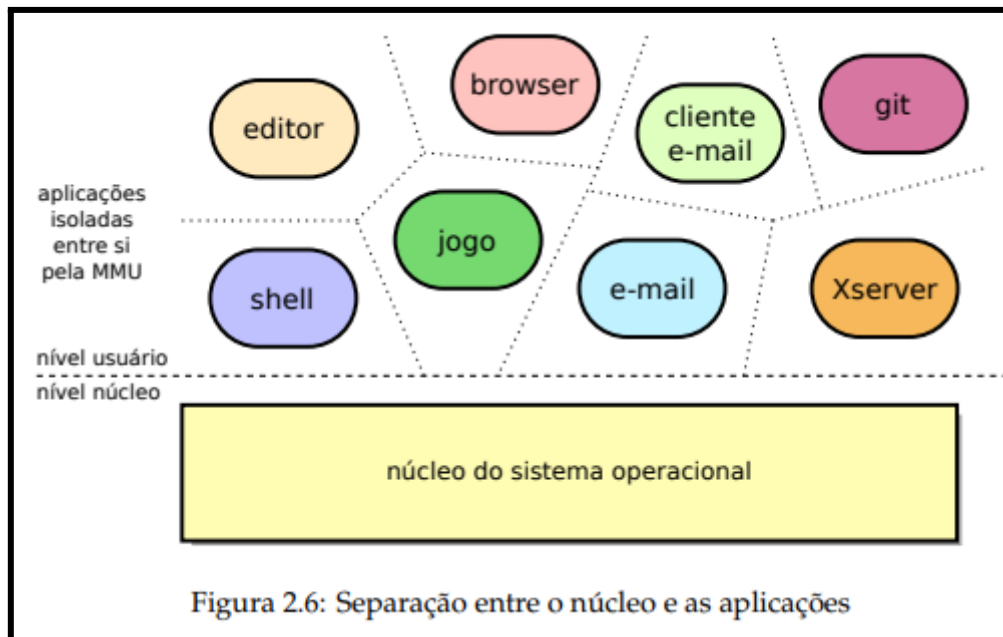
Nível usuário: neste nível, também chamado **userspace**, **somente um subconjunto das instruções do processador e registradores estão disponíveis**. Por exemplo, **instruções consideradas “perigosas”, como RESET** (reiniciar o processador) e **IN/OUT** (acessar portas de entrada/saída), são proibidas. **Caso o código em execução tente executar uma instrução proibida, o hardware irá gerar uma exceção**, desviando a execução para uma **rotina de tratamento dentro do núcleo**. Essa rotina irá tratar o erro, provavelmente abortando o programa em execução.

Dessa forma, o **núcleo do sistema operacional, bem como drivers e o código de inicialização**, executam em **modo núcleo**, enquanto os **programas utilitários e os aplicativos executam em modo usuário**. Os flags que definem o nível de privilégio só podem ser modificados por código

executando no nível núcleo, o que impede usuários maliciosos de contornar essa barreira de proteção.

Além da proteção oferecida pelos níveis de privilégio, **o núcleo do sistema operacional pode configurar a unidade de gerência de memória (MMU) para criar uma área de memória exclusiva para cada aplicação**, isolada das demais aplicações e do núcleo. **As aplicações não podem contornar essa barreira de memória**, pois a configuração da MMU só pode ser alterada em modo supervisor, ou seja, pelo próprio núcleo.

Com a proteção provida pelos níveis de privilégio do processador e pelas áreas de memória exclusivas configuradas pela MMU, obtém-se um modelo de execução confinada, no qual as aplicações executam isoladas umas das outras e do núcleo, conforme ilustrado na Figura a seguir. Esse modelo de execução é bastante robusto, pois isola os erros, falhas e comportamentos indevidos de uma aplicação do restante do sistema, proporcionando mais estabilidade e segurança.



Interrupções de Software

O confinamento de cada aplicação em sua área de memória, imposto pela MMU aos acessos em memória em nível usuário, provê robustez e confiabilidade ao sistema, pois garante que uma aplicação não poderá interferir nas áreas de memória de outras aplicações ou do núcleo. Entretanto, **essa proteção introduz um novo problema: como invocar, a partir da aplicação, as rotinas oferecidas pelo núcleo para o acesso ao hardware e demais serviços do SO?** Deve-se lembrar que o código do núcleo reside em uma área de memória inacessível à aplicação, **então operações de desvio como jump e call não funcionariam.**

A solução desse problema está no mecanismo de interrupção, apresentado anteriormente. Quando ocorre uma **interrupção**, a **execução é desviada para um endereço predefinido** (desvio incondicional) e o **processador passa a operar em nível núcleo** (elevação de privilégio). Os processadores atuais implementam uma **instrução específica que usa esse mesmo mecanismo para ativar rotinas do núcleo**. **Ao ser executada, essa instrução comuta o processador para o nível núcleo e executa o código contido em uma rotina pré definida**, de forma similar a uma interrupção de hardware. Por essa razão, esse mecanismo é denominado **interrupção de software, ou trap**.

A ativação de uma **rotina do núcleo** usando esse mecanismo é denominada **chamada de sistema (system call ou syscall)**. Os sistemas operacionais definem **chamadas de sistema para todas as operações envolvendo o acesso a recursos de baixo nível** (periféricos, arquivos, alocação de memória, etc.) ou **abstrações lógicas** (criação e encerramento de tarefas, operadores de sincronização, etc.). Geralmente as **chamadas de sistema são oferecidas para as aplicações em modo usuário através de uma biblioteca do sistema (system library)**, que prepara os parâmetros, invoca a chamada e, no retorno desta, **devolve à aplicação os resultados obtidos**.

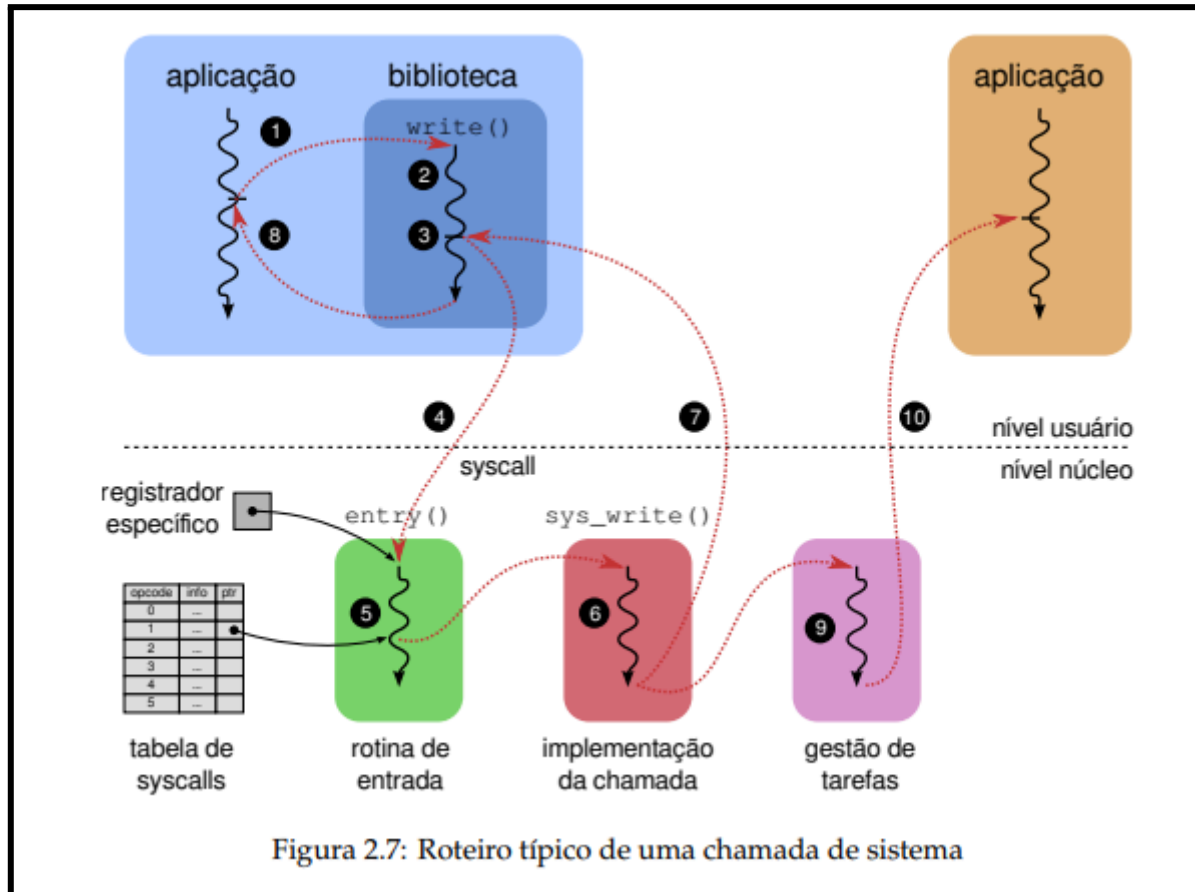
Nos processadores modernos a chamada de sistema e seu retorno são feitos usando instruções específicas como **sysenter/sysexit** (x86 32 bits), ou **syscall/sysret** (x86 64 bits). **Antes de invocar a chamada de sistema, alguns registradores do processador são preparados com valores específicos**, como o **número da operação desejada (opcode)**, geralmente no **registrador AX**, o **endereço dos parâmetros da chamada**, etc. As regras para o preenchimento dos registradores são específicas para cada chamada de sistema, em cada sistema operacional.

Exemplos de chamadas de sistema no Linux:

- **read**: lê a partir de um descritor de arquivo.
- **write**: escreve para um descritor de arquivo.
- **mmap**: mapeia arquivos e dispositivos na memória.
- **fstat**: devolve informações sobre um arquivo.
- **mprotect**: modifica a proteção de uma região de memória.
- **access**: verifica se o processo pode acessar um arquivo.
- **open**: abre ou cria um arquivo.
- **close**: fecha um descritor de arquivo.
- **brk**: modifica o tamanho do segmento de dados.

Tratamento de Interrupções de Software

A Figura a seguir detalha o **funcionamento básico da chamada de sistema write**, que escreve dados em um arquivo previamente aberto. A seguinte sequência de passos é realizada:



1. aplicação invoca a função `write()` da biblioteca de sistema;
2. a função `write()` preenche registradores CPU com os parâmetros e escreve o opcode da `syscall` no registrador AX.
3. função `write()` invoca uma chamada de sistema (`syscall`);
4. processador alterna para modo núcleo e executa a rotina `entry()`;
5. a rotina recebe o opcode da operação via registrador AX, consulta a tabela de chamadas de sistema e invoca a operação requisitada;
6. a operação `sys write()` obtém, valida e processa os parâmetros;
7. ao final, armazena o retorno e devolve o controle para `write()`;
8. a função `write()` devolve controle para aplicação;
9. caso a operação solicitada não possa ser concluída imediatamente, a função `sys_write` passa o controle para a gerência de tarefas, ao invés de retornar para a aplicação solicitante. Isto ocorre, por exemplo, quando é solicitada a leitura de uma entrada do teclado.

10. Na sequência, a **gerência de tarefas suspende a execução da aplicação solicitante e devolve o controle do processador a alguma outra aplicação que também esteja aguardando o retorno de uma chamada de sistema e cuja operação solicitada já tenha sido concluída.**

Mascaramento de interrupções

O que é?

- **Bloquear (ignorar temporariamente) interrupções**

Problemas:

- **Dispositivos podem não ser atendidos**
- **Perda de dados (ex: buffer cheio)**
- **Sistema pode travar ou ficar lento**
- **Eventos importantes podem ser ignorados**

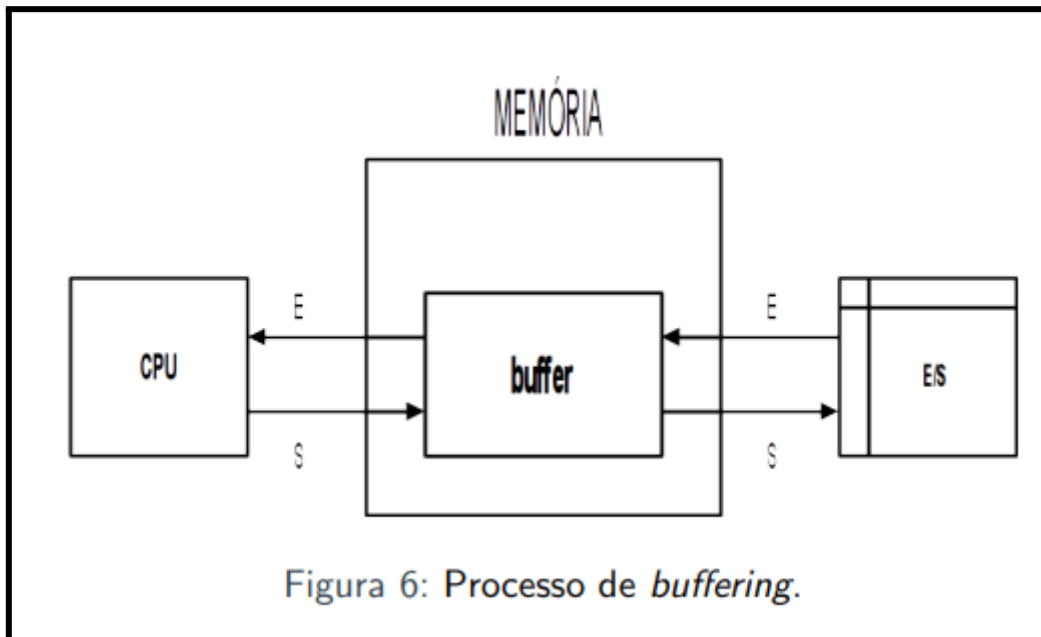
Se ignorar por muito tempo:

- **Pode causar falhas graves no sistema**
- **Perda de sincronização com hardware**

Como evitar:

- **Manter interrupções desativadas por pouco tempo**
- **Usar mecanismos mais seguros (ex: seções críticas bem controladas)**
- **Priorizar interrupções importantes**

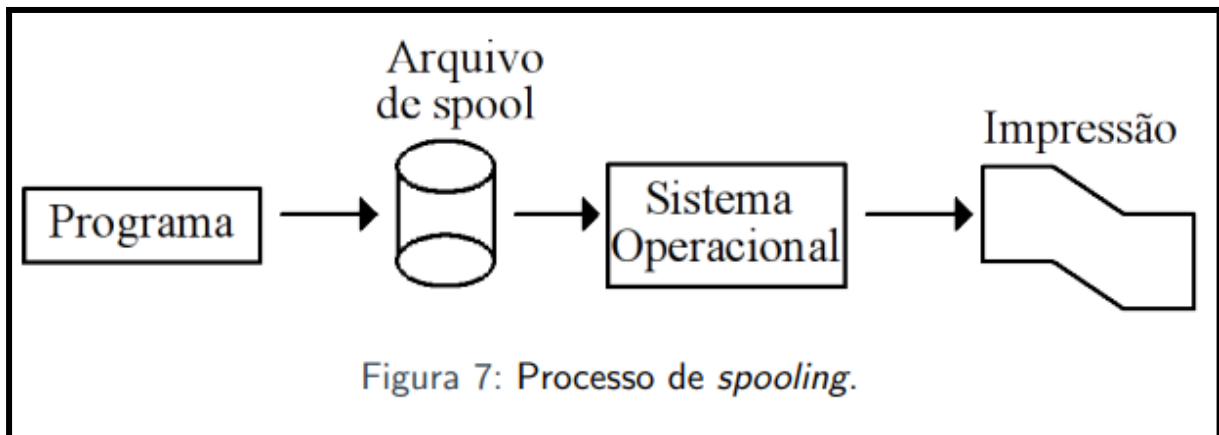
Buffering



- **Buffering** é uma técnica que utiliza uma área de memória (buffer) para armazenar temporariamente dados durante operações de entrada e saída. Ele serve para compensar a diferença de velocidade entre a CPU e os dispositivos de E/S, já que a CPU é muito mais rápida. Com o uso de buffers, a CPU não precisa esperar diretamente pelos dispositivos, podendo continuar a execução enquanto os dados são transferidos aos poucos.
- **Buffer: Armazenamento temporário de dados na RAM**, utilizado para equilibrar a diferença de velocidade entre 2 dispositivos ou processos.
- Geralmente utilizado para otimização!

Spooling

- Simultaneous Peripheral Operating On Line



- Serve para não travar outros processos.
- É uma **técnica de gerenciamento de dados** que **armazena tarefas temporariamente em um buffer** ou em um **arquivo spool** enquanto **aguardam o processo por periféricos lentos**, como impressoras.
 - Faz uma **fila** para os processos.
- Isso libera a CPU, permitindo multitarefa.

Cache

- Caching é o uso de uma memória mais rápida para armazenar cópias de dados frequentemente acessados. Isso melhora o desempenho porque evita acessos repetidos a dispositivos mais lentos, como o disco. Quando um dado já está no cache, ele pode ser acessado muito mais rapidamente pela CPU, reduzindo o tempo de resposta do sistema.

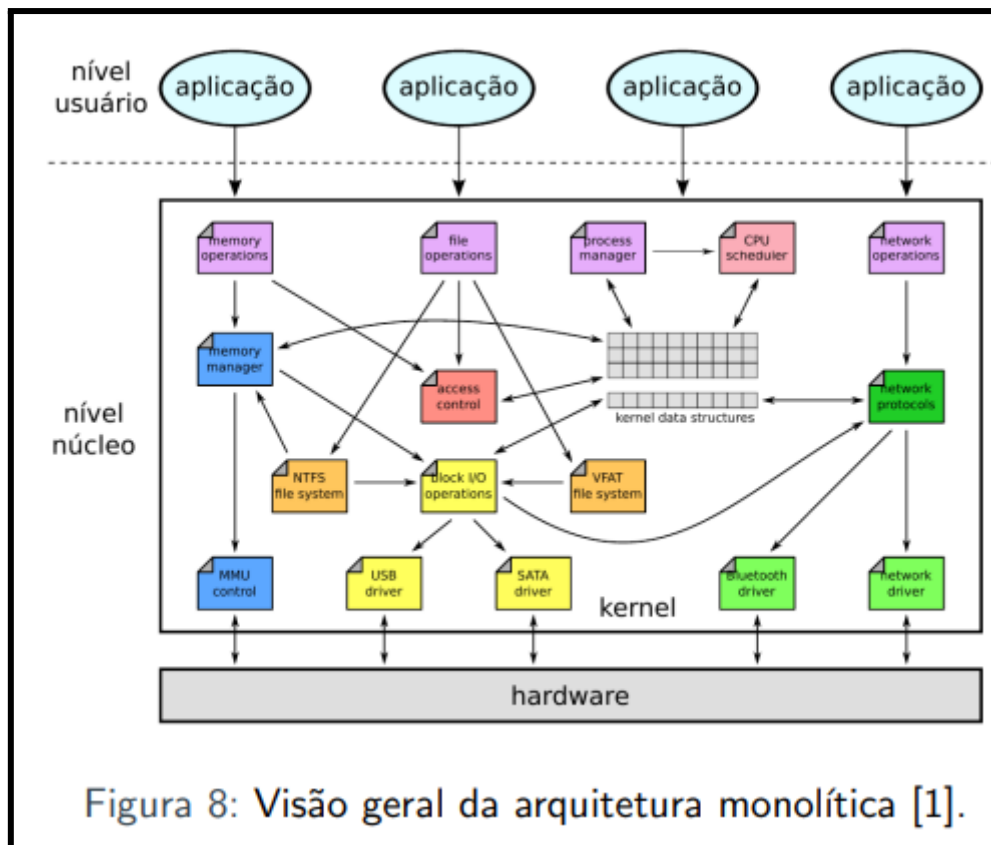
Arquiteturas de SOs

- Os atuais Sistemas Operacionais tendem a ser complexos e oferecem vários serviços.
- Suportam uma variedade de recursos de hardware e software.
- A **arquitetura do sistema operacional ajuda a gerenciar essa complexidade**:
 - **organizando componentes de sistema operacional e**
 - **como esses componentes são executados.**
- **Principais arquiteturas**
 - **Sistemas monolíticos** (ex: FreeBSD, Linux, MS-DOS).
 - **Sistemas micro núcleo - microkernel** (ex: Minix, GNU Mach).

- Sistemas em camadas (ex: MULTICS).
- **Sistemas híbridos** (ex: Windows NT, Mac OS).
- **Maquinas virtuais** (ex: Xen, KVM, Virtual Box, VMWare, JVM).
- **Contêineres** (ex: Docker e Kubernetes).
- Sistemas exonúcleo (ex: Nemesis - ideia de LibOS).
- Sistemas uni núcleo (ex: MirageOs, OSv).

Sistemas Monolíticos

- Estruturam **todo o kernel como um único programa em modo núcleo**, onde rotinas interagem livremente.
- **Extremamente eficiente** devido à baixa sobrecarga entre módulos, contudo, **falhas em um módulo podem derrubar todo o sistema**.
- Outro **problema** relacionado às arquiteturas monolíticas diz respeito ao **processo de desenvolvimento**. Como os componentes do sistema têm acesso direto uns aos outros, podem ser fortemente interdependentes, **tornando a manutenção e evolução do núcleo mais complexas**. Como as dependências e pontos de interação entre os componentes podem ser pouco evidentes, **pequenas alterações na estrutura de dados de um componente podem ter um impacto inesperado em outros componentes**, caso estes acessem aquela estrutura diretamente.
 - Ou seja, **difícil manutenção e baixa modularidade!!**



Sistemas Micronúcleo

- **Minimizam o código executado no modo núcleo**, movendo serviços como drivers, sistemas de arquivos e rede para o espaço de usuário.
- **Menor desempenho**, devido à alta troca de mensagens, porém **aumenta a confiabilidade e segurança**, pois uma falha em um serviço (tipo drivers) não trava o sistema inteiro.
 - **Alta modularidade, mais seguro e estável!**
- Ideal para sistemas embarcados.

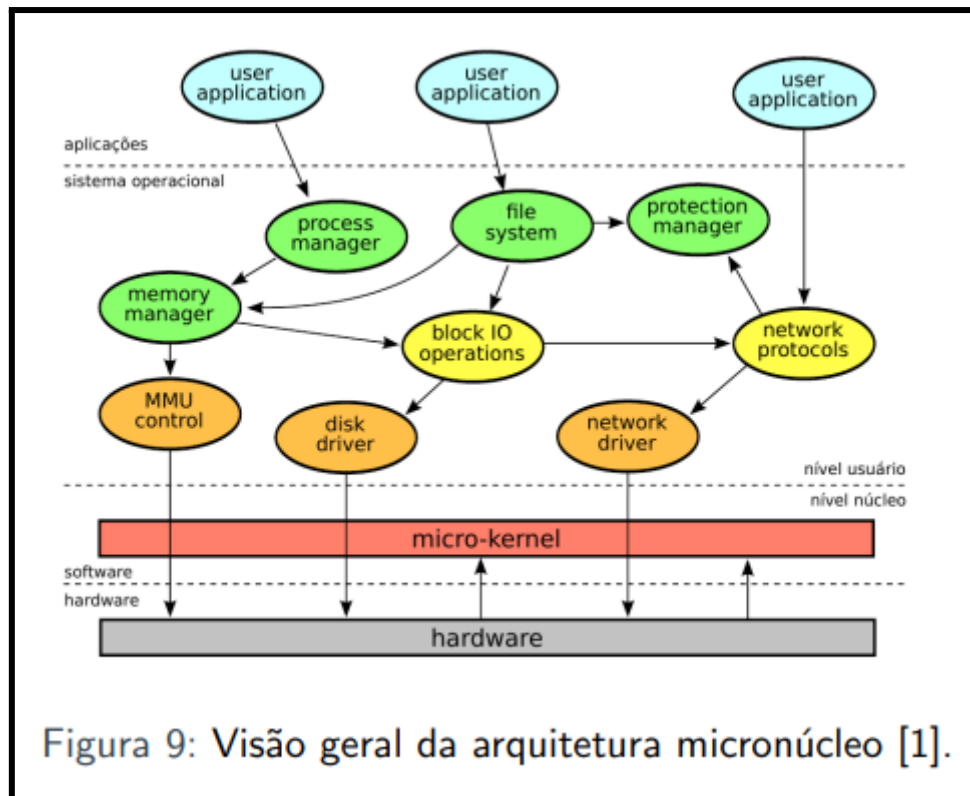
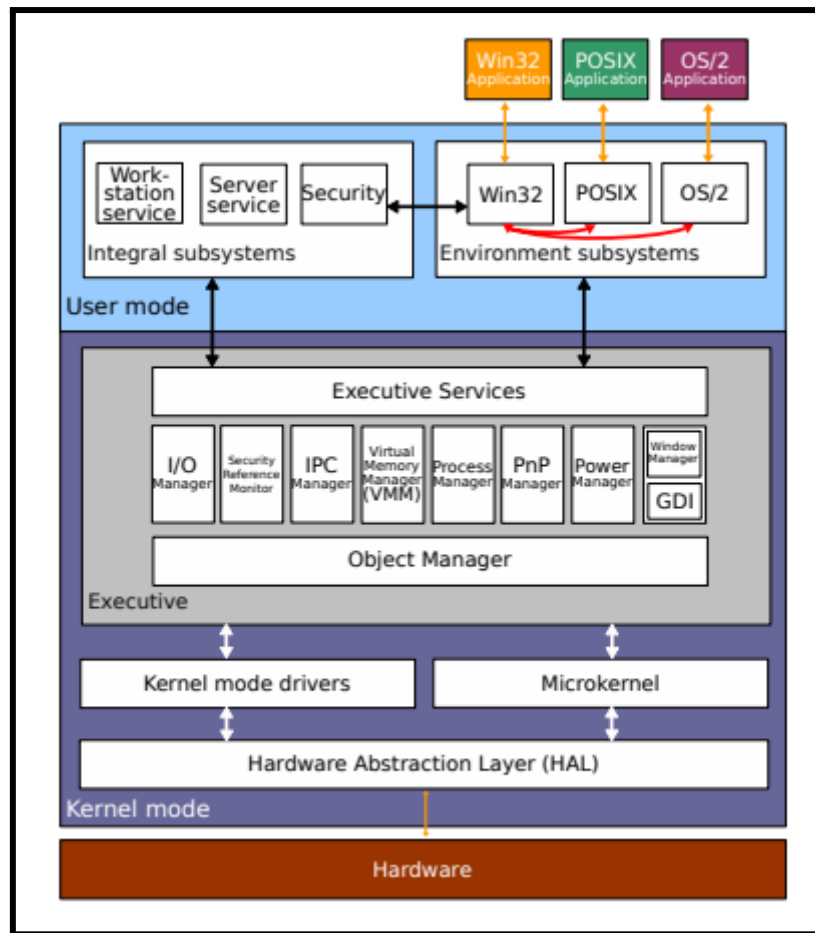


Figura 9: Visão geral da arquitetura micronúcleo [1].

Sistemas Híbridos

Apesar das boas propriedades de modularidade, flexibilidade e robustez proporcionadas pelos micronúcleos, sua adoção não teve o sucesso esperado devido ao baixo desempenho. **Uma solução encontrada para esse problema (dos micronúcleos) consiste em trazer de volta ao núcleo os componentes mais críticos, para obter melhor desempenho.** Essa abordagem intermediária entre o núcleo monolítico e micronúcleo é denominada **núcleo híbrido**. É comum observar também nos núcleos híbridos uma influência da arquitetura em camadas.

Vários sistemas operacionais atuais empregam núcleos híbridos, entre eles o **Microsoft Windows**, a partir do Windows NT. As primeiras versões do Windows NT podiam ser consideradas micronúcleo, mas a partir da versão 4 vários subsistemas foram movidos para dentro do núcleo para melhorar seu desempenho, transformando-o em um núcleo híbrido. Os sistemas **MacOS** e **iOS** da **Apple** também adotam um **núcleo híbrido** chamado XNU (de X is Not Unix), construído a partir dos núcleos Mach (micronúcleo) e FreeBSD (monolítico). A figura 3.3 mostra a arquitetura interna do núcleo usado no SO Windows 2000.



Máquinas Virtuais

Uma máquina virtual é uma **camada de software que “transforma” um sistema em outro**, ou seja, que **usa os serviços** fornecidos por um sistema operacional (ou pelo hardware) **para construir a interface de outro sistema**. Por exemplo, o ambiente (JVM - Java Virtual Machine) **usa os serviços de um sistema operacional convencional** (Windows ou Linux) **para construir um computador virtual que processa bytecode, o código binário dos programas Java**. Da mesma forma, o ambiente VMWare permite executar o sistema Windows sobre um sistema Linux subjacente, ou vice-versa.

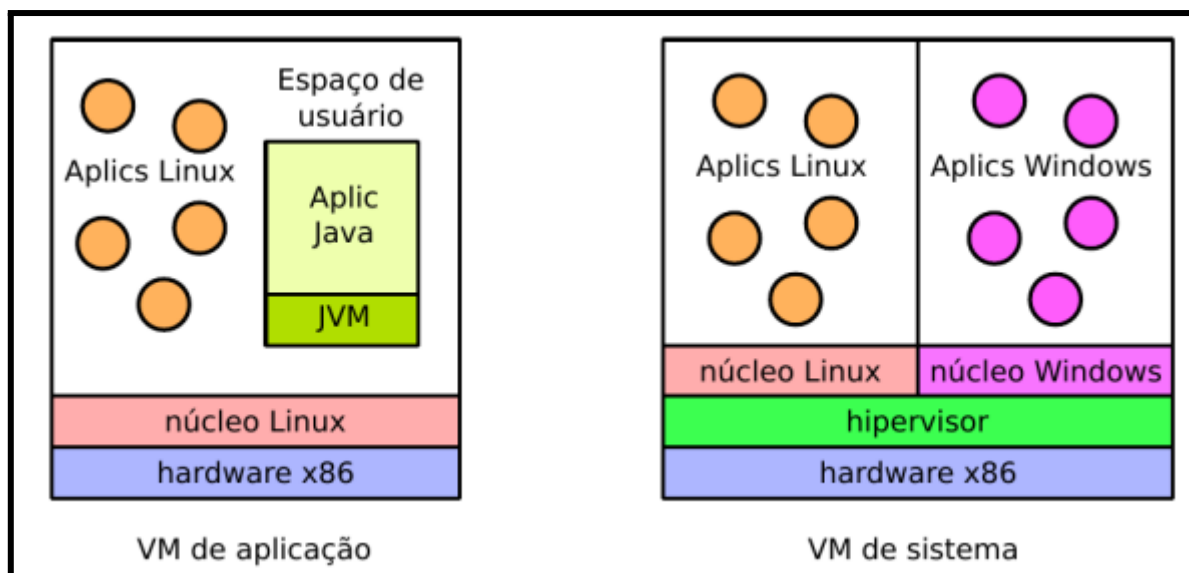
Um **ambiente de máquinas virtuais** consiste de **três partes básicas**, que podem ser observadas a seguir:

- **O sistema real, ou sistema hospedeiro (host)**, que contém os recursos reais de hardware e software do sistema;
- a **camada de virtualização**, denominada **hipervisor ou monitor de virtualização** (VMM - Virtual Machine Monitor), que constrói a máquina virtual a partir dos recursos do sistema real;

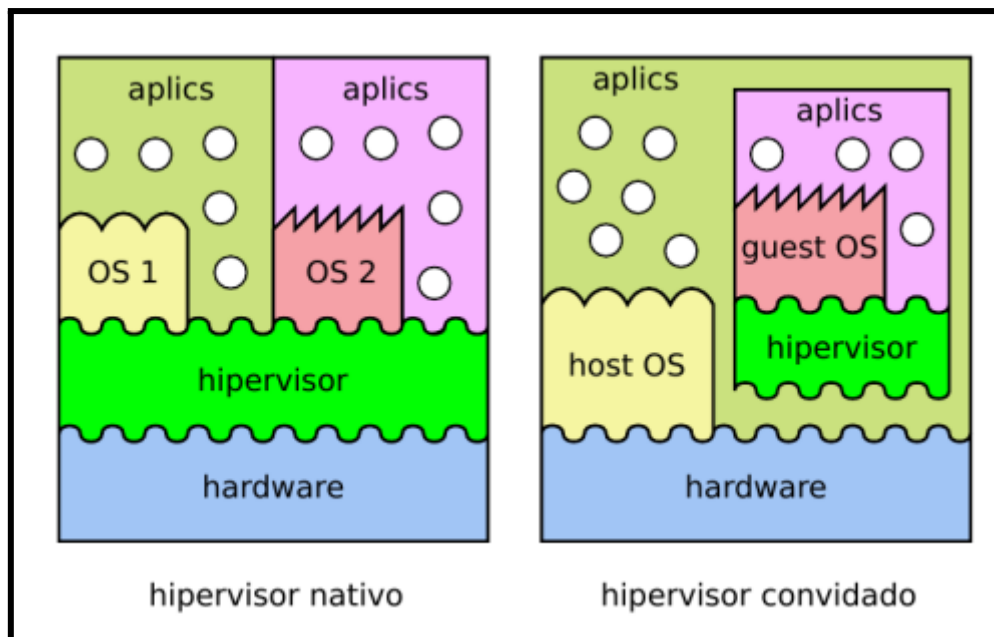
- o **sistema virtual, ou sistema convidado (guest)**; em alguns casos, vários sistemas virtuais podem coexistir, executando sobre o mesmo sistema real.

Existem diversas possibilidades de uso da virtualização, levando a vários tipos de hipervisor, por exemplo:

- **Hipervisor de aplicação:** suporta a execução de uma única aplicação em uma linguagem específica. As máquinas virtuais **JVM** (Java Virtual Machine) e **CLR** (Common Language Runtime, para C#) são exemplos típicos dessa abordagem.
- **Hipervisor de sistema:** suporta a execução de um sistema operacional convidado com suas aplicações. Esta categoria se subdivide em:



- **Hipervisor nativo:** executa diretamente sobre o hardware, virtualizando os recursos deste para construir máquinas virtuais, onde executam vários sistemas operacionais com suas respectivas aplicações. Um exemplo deste tipo de hipervisor é o ambiente VMware ESXi Server.
- **Hipervisor convidado:** executa sobre um sistema operacional hospedeiro. Os hipervisores VMware Workstation e VirtualBox implementam essa abordagem.



Contêineres

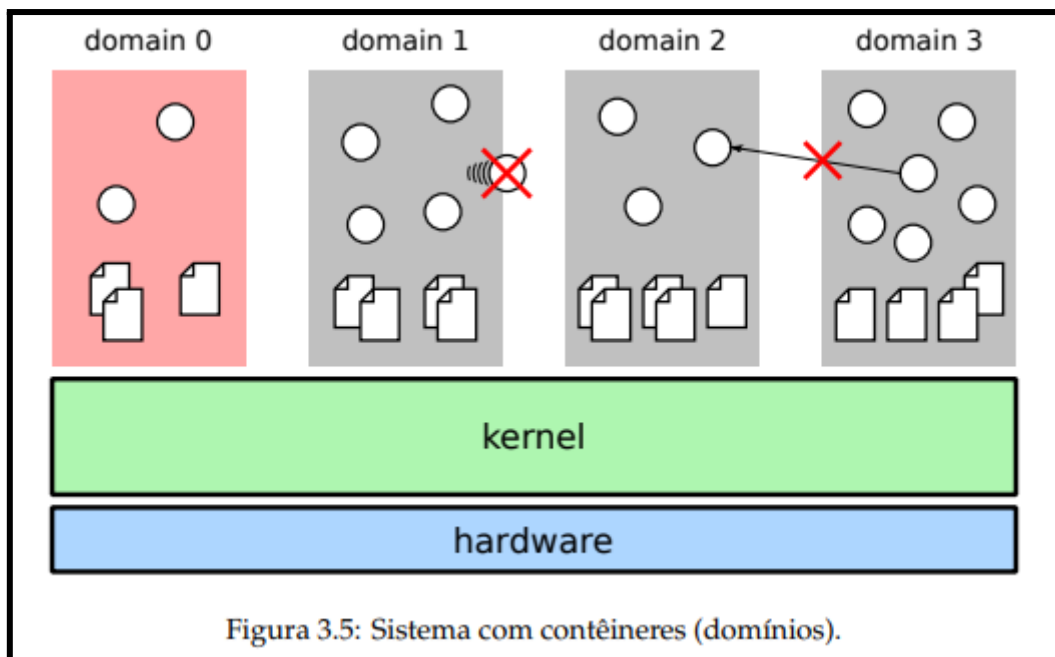
Além da virtualização, outra forma de isolar aplicações ou subsistemas em um sistema operacional consiste na **virtualização do espaço de usuário** (userspace). Nesta abordagem, denominada **servidores virtuais ou contêineres**, o **espaço de usuário do sistema operacional é dividido em áreas isoladas denominadas domínios ou contêineres**. A cada domínio é alocada uma parcela dos recursos do sistema operacional, como memória, tempo de processador e espaço em disco.

Para garantir o **isolamento entre os domínios**, cada domínio tem sua **própria interface de rede virtual** e, portanto, seu **próprio endereço de rede**. Além disso, na maioria das implementações, **cada domínio tem seu próprio espaço de nomes para os identificadores de usuários, processos e primitivas de comunicação**. Assim, é possível encontrar um usuário pedro no domínio d3 e outro usuário pedro no domínio d7, sem relação entre eles nem conflitos. **Essa noção de espaços de nomes distintos se estende aos demais recursos do sistema**: identificadores de processos, semáforos, árvores de diretórios, etc. Entretanto, **o núcleo do sistema operacional é o mesmo para todos os domínios**.

Processos em um mesmo domínio podem interagir entre si normalmente, mas **processos em domínios distintos não podem ver ou interagir entre si**. Além disso, **processos não podem trocar de domínio nem acessar recursos de outros domínios**. Dessa forma, **os domínios são vistos como sistemas distintos, acessíveis somente através de seus endereços de rede**. Para fins de gerência, normalmente é definido um domínio d0, chamado de domínio inicial, privilegiado ou de gerência,

cujos processos têm visibilidade e acesso aos processos e recursos dos demais domínios.

A figura a seguir mostra a estrutura típica de um ambiente de contêineres. Nela, pode-se observar que um **processo pode migrar de d0 para d1**, mas que os processos em **d1 não podem migrar para outros domínios**. A comunicação entre processos confinados em domínios distintos (d2 e d3) também é proibida.



O **núcleo Linux** oferece diversos mecanismos para o **isolamento de espaços de recursos**, que são usados por plataformas de gerenciamento de contêineres como **Linux Containers (LXC)**, **Docker** e **Kubernetes**.

Sistemas Exonúcleo

Em um sistema convencional, as aplicações executam sobre uma pilha de serviços de abstração e gerência de recursos providos pelo sistema operacional. Esses serviços facilitam a construção de aplicações, mas **podem constituir uma sobrecarga significativa para o desempenho do sistema**. Para cada abstração ou serviço utilizado é necessário interagir com o núcleo através de chamadas de sistema, que **impactam negativamente o desempenho**. Além disso, os serviços providos pelo núcleo devem ser **genéricos o suficiente para atender a uma vasta variedade de aplicações**. Aplicações com demandas muito específicas, como um servidor Web de alto desempenho, nem sempre são atendidas da melhor forma possível.

Os **exonúcleos (exokernels)** tentam trazer uma resposta a essas questões, **reduzindo ainda mais os serviços oferecidos pelo núcleo**. Em

um sistema exonúcleo, o **núcleo do sistema apenas proporciona acesso controlado aos recursos do hardware**, mas não implementa nenhuma abstração. Por exemplo, o núcleo provê acesso compartilhado à interface de rede, mas não implementa nenhum protocolo. **Todas as abstrações e funcionalidades de gerência que uma aplicação precisa terão de ser implementadas pela própria aplicação, em seu espaço de memória.**

Para simplificar a construção de aplicações sobre exonúcleos, são usados sistemas operacionais “de biblioteca” ou LibOS (Library Operating System), que implementam as abstrações usuais, como memória virtual, sistemas de arquivos, semáforos e protocolos de rede. Um LibOS nada mais é que um conjunto de bibliotecas compartilhadas usadas pela aplicação para prover os serviços de que esta necessita. Diferentes aplicações sobre o mesmo exonúcleo podem usar LibOS diferentes, ou implementar suas próprias abstrações. A Figura a seguir ilustra a arquitetura de um exonúcleo típico.

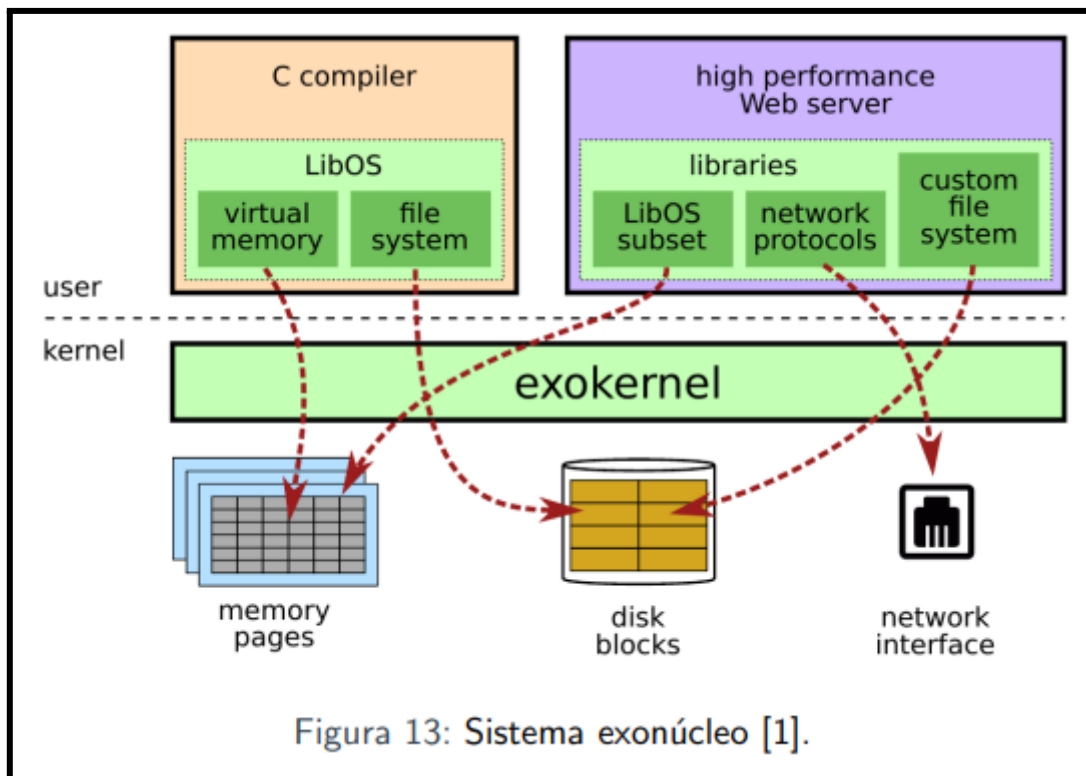


Figura 13: Sistema exonúcleo [1].

É importante observar que um **exonúcleo é diferente de um micronúcleo**, pois neste último as **abstrações são implementada por um conjunto de processos servidores independentes e isolados entre si**, enquanto **no exonúcleo cada aplicação implementa suas próprias abstrações (ou as incorpora de bibliotecas compartilhadas).**

Até o momento não existem exonúcleos em produtos comerciais, as implementações mais conhecidas são esforços de projetos de pesquisa, como **Aegis/ExOS** e **Nemesis**.

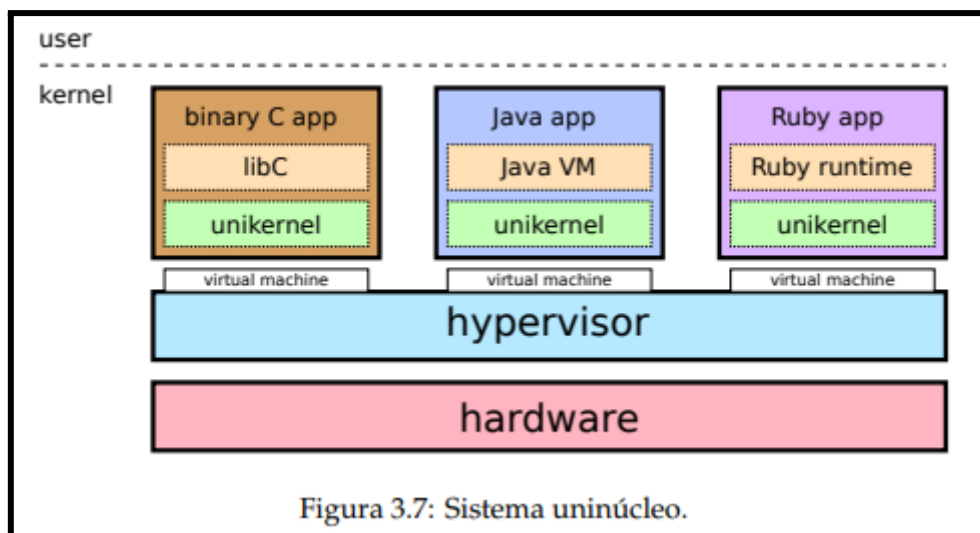
Sistemas Uninúcleo

Nos **sistemas uninúcleo (unikernel)**, um **núcleo de sistema operacional**, as **bibliotecas** e uma **aplicação** são **compilados e ligados entre si**, formando um **bloco monolítico de código**, que executa em um **único espaço de endereçamento**, em **modo privilegiado**. Dessa forma, o **custo da transição aplicação/núcleo** nas chamadas de sistema **diminui muito**, gerando um **forte ganho de desempenho**.

Outro ponto positivo da abordagem uninúcleo é o fato de **incluir no código final somente os componentes necessários para suportar a aplicação-alvo e os drivers necessários para acessar o hardware-alvo**, levando a um sistema compacto, que pode ser lançado rapidamente.

A estrutura de uninúcleo se mostra **adequada para computadores que executam uma única aplicação**, como **servidores de rede (HTTP, DNS, DHCP)** e **serviços em ambientes de nuvem computacional** (bancos de dados, Web Services). De fato, o **maior interesse em usar uninúcleos vem do ambiente de nuvem**, pois os “computadores” da nuvem são na verdade máquinas virtuais com hardware virtual simples (com drivers padronizados) e que executam serviços isolados.

Os sistemas **OSv** e **MirageOS** são bons exemplos de uninúcleos. Como eles foram especialmente concebidos para as nuvens, são também conhecidos como “**CloudOS**”. A Figura a seguir mostra a arquitetura desses sistemas.



Resumo

Arquitetura	Benefícios	Deficiências
Monolítica	Alto desempenho, comunicação direta entre módulos	Difícil manutenção, baixa modularidade
Micronúcleo	Alta modularidade, mais seguro e estável	Overhead de comunicação (mais lento)
Híbrida	Equilíbrio entre desempenho e modularidade	Mais complexa de implementar
Máquinas Virtuais	Isolamento total, portabilidade, múltiplos SO	Alto consumo de recursos, menor desempenho
Contêineres	Leves, rápidos, eficiente uso de recursos	Menor isolamento que VM, dependem do SO host

Processos

[https://www.kufunda.net/publicdocs/Sistemas%20Operacionais%20Modernos%20\(Andrew%20S.%20Tanenbaum,%20Herbert%20Bos\).pdf](https://www.kufunda.net/publicdocs/Sistemas%20Operacionais%20Modernos%20(Andrew%20S.%20Tanenbaum,%20Herbert%20Bos).pdf)

Introdução

- Uma das funcionalidades do SO é o gerenciamento de aplicação.
- As **aplicações quando estão em execução** são denominadas de **Processos**.
- **Processos:**
 - **Um processo é um programa em execução.**
 - Um processo é composto por: **código, dados e contexto de execução (stack e heap).**
 - Um mesmo programa pode possuir várias instâncias em execução, com mesmo código, mas dados e contexto de execução diferentes.
 - Processos se referem a:
 - Programas do usuário
 - Programas do sistema (**daemons**)

- **Gerenciamento de Processos pelo SO**
 - **criar e finalizar** processos.
 - **suspender e retomar** processos.
 - **bloquear e ativar** processos.
 - **alterar a prioridade** de processos.
 - **selecionar e despachar** processos (para CPU).
 - **viabilizar a comunicação** entre processos (IPC).

Ciclos

- Criação.
- Execução.
- Término.

Criação

- **Como são criados os processos?**
 - Execução de chamada ao sistema de criação de processos.
- **Quando são criados os processos?**
 - **Início do sistema.**
 - **Solicitação usuário para criar um novo processo.**
 - **Solicitação de outro processo para criar um novo processo.**
 - **Início de um job (tarefa) em lote.**

Execução

- **CPU bound** (orientados a CPU)
 - São processos que **gastam a maior parte do tempo usando a CPU**, fazendo cálculos.
 - Exemplos:
 - Renderização de gráficos
 - Cálculos matemáticos pesados
 - Simulações
- **I/O bound** (orientados a E/S)
 - São processos que **passam a maior parte do tempo esperando operações de entrada/saída**, e não usando a CPU.
 - Exemplos:
 - Download de arquivos
 - Leitura/gravação em disco
 - Aplicações que acessam banco de dados

Término

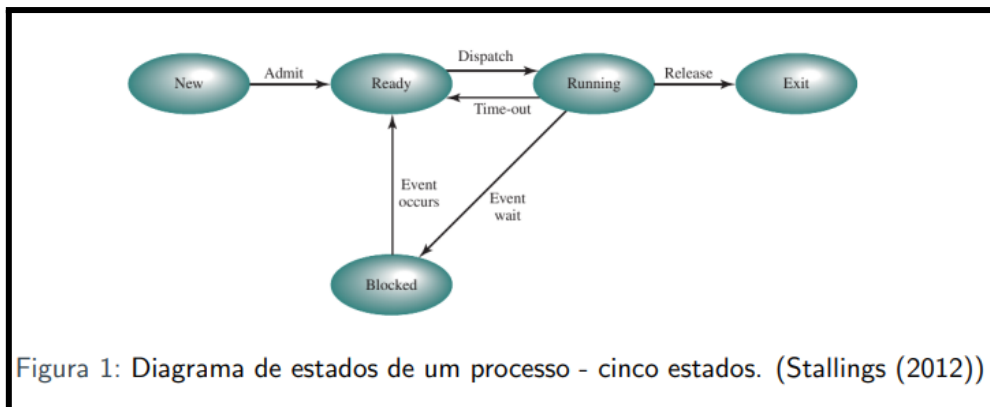
- Saída normal (voluntária).
- Saída por erro (voluntária).
- Erro fatal (involuntário).
- Cancelamento por um outro processo.

Relacionamento entre Processos

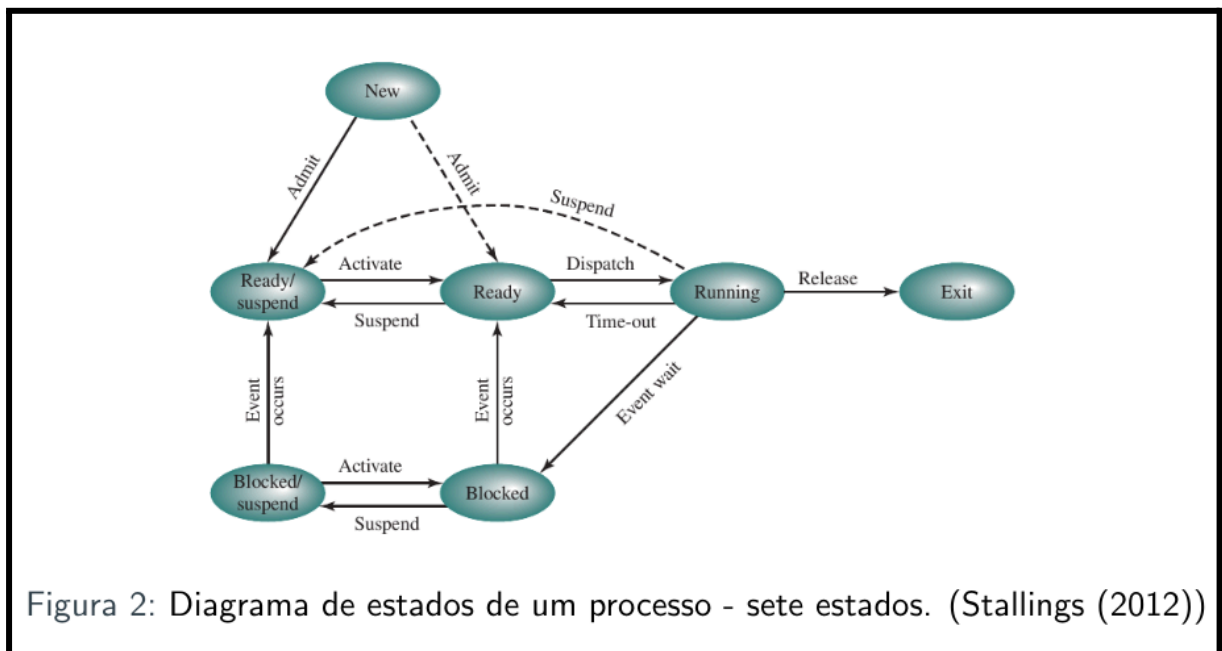
- **Processos Independentes**
 - Não apresentam relacionamento com outros processos.
 - Por exemplo, no MS Windows não há relação hierárquica.
- **Grupo de processos**
 - Apresentam uma relação entre os processos.
 - Por exemplo, no Linux há uma relação hierárquica de filiação.
- **Hierarquia de processos**
 - Processo criador é denominado pai.
 - Processo criado e denominado filho.
 - Questões:
 - Quais relações são mantidas entre pai e filho?
 - O filho herda do pai:
 - descritores de arquivos abertos
 - variáveis de ambiente
 - diretório atual
 - Memória é copiada (ou compartilhada via copy-on-write)
 - O que fazer ao término de um processo?

Estados

- **New (Novo)**: criado mas não admitido para a execução.
- **Ready (Pronto/Apto)**: disponível para a execução.
- **Running (Execução)**: em execução.
- **Blocked (Bloqueado)**: esperando por um evento (por exemplo, E/S).
- **Exit (Término)**: execução finalizada ou abortada.



- Diagrama com **7 estados**, adição de dois novos estados:
 - **Ready/Suspend (Pronto/Suspensão)**: Processo na memória secundária e disponível para execução.
 - **Blocked/Suspend (Bloqueado/Suspensão)**: Processo na memória secundária e esperando um evento.



Transição de Estados

- **New → Ready**
 - **Processo foi criado e carregado na memória**
- **Ready → Running**
 - **Escalonador escolhe o processo para usar a CPU**
- **Running → Exit**
 - **Processo finaliza**
- **Running → Blocked**
 - **Precisa de E/S (ex: ler disco)**
- **Running → Ready:**

- perde CPU (preempção)
- Blocked → Ready
 - E/S terminou, ele pode voltar a competir pela CPU
- New → Ready/Suspend
 - Processo foi criado, mas não há memória disponível, então fica fora da RAM.
- Ready → Ready/Suspend
 - Processo é retirado da memória (falta de RAM).
- Ready/Suspend → Ready
 - Processo volta para a memória e pode executar.
- Blocked → Blocked/Suspend
 - Processo bloqueado é removido da memória.
- Blocked/Suspend → Blocked
 - Processo volta para a memória, mas ainda espera E/S.
- Blocked/Suspend → Ready/Suspend
 - E/S termina enquanto o processo ainda está fora da memória.

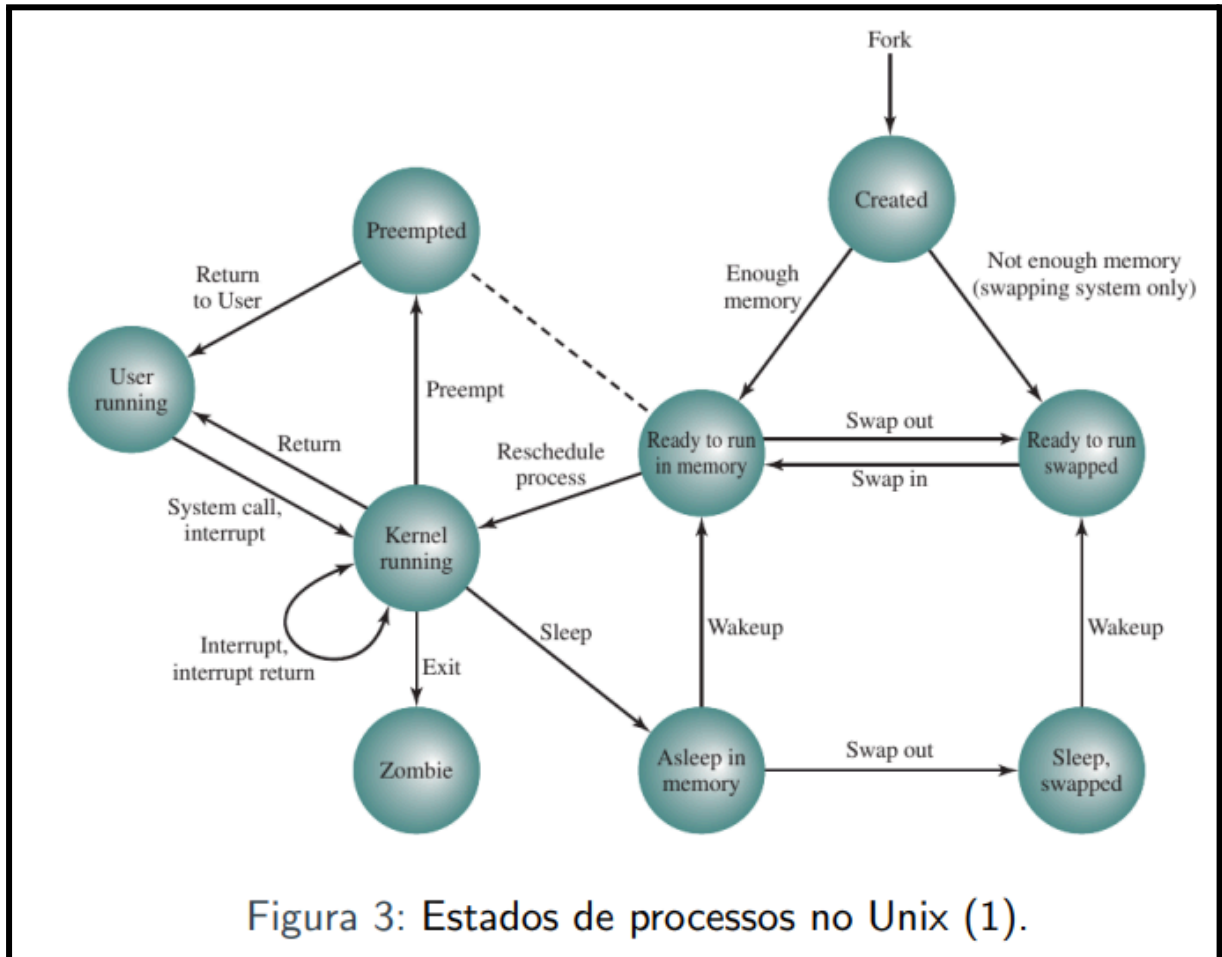
Estados de Processos no Unix

- Created
 - Processo acabou de ser criado (via *fork*), ainda não começou a executar
- Ready to run in memory
 - Processo está pronto para executar e está na RAM
- Ready to run swapped
 - Processo está pronto para executar, mas está fora da memória (no disco)
- Kernel running
 - Processo está executando código do sistema operacional (modo kernel)
- User running
 - Processo está executando código do programa do usuário (modo usuário)
- Preempted
 - Processo foi interrompido (perdeu a CPU temporariamente) e vai ser reescalonado
- Asleep in memory
 - Processo está bloqueado (esperando E/S) e ainda está na memória
- Sleep, swapped

- Processo está bloqueado e também foi removido da memória (swap)

- **Zombie**

- Processo já terminou, mas ainda mantém informações para o processo pai (ainda não foi coletado com *wait*)



- **Created → Ready to run in memory**

- Processo foi criado (fork) e há memória suficiente para colocá-lo na RAM

- **Created → Ready to run swapped**

- Processo foi criado, mas não há memória suficiente, então vai direto para o disco (swap)

- **Ready to run in memory → Kernel running**

- Processo é escalonado para execução (entra no kernel para ser executado)

- **Kernel running → User running**

- Processo sai do kernel e continua executando em modo usuário

- **User running → Kernel running**

- Ocorre uma chamada de sistema (system call) ou interrupção
- **Kernel running → Asleep in memory**
 - Processo solicita uma operação de E/S e fica bloqueado (sleep)
- **Asleep in memory → Ready to run in memory**
 - Evento esperado ocorre (wakeup), então o processo pode voltar a executar
- **Ready to run in memory → Ready to run swapped**
 - Processo pronto é removido da memória (swap out)
- **Ready to run swapped → Ready to run in memory**
 - Processo volta para a memória (swap in)
- **Asleep in memory → Sleep, swapped**
 - Processo bloqueado é removido da memória
- **Sleep, swapped → Ready to run swapped**
 - Evento ocorre (wakeup), mas o processo ainda está fora da memória
- **Kernel running → Preempted**
 - Processo perde a CPU por preempção
- **Preempted → User running**
 - Processo volta a executar em modo usuário
- **Kernel running → Zombie**
 - Processo termina, mas ainda mantém informações para o processo pai

Implementação

BCP

- **Bloco de Controle de Processos (BCP)**
 - **BCP é usado pelo SO para gerenciar os processos.**
 - É uma estrutura de dados mantida no núcleo do SO.
 - Os BCP são mantidos em uma **Tabela de Controle de Processos**.
 - Exemplo de BCP: task struct no Linux .

Process management	Memory management	File management
Registers	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Program status word	Pointer to stack segment info	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

Figura 4: Campos de um bloco de controle de processos.

- Bom decorar alguns!

Na prática, ele é utilizado assim:

- **Criação do processo**

- **O SO cria um BCP e preenche com:**
 - estado inicial (novo/pronto)
 - PID
 - informações de memória

- **Escalonamento**

- O escalonador usa os BCPs da fila de prontos para decidir:
 - qual processo vai executar

- **Execução**

- O BCP guarda:
 - contador de programa
 - registradores
 → isso permite continuar a execução corretamente

- **Troca de contexto**

- Quando troca de processo:
 - o SO salva o estado atual no BCP
 - e carrega o estado de outro BCP
 → isso permite alternar entre processos

- **Gerenciamento de estados**

- O estado do processo (pronto, executando, bloqueado...) fica no BCP
- O SO atualiza esse campo conforme eventos

- **Encerramento**

- O BCP ainda guarda:
 - status de saída
- Depois é removido da tabela de processos

Troca de Contexto

- É o processo em que o sistema operacional **interrompe um processo que está usando a CPU e coloca outro no lugar.**
- **Por que isso acontece?**
 - Porque:
 - existe **mais de um processo** no sistema
 - a CPU precisa ser **compartilhada**
- **Principais motivos:**
 - fim do **quantum** (tempo de CPU)
 - processo pediu **E/S**
 - **chegou um processo de maior prioridade**
 - ocorreu uma **interrupção**
- **Como a troca de contexto acontece (passo a passo)**
 - Imagine:
 - Processo A está executando
 - Processo B está pronto

1. Interrupção

Algo acontece (ex: fim do tempo)

2. Salvar o processo atual (A)

O SO pega o estado atual de A e salva no **BCP dele**:

- registradores
- contador de programa
- estado

Isso garante que ele possa continuar depois exatamente de onde parou

3. Atualizar estado

- A → vai para **pronto** ou **bloqueado**

4. Escolher próximo processo

O escalonador escolhe B

5. Restaurar o próximo processo (B)

O SO pega o BCP de B e:

- carrega registradores
- carrega contador de programa

6. Executar B

Agora B continua execução como se nunca tivesse parado

Listando Processos

- Os comandos **ps, top e htop** possibilitam **visualizar os processos em execução**.
- Algumas das informações providas são:
 - **pid**: identificação do processo.
 - **user**: usuário que iniciou o processo.
 - **pr**: prioridade do processo (escalonamento).
 - **ni**: “nice” do processo (escalonamento).
 - **virt (VSZ), res (RSS), shr, %MEM**: uso de memória pelo processo (total de memória usada, memória usada em RAM, memória compartilhada, memória RAM disponível).
 - **S ou STAT**: estado do processo.
 - **TIME+**: total de tempo do processo desde sua inicialização.
 - **COMMAND**: nome do programa.
 - **TTY**: terminal associado ao processo.
 - **%CPU**: tempo de CPU.
- Principais estados (S):
 - **R (RUNNING)**: em execução.
 - **S (INTERRUPTIBLE SLEEP)**: esperando por um evento.
 - **I (IDLE TASK)**: usado por threads ociosas em nível núcleo.
 - **D (UNINTERRUPTIBLE SLEEP)**: esperando por E/S mas não pode ser interrompido.
 - **T (STOPPED)**: processo suspenso (sinal de controle - CTRL + Z ou depuração).

- **Z (ZUMBI)**: processo finalizado, mas não “limpo” pelo processo-pai.

CPU bound x IO bound

```
/* Processo CPU bound */
int main ( ) {

    int a = 0 ;
    while (1){
        a = a + 1 ;
        if (a == 32000) a = 0 ;
    }
    return 0 ;
}
```

```
/* Processo IO bound */
int main ( ) {
    char pnome [ 30 ] ;
    char unome [ 30 ] ;
    printf ("Digite primeiro nome : ");
    scanf ("%s", pnome);
    printf ("Digite ultimo nome : ");
    scanf ("%s", unome);
    printf ("\t%s, %s\n ", unome, pnome);
    return 0;
}
```

Fork

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>      // fork()
#include <sys/types.h>   // pid_t

int main(){
    pid_t  pid;
    int    valor = 0;
    pid = fork();        /* cria um processo e devolve pid do filho
                           para o pai e 0 para o filho */

    if (pid){            /* trecho executado pelo pai */
        printf("Eu Sou o Processo Pai - Filho %d \n", pid);
        valor = 5;
        printf("Valor: %d \n", valor);
    } else {             /* trecho executado pelo filho */
        printf("Eu Sou o Processo Filho - Filho %d \n", pid);
        valor = 10;
        printf("Valor: %d \n", valor);
    }

    exit(0);
}
```

- **#include <unistd.h>**: A principal biblioteca para chamadas de sistema POSIX. Contém **fork, exec, getpid e getppid**.
- **#include <sys/types.h>**: Define tipos de dados fundamentais, como **pid_t**.
- **fork()**: Cria um novo processo (filho) duplicando o processo atual (pai).
 - Retorna 0 no processo filho.
 - Retorna o PID do filho no processo pai.
 - Retorna -1 em caso de erro.

Exec

```
#include <unistd.h>      // execl()
#include <stdio.h>

int main() {
    printf("Antes do exec\n");
    execl("/bin/ls", "/bin/ls", "-r", "-t", "-l", (char *) 0);
    printf("Depois do exec\n");

    return 0;
}
```

```
#include <unistd.h>      // execv()
#include <stdio.h>

int main() {
    printf("Antes do exec\n");
    char *cmd[] = { "ls", "-l", (char *)0 };
    execv ("/bin/ls", cmd);
    printf("Depois do exec\n");

    return 0;
}
```

- **As funções da família exec substituem a imagem do processo atual por um novo programa**, mas cada uma recebe os argumentos de forma diferente e possui regras distintas para localizar o executável:
 - **execl (l = list):**
 - Os argumentos são passados como uma lista individual de strings diretamente na chamada da função. O último argumento deve ser obrigatoriamente NULL.
 - Exemplo: `execl("/bin/ls", "ls", "-l", "/home", NULL);`
 - Caminho: Você deve fornecer o caminho completo do executável (ex: /bin/ls).
 - **execv (v = vector):**
 - Os argumentos são passados através de um vetor (array) de strings (`char* argv[]`). O último elemento do array deve ser NULL.
 - Exemplo:
 - `char *args[] = {"ls", "-l", "/home", NULL};`
 - `execv("/bin/ls", args);`

- **Caminho:** Também exige o caminho completo do executável.
- **execvp (p = path):**
 - É a versão mais flexível e a que você usou no shell.cpp. Ela combina o uso de vetor (v) com a busca automática no PATH do sistema (p).
 - Exemplo:
 - `char *args[] = {"ls", "-l", NULL};`
 - `execvp("ls", args);` // O SO procura "ls" em /bin, /usr/bin, etc.
 - Vantagem: Você não precisa saber onde o comando está instalado (ex: basta passar "ping" em vez de "/usr/bin/ping").

Zumbis

```
#include <unistd.h>    // _exit()

int main(){
    for(int i=0; i<5; i++){
        if (fork() == 0) { // processo filho
            _exit(0); // and exit
        }
    }

    while(1);    // processo pai continua executando
}
```

- **exit(int status):** Encerra o processo imediatamente, retornando um código de status para o pai.

Eliminado Zumbis

```
#include <unistd.h>    // _exit(), sleep()
#include <sys/types.h>  // pid_t
#include <sys/wait.h>   // wait()
#include <stdio.h>

int main(){
    pid_t pid;
    int status;

    for(int i=0; i<5; i++){
        if (fork() == 0) { // processo filho
            _exit(0); // and exit
        }
    }

    do {
        pid = wait(&status);
        printf("-- Pai detecta processo %d foi finalizado com estado %d. \n", pid,
            WEXITSTATUS(status));
    } while (pid > 0); // pid == -1 indica que o pai nao tem mais filhos

    printf ("Pai finalizado. \n");
}
```



- **wait(int *status)**: Suspende a execução do pai até que qualquer um de seus filhos termine.
- **waitpid(pid_t pid, int *status, int options)**: Mais flexível que o wait. Permite esperar por um filho específico ou usar a opção **WNOHANG** para verificar se o filho terminou sem bloquear o pai (usado no shell.cpp para processos em background).
- **getpid()**: Retorna o ID do processo atual.
- **getppid()**: Retorna o ID do processo pai.
- **Macros de Status** (Usadas com wait/waitpid):
 - **WIFEXITED(status)**: Retorna verdadeiro se o filho terminou normalmente (via exit ou return).
 - **WEXITSTATUS(status)**: Extrai o código de retorno (ex: 0 para sucesso) se WIFEXITED for verdadeiro.
 - **WIFSIGNALED(status)**: Retorna verdadeiro se o filho foi encerrado por um sinal (ex: Ctrl+C).
 - **WTERMSIG(status)**: Identifica qual sinal causou o encerramento.

Órfãos

- Um processo órfão é um processo cujo pai foi finalizado ou terminado.
- Processos órfãos são terminados ou adotados pelo processo init.

- Init também é o primeiro processo (PID 1)
- **Órfão não intencional:** processo pai finaliza ou termina inesperadamente e os filhos são finalizados por um mecanismo de proteção contra órfãos acidentais da sessão/aplicação.
- **Órfão intencional:** processo desassociado da sessão/aplicação e executa em segundo plano; pai finaliza e os filhos são re-parenting.

Mais comandos

- **kill:** envia um sinal para o processo.
 - \$ kill -SIGKILL 26004
- **renice:** altera a prioridade de um processo.
 - \$ renice -n -10 -p 28990
- **pidof:** devolve o(s) pid(s) de um processo a partir do nome.
 - \$ pidof kate
- **chrt:** manipula atributos de tempo real de um processo.
 - \$ chrt -p 28990
- **ps tree:** mostra a árvore de processos.
 - \$ ps tree
- **watch:** executa um programa periodicamente.
 - \$ watch -n 1 'ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem'
- **lsof:** mostra os arquivos abertos por processo.
 - \$ lsof -p 28990

Threads

Introdução

- O que é uma thread?
 - **um fluxo em execução dentro de um processo.**
 - **um processo leve** (lightweight process).
 - unidade básica que o SO aloca tempo de processador.
- Por que usar threads?
 - **Dividir tarefas de uma aplicação** (ex: navegador Web).
 - **Compartilhamento de espaço de endereçamento.**
 - **Tempo de criação e finalização.**
 - **Concorrência e paralelismo.**

Implementação

- Modelos clássicos:

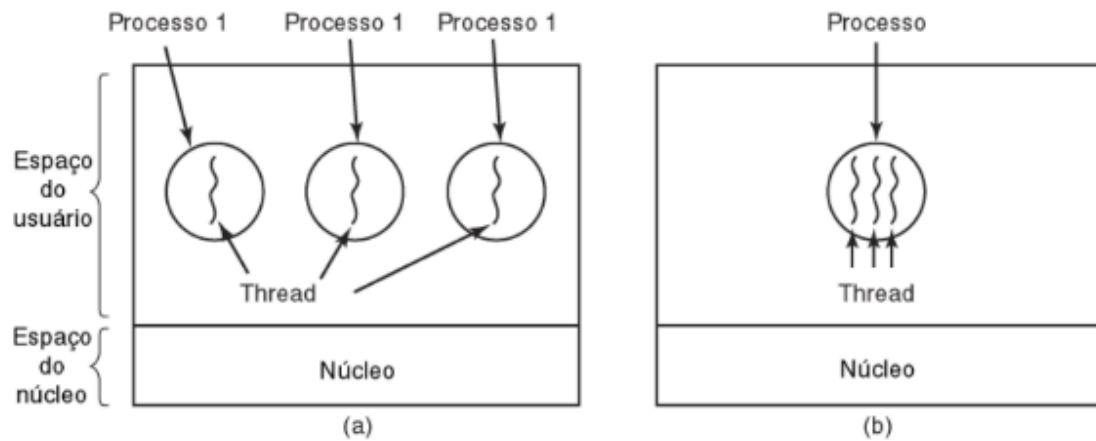


Figura 3: (a) Processo com uma thread. (b) Processo com três threads [3].

- Quais recursos as threads compartilham em um processo?

Compartilhado entre threads	Exclusivo por thread
Espaço de endereçamento.	Thread ID
Variáveis globais.	Contador de programa
Arquivos abertos.	Registradores
Processos filhos.	Pilha (variáveis locais e endereços de retorno)
Alarmes pendentes.	Estado
Sinais e tratadores de sinais.	Prioridade
Estatísticas de execução.	
Permissões de acesso.	
Heap	

- Modelos clássicos - Detalhes:

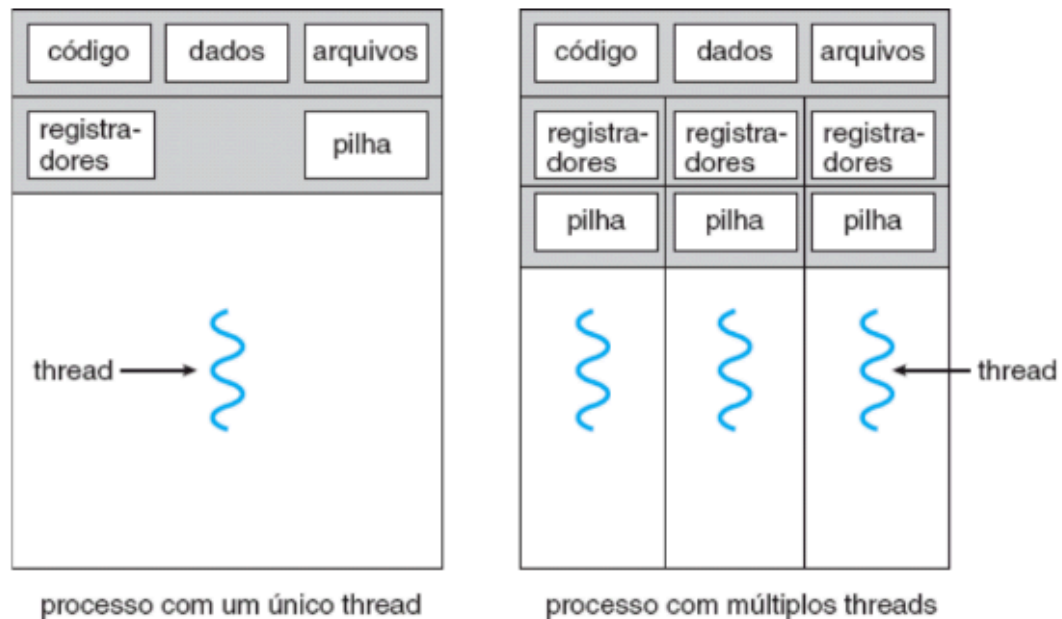
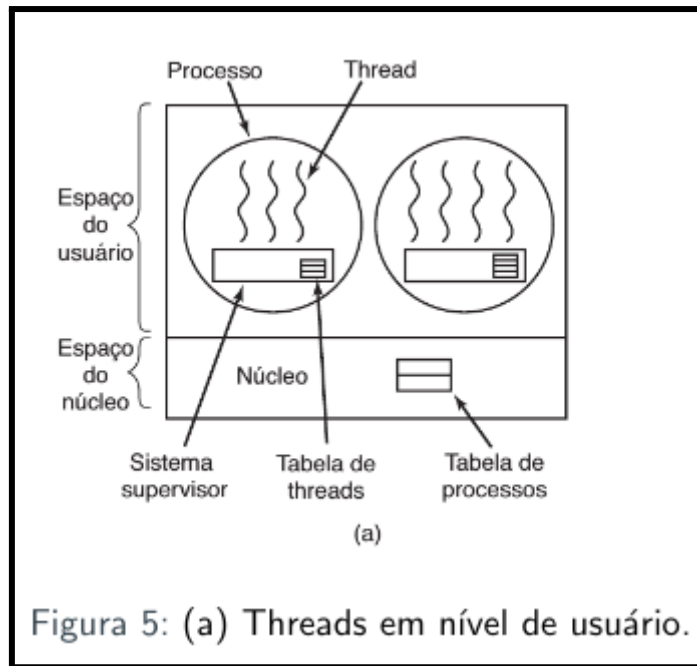


Figura 4: (a) Processo com uma thread. (b) Processo com três threads [2].

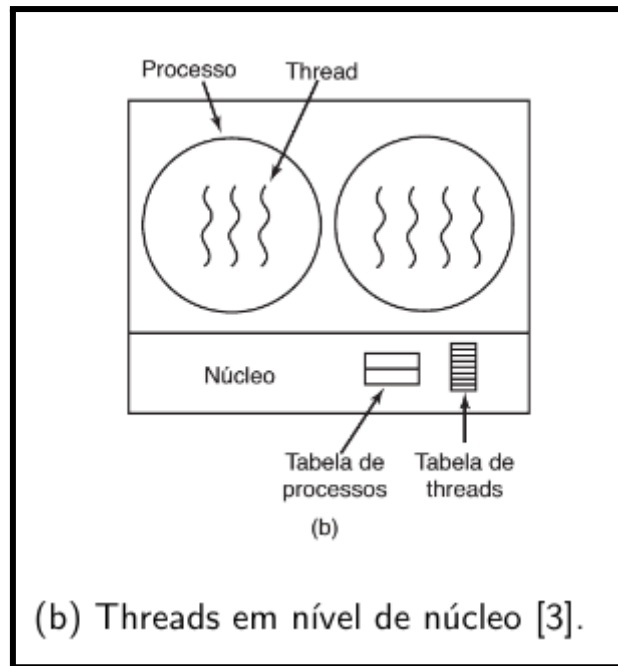
Nível de Usuário

- Executam operações de suporte a threads no espaço do usuário.
- SO mapeia threads de um processo multithread para um único contexto de execução.
- Gerenciadas pela aplicação (biblioteca)
- Vantagens:
 - Escalonamento controlado pela biblioteca possibilita otimizar o desempenho.
 - Chaveamento de contexto em nível de usuário.
 - Portabilidade maior.
- Desvantagens:
 - Núcleo considera o processo multithread como um único thread.
 - se uma bloqueia → todas bloqueiam
 - Operações de E/S podem bloquear todas as threads.
 - Escalonamento limitado a um único núcleo.



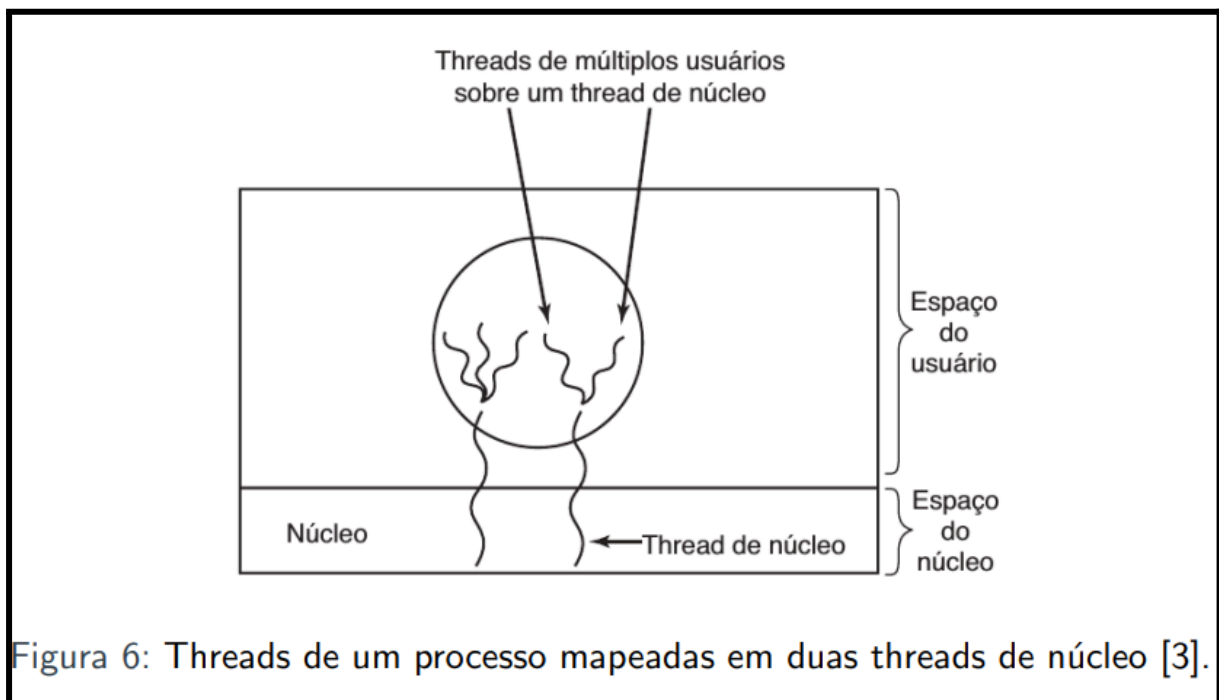
Nível de Núcleo

- Mapeia cada thread para o seu próprio contexto de execução.
- O núcleo reconhece cada thread individualmente.
- Gerenciadas pelo sistema operacional
- Vantagens:
 - paralelismo.
 - divisão justa do processador.
- Desvantagens:
 - custo do chaveamento de contexto.
 - menor portabilidade.

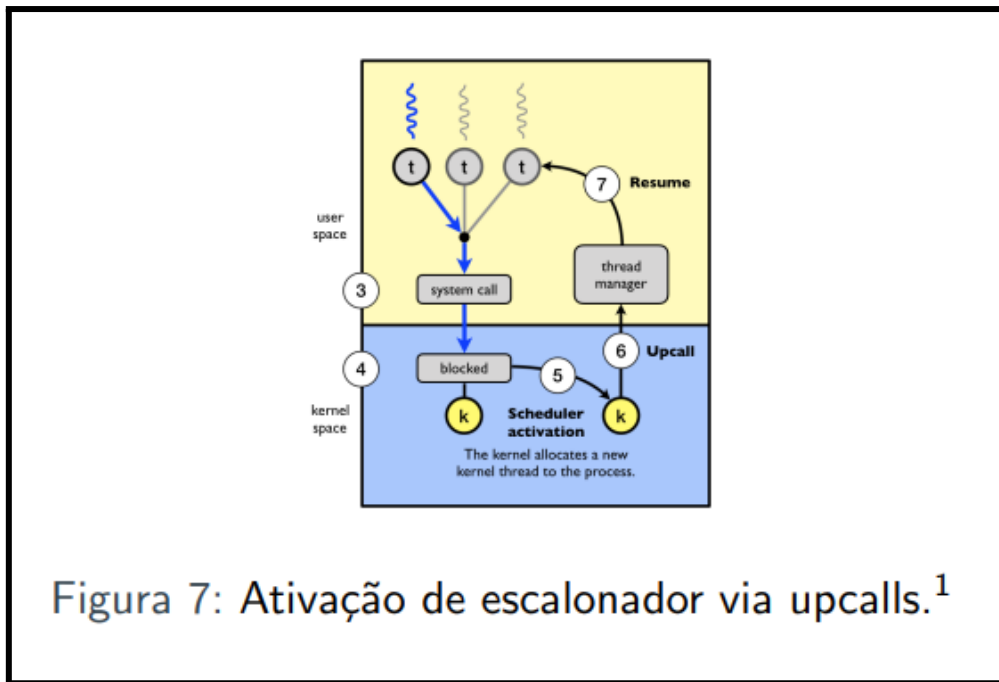


Implementação Híbrida

- Consiste em mapear números arbitrários de threads de usuário para threads de núcleo.

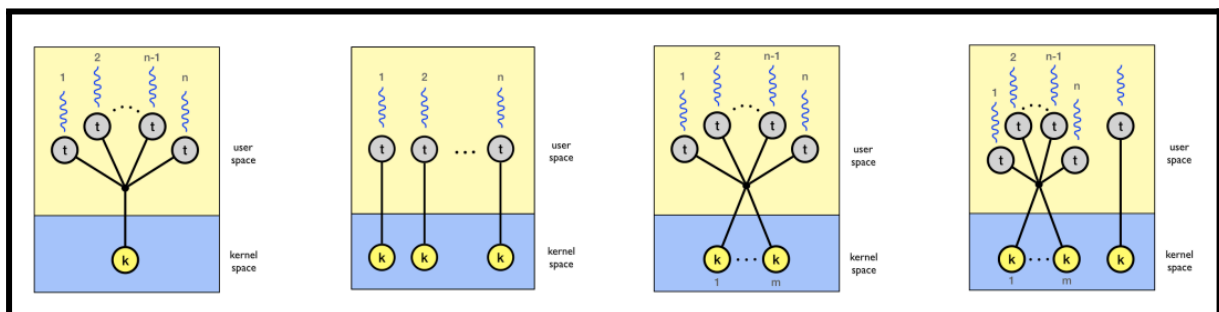


- Necessitam de comunicação para manter o número apropriado de threads de núcleo alocados à aplicação.
- O mecanismo de ativação do escalonador oferece **upcalls** – um mecanismo de comunicação do núcleo para a biblioteca de threads.



Modelos de Mapeamento

- **Muitos-para-um**: Solaris Green Threads, GNU Portable Threads.
 - várias threads → 1 thread do kernel
 - rápido, mas sem paralelismo real
- **Um-para-um**: Windows NT/XP/2000, Linux, Solaris 9 em diante
 - 1 thread usuário → 1 thread kernel
 - paralelismo real, mais custo
- **Muitos-para-muitos** (dois níveis): Solaris <= 8, Windows NT/2000 com fiber.
 - várias threads usuário → várias kernel
 - equilíbrio entre desempenho e flexibilidade



Programação

POSIX e Threads

- **POSIX** (IEEE 1003.1c) especifica uma **API para criação e sincronismo de threads**.
- A **API** especifica o comportamento da biblioteca de threads, a implementação fica para o desenvolvimento da biblioteca.
- Comum em sistemas operacionais UNIX (Solaris, Linux, Mac OS X).
- Os **threads que usam a API de thread POSIX** são chamados de **Pthreads**.

Threads no Linux

- Aloca o mesmo bloco de controle para threads e processos.
- Faz uso da chamada de sistema **clone**.
- As **threads compartilham o espaço de endereçamento do processo**.
- Os **estados que as threads assumem são os mesmos dos processos**.
- As **Pthreads são implementadas como nível núcleo**.

Biblioteca Pthreads

Criar Novas Threads

```
int pthread_create (pthread_t *thread ,
                   const pthread_attr_t *attr ,
                   void * (*start_routine)(void *),
                   void *arg);
```

- **thread**: identificador da thread.
- **attr**: atributos de configuração da thread. Se NULL, usa os padrões.
- **start_routine**: função com o código da thread.
- **arg**: parâmetro único para passagem de valores para a thread.
- **returns**: 0 em caso de sucesso ou um código de erro.

Finaliza Threads

```
void pthread_exit (void *retval);
```

- **retval**: valor de retorno para a thread que aguarda (join).

Aguarda Término de Threads

```
int pthread_join (pthread_t thread ,
                 void **retval);
```

- **thread**: identificador da thread a ser aguardada.
- **retval**: valor devolvido pela função **pthread exit**.
- **returns**: 0 em caso de sucesso ou um código de erro.
- Exemplo de Criação e Finalização de Threads:

```
#include <stdio.h>
#include <stdlib.h> // exit
#include <unistd.h> // sleep
#include <pthread.h> // pthread_t, pthread_create(), pthread_join(), pthread_exit()

/** funcao que implementa o fluxo da thread */
void *print_message (void *ptr)
{
    char *message;
    message = (char *) ptr; //casting - coersao explicita
    sleep(2);
    printf("%s \n", message);

    //pthread_exit(NULL);
    return NULL;
}
```

```
/* fluxo principal - thread 'main' */
int main() {
    /* Declara duas threads */
    pthread_t thread1, thread2;
    //pthread_t thread[2];

    char *msg1 = "Eu sou a Thread 1.";
    char *msg2 = "Eu sou a Thread 2.";

    int iret1, iret2;

    /* Cria duas threads independentes, cada uma executa a funcao */
    iret1 = pthread_create( &thread1, NULL, print_message, (void*) msg1);
    iret2 = pthread_create( &thread2, NULL, print_message, (void*) msg2);

    printf("Thread 1 foi criada com sucesso?: %d\n", iret1);
    printf("Thread 2 foi criada com sucesso?: %d\n", iret2);

    /* Aguarda threads finalizarem antes de continuar. */
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("MAIN - FIM\n");

    //pthread_exit(0);
    exit(0);
}
```

Cancelamento de Threads

```
int pthread_cancel (pthread_t thread);
```

- **thread**: identificador da thread.
- **returns**: 0 em caso de sucesso ou um código de erro.

Cria Pontos de Cancelamento de Threads

```
void pthread_testcancel(void);
```

Ativa ou Desativa o Cancelamento de Threads

```
int pthread_setcancelstate (int state ,  
                           int *oldstate);
```

- **state**: modifica o estado para **PTHREAD_CANCEL_ENABLE** ou **PTHREAD_CANCEL_DISABLE**.
- **oldstate**: armazena o estado antigo.
- **returns**: 0 em caso de sucesso ou um código de erro.

Modifica o Tipo de Cancelamento de Threads

```
int pthread_setcanceltype (int type ,  
                          int *oldtype);
```

- **type**: modifica o tipo de cancelamento para **PTHREAD_CANCEL_DEFERRED** ou **PTHREAD_CANCEL_ASYNCHRONOUS**.
- **oldstate**: armazena o antigo tipo.
- **returns**: 0 em caso de sucesso ou um código de erro.
- Exemplo de Cancelamento de Threads:

```

#include <pthread.h>    // pthread_testcancel(), pthread_setcancelstate(),
                        pthread_cancel()
#include <stdio.h>
#include <unistd.h>

/* tarefa para ser executada como thread e testar o cancelamento */
void *task_do_nothing (void *parm)
{
    int    i = 1;
    printf("THREAD - thread executada.\n");
    //pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);
    /* altera o tipo de cancelamento para assincrono - padrao deferred (adiado) */
    //pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);

    for (;;) {
        printf("THREAD - thread em laço %d\n", i);    //ponto de cancelamento

        /* testa o cancelamento */
        //pthread_testcancel();

        sleep(1);
        if (i == 5) {
            /* Como o opcao de cancelamento foi desabilitada, ativar agora */
            printf("THREAD - ativa o estado de cancelamento\n");
            pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
        }
    } // for

    return NULL;
}

```

```

/* fluxo principal (thread principal) */
int main(int argc, char **argv)
{
    pthread_t thread;
    int status = 0;

    printf("MAIN    - Iniciando a thread principal (main)\n");

    /* cria a thread com atributos padroes */
    printf("MAIN    - Cria a thread com atributos padroes \n");
    status = pthread_create(&thread, NULL, task_do_nothing, NULL);

    /* aguarda um tempo antes de solicitar o cancelamento */
    sleep(2);

    printf("MAIN    - Cancelar a thread\n");
    status = pthread_cancel(thread);
    printf("MAIN    - Retorno de pthread_cancel %d\n", status);

    pthread_join(thread, NULL);
    printf("MAIN    - Main completed\n");

    pthread_exit(NULL);

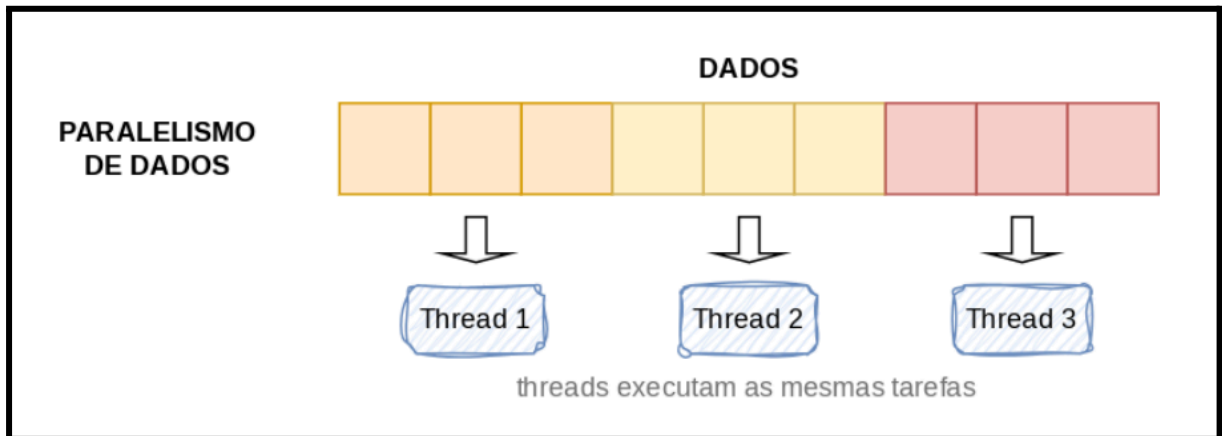
    return 0;
}

```

Paralelismo com Threads

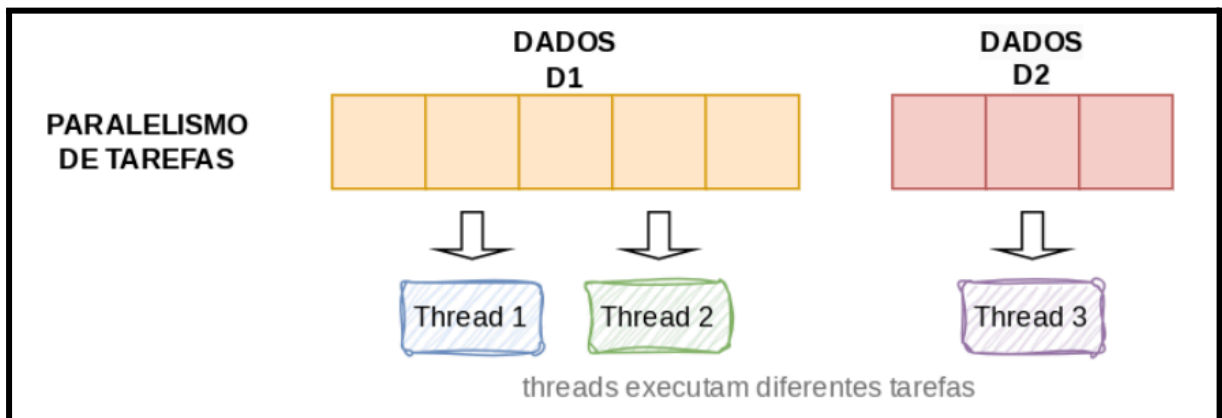
- Paralelismo de Dados

- Execução concorrente de uma **mesma tarefa em diferentes elementos de dados**.



- **Paralelismo de Tarefas**

- Execução concorrente de diferentes tarefas nos mesmos ou diferentes dados.



Gerência de Processos

Introdução

- A gerência de processador depende de dois principais módulos:
- **Escalonador (scheduler):**
 - seleciona um processo.
 - considera uma política de seleção.
 - favorece tipos de processos (IO/CPU bound).
- **Despachante (dispatcher):**
 - realiza a troca de contexto.
 - grava o contexto do processo atual.
 - recupera o contexto do processo que executará.

Escalonamento

- O escalonador é classificado quanto ao seu nível:
 - **Longo prazo** (long-term scheduler).
 - **Médio prazo** (medium-term scheduler).
 - **Curto prazo** (short-term/CPU scheduler).
- **Escalonador de Longo Prazo**
 - Executado quando um novo processo é criado.
 - Realiza o controle de admissão - quando um novo processo está apto para execução.

- Controla o grau de multiprogramação do sistema, isto é, **controla se o número de processos ativos afeta negativamente o tempo de uso do processador por processo.**

- **Escalonador de Médio Prazo**

- Associado à Gerência de Memória.
 - **mecanismo de swapping.**
 - **Suspende/retoma processos (swap)**
- Suporte adicional à multiprogramação.
 - grau de multiprogramação efetiva (aptos e aptos-suspensos).

- **Escalonador de Curto Prazo**

- **Seleciona o processo na lista de prontos para usar o processador.**
- Executado com alta frequência (curto prazo).
- Aciona o módulo do **despachante**.

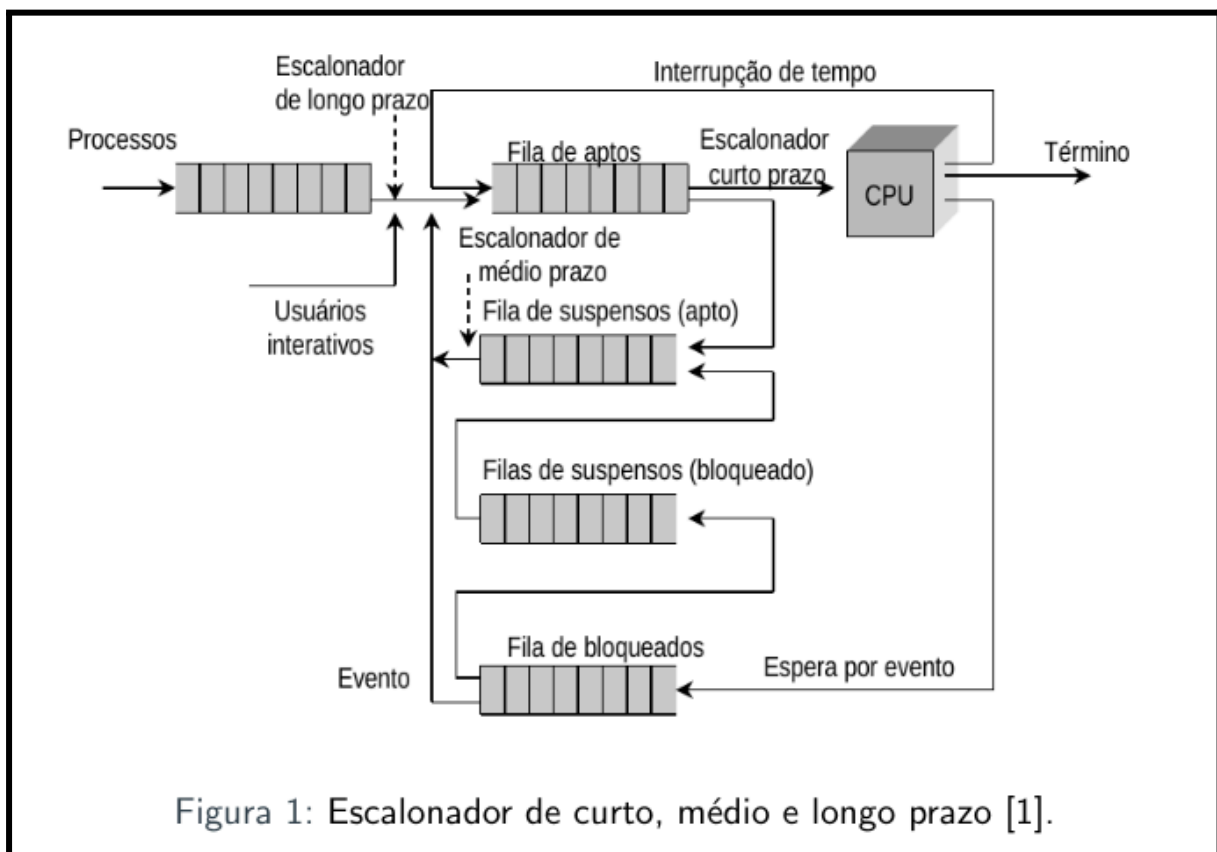
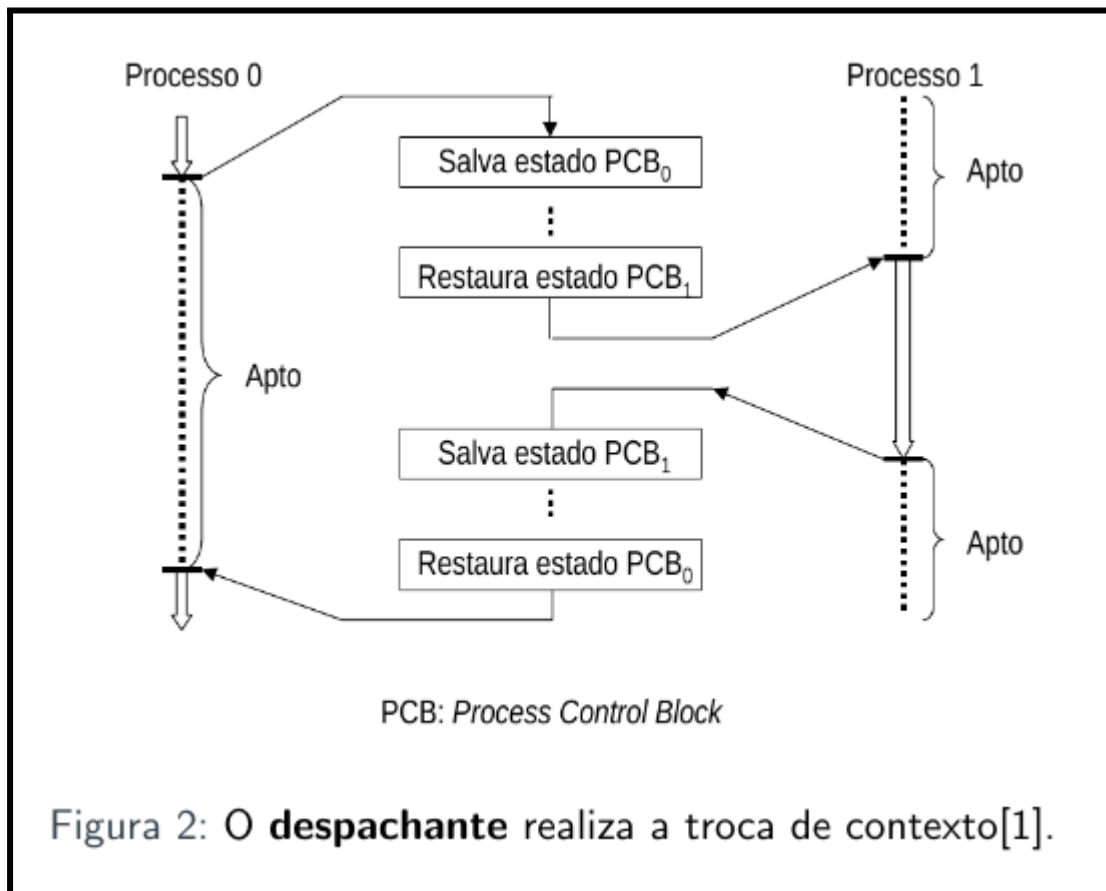


Figura 1: Escalonador de curto, médio e longo prazo [1].

- O **escalonador de CPU** (CPU scheduler) é a **entidade responsável por selecionar um processo pronto para executar no processador (CPU).**

- O principal objetivo é prover acesso à CPU de forma justa entre os processos prontos.
- SOs possuem requisitos diferentes para uso de tempo da CPU.
 - sistemas de lote;
 - tempo compartilhado;
 - tempo real;
 - portáteis.
- Objetivos do escalonamento
 - Maximizar o uso do processador (CPU utilization).
 - manter a CPU ocupada.
 - Maximizar a taxa de trabalho do sistema (throughput).
 - número de processos executados por unidade de tempo.
 - Minimizar o tempo de execução (turnaround time).
 - tempo total de execução de um processo.
 - Minimizar o tempo de espera (waiting time).
 - tempo total de espera na lista de prontos.
 - Minimizar o tempo de resposta (response time).
 - tempo entre uma requisição e seu início.
- Quando executar o escalonador?
 - Considerar as características do escalonador.
 - preemptivo ou não-preemptivo, uso ou não de prioridades, ...
 - CPU livre e processos aptos a executar.
 - Criação e término de processos.
 - Processos de alta prioridade estar na fila de aptos.
 - Interrupção de relógio (limite de tempo por processo).
 - Interrupção de hardware (E/S).
 - Interrupção de software (chamadas de sistema).
 - Interrupção de falta de página em memória.
 - Exceções.

Despachante (Dispatcher)



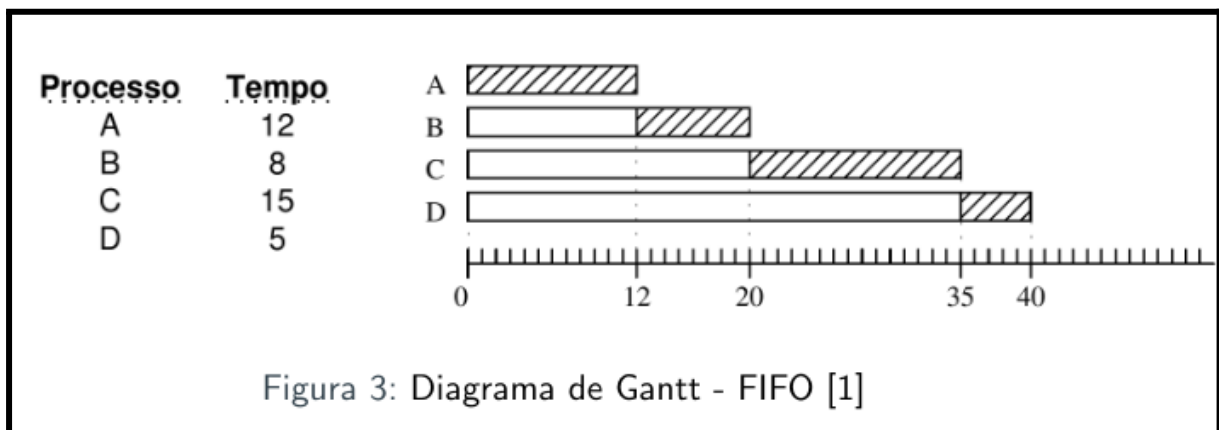
Algoritmos de Escalonamento

- Um **algoritmo de escalonamento seleciona um processo apto para executar na CPU.**
- **Algoritmos não preemptivos:**
 - **First-In First-Out (FIFO)** ou First-Come First-Served (FCFS).
 - **Shortest Job First (SJF)** ou Shortest Process Next (SPN).
- **Algoritmos preemptivos:**
 - **Shortest Remaining Time First (SRTF).**
 - **Round Robin (Circular).**
 - **Prioridades.**

FIFO

- **FIFO - First In First Out.**
 - Também denominado de First-Come, First-Served (FCFS).
 - **Simplicidade de implementação (fila simples).**

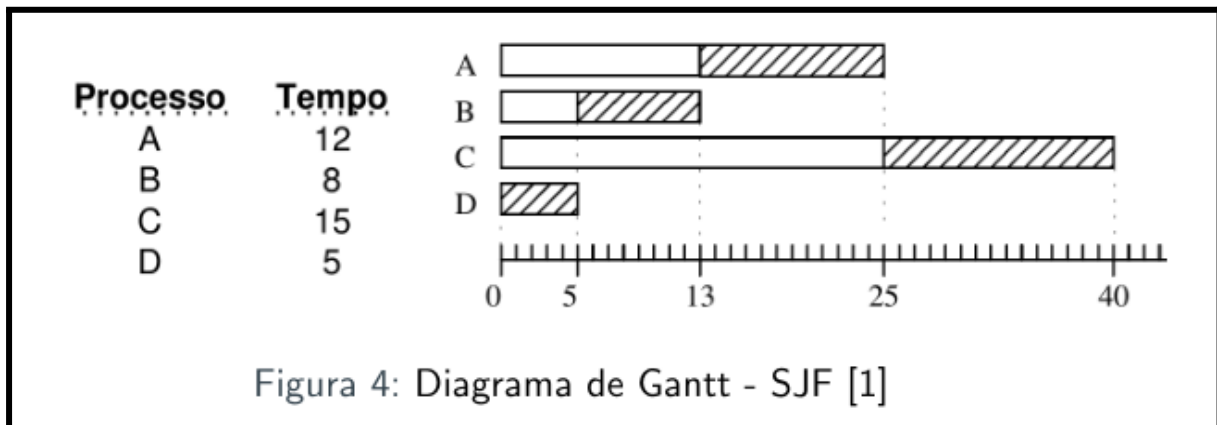
- **Favorece processos CPU-bound.**
- Operação:
 - processos que se tornam aptos são **inseridos no final da fila.**
 - O processo no início da fila é o próximo a executar.
 - O processo executa até que:
 - **finalize a sua execução.**
 - **libere o processador explicitamente (yield).**
 - **faça uma chamada de sistema (estado = bloqueado).**
- Exemplo:
 - Análise de operação - **Tempo de Espera:**
 - **Ordem A-B-C-D** = $(0 + 12 + 20 + 35)/4 = 16,75$ u.t.
 - Somar quando cada processo começou!
 - **Ordem D-A-B-C** = $(0 + 5 + 17 + 25)/4 = 11,7$ u.t.



SJF - Shortest Job First

- Também denominado de Shortest Process Next (SPN).
- **Provê o menor tempo médio de espera para um conjunto de processos** (algoritmo ótimo).
- **Favorece processos IO-bound.**
- Operação:
 - **Seleciona o processo apto com o menor tempo para uso de CPU** (ciclo de processador).
- **Problema:** determinar o tempo do próximo ciclo de CPU de cada processo.
- **Uso:** processos batch ou fazer uso de previsão.
- Exemplo:

- Análise de operação - **Tempo de Espera:**
 - Ordem D-B-A-C $(0 + 5 + 13 + 25)/4 = 10,75$ u.t.



Shortest Remaining Time First (SRTF)

- Também denominado de **SJF Preemptivo**.
- **Favorece processos IO bound.**
- Pode causar **inanição** (starvation).
- Operação:
 - **Seleciona o processo apto com o menor tempo.**
 - **Se o processo em execução não for o de menor tempo, é retirado e colocado na fila de aptos.**

Exemplo:

- Análise de operação - **Tempo de Espera:**

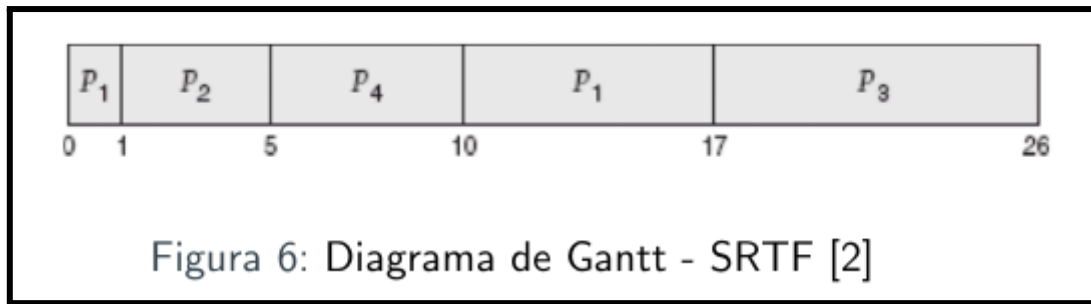
- Considere a tabela:

■

Processos	Tempo de Chegada	Duração de Ciclo
P1	0	8
P2	1	4
P3	2	9
P4	3	5

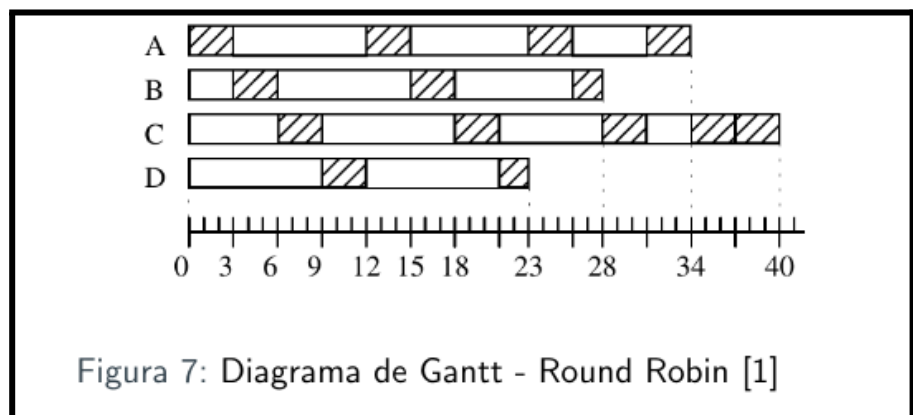
- Tempo médio de espera:

- $[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)]/4 = 26/4 = 6,5$ u.t. (SJF: 7,75 ut)



Round Robin (RR)

- Similar ao algoritmo FIFO.
 - **tempo limite máximo de CPU** (time-slice, **quantum**).
- A fila de processos aptos é uma fila circular.
- Mecanismo para delimitar as fatias de tempo.
 - **Interrupção de tempo**
- Operação:
 - **Processos que se tornam aptos são inseridos no final da fila.**
 - O processo no início da fila é o próximo a executar.
 - **O processo executa até que:**
 - (a) **finalize a sua execução.**
 - (b) **libere o processador explicitamente (yield).**
 - (c) **faça uma chamada de sistema** (estado = bloqueado).
 - (d) **fatia de tempo (quantum) é esgotada.**
- Exemplo:
 - Análise de operação - **Tempo de Espera:**
 - Duração dos processos (A=12, B=8, C=15, D=5)
 - Quantum = 3 ut
 - Ordem inicial: A-B-C-D.

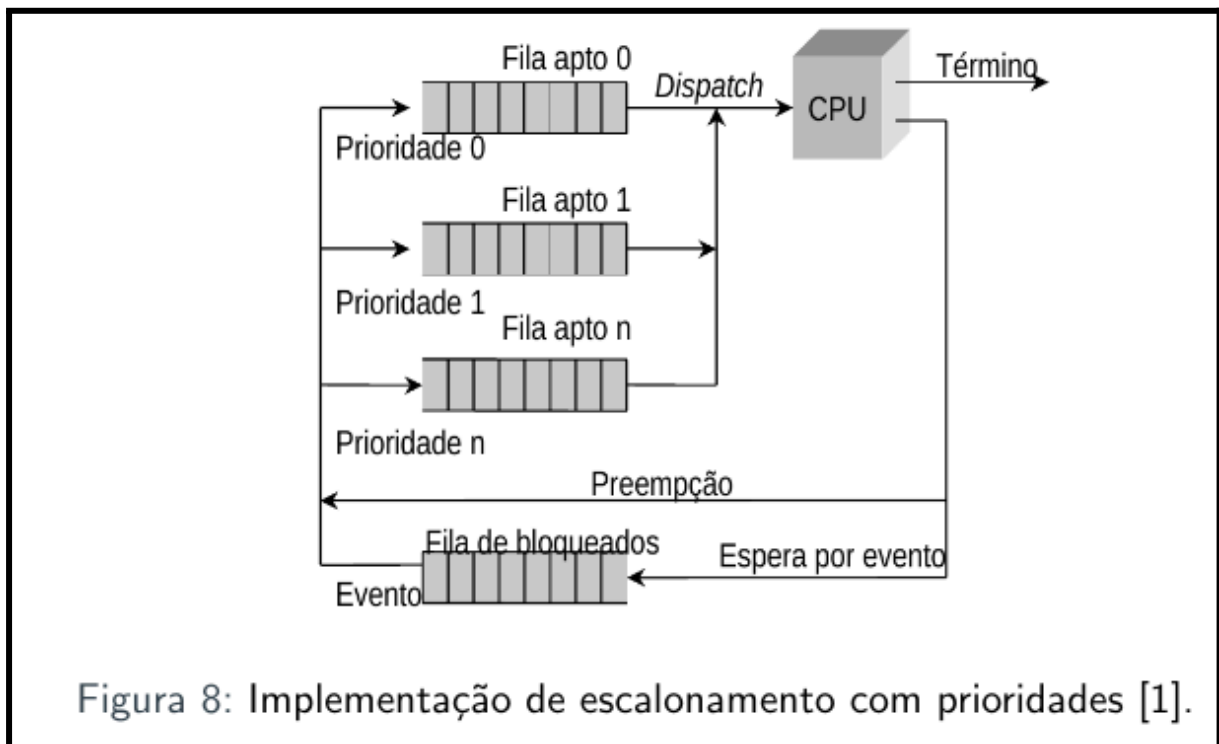


- Tempo de espera médio:
 - $A (9+8+5), B (3+9+8), C (6+9+7+3), D (9+9) = 85/4 = 21,25$ ut

- Questões sobre o quantum:
 - Pequeno ou grande?
 - Fixo ou variável?
 - Idêntico ou não para todos os processos?
- **Problema de dimensionamento do quantum:**
 - (a) sobrecarga e tempo de resposta.
 - (b) tempo de chaveamento e tempo de ciclo de processador.
- **Problema de favorecer processos CPU-bound:**
 - (a) IO-bound não usam todo o seu quantum.
 - (b) Há como compensar essa desigualdade?

Escalonamento com Prioridades

- **Processos aptos de maior prioridade têm preferência.**
- Ocorre **preempção** se um processo apto for de maior prioridade **que atualmente em execução.**
- Existência de prioridades implicam em preempção, **mas podemos ter prioridade não-preemptiva.**
- **Escalonador seleciona o processo de mais alta prioridade considerando uma política (RR, FIFO, SJF).**
- **Múltiplas filas associadas ao estado apto.**
- Cada fila com uma prioridade.
- Cada fila com política própria de escalonamento (FIFO, RR).



- **Política FIFO**
 - **processo é preemptado** – insere no início de sua fila de prioridade.
 - **processo de bloqueado para apto** – insere no final da fila de sua prioridade.
 - **processo troca de prioridade** – insere no final da fila de sua nova prioridade.
 - **processo cede processador** – insere no final da fila de sua prioridade.
- **Prioridade estática:**
 - **prioridade não se modifica durante a vida do processo.**
- **Prioridade dinâmica:**
 - **prioridade ajustada considerando o estado da execução do processo e/ou do sistema.**
- **Exemplo:**
 - (a) ajustar em função do uso efetivo do quantum:
 - quantum = 100ms
 - Processo A usou 10ms – nova prioridade = $1/0.1 = 10$
 - Processo B usou 50ms – nova prioridade = $1/0.5 = 2$

Comunicação entre Processos (IPC)

- **Processos podem ser:**
 - **Independentes:** não compartilham dados com outros processos.
 - **Cooperativos:** podem afetar ou ser afetados por outros processos em execução no sistema.
- Os processos possuem seu próprio espaço de endereçamento, mas há situações que precisam enviar/receber/acessar informações de outros processos.
- **Mecanismo de IPC (Interprocess Communication)**
- **Motivos:**
 - **Compartilhamento de informações**
 - **Aumento de desempenho na computação**
 - **Modularidade**
 - **Conveniência**

- Vários mecanismos de IPC originaram-se do tradicional **IPC do UNIX**.

- **Dois modelos**

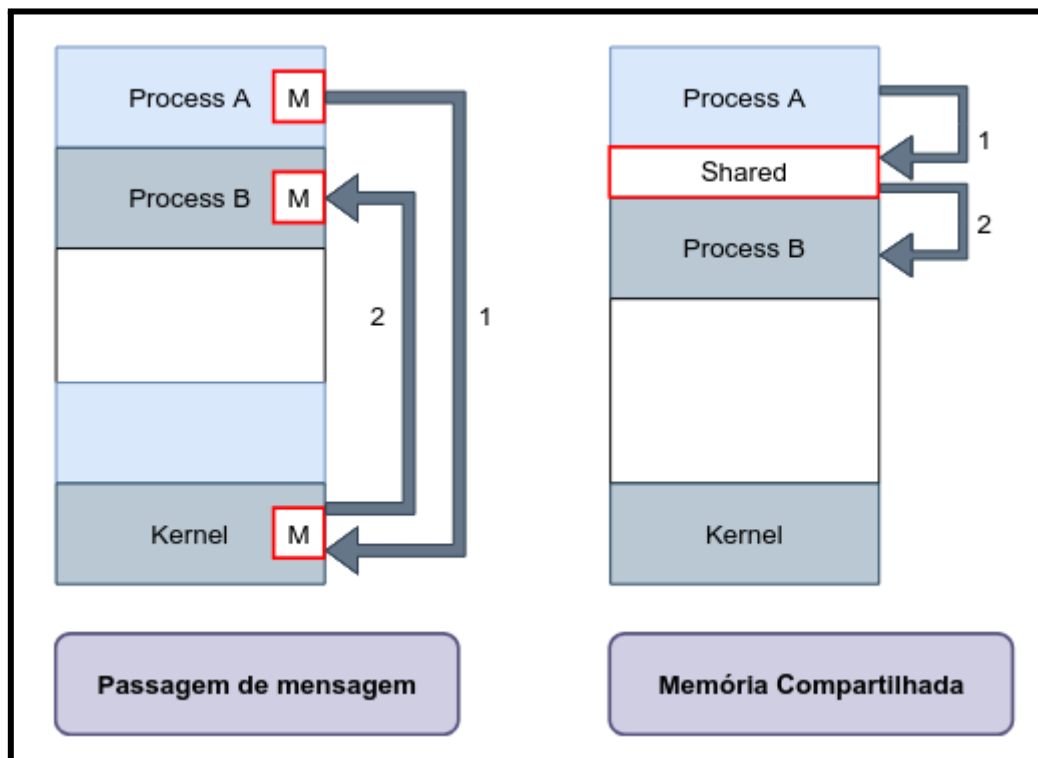
- **Troca de mensagem**
- **Memória compartilhada**

- **Troca de Mensagem**

- Ocorre por meio de troca de mensagens entre os processos.
- Útil para trocar **quantidades menores de dados**.
- **Mais fácil de implementar**.
- Normalmente implementado com **uso de chamadas de sistema**.

- **Memória Compartilhada**

- Uma região de memória é compartilhada entre os processos.
- A troca de informações ocorre por **leitura/escrita de dados nessa região**.
- **Mais rápida**, chamadas de sistemas só criam a região.



- **Mecanismos por troca de mensagens**

- **Pipe**
- **Fifo**
- **Filas de mensagens (mqueue)**

- **Sockets**
- **RPC**
- **Mecanismos por memória compartilhada**
 - **Memória compartilhada (shared memory)**

Sinais

- Um dos primeiros mecanismos de comunicação interprocessos disponíveis em sistemas **UNIX**.
- São interrupções de software que notificam ao processo que um evento ocorreu, são utilizados pelo núcleo.
 - **Sinais são notificações assíncronas enviadas a processos para informar eventos.**
- **Permitem somente o envio de uma palavra de dados** (código do sinal (1 a 64)) para outros processos.
- Não permitem que processos especifiquem dados para trocar com outros processos.
- Dependem do **SO** e das interrupções geradas por software suportadas pelo processador do sistema.
- Quando ocorre um sinal, o **SO** determina qual processo deve receber o sinal e como esse processo responderá ao sinal.
- **Quando os sinais são gerados?**
 - Criados pelo núcleo em resposta a **interrupções e exceções**, os sinais são **enviados a um processo ou thread**.
 - Em consequência da **execução de uma instrução** (como falha de segmentação)
 - **Em um outro processo** (como quando um processo encerra outro) ou em um **evento assíncrono**.
 - Ou pelo próprio processo.

Sinais POSIX

Sinal	Tipo	Ação default	Descrição
1	SIGHUP	Abortar	Detectada interrupção ou morte do processo controlador
2	SIGINT	Abortar	Interrupção do teclado
3	SIGQUIT	Descarregar	Sair do teclado
4	SIGILL	Descarregar	Instrução ilegal
5	SIGTRAP	Descarregar	Rastreamento/ponto de ruptura
6	SIGABRT	Descarregar	Abortar sinal de função abort
7	SIGBUS	Descarregar	Erro de barramento
8	SIGFPE	Descarregar	Exceção de ponto flutuante
9	SIGKILL	Abortar	Sinal de matar
10	SIGUSR1	Abortar	Sinal 1 definido pelo usuário
11	SIGSEGV	Descarregar	Referência inválida na memória
12	SIGUSR2	Abortar	Sinal 2 definido pelo usuário
13	SIGPIPE	Abortar	Pipe rompido: escrever para pipe com nenhum leitor
14	SIGALRM	Abortar	Sinal de temporizador da função alarm
15	SIGTERM	Abortar	Sinal de encerramento
16	SIGSTKFLT	Abortar	Falha de pilha no co-processor
17	SIGCHLD	Ignorar	Filho parado ou encerrado
18	SIGCONT	Continuar	Continuar se estiver parado
19	SIGSTOP	Parar	Parar processo
20	SIGTSTP	Parar	Parar digitado no dispositivo de terminal

- Estão definidos 64 sinais.

```

rogerio@guarani:~$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM    15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO      30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX

```

- Um **kill -9 pid** tem o mesmo efeito de um **kill -SIGKILL pid**
- Alguns sinais importantes são:
 - **SIGHUP (1)**: originalmente indicava desconexão do terminal, mas atualmente é muito utilizado para recarregar configurações de serviços e servidores sem encerrá-los.
 - **SIGKILL (9)**: encerra imediatamente um processo. Esse sinal não pode ser ignorado nem tratado pelo processo.
 - **SIGSEGV (11)**: ocorre quando um programa tenta acessar uma região inválida de memória, gerando o famoso erro "Segmentation Fault".

- **SIGTERM (15)**: solicita o encerramento “educado” de um processo, permitindo que ele finalize tarefas, salve dados e libere recursos antes de terminar.
- **Processos** podem
 - **Capturar**: especificando uma rotina que o SO chama quando entrega o sinal.
 - **Ignorar**: neste caso **depende da ação padrão do SO** para tratar o sinal.
 - **Mascarar um sinal**: Quando um processo mascara um sinal de um tipo específico, o **SO não transmite mais sinais daquele tipo para o processo até que ele desbloqueie a máscara do sinal**.

Tratamento

- Um processo/thread pode tratar um sinal
 - **Capturando o sinal**
 - Quando um processo capta um sinal, chama o tratador para responder ao sinal.
 - **Ignorando o sinal**
 - Processos podem ignorar todos, **exceto os sinais SIGSTOP e SIGKILL**.
 - **Executando a ação default do SO**
 - Ação definida pelo núcleo para esse sinal.
- **Ações default**
 - **Abortar**: terminar imediatamente.
 - **Descarga de memória**: copia o contexto de execução antes de sair para um arquivo do núcleo (**memory dump**).
 - **Ignorar**.
 - **Parar** (isto é, **suspender**).
 - **Continuar** (isto é, **retomar**).

Bloqueio

- Um processo ou thread pode **bloquear um sinal**.

- O sinal não é entregue até que o processo/thread pare de bloqueá-lo.
- Enquanto o tratador de sinal estiver em execução, os sinais desse tipo são bloqueados por default.
- Ainda é possível receber sinais de um tipo diferente (não bloqueados).
- Os sinais comuns não são enfileirados.
- Os sinais de tempo real podem ser enfileirados.

Troca de Mensagens

- O SO fornece mecanismos para permitir que os processos cooperativos se comuniquem entre si por meio de troca de mensagens.
- Comunicação e sincronização de ações sem compartilhar o mesmo espaço de endereços, por exemplo, entre processos locais ou em um ambiente distribuído.
- Sistema de mensagem – processos se comunicam entre si trocando mensagens sem o uso de variáveis compartilhadas.
- Há duas operações básicas
 - send (mensagem) – tamanho da mensagem fixo ou variável
 - receive (mensagem)
- Se P e Q quiserem se comunicar, eles precisam
 - estabelecer um enlace(link) de comunicação entre eles.
 - trocar mensagens por meio de send/receive.

Sincronismo no Envio/Recebimento

- A passagem de mensagens pode ser com bloqueio ou sem bloqueio.
- Bloqueio é considerado síncrono (blocking)
 - Envio com bloqueio: emissor é bloqueado até que a mensagem é recebida.
 - Recepção com bloqueio: receptor é bloqueado até que a uma mensagem esteja disponível.
- Não bloqueio é considerado assíncrono (nonblocking)

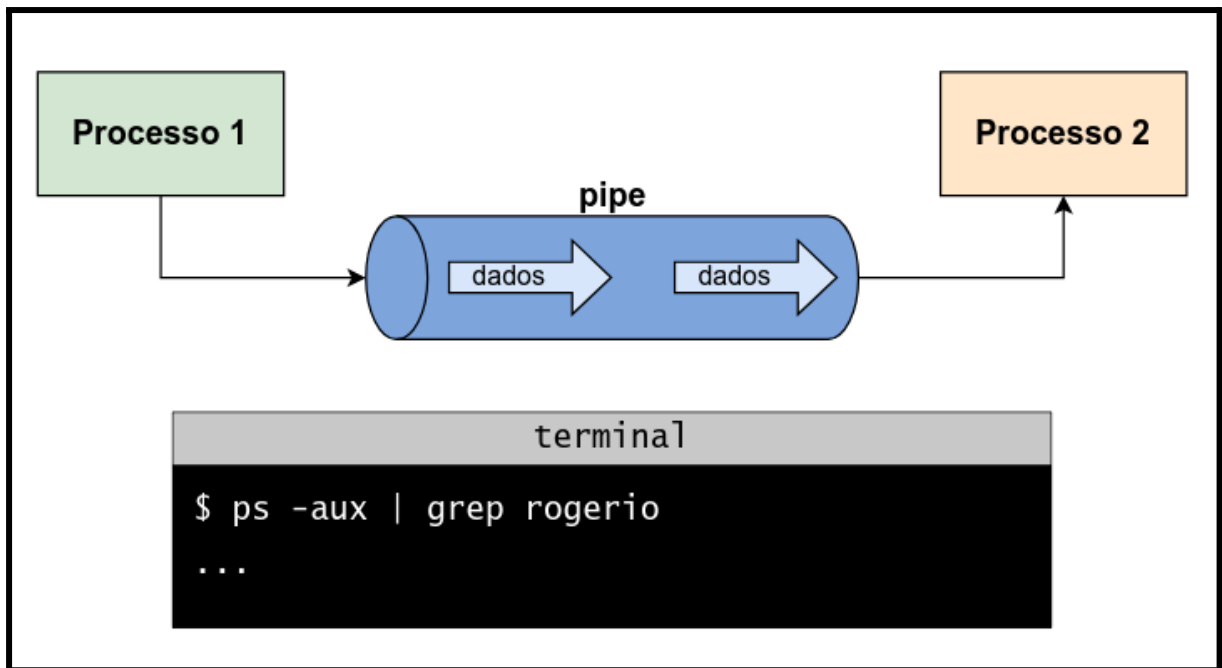
- **Envio sem bloqueio**: emissor envia a mensagem e continua sua execução.
- **Recepção sem bloqueio**: receptor tenta receber a mensagem e continua executando caso não exista nenhuma disponível.

Pipes e Fifos

- **Pipes e Fifos** são canais para comunicação entre processos, geralmente criados por chamadas de sistema. Os dados são tratados como se estivessem em uma **fila**.
- **Pipes**:
 - Não possuem nome e são herdados de um processo.
- **Fifos** (Named pipes):
 - Continuam existindo mesmo depois que o processo terminar.

Pipes

- **O acesso ao pipe é controlado por descritores de arquivo**
 - Podem ser passados entre processos relacionados (por exemplo, pai e filho).
 - São mais adequados quando:
 - há comunicação entre **processos com relação de parentesco (pai e filho)**;
 - a comunicação é **temporária**.
- **Pipes nomeados (FIFOs)**
 - Podem ser acessados por meio da árvore de diretório.
 - São mais adequados quando:
 - **processos independentes precisam se comunicar**;
 - **a comunicação deve persistir no sistema de arquivos**.
- **Limitação: buffer de tamanho fixo**



- **int pipe (int pipefd[2]);**
 - **pipe()** cria um pipe, um canal de dados unidirecional que pode ser usado em IPC.
- O array **pipefd** é usado para retornar dois descritores de arquivo
 - **pipefd[0]** referencia o lado de **leitura**.
 - **pipefd[1]** referencia o lado de **escrita**.
- Os dados escritos são colocados em um buffer pelo núcleo até ser lido pelo lado de leitura.


```

#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

/* Programa principal */
int main(void) {
    pid_t pid;
    int mypipe[2];
    char buffer[40];

    /* Criar o pipe. */
    if (pipe(mypipe)) {
        fprintf(stderr, "Falha ao criar o Pipe.\n");
        return EXIT_FAILURE;
    }

    /* Criar o processo filho. */
    pid = fork();
    if (pid < (pid_t) 0) {
        /* pid negativo, falha no fork. */
        fprintf(stderr, "Falha ao executar o Fork.\n");
        return EXIT_FAILURE;
    }

    } else if (pid == (pid_t) 0) {
        /* No processo filho. */
        close(mypipe[1]);
        read(mypipe[0], buffer, sizeof(buffer));
        printf("FILHO: ...%s\n", buffer);
        fflush(stdout);
        return EXIT_SUCCESS;
    } else {
        /* Processo pai. */
        close(mypipe[0]);
        printf("PAI: Digite algo para enviar: ");
        scanf("%40[^\n]", buffer);
        write(mypipe[1], buffer, sizeof(buffer));

        wait(NULL);
        return EXIT_SUCCESS;
    }
}

```

```

$gcc simple-pipe.c -osimple-pipe
$./simple-pipe
PAI: Digite algo para enviar: Olá filho.
FILHO: ...Olá filho.
$

```

FIFO (Named Pipes)

- FIFOs são canais nomeados (named pipes).
- É possível **criar pipes nomeados** usando o comando **mkfifo**.

```

$mkfifo mypipe
$ls -l
total 0
prw-rw-r-- 1 rodrigo rodrigo 0 Abr 17 14:28 mypipe
$echo "Armazene essa msg no fifo." > mypipe
$

```

teste : bash — Konsole <2>

```

File Edit View Bookmarks Settings Help
$cat < mypipe
Armazene essa msg no fifo.
$

```

- A mesma chamada de sistema está disponível em linguagem de programação
 - **int mkfifo** (const char *filename, mode_t mode)

```

/**
 * Lê mensagens de um FIFO e exibe na tela
 */
#include <stdio.h> // standard io
#include <stdlib.h> // standard lib
#include <errno.h> // number of last error
#include <sys/stat.h> // data returned stat() function (mkfifo)
#include <unistd.h> // unix standard
#include <fcntl.h> // file control options
#include <string.h> // string operations

#define SERVER_FIFO "/tmp/serverfifo"

int main (int argc, char **argv)
{
    int fd_server, num_bytes_read;
    char buf [512];

    // cria um FIFO se inexistente
    if (mkfifo (SERVER_FIFO, 0664) == -1) {
        perror ("mkfifo");
        exit (1);
    }

    // abre um FIFO
    if ((fd_server = open (SERVER_FIFO, O_RDONLY)) == -1)
        perror ("open");

    // lê e trata mensagens do FIFO
    while (1) {
        memset (buf, '\0', sizeof (buf));
        num_bytes_read = read (fd_server, buf, sizeof (buf));
        switch (num_bytes_read) {
            case -1:
                perror ("-- read error"); break;
            case 0:
                printf ("-- None data...close and reopen fifo --\n");
                close (fd_server);
                fd_server = open (SERVER_FIFO, O_RDONLY);
                break;
            default:
                printf ("Received %d bytes: %s\n", num_bytes_read, buf);
        }
    }

    if (close (fd_server) == -1)
        perror ("close");
}

```

```

/**
 * Escreve N mensagens para o FIFO
 */
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

#define SERVER_FIFO "/tmp/serverfifo"
#define N 5

char buf [512];

int main (int argc, char **argv)
{
    int fd_server; // descritor para o FIFO

    if ((fd_server = open (SERVER_FIFO, O_WRONLY)) == -1) {
        perror ("open error: server fifo");
        return 1;
    }

    int value = 0;
    while (value < N) {
        // cria mensagens incrementalmente
        sprintf (buf, "Message %ld - Number %d", (long) getpid (), value++);
        printf ("Sending: %s\n", buf);

        // envia mensagem para o fifo
        write (fd_server, buf, strlen (buf));

        sleep(1); // somente para visualizacao
    }

    if (close (fd_server) == -1) {
        perror ("close error:server fifo");
        return 2;
    }

    return 0;
}

```

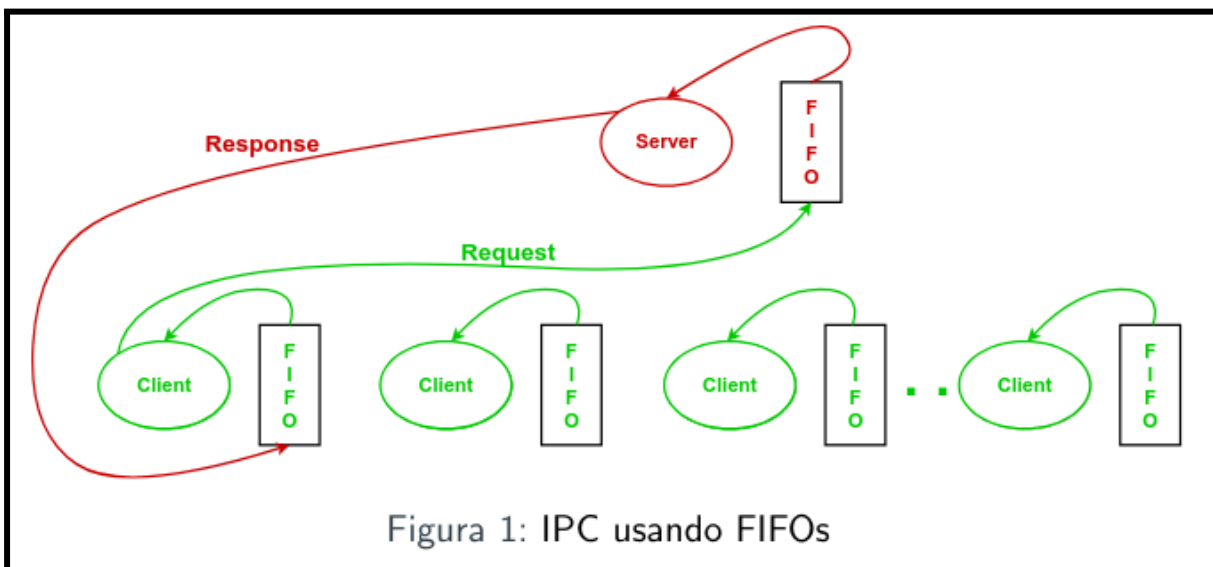


Figura 1: IPC usando FIFOs

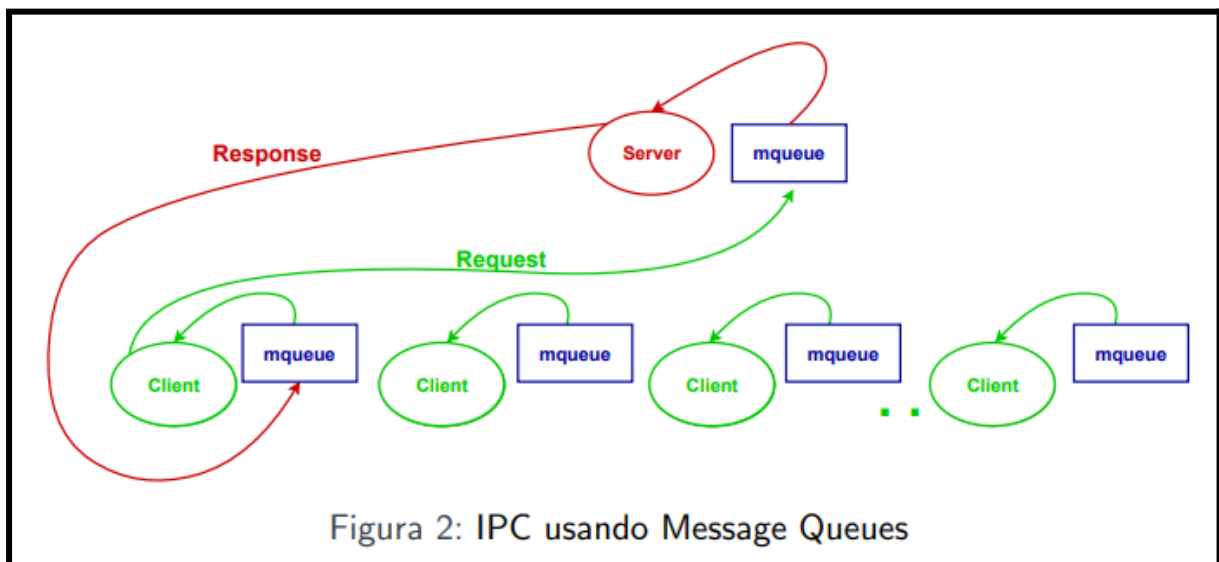
Fila de Mensagens

- **Permitem que os processos transmitam informações** que são compostas por um tipo de mensagem e por uma área de dados de tamanho variável.
- **Armazenadas em filas de mensagens**, permanecem até que um processo esteja preparado para recebê-las.
- **Processos relacionados podem procurar um identificador de fila de mensagens** em um arranjo global de descritores de fila de mensagens.

- O descritor de fila de mensagens contém:
 - Fila de mensagens pendentes;
 - Fila de processos em espera de mensagens;
 - Fila de processos em espera para enviar mensagens;
 - Dados que descrevem o tamanho e o conteúdo da fila de mensagens.

POSIX Message Queue

- Disponível no Linux desde a versão 2.2.6.
- São identificadas por nomes definidos por uma cadeia de caracteres (string).
- No Linux, os nomes iniciam-se com /
- Qualquer processo que conheça o nome e tenha permissões, pode enviar e receber mensagens.
- No Linux, usa-se a biblioteca de tempo real para compilar (-lrt) e os nomes das funções iniciam-se com mq_
- Funções básicas (mqqueue.h)
 - mq open: abre uma fila POSIX.
 - mq close: fecha o descritor da fila.
 - mq send: envia uma mensagem para uma fila.
 - mq receive: recebe uma mensagem de uma fila.
 - mq unlink: remove uma fila.
 - mq setattr: configura atributos de uma fila.



Sockets

- Possibilita a comunicação entre processos locais e remotos.

- Comunicação bidirecional.
- Deve-se especificar:
 - **domínio de comunicação** (ex: AF_UNIX, AF_INET).
 - **tipo de comunicação** (ex: SOCK_STREAM, SOCK_DGRAM).
- No linux, ao criar um socket, é devolvido um descritor de arquivo.
- Funções básicas:
 - **socket**: cria e devolve o descritor do socket.
 - **connect**: estabelece conexão com o servidor.
 - **bind**: associa um endereço de protocolo local (ex:porta) ao socket.
 - **listen**: define o socket como **passivo (socket do servidor)** e limita o tamanho da fila interna de conexões.
 - **accept**: aguarda conexões e devolve um descritor de socket para uma conexão.
 - **send**: envia dados por stream sockets.
 - **recv**: recebe dados por stream sockets.
 - **close**: fecha a comunicação entre cliente e servidor.

Socket Unix

- **Comunicação entre processos em uma mesma máquina.**
- Socket UNIX é conhecido por um pathname.
- Um servidor mapeia o pathname para o socket.
- Há três tipos de endereçamento: **pathname, abstract e unnamed.**
- Provém comunicação bidirecional ao usar stream sockets.
- Clientes usando sockets mantêm conexão individual com o servidor.

Servidor: aguarda conexão e recebe mensagens do cliente.

Fonte: https://troydhanson.github.io/network/Unix_domain_sockets.html

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/un.h>

//char *socket_path = "./mysocket";
char *socket_path = "\\myabstractsocket";

int main(int argc, char *argv[]) {
    int server_socket, // descritor do socket
        client_socket, // socket da conexão do cliente
        received_bytes; // bytes recebidos
    struct sockaddr_un addr; // endereço socket
    char buf[100]; // buffer de comunicação

    /* cria um socket AF_UNIX do tipo SOCK_STREAM */
    if ( (server_socket = socket(AF_UNIX, SOCK_STREAM, 0)) == -1) {
        perror("socket error");
        exit(-1);
    }

    /* configura endereço do socket */
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;

    if (*socket_path == '\\0') {
        *addr.sun_path = '\\0';
        strncpy(addr.sun_path+1, socket_path+1, sizeof(addr.sun_path)-2);
    } else {
        strncpy(addr.sun_path, socket_path, sizeof(addr.sun_path)-1);
        unlink(socket_path); // desvincular path se existe
    }

    /* mapeia o socket para o socket_path */
    if (bind(server_socket, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
        perror("bind error");
        exit(-1);
    }

    /* configura para aguardar conexões */
    if (listen(server_socket, 5) == -1) {
        perror("listen error");
        exit(-1);
    }

    while (1) {
        /* aguarda conexões dos clientes */
        if ( (client_socket = accept(server_socket, NULL, NULL)) == -1) {
            perror("accept error");
            continue;
        }

        /* lê dados enviados pelos clientes */
        while ( (received_bytes = read(client_socket, buf, sizeof(buf))) > 0 ) {
            printf("read %u bytes: %s\n", received_bytes, buf);
        }

        /* trata erros */
        if (received_bytes == -1) {
            perror("read");
            exit(-1);
        } else if (received_bytes == 0) {
            printf("EOF\n");
            close(client_socket);
        }
    }

    return 0;
}
```

Cliente: conecta e envia mensagens para o servidor.

Fonte: https://troydhanson.github.io/network/Unix_domain_sockets.html

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>

/* especificar o path para mapear o socket */
//char *socket_path = "./mysocket";
char *socket_path = "\\myabstractsocket";

int main(int argc, char *argv[]) {
    int client_socket, // descritor para o socket (file descriptor)
        sent_bytes; // número de bytes enviados
    struct sockaddr_un addr; // estrutura de endereço socket unix
    char buf[100]; // buffer para troca de mensagens

    /* cria socket UNIX do tipo SOCK_STREAM */
    if ( (client_socket = socket(AF_UNIX, SOCK_STREAM, 0)) == -1) {
        perror("socket error");
        exit(-1);
    }

    /* preenche estrutura de endereço */
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;

    if (*socket_path == '\\0') { // socket não mapeado no sistema de arquivos
        *addr.sun_path = '\\0';
        strncpy(addr.sun_path+1, socket_path+1, sizeof(addr.sun_path)-2);
    } else {
        strncpy(addr.sun_path, socket_path, sizeof(addr.sun_path)-1);
    }

    /* conecta com o processo servidor */
    if (connect(client_socket, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
        perror("connect error");
        exit(-1);
    }

    /* lê dados do terminal e envia via sockets */
    while ( (sent_bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0 ) {
        if (write(client_socket, buf, sent_bytes) != sent_bytes) {
            if (sent_bytes > 0) fprintf(stderr, "partial write");
            else {
                perror("write error");
                exit(-1);
            }
        }
    }

    return 0;
}
```

Troca de mensagem entre pai e filho com *socketpair*.

```
/** Demonstracao de como usar unnamed sockets para comunicação entre
    processos pai e filho.
    int socketpair(int domain, int type, int protocol, int sv[2]);
    **/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <sys/types.h>

#define HELLOREQUEST "Hello, how are you?"
#define HELLORESPONSE "I am fine, thanks."

int main()
{
    int sockets[2], pid;
    char buf[1024];

    /* cria um par de sockets de domínio UNIX e tipo STREAM */
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sockets) < 0) {
        perror("opening stream socket pair");
        exit(1);
    }

    pid = fork();

    if (pid) { /* Processo PAI */
        /* PAI usa o sockets[1] para enviar/receber */
        close(sockets[0]);

        if (read(sockets[1], buf, 1024) < 0)
            perror("error reading message");

        printf("PAI recv: %s\n", buf);

        if (write(sockets[1], HELLORESPONSE, sizeof(HELLORESPONSE)) < 0)
            perror("error writing message");

        close(sockets[1]);
    }
    else { /* Processo FILHO. */
        /* FILHO usa o sockets[0] para enviar/receber */
        close(sockets[1]);

        if (write(sockets[0], HELLOREQUEST, sizeof(HELLOREQUEST)) < 0)
            perror("error writing message");

        if (read(sockets[0], buf, 1024) < 0)
            perror("error reading message");

        printf("FILHO recv: %s\n", buf);

        close(sockets[0]);
    }

    return 0;
}
```

Socket TCP/IP

- Comunicação entre processos remotos ou local usando endereço de loopback (127.0.0.0/8).
- **Socket é associado a endereço IP e porta.**
- Exemplo: chat

Memória Compartilhada

- Dois ou mais processos utilizam a **região de memória compartilhada**, conectando-a no seu espaço de endereçamento.
- **Deve-se ter a garantia de que os dois processos não estejam gravando dados no mesmo local simultaneamente.**
- Exemplo: Problema Produtor-Consumidor
- Vantagens:
 - **Melhora o desempenho de processos que acessam frequentemente dados compartilhados.**
 - **Os processos podem compartilhar a mesma quantidade de dados que podem endereçar.**
 - Mais adequada para comunicação local (mesma máquina)
- Interface padronizada

- **Memória compartilhada System V.**
- **Memória compartilhada POSIX.**
 - **Não permite que os processos mudem privilégios de um segmento de memória compartilhada.**
- **Funções para uso de memória compartilhada POSIX**
 - **shm_open:** cria ou abre um objeto de memória compartilhada.
 - **shm_unlink:** remove um objeto de memória compartilhada.
 - **ftruncate:** especifica o tamanho de segmento de memória compartilhada.
 - **mmap:** mapeia o objeto de memória compartilhada dentro do espaço de endereçamento do processo.
 - **munmap:** desassocia o objeto de memória compartilhada do espaço de endereçamento do processo.
 - **close:** fecha o descritor alocado por shm_open.
- **Funções para uso de memória compartilhada SYSTEM V**
 - **shmget:** aloca um segmento de memória compartilhada.
 - **shmat:** anexa um segmento de memória compartilhada a um processo.
 - **shmctl:** altera os atributos de um segmento de memória compartilhada.
 - **shmdt:** desacopla um segmento de memória compartilhada.

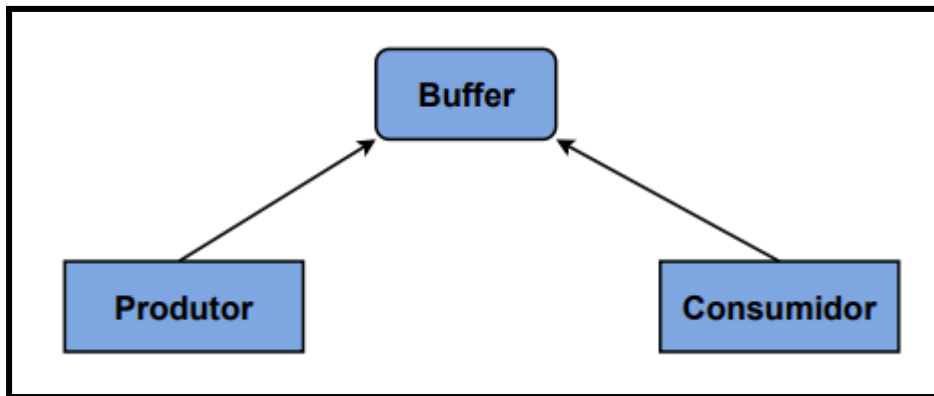
Implementação de Memória Compartilhada - POSIX

- **Trata a região da memória compartilhada como um arquivo.**
- **As molduras de página de memória compartilhada são liberadas quando o arquivo é apagado.**
- **o tmpfs (sistema de arquivo temporário) armazena esses arquivos.**
 - Geralmente montado no Linux em /dev/shm.
 - As páginas do tmpfs podem ser trocadas.
 - É possível definir as permissões.
 - O sistema de arquivo não exige formatação.

Problema Produtor-Consumidor

- **Paradigma para processos em cooperação**
- **Processo produtor produz informações que são consumidas por um processo consumidor**
 - **Buffer ilimitado** não impõe limite prático sobre o tamanho do buffer

- **Buffer limitado** assume que existe um tamanho de buffer fixo



Sincronização e Concorrência

Introdução

- **Programação Concorrente**

- Consiste em um conjunto de processos sequenciais que executam concorrentemente.
- A concorrência implica em: disputa, execução simultânea e cooperação entre processos.
- A cooperação implica em compartilhar dados por meio de memória ou troca de mensagens.
- No entanto, o acesso a dados compartilhados sem controle pode resultar em inconsistências ou comportamentos não esperados.

Problemas

- Exemplo 1 – Produtor-Consumidor
 - Há processos que produzem dados e armazenam em uma variável compartilhada.
 - Há processos que consomem os dados armazenados na variável compartilhada.
- Exemplo 2 – Região com variáveis compartilhadas
 - Considere o incremento de uma variável.

```
1  int x = 10;  
2  
3  // código compartilhado entre dois processos  
4  x = x + 1;
```


- Execução concorrente dos processos P1 e P2:

Execução 1:	Execução 2:
1 P1: MOV x, ACC	1 P1: MOV x, ACC
2 P1: INC ACC	2 P1: INC ACC
3 P1: MOV ACC, x	3 % P1 escalonado
4 % P1 escalonado	4 P2: MOV x, ACC
5 P2: MOV x, ACC	5 P2: INC ACC
6 P2: INC ACC	6 P2: MOV ACC, x
7 P2: MOV ACC, x	7 % P2 escalonado
8 % P2 escalonado	8 P1: MOV ACC, x

○

Problema da Região Crítica

- **Região Crítica**

- Trecho de código com alteração de dados compartilhados.

- **Condição de corrida (race conditions)**

- Ocorre quando dois ou mais processos acessam/modificam dados compartilhados simultaneamente e o resultado depende da ordem de execução.
- Resultado da execução de um trecho de código depende da ordenação de acesso às operações desse trecho.

- **Requisitos para a solução**

- **Exclusão mútua**: dois ou mais processos não podem estar simultaneamente dentro da região crítica.
- **Progressão**: processos fora da região crítica não podem bloquear a entrada/execução de outro processo na região crítica.
- **Espera limitada**: processos não podem esperar infinitamente para entrar na região crítica. **(Justiça)**
- Processos não podem depender do número de processadores e de suas velocidades de execução com solução.

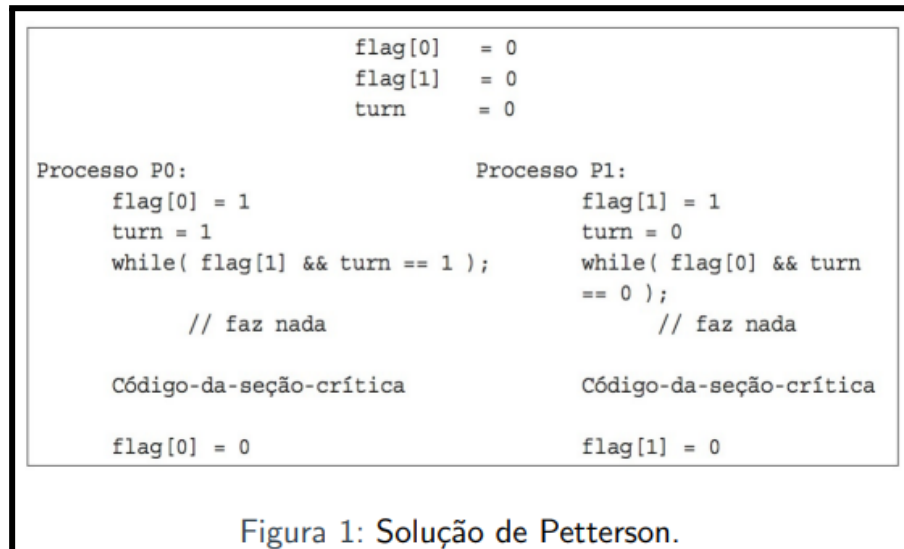
Mecanismos Básicos de Exclusão Mútua

- **Soluções em software puro**

- Executam na entrada e saída da Seção Crítica.
- Não há chamadas ao SO e interrupções especiais.
- Problemas: **complexas, espera-ocupada (busy-waiting)**.

- **Busy waiting:** É quando um processo fica continuamente verificando uma condição em loop, consumindo CPU enquanto espera.

- Exemplo:
- while (lock == 1);
- O processo fica “girando” até o lock liberar.



- **Desabilitar interrupções**

- **Desabilita interrupções antes e ativa após o acesso à Seção Crítica.**
- Solução simples para um processador.
- Usado em sistemas pequenos e dedicados (embarcados).
- Controle interno via SO.
- **Problemas: pode diminuir a eficiência (interrupções pendentes), não adequado para máquinas paralelas.**

- **Spin-lock**

- Baseada em instruções de máquina que **realizam operações atômicas**.
- A ideia central é proteger a Seção Crítica com uma variável (lock).
 - lock = 0: seção crítica livre.
 - lock = 1: seção crítica ocupada.
- Simplicidade de implementação.
- Recomendado para seções críticas pequenas.
- **Desvantagens: espera ocupada (busy-waiting), possibilidade de postergação indefinida.**

Soluções em Software Puro

Algoritmo de Dekker

- Primeira solução de software para o **problema da exclusão mútua entre dois processos**, proposta por Dekker (1965). Combina duas ideias: **flags de intenção (quer_entrar[])** para sinalizar interesse, e uma **variável vez** para desempatar quando os dois querem entrar **ao mesmo tempo** — o processo que não tem a vez cede e espera. **Elimina deadlock, livelock e starvation.**

```
/* VARIÁVEIS COMPARTILHADAS */

int vez = 0;           // 0 = vez de P0, 1 = vez de P1
bool quer_entrar[2] = {false, false};

PROCESSOS

PROCESSO P0

while (1) {
    /* protocolo de entrada */
    quer_entrar[0] = true;

    while (quer_entrar[1]) {
        if (vez == 1) {
            // não é minha vez: cedo
            quer_entrar[0] = false;
            while (vez == 1); // espera
            quer_entrar[0] = true;
        }
    }

    /* ---- seção crítica ---- */

    /* protocolo de saída */
    vez = 1;
    quer_entrar[0] = false;

    /* seção não-crítica */
}

PROCESSO P1

while (1) {
    /* protocolo de entrada */
    quer_entrar[1] = true;

    while (quer_entrar[0]) {
        if (vez == 0) {
            // não é minha vez: cedo
            quer_entrar[1] = false;
            while (vez == 0); // espera
            quer_entrar[1] = true;
        }
    }

    /* ---- seção crítica ---- */

    /* protocolo de saída */
    vez = 0;
    quer_entrar[1] = false;

    /* seção não-crítica */
}
```

- **Como funciona o desempate:** Se P0 e P1 levantam a flag ao mesmo tempo, **o que não tem a vez abaixa a flag**, espera passivamente até a vez mudar, e então rearma a flag. O detentor da vez segue em frente. **Na saída, quem acabou de usar a SC transfere a vez para o outro** — garantindo que nenhum processo entre duas vezes seguidas antes do outro ter uma chance.

Algoritmo de Petterson

- Solução mais elegante de Gary Peterson (1981), também para **dois processos**. Usa as mesmas variáveis que Dekker, mas **combina flag e vez** em um único passo de "cortesia": **ao querer entrar, o processo levanta a flag e cede a vez ao rival. Entra na SC apenas se o rival não quer entrar ou a vez voltou para si**. O código é muito mais simples que o de Dekker.

```
/* VARIÁVEIS COMPARTILHADAS */
```

```
int  vez = 0;
bool quer_entrar[2] = {false, false};
```

PROCESSOS

PROCESSO P0

```
while (1) {
    /* protocolo de entrada */
    quer_entrar[0] = true;
    vez = 1; // cede a vez a P1

    while (quer_entrar[1]
           && vez == 1);
    // espera se P1 quer E tem a vez

    /* ---- seção crítica ---- */

    /* protocolo de saída */
    quer_entrar[0] = false;

    /* seção não-crítica */
}
```

PROCESSO P1

```
while (1) {
    /* protocolo de entrada */
    quer_entrar[1] = true;
    vez = 0; // cede a vez a P0

    while (quer_entrar[0]
           && vez == 0);
    // espera se P0 quer E tem a vez

    /* ---- seção crítica ---- */

    /* protocolo de saída */
    quer_entrar[1] = false;

    /* seção não-crítica */
}
```

- Por que funciona:** **O último a escrever em vez perde o desempate** — se os dois chegam ao mesmo tempo, um sobrescreve o outro em vez, e quem sobrescreveu por último ficará travado no while. **Quem tem a vez entra; quem cedeu espera**. Na saída basta abaixar a flag: não é preciso transferir a vez, pois ela foi cedida na entrada.

Algoritmo da Padaria

- Algoritmo de Lamport (1974), inspirado nas fichas numeradas de uma padaria. **Funciona com N processos arbitrários**. **Cada processo pega um número maior que os existentes e espera até ser o menor número ativo**. Em caso de empate no número, o de menor índice ($i < j$) tem prioridade. Não requer instrução atômica especial — apenas leitura/escrita simples.

/ VARIÁVEIS COMPARTILHADAS */*

```
int N = 5; // número de processos
int numero[N] = {0, ...}; // ficha de cada processo (0 = não interessado)
bool escolhendo[N] = {false, ...}; // true enquanto está pegando número

// Comparação lexicográfica de pares (numero, id):
// (a,i) < (b,j) sse a
```

FUNÇÕES AUXILIARES

```
max(v[], n) { // retorna o maior número no vetor
    m = 0;
    for (k = 0; k < n; k++)
        if (v[k] > m) m = v[k];
    return m;
}

menor_par(a, i, b, j) { // (a,i) < (b,j) ?
    return (a < b) || (a == b && i < j);
}
```

PROCESSO P_i

```
while (1) {
    /* — protocolo de entrada — */

    // 1. Anuncia que está escolhendo número
    escolhendo[i] = true;

    // 2. Pega um número maior que todos os atuais
    numero[i] = 1 + max(numero, N);

    // 3. Finaliza escolha
    escolhendo[i] = false;

    // 4. Espera sua vez: para cada outro processo Pj...
    for (j = 0; j < N; j++) {
        if (j == i) continue;

        // 4a. Espera Pj terminar de escolher número
        while (escolhendo[j]);

        // 4b. Espera enquanto Pj tem prioridade sobre Pi
        while (numero[j] != 0
            && menor_par(numero[j], j, numero[i], i));
    }

    /* — seção crítica — */

    /* — protocolo de saída — */
    numero[i] = 0; // devolve a ficha

    /* seção não-crítica */
}
```

- **Por que precisa de escolhendo[]:** A leitura de max() e a escrita em numero[i] não são atômicas — dois processos podem ler o mesmo máximo e pegar o mesmo número. A flag escolhendo[i] sinaliza "estou no meio da escolha": qualquer outro processo deve esperar eu terminar antes de comparar fichas comigo. Isso evita que alguém compare com um número ainda incompleto e passe à frente indevidamente.

Spin-lock

- Solução em alto nível:

```
1 /* Entrada da Secao Critica */
2 while (lock == 1);      // se verdadeiro, SC ocupada
3 lock = 1;
4
5 // Secao Critica ...
6
7 /* Saida da Secao Critica */
8 lock = 0;
```

- A manipulação da **variável lock** (comparação/leitura/escrita) deve ocorrer de **forma atômica**.
- Duas instruções comuns para implementar são:
 - **test-and-set (TSL)**: Copia o valor de memória para um registrador e armazena um valor diferente de 0 na memória.
 - **swap (SWAP ou XCHG)**: Troca o valor de duas posições (memórias, registrador) atômicamente.

Spin-Lock - TSL

```
1 entradaSC:
2   TSL RX, lock      % copia lock para RX, armazena valor diferente 0 em lock
3   CMP RX, 0         % compara RX com 0
4   JNE entradaSC     % se diferente, ir para entrada da SC
5   RET               % retorna
6
7 saidaSC:
8   MOV lock, 0       % atribui 0 a lock
9   RET
```

Execução 1: lock = 0

RX = 0 e lock = 2 (linha 2)
RX == 0 ? (linha 3)
Verdade (linha 4)
entrando SC (linha 5)

Processo na SC

lock = 0 (linha 8)
saindo SC (linha 9)

Execução 2: lock = 1

RX = 1 e lock = 5 (linha 2)
RX == 0 ? (linha 3)
Falso (linha 4)
indo entradaSC (linha 1)

Processo repete até que lock seja 0

Spin-Lock - XCHG

```
1  entradaSC:
2  MOV  RX, 1          % atribui 1 a RX
3  XCHG RX, lock       % troca conteudo RX e lock
4  CMP  RX, 0          % compara RX com 0
5  JNE  entradaSC      % se diferente, ir para entrada da SC
6  RET                % retorna
7
8  saidaSC:
9  MOV  lock, 0        % atribui 0 a lock
10 RET                % retorna
```

Execução 1: lock = 0

RX = 1 (linha 2)
RX = 0 e lock = 1 (linha 3)
RX == 0 ? (linha 4)
Verdade (linha 5)
entrando SC (linha 6)

Processo na SC

lock = 0 (linha 9)
saindo SC (linha 10)

Execução 2: lock = 1

RX = 1 (linha 2)
RX = 1 e lock = 1 (linha 3)
RX == 0 ? (linha 4)
Falso (linha 5)
indo entradaSC (linha 1)

Processo repete até que lock seja 0

Mutex

- Tipo abstrato de dados: valor lógica e fila de processos.
- A **variável do tipo Mutex pode assumir: livre e ocupado**.
- Suporta duas operações:
 - **lock**: solicitar entrada na seção crítica.
 - **unlock**: liberar seção crítica (indicar a saída).
- As operações lock e unlock devem ser atômicas.


```

1 lock(mutex):
2   if mutex == LIVRE:
3     mutex = OCUPADO
4   else:
5     bloqueia processo e insere na fila
6
7 unlock(mutex):
8   if fila vazia:
9     mutex = LIVRE
10  else:
11    libera processo no inicio da fila

```

Figura 2: Pseudocódigo para as operações lock e unlock.

```

1 Mutex mutex = LIVRE
2
3 lock(mutex)
4  ## código da Seção Crítica ##
5 unlock(mutex)

```

Figura 3: Protegendo a seção crítica com o Mutex.

```

1 pthread_t t[N];           // define N threads
2 pthread_mutex_t mutex;    // variavel mutex
3 int counter;              // variavel compartilhada
4
5 void* task(void *arg)
6 {
7   pthread_mutex_lock(&mutex); // inicio SC
8   counter += 1;
9   pthread_mutex_unlock(&mutex); // saida SC
10 }
11
12 int main(void)
13 {
14   /* inicializa mutex — inicializado como LIVRE */
15   pthread_mutex_init(&mutex, NULL);
16
17   /* inicializa threads */ ...
18   /* aguarda todas as threads finalizarem */ ...
19
20   /* destroi mutex */
21   pthread_mutex_destroy(&mutex);
22   ...
23 }

```

Figura 4: Código em C com pthreads usando mutex.

Semáforos

- Tipo abstrato de dados: valor inteiro e fila de processos.
- Suporta duas operações:
 - **P()**: decrementa valor inteiro, **bloqueia processo se valor for negativo (insere fila)**.
 - **V()**: incrementa valor inteiro, **libera processo no início da fila (se existir)**.

- **P()** - proben (testar/tentar) e **V()** - verhogen (incrementar).
- As operações **P()** e **V()** devem ser atômicas.

```

1 P(S):
2   S.valor = S.valor - 1
3   if S.valor < 0:      # valor negativo
4       bloqueia processo e insere na fila
5
6 V(S):
7   S.valor = S.valor + 1
8   if len(S.fila) > 0:  # fila nao vazia
9       libera processo no inicio da fila

```

Figura 5: Pseudocódigo para as operações P() e V().

```

1 Semaforo s
2 s.init_sem(1) #inicializa com 1
3 ...
4
5 P(s)
6 ## codigo da Secao Critica ##
7 V(s)

```

Figura 6: Usando semáforo para proteção de seção crítica.

- **Considerações e usos de semáforos:**
 - **Semáforos podem ser usados também para sincronização/sinalização entre processos.**
 - Exemplos: relações de precedência de execução, execução após um evento específico, barreiras.
 - Em geral:
 - **para exclusão mútua:** semáforo **inicializado com valor 1.**
 - **para sinalização:** semáforo **inicializado com valor 0.**
 - **para controle de múltiplas instâncias de recurso:** semáforo **inicializado com valor maior que 1.**
 - Ex: 5 impressoras disponíveis.
 - **Mutex:** mesmo processo executa o lock() e unlock().
 - **Semáforo:** processos distintos podem realizar o P() e V().

Usando semáforos para sinalização

```
1  Semaforo s
2  s.init_sem (0)
3  ...
4  # compartilhar semaforo com as threads
```

Thread 1

```
1  # codigo da Thread 1
2  ...
3  ...
4  # thread aguardando Thread 2
5  P(s)
6  ... # codigo executa somente apos
      sinalizacao
```

Thread 2

```
1  # codigo da Thread 2
2  ...
3  ...
4  # sinaliza para Thread 1
5  V(s)
6  ...
7  # continua a execucao
```

Monitores

- Tipo abstrato de dados em nível de linguagem.
- Gerenciar variáveis compartilhados dentro do Monitor, isto é, encapsular variáveis e acesso somente via métodos.
- As operações do Monitor provêm exclusão mútua.
- Uso de variáveis de condição e operações wait() e signal().
- Variável de condição: associada a uma fila de processos bloqueados.
- wait(): bloqueia o processo e adiciona na fila.
- signal(): acorda o primeiro processo na fila.

```
1 public class ResourceMonitor {
2     private String buffer;
3     private boolean empty = true;
4
5     public synchronized String take() {
6         while (empty) { // aguarda se buffer vazio
7             try {
8                 wait();
9             } catch (InterruptedException e) {}
10        }
11        /* remove valor de buffer */
12        empty = true;
13        notifyAll(); // notifica os produtores
14        return buffer;
15    }
16
17    public synchronized void put(String buffer) {
18        while (!empty) { // aguarda se buffer cheio
19            try {
20                wait();
21            } catch (InterruptedException e) {}
22        }
23        /* armazena valor em buffer */
24        empty = false;
25        this.buffer = buffer;
26        notifyAll(); // notifica os consumidores
27    }
28 }
```

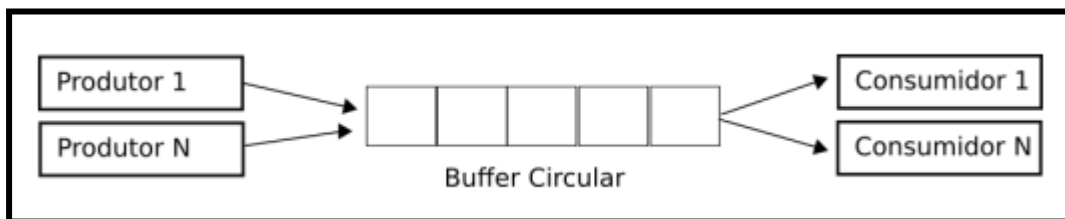
Figura 8: Exemplo de monitor em Java para Problema do Produtor/Consumidor.

Problemas Clássicos

- **Produtores/Consumidores.**
- **Leitores/Escritores.**
- **Jantar dos Filósofos.**
- **Barbeiro Dorminhoco.**

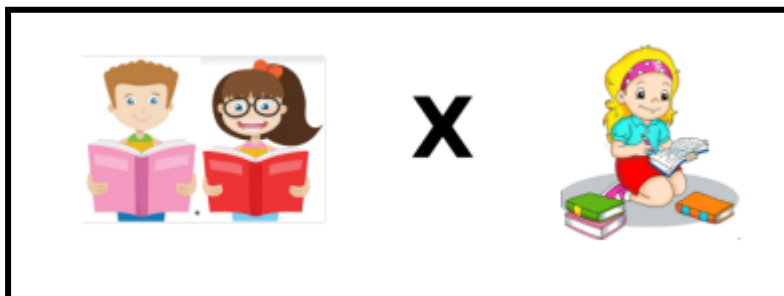
Produtores/Consumidores

- **Dois tipos de processos (threads):**
 - **Produtores:** produzem e armazenam itens em um buffer.
 - **Consumidores:** retiram e consomem itens de um buffer.
- **A variável buffer é compartilhada.**
- **Restrições:**
 - **Produtores** devem aguardar se o buffer estiver cheio.
 - **Consumidores** devem aguardar se o buffer estiver vazio.
 - **Exclusão mútua** para manipulação do buffer.



Leitores/Escritores

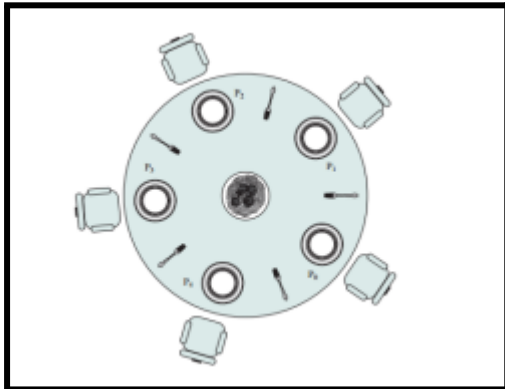
- **Dois tipos de processos (threads):**
 - **Leitores:** Realizam a leitura de um recurso compartilhado.
 - **Escritores:** Realizam a escrita em um recurso compartilhado.
- **Restrições:**
 - **Os leitores** podem acessar o recurso simultaneamente.
 - **Os escritores** devem acessar o recurso exclusivamente.



Jantar dos Filósofos

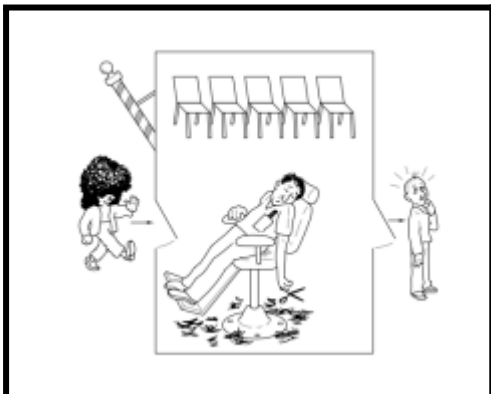
- Os filósofos (processos) estão em uma mesa circular.
- Os filósofos pensam, sentem fome e comem.

- Mesa: 5 pratos de macarrão e um garfo (recurso) entre cada prato.
- Para comer, os filósofos precisam de dois garfos.
- Ao sentir fome, os filósofos pegam os garfos e, se conseguirem, comem.
- Ao terminar de comer, os filósofos devolvem os dois garfos.



Barbeiro Dorminhoco

- Barbearia com 1 cadeira para atender e N para esperar.
- O barbeiro (processo) está atendendo ou dormindo (não há clientes).
- Os clientes (processos) chegam na barbearia e, se o barbeiro está dormindo, o acordam. Se está atendendo, os clientes esperam em uma das cadeiras (se disponível), caso contrário, vão embora.
- O barbeiro volta a dormir quando não há mais clientes.



Impasse e Inanição

Introdução

- **Recursos**
 - Exemplos de recursos: área de memória, impressora, estrutura de dados.

- **Processos precisam de acesso a recursos em determinada ordem.**
- Exemplo de **problema**:
 - Processo P está usando um recurso A.
 - Processo Q está usando um recurso B.
 - Processo Q solicita o recurso A e é bloqueado.
 - Processo P solicita o recurso B e é bloqueado.

- **Definições**

- **Impasse (Deadlock)**

- Dois ou mais processos aguardam eventos que dependem dos outros processos.
 - Ex: dois ou mais processos tentam entrar na seção crítica ao mesmo tempo.

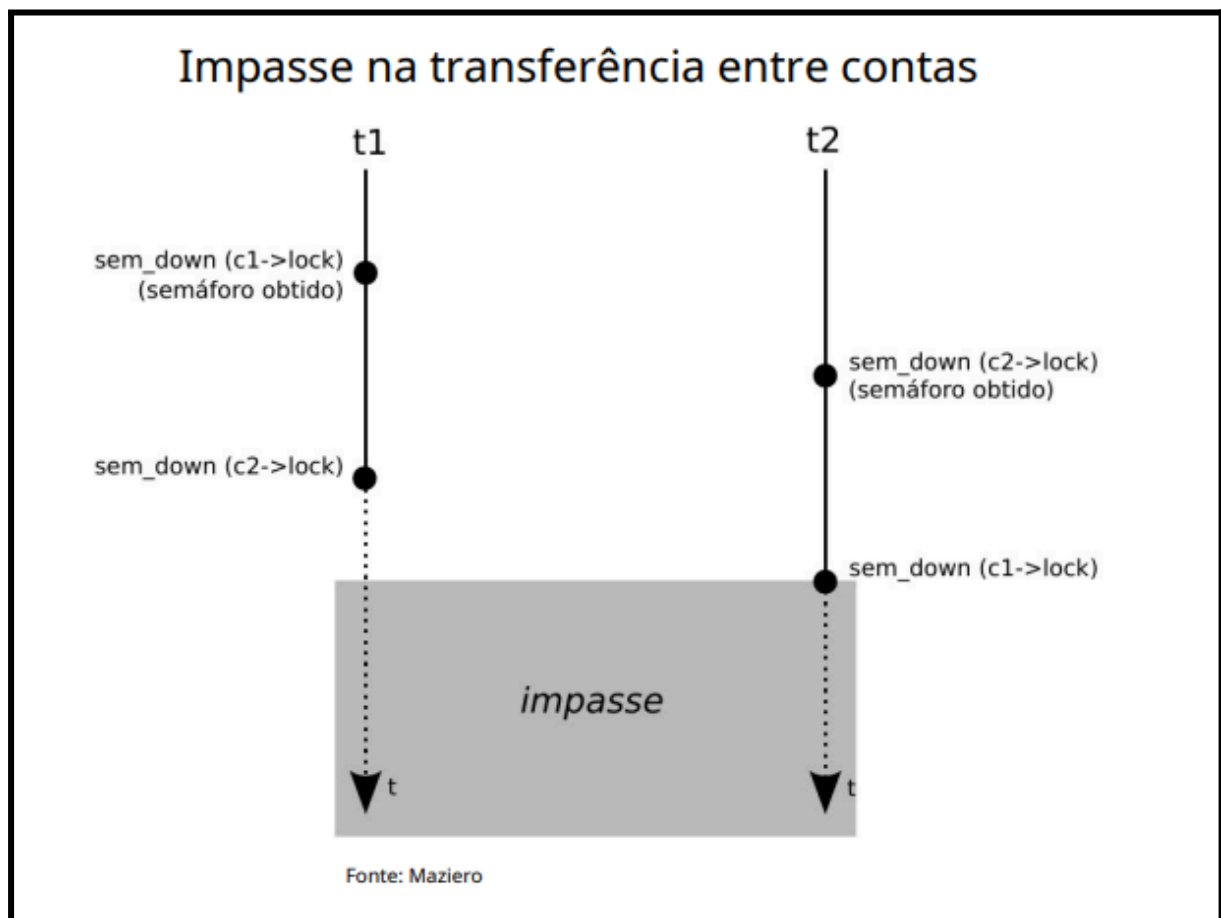
- **Inanição ou adiamento indefinido (Starvation)**

- Um processo é negligenciado continuamente pelo escalonador, assim não consegue proceder sua execução.

- **Livelock**

- Dois ou mais processos alteram seus estados continuamente sem fazer trabalho útil em função de alterações de outros processos.
 - **Os processos continuam executando, mas nenhum avança.**
 - Exemplo:
 - Dois processos cedendo passagem um ao outro indefinidamente.

Impasses



- Quatro condições obrigatórias:
 - **Exclusão mútua**
 - Todo recurso está associado exclusivamente a um processo.
 - **Posse e espera**
 - Processos podem manter recursos e solicitar novos recursos.
 - **Não preempção**
 - Recursos não podem ser preemptados, isto é, tomados de um processo.
 - **Espera circular**
 - Processos formam uma cadeia circular de posse e espera por um recurso.

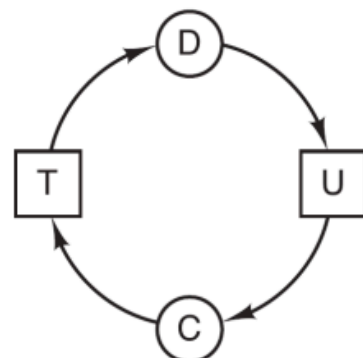
■ Grafo de Alocação de Recursos



(a)



(b)

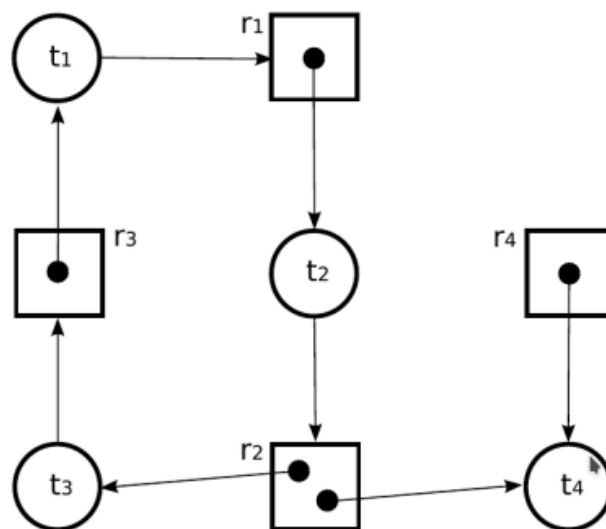


(c)

Grafos de alocação de recursos:
 (a) processo de posse de um recurso.
 (b) Solicitação de um recurso.
 (c) Impasse.

Fonte: Tanenbaum

■ Observando o grafo, há um impasse?



Fonte: Maziero

t: tarefa
 r: recurso

R:

- t1 espera r1

- r1 está com t2
- t2 espera r2
- r2 está com t3
- t3 espera r3
- r3 está com t1

Forma-se o ciclo:

$t1 \rightarrow t2 \rightarrow t3 \rightarrow t1$

Resumo

Condição de Impasse	Presente?	Evidência
Exclusão mútua	✓	r1 e r3 possuem apenas uma instância
Posse e espera	✓	t1, t2 e t3 seguram recursos enquanto aguardam outros
Não preempção	✓	recursos não podem ser retirados à força
Espera circular	✓	$t1 \rightarrow t2 \rightarrow t3 \rightarrow t1$

Há um impasse (deadlock) entre t1, t2 e t3.

Técnicas de Tratamento

- Técnicas de **tratamento de impasses**
 - **Algoritmos do Avestruz**
 - Ignora o impasse.
 - **Prevenção de impasses**
 - Negar ao menos uma das condições de impasse.
 - **Evitação de impasses**
 - Alocar cuidadosamente os recursos para evitar impasses.
 - **Deteccção e recuperação de impasses**
 - Detectar o impasse e aplicar medidas para resolvê-lo.

Algoritmo do Avestruz

- **Ignora a existência do problema de impasses.**
- UNIX e Windows adotam essa técnica para lidar com os impasses.
- Poucos impasses x custo de prevenção.

Prevenção de Impasses

- Consiste em impedir que pelo menos uma das quatro condições necessárias para ocorrência de deadlock aconteça.
- Desvantagem: pode reduzir significativamente o desempenho do sistema.
- Negar a exclusão mútua
 - Compartilhar recursos quando possível.
 - Exemplo: técnica de spooling.
- Negar a posse e espera
 - Processo deve solicitar todos os recursos de uma vez.
 - Usar apenas um recurso por vez.
 - Obter todos os recursos antes de executar.
 - Estabelecer um limite de tempo para posse e espera.
- Negar a não-preempção
 - Retirar um recurso de um processo.
 - Interessante para situações em que o estado do processo pode ser salvo e restaurado.
- Negar a espera circular
 - Ordenação de todos os recursos do sistema.
 - Processos solicitam recursos somente nessa ordem.

Evitação de Impasses

- Consiste em evitar o impasse pela alocação cuidadosa de recursos.
- Vantagem: melhor aproveitamento dos recursos.
- Desvantagem: implementação mais complexa e custo de monitoramento.
- Estado seguro x estado inseguro.
 - Um estado é seguro se a partir dele é possível realizar a alocação de recursos aos processos e evitar assim o impasse.

Estado Seguro			Estado Inseguro		
	posse	total		posse	total
P _A	3	9	P _A	3	9
P _B	2	4	P _B	2	4
P _C	2	7	P _C	2	7
Disponível: 3 recursos			Disponível: 2 recursos		

○

● Algoritmo do Banqueiro

F

P

S

B

E = (4 2 3 1)

F

P

S

B

D = (2 1 0 0)

F: Unidades de fita

P: Plotters

S: Scanners

B: Blu-rays

F

P

S

B

P1

P2

P3

0

0

1

0

2

0

0

1

0

1

2

0

Matriz de alocação

F

P

S

B

P1

P2

P3

2

0

0

1

1

0

1

0

2

1

0

0

Matriz de requisições

E: recursos existentes

D: recursos disponíveis

Matrizes: cada linha representa os recursos para um processo.

Situação 1: P3 solicita 1 plotter.

$$E = \begin{pmatrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ & 4 & 2 & 3 & 1 \end{pmatrix}$$

$$D = \begin{pmatrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ & 2 & 0 & 0 & 0 \end{pmatrix}$$

F: Unidades de fita
P: Plotters
S: Scanners
B: Blu-rays

$$\begin{matrix} \text{P1} \\ \text{P2} \\ \text{P3} \end{matrix} \left\{ \begin{matrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 2 & 2 & 0 \end{matrix} \right\}$$

Matriz de alocação

$$\begin{matrix} \text{P1} \\ \text{P2} \\ \text{P3} \end{matrix} \left\{ \begin{matrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 0 & 0 & 0 \end{matrix} \right\}$$

Matriz de requisições

E: recursos existentes

D: recursos disponíveis

Matrizes: cada linha representa os recursos para um processo.

Situação 2: P1 solicita 1 fita.

$$E = \begin{pmatrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ & 4 & 2 & 3 & 1 \end{pmatrix}$$

$$D = \begin{pmatrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ & 1 & 1 & 0 & 0 \end{pmatrix}$$

F: Unidades de fita
P: Plotters
S: Scanners
B: Blu-rays

$$\begin{matrix} \text{P1} \\ \text{P2} \\ \text{P3} \end{matrix} \left\{ \begin{matrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ 1 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{matrix} \right\}$$

Matriz de alocação

$$\begin{matrix} \text{P1} \\ \text{P2} \\ \text{P3} \end{matrix} \left\{ \begin{matrix} & \text{F} & \text{P} & \text{S} & \text{B} \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{matrix} \right\}$$

Matriz de requisições

E: recursos existentes

D: recursos disponíveis

Matrizes: cada linha representa os recursos para um processo.

Deteção e Recuperação

- Deteção por inspecção ao grafo de alocação de recursos.
- O grafo deve ser mantido pelo sistema, isto é, atualizado nas alocações e liberações de recursos.
- Executar periodicamente um algoritmo para deteção de ciclos no grafo.

- **Problema: custo computacional do algoritmo.**

- **Recuperação**

- **Eliminar tarefas:**

- Eliminar todas ou selecionar tarefas.
 - Critérios para eliminar tarefas.

- **Reversão de estado:**

- Gravar o estado periodicamente dos processos e retomar a partir de um estado seguro.

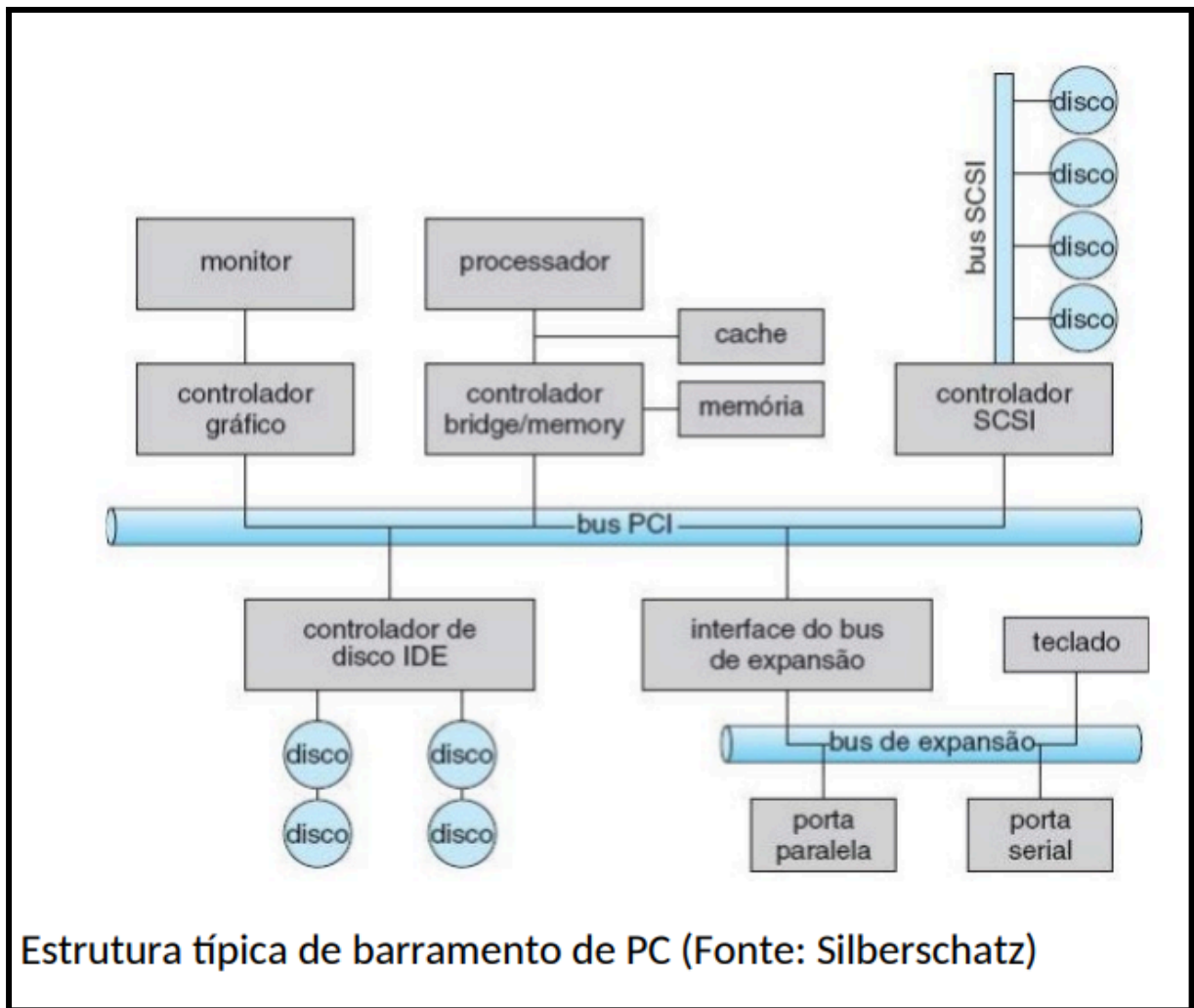
Entrada e Saída

Introdução

- Duas principais tarefas de computadores são:
 - **Processamento e Entrada/Saída (E/S).**
- **Sistema Operacional** deve:
 - **Gerenciar e controlar os dispositivos de E/S.**
 - Encapsular os detalhes e especificidades de cada dispositivo.
 - Possibilitar a adição de uma variedade de novos/existentes dispositivos.

Princípios Básicos de Hardware

- **Periféricos:** dispositivos externos conectados ao computador que possibilitam E/S.
- **Interfaces:** componentes de hardware que conectam o periférico ao computador.
- **Porta:** Ponto de conexão de um dispositivo com o computador.
- **Barramentos (bus):** ligações internas compartilhadas por dispositivos para transferência de dados/comunicação.
- **Controladores:** conjunto de componentes eletrônicos que podem operar uma porta, um barramento ou dispositivo.



Controladores

- **Simplex**: chip interno, por exemplo, controlador de porta serial.
- **Externo**: placa externa com processador, memória privada, código.
- **Embutido**: circuito embutido com processador e microcódigo.
- Os controladores possuem **registradores de dados, estado e controle usados para comunicação com a CPU** (lê e escreve para esses registradores).

Registradores

- **data-in e data-out**: obter e enviar dados para o dispositivo.
- **status**: indicar estado do dispositivo (ocioso, pronto leitura, ocupado, erro, ...)
- **control**: enviar comandos ou alterações de configurações (taxa de transferência, modo de operação, ...)
- Registradores são acessados como posições de memória:

- **E/S mapeada em espaço de E/S.**
- **E/S mapeada em espaço de memória.**

intervalo de endereços de I/O (hexadecimal)	dispositivo
000–00F	controlador de DMA
020–021	controlador de interrupções
040–043	timer
200–20F	controlador de jogos
2F8–2FF	porta serial (secundária)
320–32F	controlador de disco rígido
378–37F	porta paralela
3D0–3DF	controlador gráfico
3F0–3F7	controlador de drive de disquete
3F8–3FF	porta serial (primária)

Exemplos de endereços de E/S.

Mapeamento em Espaço de Memória

- Único espaço de endereçamento.
- No projeto, reservam-se endereços para acesso a controladores.
- **Os registradores dos dispositivos recebem endereços da memória principal.**
- A CPU acessa o dispositivo usando instruções normais de leitura e escrita.
- Vantagem:
 - **Não exige instruções especiais.**
- Desvantagem:
 - **Consome parte do espaço de endereçamento da memória.**

Mapeamento em Espaço de E/S

- Espaço de endereçamento distinto.
- **Os dispositivos possuem um espaço próprio de endereços.**
- **São usadas instruções específicas (IN, OUT).**
- Vantagem:
 - **Não ocupa espaço de memória.**
- Desvantagem:
 - **Exige instruções especiais.**

Mapeado em memória	Mapeado em entrada e saída
Le_status: mov AL, 315H	Le_status: in AL, 315H
Print_char: mov AL, 65H mov 312H, AL	Print_char: mov AL, 65H out 312H, AL

Dispositivos de E/S

- Dispositivos de E/S podem ser classificados:
 - **Orientados a caractere**
 - Unidade básica de transferência é o caractere. Por exemplo, teclado.
 - **Orientados a bloco**
 - Unidade de transferência é um bloco de caracteres. Por exemplo, disco.
- **Device Driver**
 - Módulo adicionado ao sistema para tratar um dispositivo específico ou uma categoria de dispositivos similares.

Técnicas de E/S

- **E/S programada.**
- **E/S orientada a interrupções.**
- **Acesso direto à memória.**

E/S Programada

- **A interação entre CPU e controlador é realizada pelo programador.**
- Operação: **envia o comando à controladora e espera a conclusão.**
- A Controladora atualiza constantemente o estado da operação.
- **A CPU verifica o estado continuamente (busy waiting).**
- **Usar polling:** CPU intercala outros processamentos e consulta o estado periodicamente.
- Vantagens
 - **Simples de implementar.**

- **Baixo custo de hardware.**
- **Desvantagens**
 - **CPU fica ocupada verificando o dispositivo continuamente.**
 - **Baixa eficiência.**

E/S Orientada a Interrupções

- **A CPU é interrompida após o módulo realizar a E/S.**
- A **CPU apenas inicia a operação de E/S** e fica livre para realizar outros processos.
- Faz uso de uma rotina de **tratamento de interrupção** para processar a E/S.
- A CPU atua como uma intermediária na transferência, pois toda palavra lida ou escrita pelo dispositivo envolve acesso à CPU.
- **Vantagens**
 - **CPU pode executar outras tarefas.**
 - **Melhor aproveitamento do processador.**
- **Desvantagens**
 - **Sobrecarga para tratar interrupções.**
 - **Muitas interrupções podem degradar desempenho.**

Acesso Direto à Memória

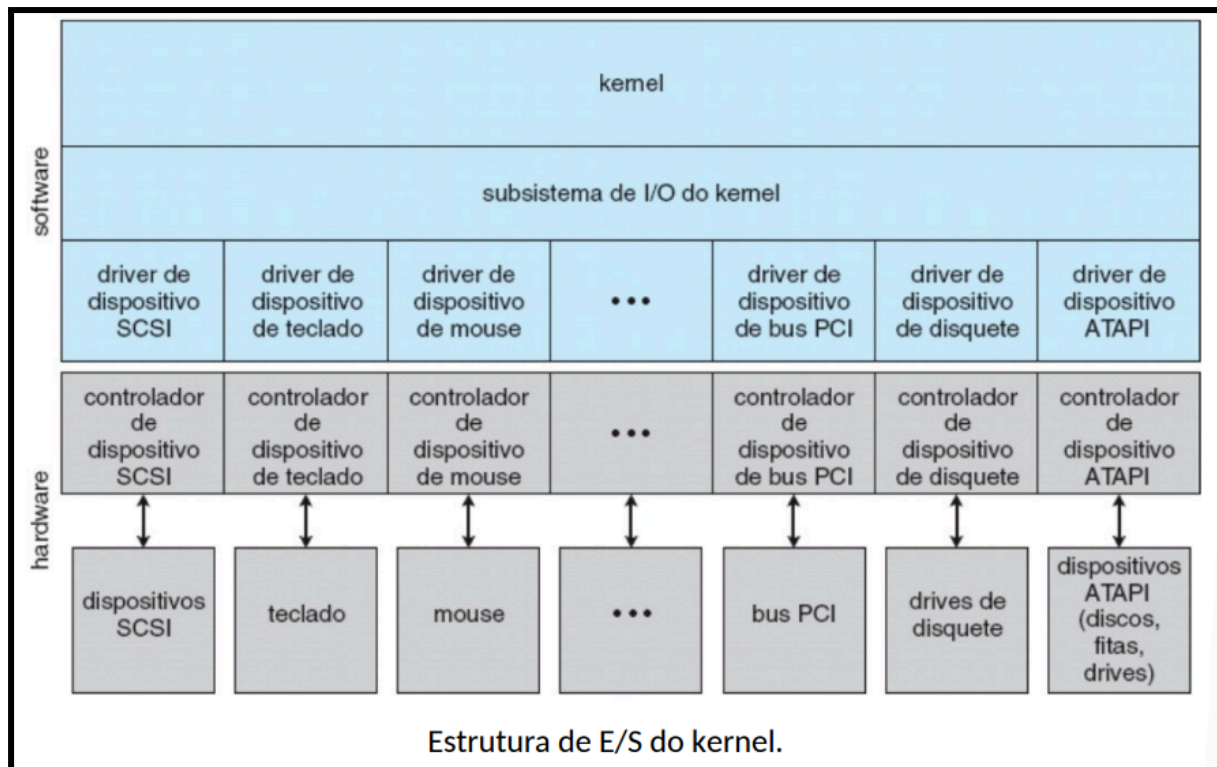
- Faz uso de um módulo de hardware denominado **DMA (Direct Memory Access)**.
- **Transfere diretamente blocos de dados entre a memória e o módulo de E/S.**
- A CPU atua somente no início e no final da operação.
- **Vantagens**
 - **CPU praticamente não participa da transferência.**
 - **Alta velocidade.**
 - **Ideal para grandes volumes de dados.**
- **Desvantagens**
 - **Hardware mais complexo.**
 - **Necessita controlador DMA.**

Gerência de E/S

- **Objetivos**

- **Eficiência**
- **Generalidade**
- **Abstrair detalhes do serviço**
- **Oferecer operações de alto nível:** read, write, open e close.
- **Envolve aspectos de hardware e software.**

Característica	Variação	Exemplo
Transferência de dados	caractere bloco	terminal disco
Método de acesso	sequencial aleatório	modem CD-ROM
Modo de transferência	síncrono assíncrono	fita Teclado
Compartilhamento	dedicado compartilhado	fita teclado
Velocidade do dispositivo	latência tempo de busca taxa de transferência atrasos entre operações	
Direção de E/S	somente leitura somente escrita leitura e escrita	CD-ROM controlador gráfico disco rígido
Características dos dispositivos E/S. (Fonte: Silberschatz)		



- **Visão geral do software E/S:**

- **Tratador de interrupção**

- Aciona o driver ao final da operação de transferência.

- **Driver de dispositivo**

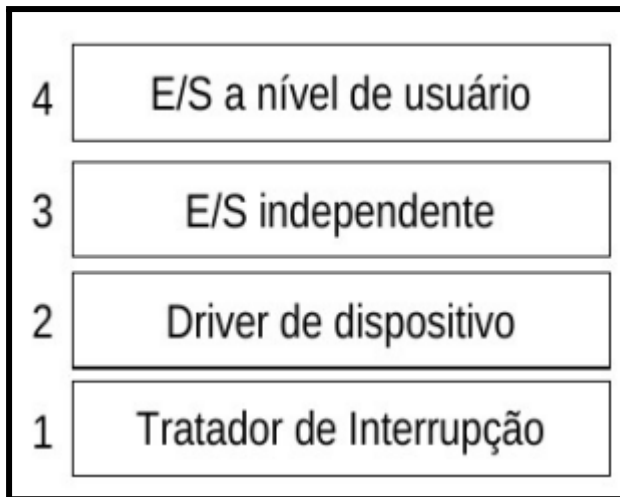
- Configuração controladora e estado.
 - Recebimento de requisições.

- **E/S independente do dispositivo**

- Escalonamento, nomeação.
 - Bufferização, proteção.

- **E/S nível de usuário**

- Chamadas de E/S.
 - Formatação de E/S.



Driver de Dispositivo

- **Conjunto de estruturas de dados e funções que controlam um ou mais dispositivos.**
- Possui uma interface bem definida com o núcleo.
- Fornecido pelo fabricante do periférico.
- **O sistema operacional utiliza uma interface padrão para que todos os dispositivos possam ser acessados da mesma forma, independentemente do fabricante ou da tecnologia utilizada.**
- Se cada driver tivesse sua própria interface:
 - O kernel teria que conhecer as particularidades de cada dispositivo.
 - A manutenção do SO seria muito mais complexa.
 - Seria difícil adicionar novos dispositivos.
- Vantagens da interface padrão:
 - **Modularidade.**
 - **Facilidade de manutenção.**
 - **Compatibilidade entre dispositivos.**
 - Isolar código específico de dispositivo.
 - **Facilitar adição de novos drivers.**
 - Não é necessário alterar o núcleo.
 - Núcleo possui uma visão uniforme de todos os dispositivos via interface bem definida.
- Operação
 - **Núcleo aciona driver para:**
 - Configurar os dispositivos.
 - Realizar acessos de leitura e escrita.

- Controlar e obter informações de estado.
- Tratamento de interrupções.

- **Driver aciona núcleo para:**

- Efetuar gerenciamento de buffers.
- Escalonamento de requisições.

E/S Independente do Dispositivo

- **Controla a parte independente do dispositivo e interage com o driver para manipular a parte dependente.**

- Responsável por:

- Distribuição uniforme dos nomes.
- Proteção.
- Fornecer uma interface aos processos usuários.
- Gerenciamento e desempenho do sistema de E/S.

- Subsistema de E/S – **Linux**

- **Nomes dos dispositivos no sistema de arquivos.**

- **“c” orientados a caractere e “b” a bloco.**

- Major number e minor number.

- **Major number:** Identifica o driver do dispositivo (por exemplo, qual driver de disco ou placa de rede está sendo usado).
- **Minor number:** Identifica o dispositivo específico ou a partição controlada por esse driver.

```
crw-rw-rw- 1 root   root    1, 3   Feb 23 1999  null
crw----- 1 root   root   10, 1   Feb 23 1999  psaux
crw----- 1 rubini tty     4, 1   Aug 16 22:22 tty1
crw-rw-rw- 1 root   dialout 4, 64  Jun 30 11:19 ttyS0
crw-rw-rw- 1 root   dialout 4, 65  Aug 16 00:00 ttyS1
crw----- 1 root   sys      7, 1   Feb 23 1999  vcs1
crw----- 1 root   sys     7, 129 Feb 23 1999  vcsa1
crw-rw-rw- 1 root   root     1, 5   Feb 23 1999  zero
```

- Subsistema de E/S – **Funcionalidades**

- **Escalonamento de E/S**

- Determinar a melhor ordem para o atendimento das requisições E/S.

- Dividir de forma justa o acesso a periféricos.

- **Buffering**

- Área de armazenamento temporário de dados.

- **Cache**

- Acesso rápido aos dados.

- **Spooling**

- Controlar acesso a dispositivos que atendem apenas uma requisição por vez.

- **Reserva de dispositivos**

- Controlar acesso exclusivo a dispositivos (alocação, sincronização).

E/S em Nível de Usuário

- **Bibliotecas e chamadas de sistema para aplicações do usuário.**

- Interface de programação

- **Síncrona**

- **Bloqueante**: O processo é suspenso até a conclusão da operação.
- **Não bloqueante**: Retorna imediatamente com os dados disponíveis no momento.

- **Assíncrona**

- O processo continua em execução enquanto E/S é realizada (sinalização x polling).

Resumo

- **Ciclo de vida de uma requisição read de E/S** (Fonte: Silberschatz).

- **Subsistema de E/S:**

- Gerenciamento do espaço de nomes para arquivos e dispositivos.
- Controle de acesso, operação.
- Alocação de espaço no sistema de arquivo.
- Alocação de dispositivo.
- Buffering, caching e spooling.
- Escalonamento E/S.
- Monitoramento de estado do dispositivo, tratamento de erro e recuperação de falha.

- Inicialização e configuração dos drivers de dispositivos.

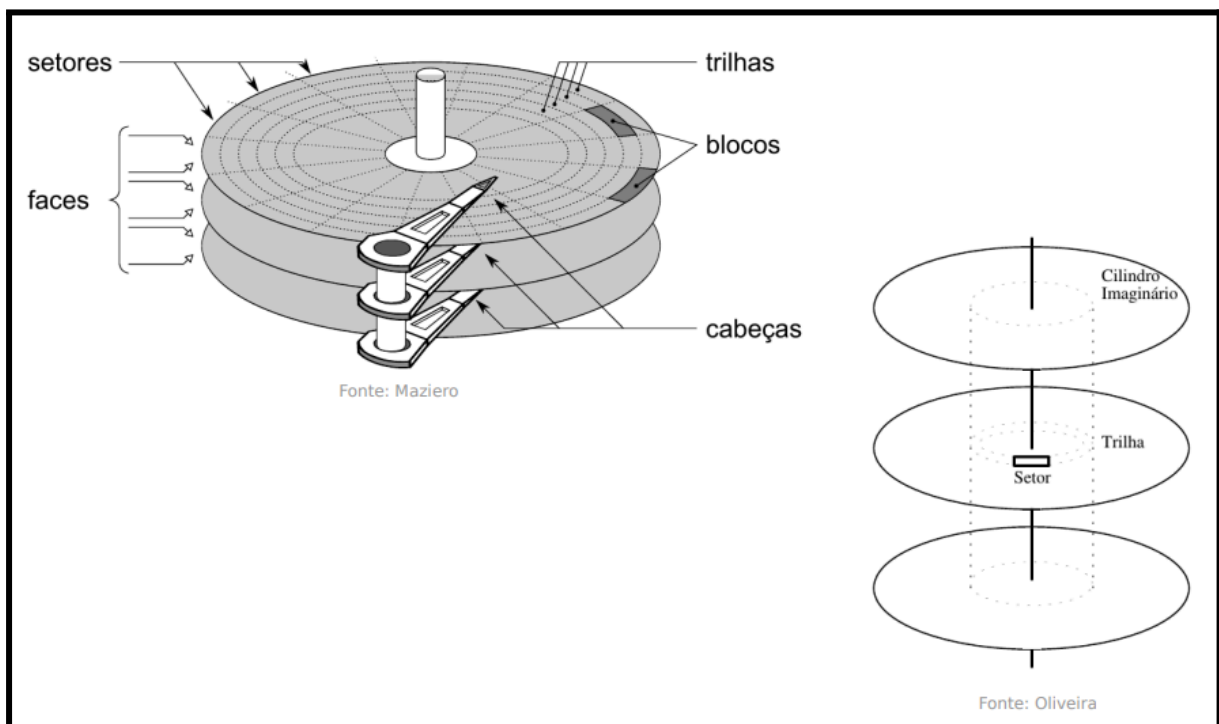
Discos Rígidos

Introdução

- Discos Rígidos (Hard Disks – HD) ou Discos Magnéticos **são dispositivos de E/S para armazenamento persistente de dados.**

Estrutura do Disco

- Composto por discos metálicos sobrepostos que giram em rotações aproximadas entre 5000 a 15000 rpms.
- Possuem uma **cabeça de leitura para cada face** e as **faces são divididas em trilhas e setores**. O conjunto de trilhas sobreposto em diferentes discos formam os cilindros.
- Setor(es) em uma trilha formam um **bloco lógico** (unidade básica de leitura e escrita).



- Unidades de disco são endereçadas como agregados unidimensionais de blocos lógicos.
- **Cada bloco lógico é a menor unidade de transferência.**
- O agregado unidimensional de blocos lógicos é mapeado nos setores do disco sequencialmente.

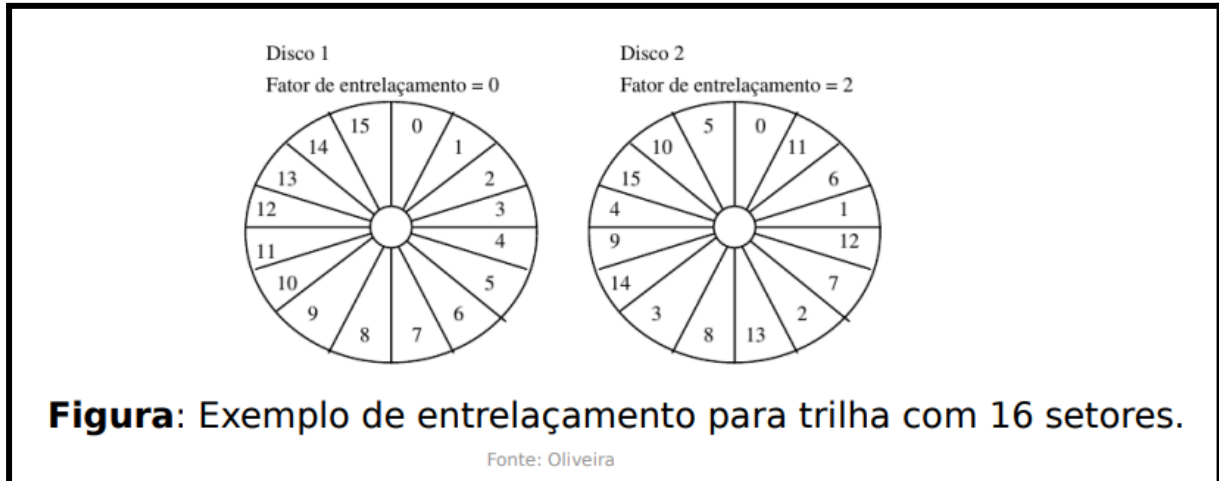
- **Setor 0 é o primeiro setor da primeira trilha no cilindro mais externo.**
- O mapeamento prossegue na ordem por essa trilha, depois o restante das trilhas nesse cilindro, e depois pelo restante dos cilindros de fora para dentro.

Tempo de Acesso

- **Tempo de acesso** é dado por:
 - $t_{\text{acesso}} = t_{\text{busca}} + t_{\text{latência}} + t_{\text{transferência}}$
- **t_{busca} (seek time):** posicionar o cabeçote no cilindro para o acesso.
 - Mais custoso.
- **$t_{\text{latência}}$ (latency time):** posicionar o cabeçote no setor para o acesso.
- **$t_{\text{transferência}}$ (transfer time):** tempo de transferência dos dados.

Entrelaçamento

- **Entrelaçamento (interleaving):** técnica para numerar os setores de forma não contígua em uma trilha. Usa-se um espaçamento fixo entre eles.

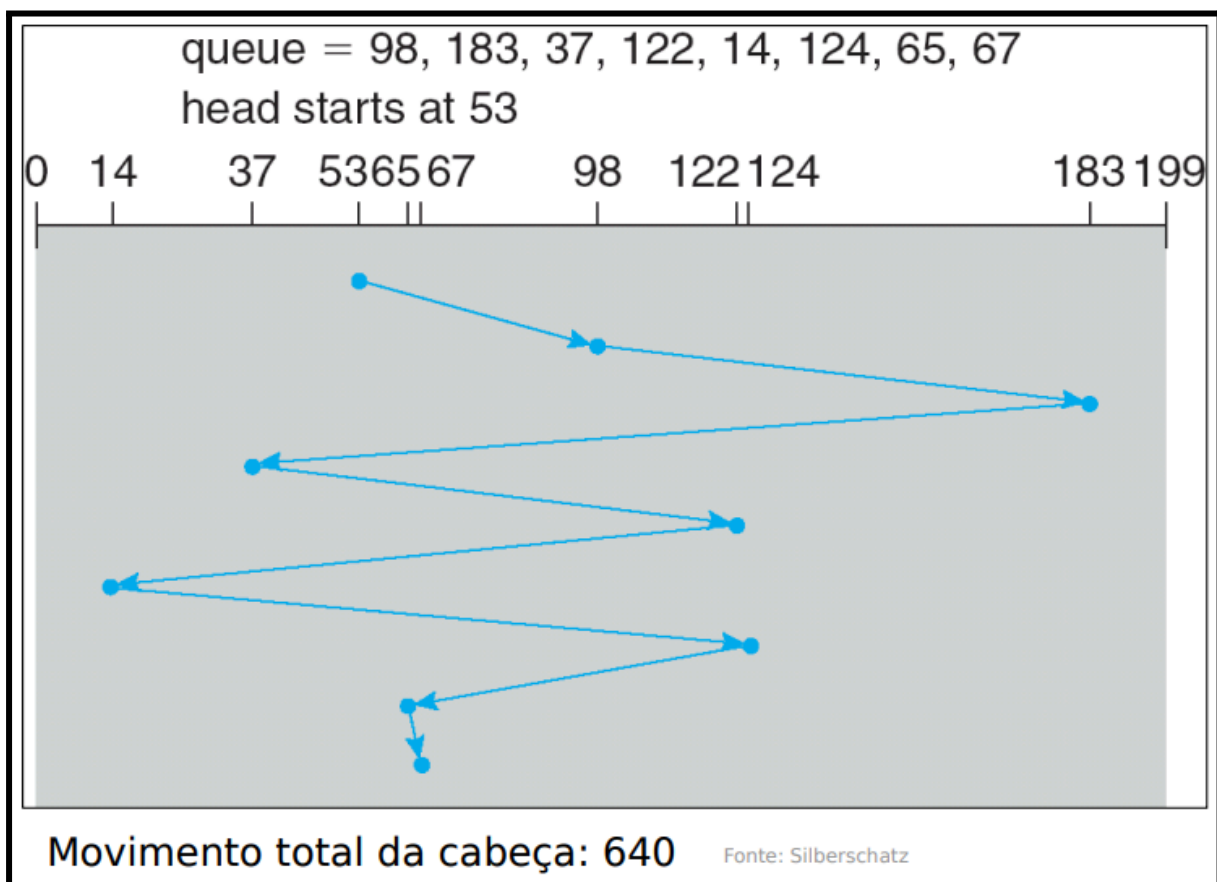


Escalonamento de Disco

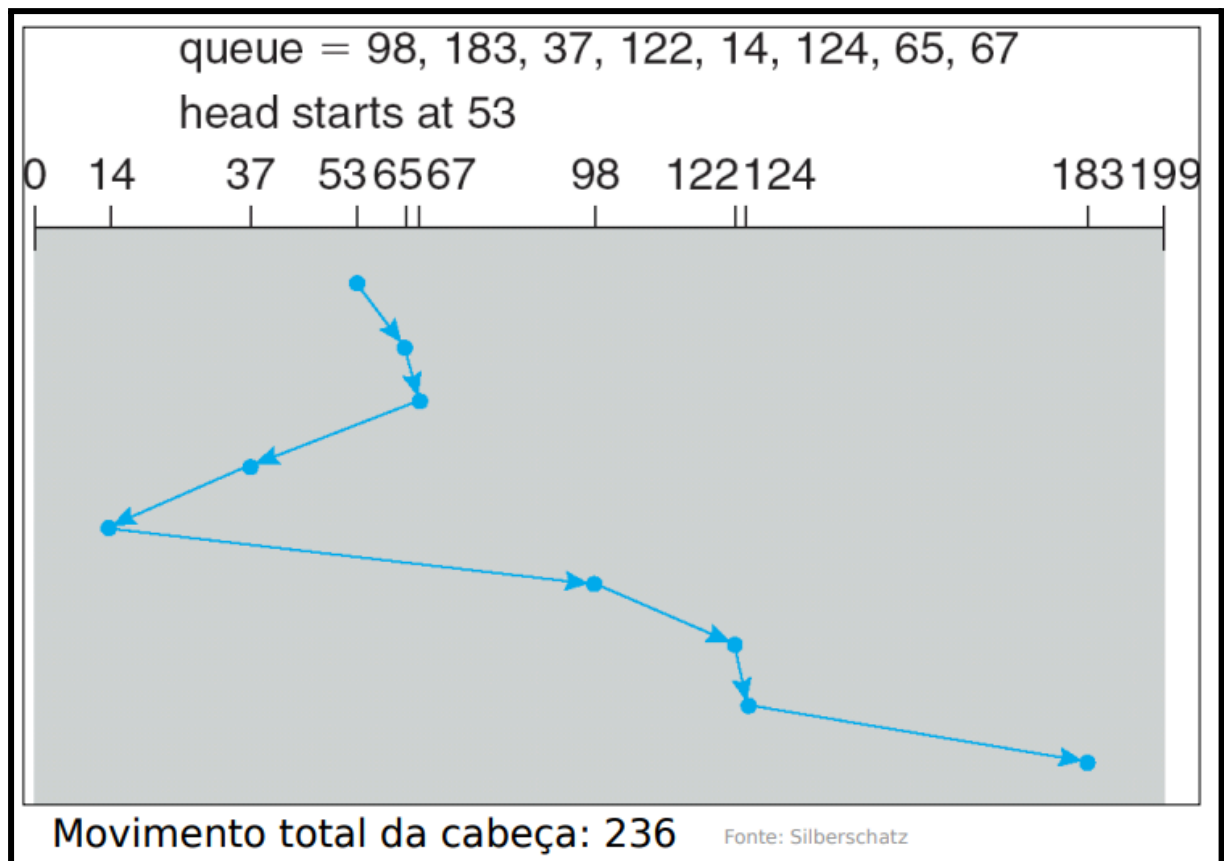
- Para otimizar o tempo de acesso:
 - **Minimizar os movimentos do cabeçote de leitura e escrita.**
 - **Maximizar o número de bytes transferidos (throughput).**

- O **escalonamento de disco** consiste em **algoritmos para selecionar e organizar um conjunto de requisições de acesso** para otimizar o tempo de acesso.
- Existem vários algoritmos para escalonar o atendimento das solicitações de E/S de disco.
- Ilustramos com uma fila de solicitação (0-199).
 - 98, 183, 37, 122, 14, 124, 65, 67
 - Posição inicial do ponteiro na cabeça: 53

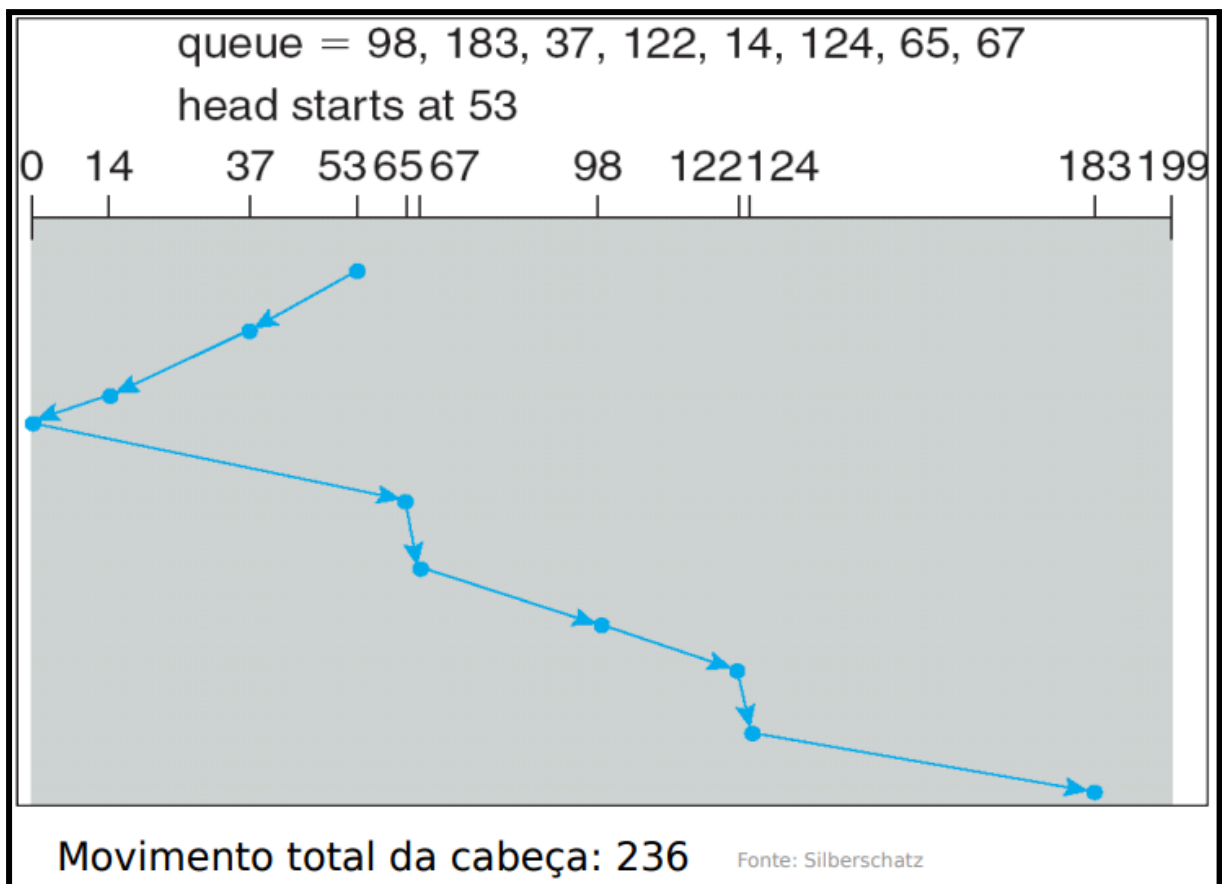
FCFS (First-Come First-Served)



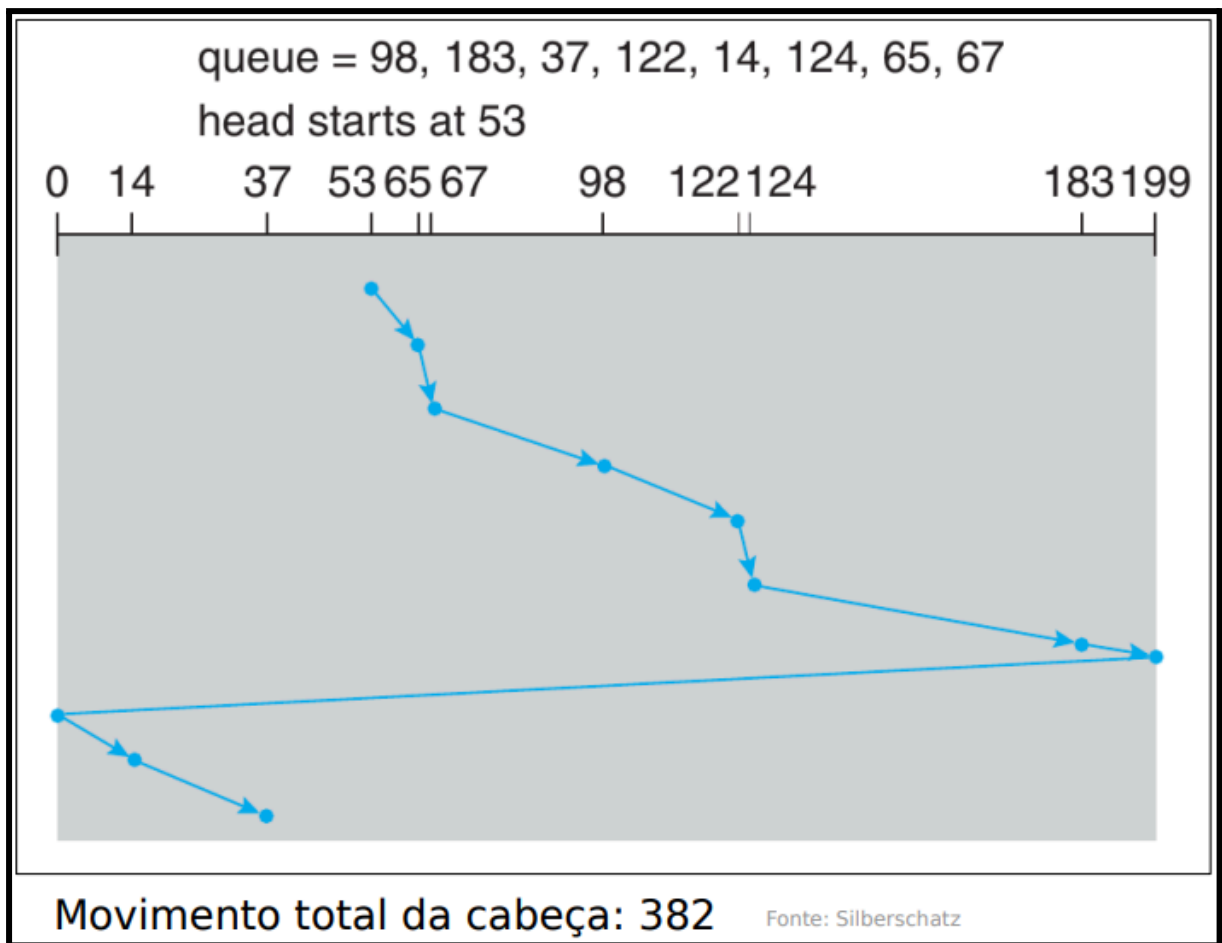
SSTF (Shortest Seek Time First)



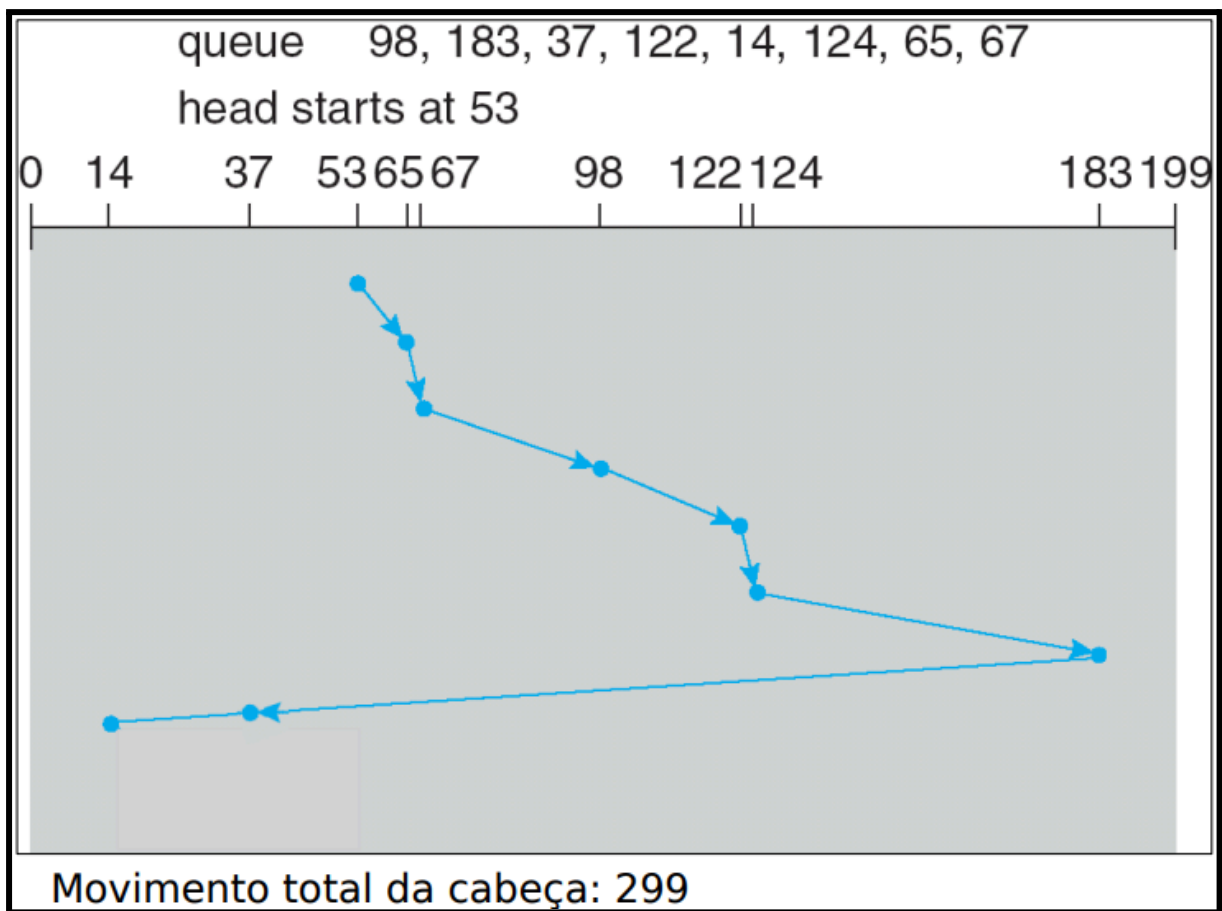
SCAN (Alg. do Elevador)



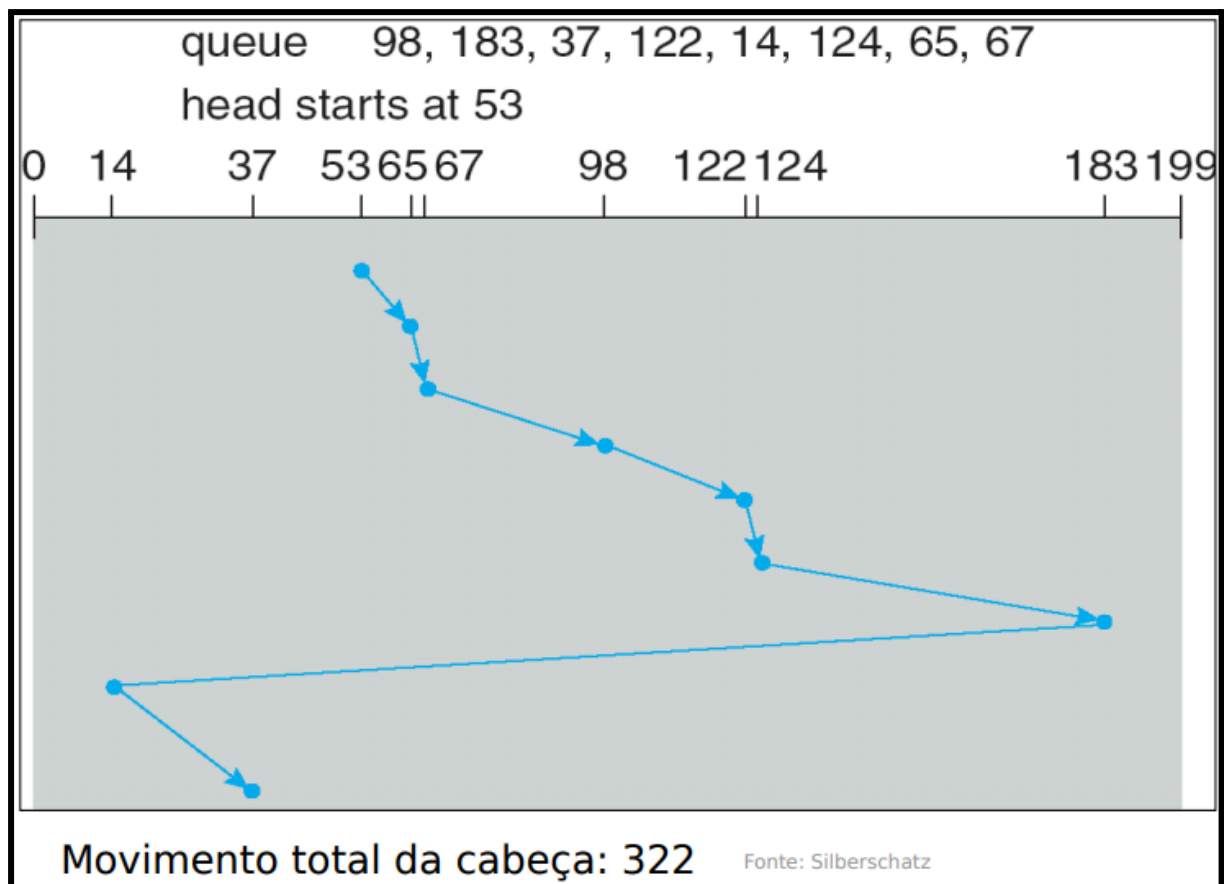
C-SCAN (Alg. do Elevador)



LOOK (Alg. do Elevador)



C-LOOK (Alg. do Elevador)



Resumo

- SSTF é comum e tem um apelo natural
- SCAN e C-SCAN funcionam melhor para sistemas que têm cargas pesadas sobre o disco
- O desempenho depende do número e tipo de solicitações.
- Requisições para serviço de disco podem ser influenciadas pelo método de alocação de arquivo.
- O algoritmo de escalonamento de disco deve ser escrito como um módulo separado do sistema operacional, permitindo que seja substituído por um algoritmo diferente, se necessário.
- SSTF ou C-LOOK é uma escolha razoável para o algoritmo padrão.

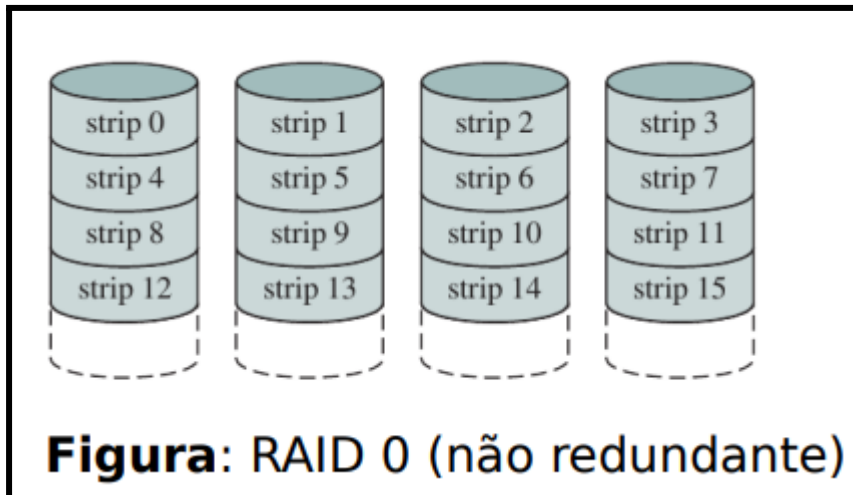
RAID

- RAID – Redundant Array of Inexpensive Disks
 - Conjunto de discos redundantes vistos pelo SO como um único disco lógico.
 - Desempenho, disponibilidade e tolerância a falhas.

- Podem ser implementados em hardware ou software.
- Diferentes níveis de RAID.

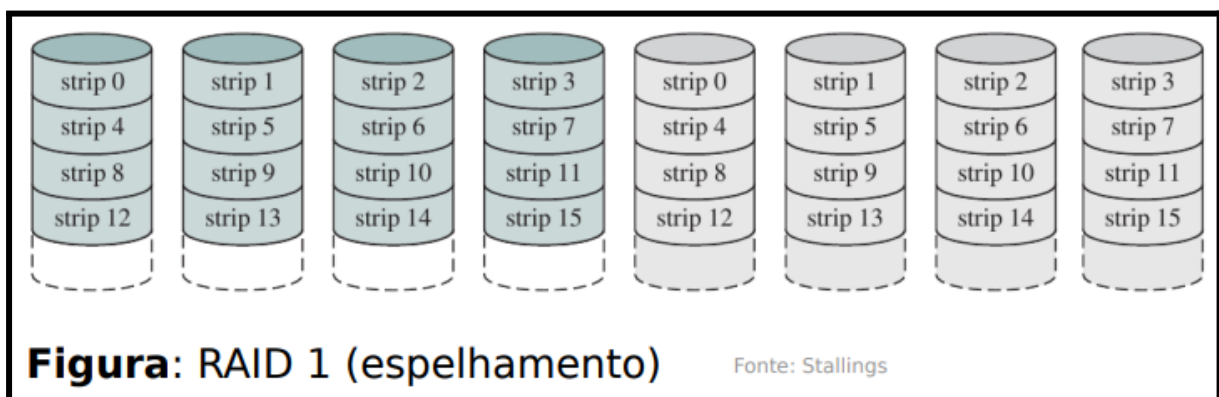
RAID 0

- Dados divididos em diferentes discos.
- Cada parte é denominada de strip.
- Técnica denominada de **stripping**.
- Vantagem: **paralelismo, desempenho.**



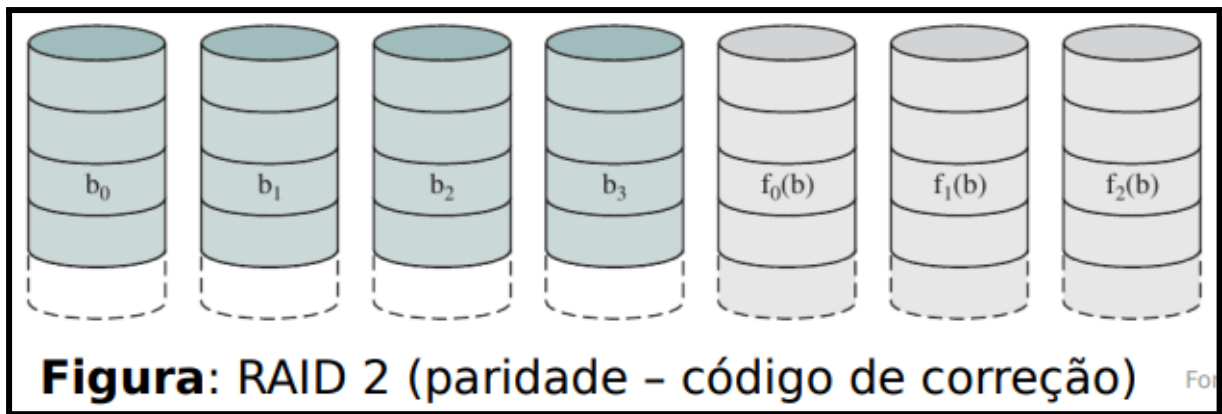
RAID 1

- Dados espelhados em diferentes discos.
- Técnica denominada de **mirroring**.
- Disponibilidade e tolerância a falhas.
- Desvantagem: **custo.**



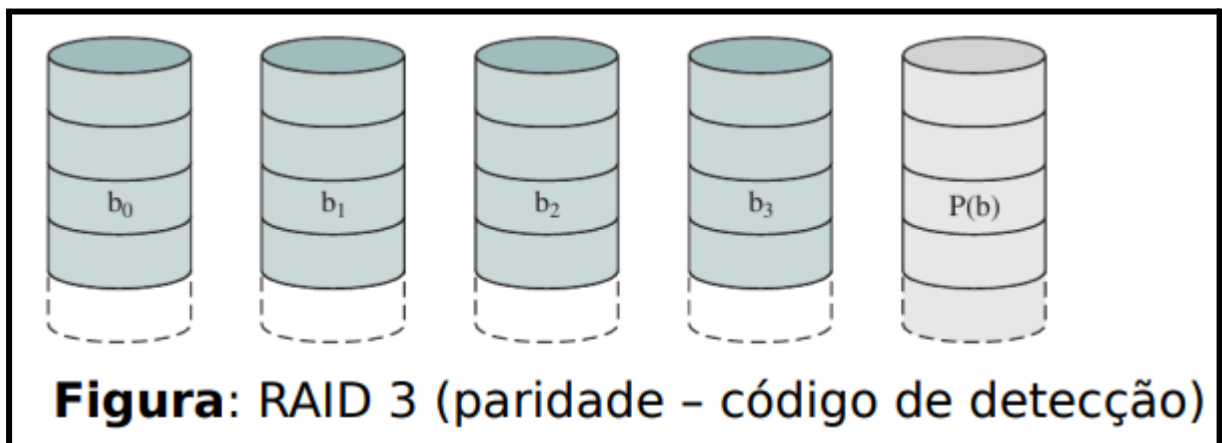
RAID 2

- Strips são pequenos (byte ou palavra).
- Tipicamente é usado Hamming Code para calcular a paridade.
- Discos sincronizados para E/S.



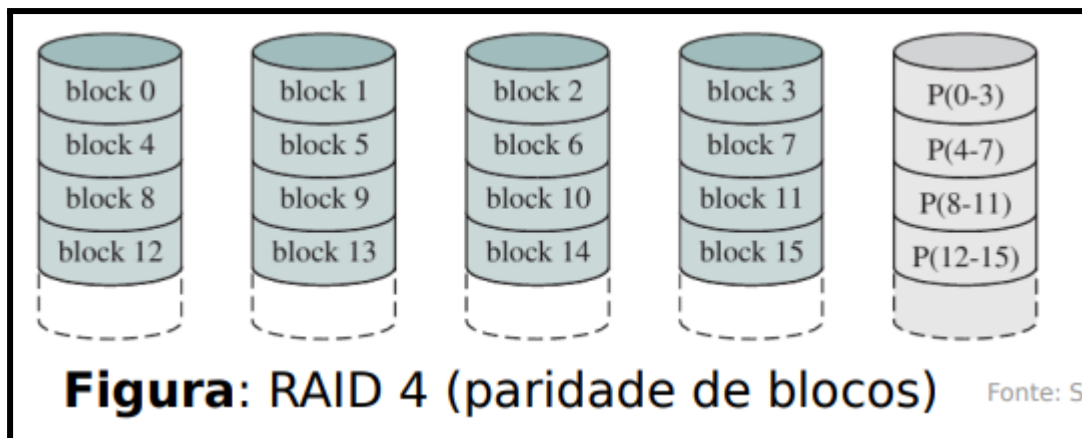
RAID 3

- Similar ao RAID 2.
- Calcula a paridade de um conjunto de bits de mesma posição em diferentes discos.
- **Apenas um disco de redundância.**
- Possível reconstruir dados de um disco.



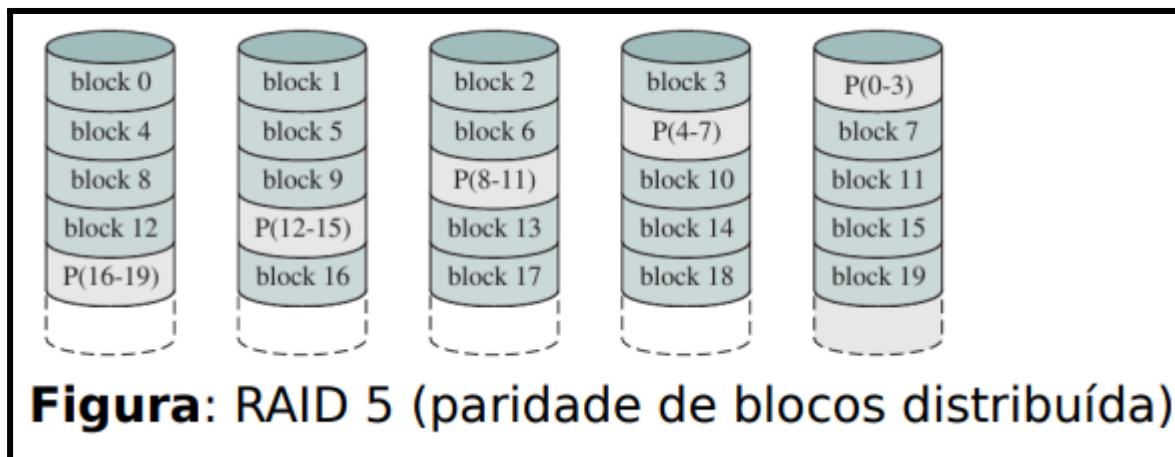
RAID 4

- **Acesso independente por discos.**
- Strips são maiores.
- Paridade dos blocos.



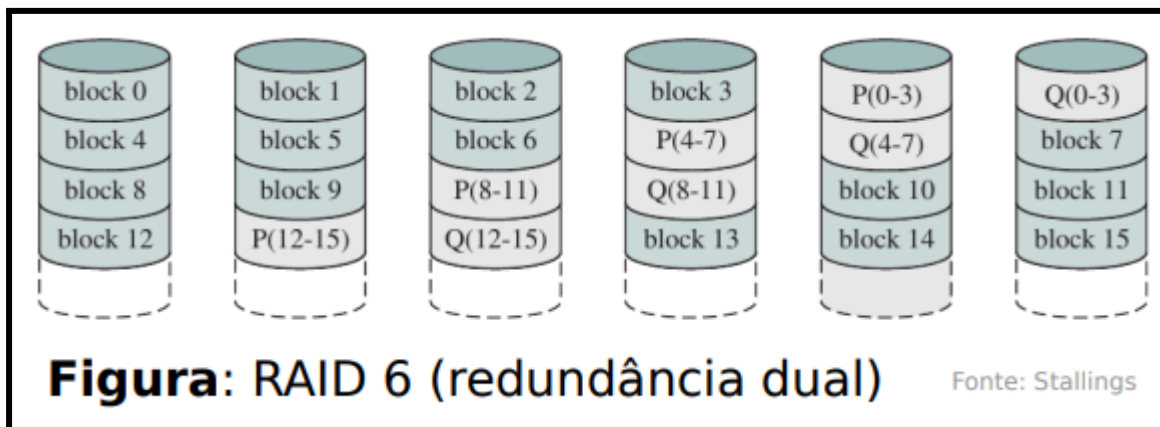
RAID 5

- Similar ao RAID 4.
- Paridade distribuída entre diferentes discos.



RAID 6

- Faz uso de dois esquemas de cálculo de paridade.
- Desvantagem é o custo para atualizar as paridades na escrita.



Sistemas de Arquivo

Fundamentos

Introdução

- **Arquivos**

- Abstração do Sistema Operacional para **armazenar e acessar dados em um meio não volátil (persistente)**.
- **Unidades lógicas de informação criadas por processos.**
- Proveem uma **forma simples, coerente e fácil de usar para acessar informações** em uma mídia de armazenamento persistente.



- **Sistemas de Arquivos**

- **Parte do Sistema Operacional que realiza a gestão de arquivos.**
- Prover mecanismos para o usuário **criar, ler, escrever, remover, localizar, organizar e proteger arquivos.**
- Otimização e concorrência de acesso.
- **Suporte a múltiplos usuários**
 - **compartilhamento e proteção**
- **Garantir a persistência e a recuperação de arquivos.**
- **Gerenciamento de espaço (livre e ocupado).**

Conceitos

- **Arquivos**

- Sequência de bytes em dispositivos físicos.

- **Diretórios**

- **Organização e referência para arquivos**, geralmente na forma hierárquica.

- **Sistemas de arquivos**

- Organização do conteúdo dos arquivos e diretórios no dispositivo físico.

- **Partições**

- **Divisão de um disco físico em discos lógicos.** Cada disco lógico possui um sistema de arquivo.

Partições

sda1	150,21 GiB				
sda3	235,16 GiB				
Partition	Type	Mount Point	Label	Size	
/dev/sda (ATA ST500LT015-1D14, 465,76 GiB)					
/dev/sda1	ext4			150,21 GiB	
/dev/sda2	ext4	/		64,00 GiB	
/dev/sda3	ext4	/home		235,16 GiB	

Disk 0					
Basic		(C:)	(F:)	(G:)	(H:)
250.00 GB	350 MB	37.97 GB	16.52 GB	103.06 GB	92.10 GB
Online	Heal	Healthy (Bc)	Healthy (F)	Healthy (Lo)	Healthy (Lo)

Diretórios e Arquivos

/	>	home	>	rodrigo
Name				
Apps				
eclipse				
>				
> ide-latest-released				
freeplane				
jabref				
JabRef-4.3.1.jar				
JabRef-icon-64.png				
PacketTracer				
pdfxchange				
tidllydesktop				
yed				
Zotero				

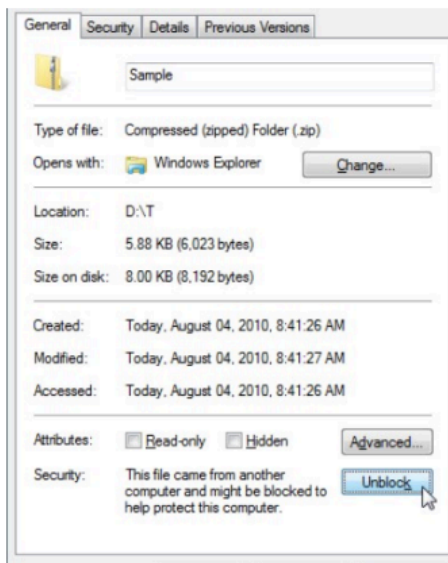
Folders	Name
Local Disk (C:)	DBEVENT
DB2	SQLLOGDIR
NODE0000	db2hist.asc
DB2	db2hist.bak
MYINST	SQLBP.1
SQL00001	SQLBP.2
DBEVENT	SQLDBCON
SQLLOGDIR	SQLDBCONF
Documents and Settings	SQLINSLK
Inetpub	SQLGCTL.LFH.1
MyFiles	SQLGCTL.LFH.2
MYINST	SQLGMR.LFH
NODE0000	SQLSGF.1
MYINST	SQLSGF.2
SQL00001	SQLSPCS.1
SQLDBDIR	SQLSPCS.2
	SQLTMPLK

DROP.BAT	E.BAT
GET.BAT	GO.BAT
INVENTOR.BAT	INVENT~1.BAT
LOOK.BAT	N.BAT
S.BAT	SAVE.BAT
TALKHELP.BAT	W.BAT

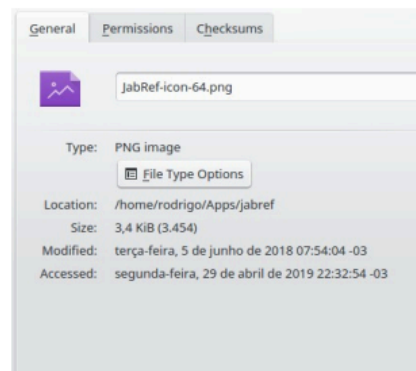
Arquivos

- **Atributos**

- **Nome**
- **Tipo**
- **Tamanho**
- **Datas:** criação, modificação, último acesso
- **Proprietário** (owner)
- **Permissões de acesso** (leitura, escrita, ...)
- **Localização**
- Outros: oculto, sistema, binário ou textual, grupo, somente leitura, ou ainda **atributos estendidos**.



Atributos de arquivos - Windows



```
# lsattr JabRef-4.3.1.jar
-----e-- JabRef-4.3.1.jar

# ls -l
```

Atributos de arquivos - Linux

• Nomes

- **O sistema de arquivos define um espaço de nomes.**
 - Conjunto de regras e convenções para identificar simbolicamente um arquivo.
- Variam de sistema para sistema.
 - **Distinção entre letras maiúsculas e minúsculas.**
 - **Obrigatoriedade ou não de uma extensão.**
 - **Tamanho máximo de nome e da extensão.**

• Formatos

- **Sequência de bytes**
 - **Núcleo reconhece poucos formatos.**
 - Estrutura conhecida pelas aplicações.
- **Arquivos de registros**
 - Sequência linear de registros de tamanho fixo ou variável, e indexados.

○ **Arquivos de texto**

- **Texto puro (plain text).**
- **Unix (\n – new line), Windows (\r\n – carriage return e new line).**

- **Arquivos binários**

- Formato próprio.
- Exemplos: executáveis, processadores de texto, imagens, etc.

- **Identificação de conteúdo**

- Uso de extensão (ex.: .jpg)
- Números mágicos (UNIX)
- Atributos adicionais
- **Tipos MIME (tipo/subtipo)**
 - application/java-archive
 - image/png
 - text/html
 - text/plain

- **Arquivos especiais**

- **Abstração de dispositivos de E/S**
 - /dev
- **Abstração de interfaces do núcleo**
 - /proc/cpuinfo
- **Canais de comunicação**
 - **sockets e pipes**

- **Operações**

- Criar
- Abrir
- Ler
- Escrever
- Fechar
- Remover
- Gerenciar atributos

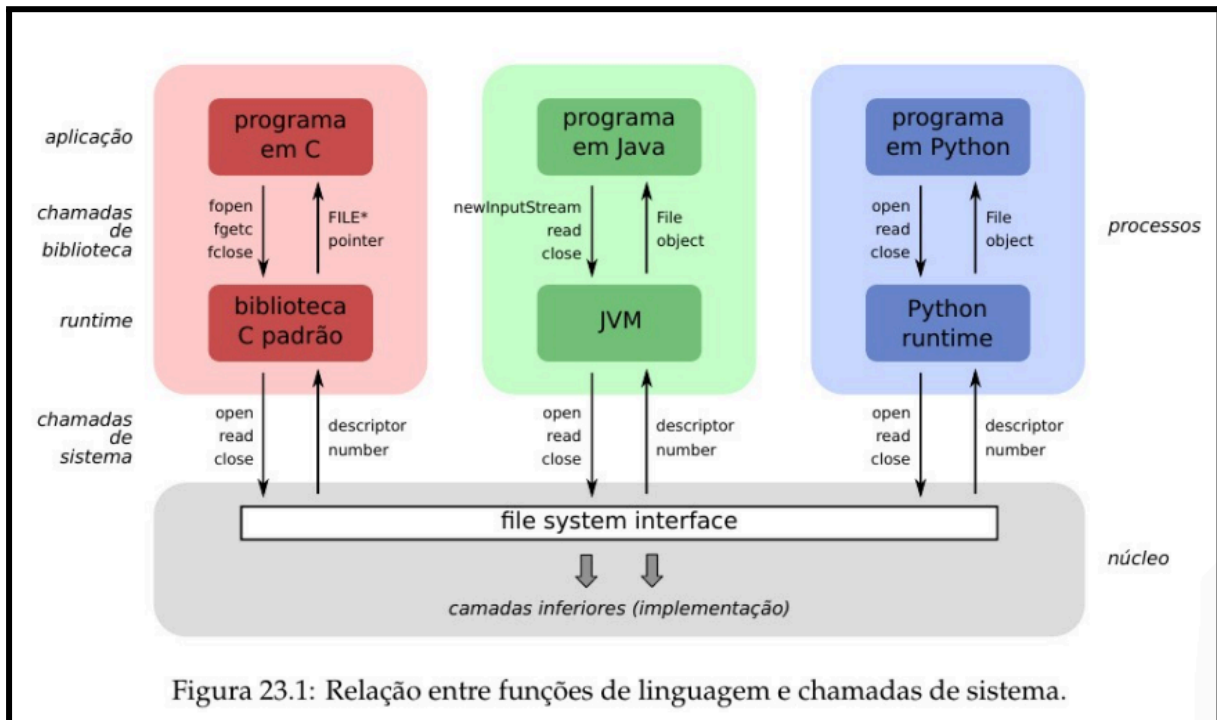
- Implementadas por bibliotecas e chamadas de sistema.

- **Interface de acesso**

- Interface de **alto nível (bibliotecas)**
- Interface de **baixo nível (chamadas de sistema)**

- **Descritores de arquivos**

- **Representação lógica de um arquivo em uso por um processo.**
- **Alto nível** (ex: FILE biblioteca C)
- **Baixo nível**
 - POSIX/UNIX: inteiro positivo.
 - WINDOWS: file handles.



- **Exemplo: Abrir Arquivo**
 - **Processo:** solicita abertura, realiza a chamada de sistema.
 - **Núcleo:**
 1. recebe chamada
 2. localiza o arquivo,
 3. verifica permissões,
 4. cria estrutura de dados na memória do núcleo,
 5. mantém uma referência de arquivos abertos,
 6. devolve à referência para o processo.
 - **Processo:** recebe descritor, cria descritor de alto nível, devolve para o código da aplicação.

Formas de Acesso

● **Acesso sequencial**

- **Acessa o arquivo de forma sequencial por meio de um ponteiro de acesso.**

- Indicação de fim de arquivo (EOF).

- **Acesso aleatório**

- Possibilita mover o ponteiro de acesso para uma posição e acessar diretamente o valor armazenado.

- **Acesso mapeado em memória**

- Mecanismo de paginação em disco.
- Mapeia arquivo em vetor de bytes ou registros.

- **Acesso indexado**

- Estrutura interna como chave/valor.
- Dados arquivos em registros com chaves (índices).

Compartilhamento

- **Travas em arquivos (locks)**
 - Todo o arquivo ou em partes.
- **Tipos de travas (rigidez)**
 - Travas obrigatórias
 - Windows
 - Travas recomendadas
 - Bibliotecas e processo pode ignorar
- **Tipos de travas (compartilhamento)**
 - Travas exclusivas (escrita)
 - Travas compartilhadas (leitura)

Semântica Acesso

- **Semântica de Compartilhamento**
 - **Imutável**
 - Conteúdo só pode ser lido.
 - **UNIX**
 - Modificações visíveis imediatamente.
 - **Sessão**
 - Processos manipulam arquivos como uma sessão.
 - **Transação**
 - Sequências de leitura e escrita tratados como transações.

Controle de Acesso

- Proprietário
- Grupo
- Permissões de acesso
- Geralmente realizado durante a abertura.

```
-rw-rw-r-- 1 rodrigo rodrigo 4,4M Mai 23 15:43 apache-qpid-jms-0.42.0-bin.tar.gz  
-rw-rw-r-- 1 rodrigo rodrigo 12K Mai 28 16:33 atividade1_SO.odt.tar.gz  
-rwxrwxr-x 1 rodrigo rodrigo 8,5K Mar 21 14:23 execanything  
drwxrwxr-x 3 rodrigo rodrigo 4,0K Mai 23 15:41 qpid-broker  
-rw-r--r-- 1 root root 608 Mai 23 15:56 readme.txt
```

Figura: Listagem de proprietário, grupo e permissões de acesso para arquivos no SO GNU/Linux.

Lista de Abertos

- No Linux, o comando **lsuf** lista os arquivos abertos no sistema.

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE NAME
dropbox	401	rodrigo	mem	REG	8,2	27000	791531 /lib/x86_64-linux-gnu/libnss_dns-2.23.so
soffice.b	4919	rodrigo	34uW	REG	8,3	475495	1187390 /home/rodrigo/....so.odp
soffice.b	4919	rodrigo	30r	FIFO	0,12	0t0	6972300 pipe
firefox	20518	rodrigo	54u	IPv4	6976354	0t0	TCP 172.16.255.143:53188→g...

Diretórios

- Conceitos
 - Os **diretórios** possibilitam organizar arquivos em grupos e **hierarquias**.
 - Também são denominados de **pastas ou folders**.
 - Possuem nomes e atributos.
 - O diretório principal é denominado de **raiz**.
 - No Linux, Unix: /
 - No Windows: \ (exemplo C:\)

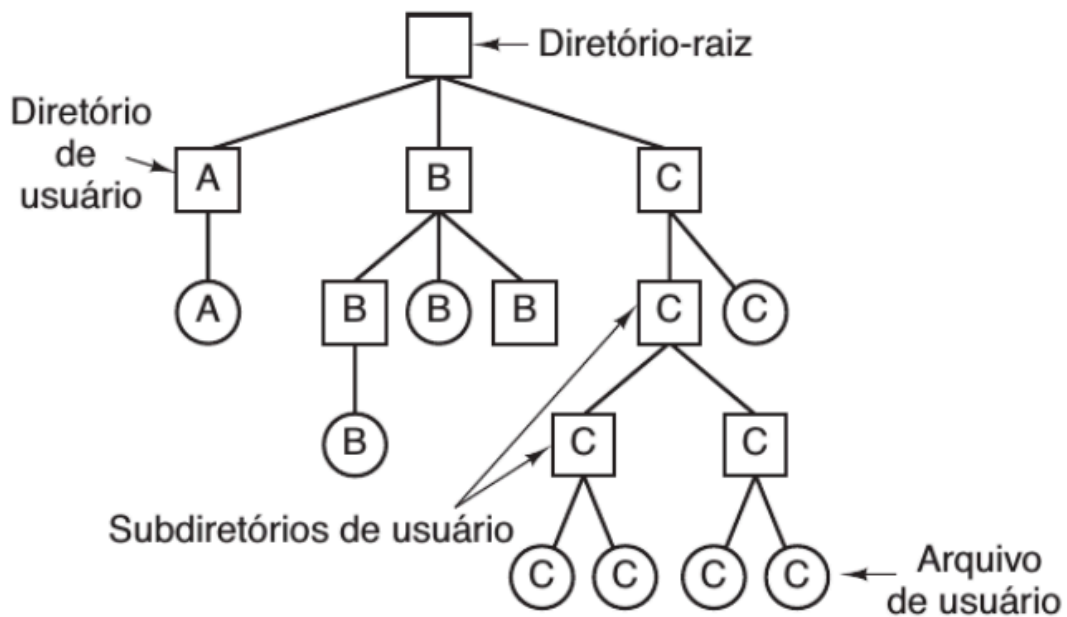


Figura: Sistema hierárquico de diretórios (Tanenbaum).

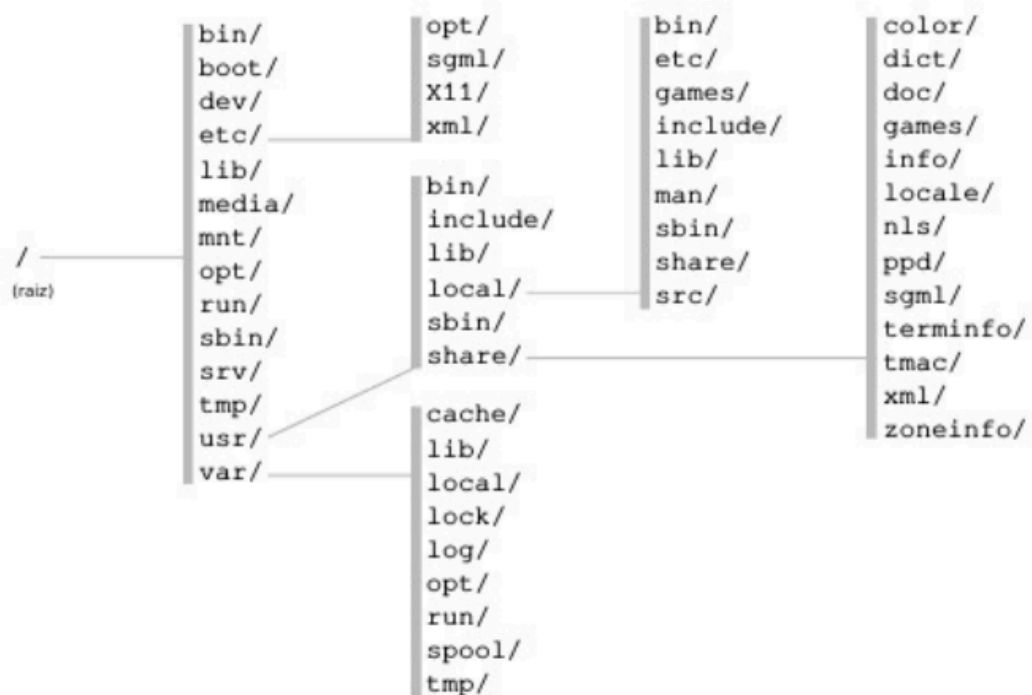
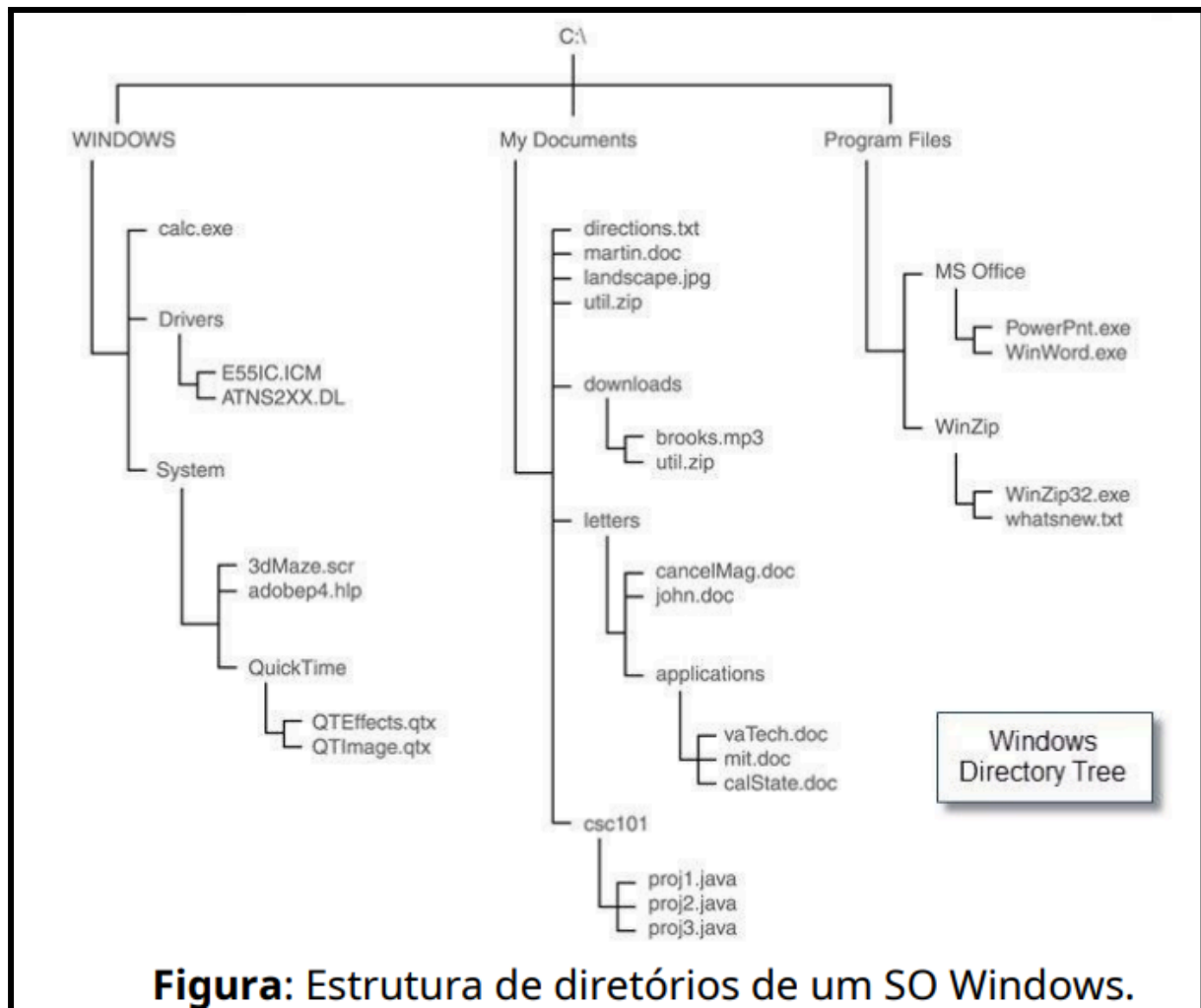


Figura: Estrutura de diretórios FHS de um SO Linux (Maziero).



Caminhos de Acesso

- **Caminho de acesso (Path)**

- **Localização de um arquivo dentro de um sistema de arquivos.**
- /home/rodrigo/disciplinas/so/aula9.pdf
- D:\Users\Rodrigo\SO\aula9.pdf
- Podem ser **absolutos (caminhos a partir da raiz)** ou **relativos (caminhos a partir da posição corrente)**.

Operações

- Exemplos de operações:
 - **create; delete:** criar e remover diretórios.
 - **opendir; closedir:** abrir e fechar diretórios.
 - **readdir:** próxima entrada de um diretório aberto.
 - **rename:** renomear diretório.

- **link**: permite arquivo aparecer em mais de um diretório.
- **unlink**: remove a entrada de um arquivo do diretório.

Atalhos

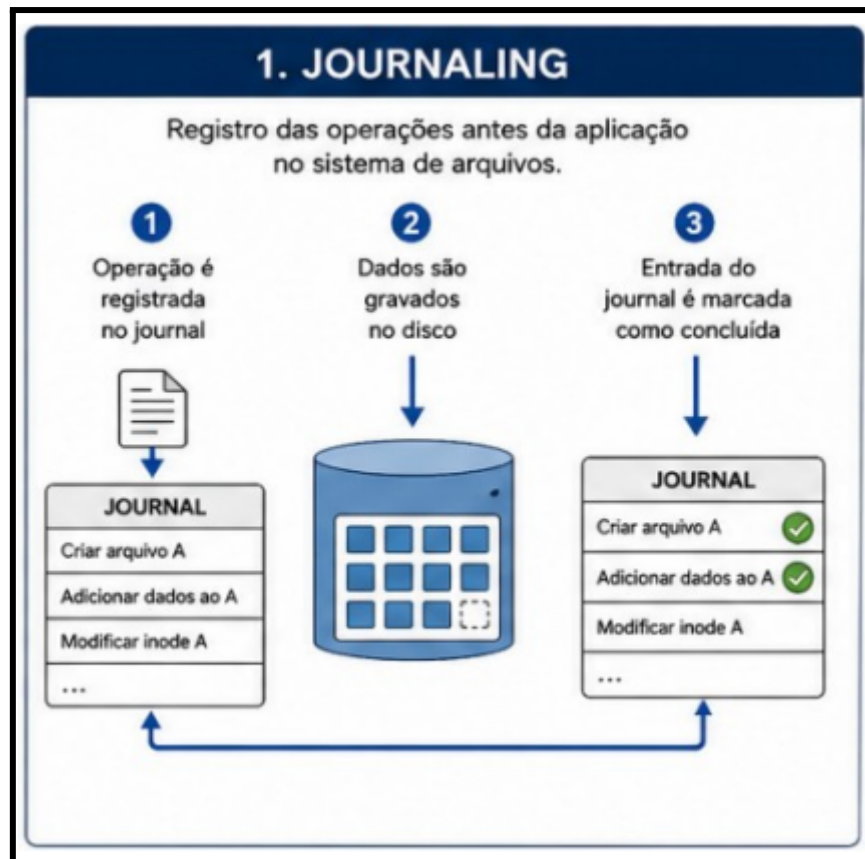
- Os **atalhos** possibilitam que um **arquivo ou diretório estejam localizados em diferentes posições no sistema de arquivos**.
- Podem ser:
 - **Simbólicos (soft link)**
 - Arquivos pequenos com a localização do arquivo original.
 - **Físicos (hard link)**
 - Referências no sistema de arquivos apontam para o local de armazenamento do arquivo.

Características de SA Modernos

- Journaling
- Copy-on-Write (CoW)
- Snapshots
- Checksums de metadados e dados
- Compressão
- Criptografia

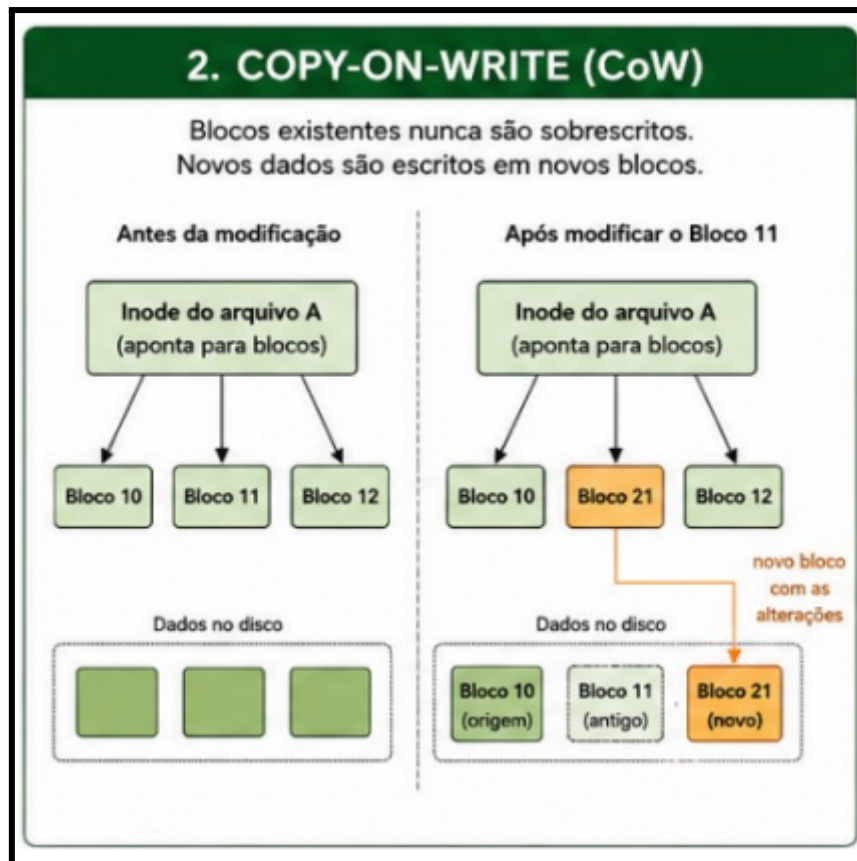
Journaling

- **Registra operações pendentes** em uma **área especial reservada no disco**, chamada **journal**, antes de aplicá-las ao sistema de arquivos.
- Garante a consistência do sistema de arquivos.
- **Pequena sobrecarga ao escrever.**
- Exemplos: **ext4, XFS, NTFS**.



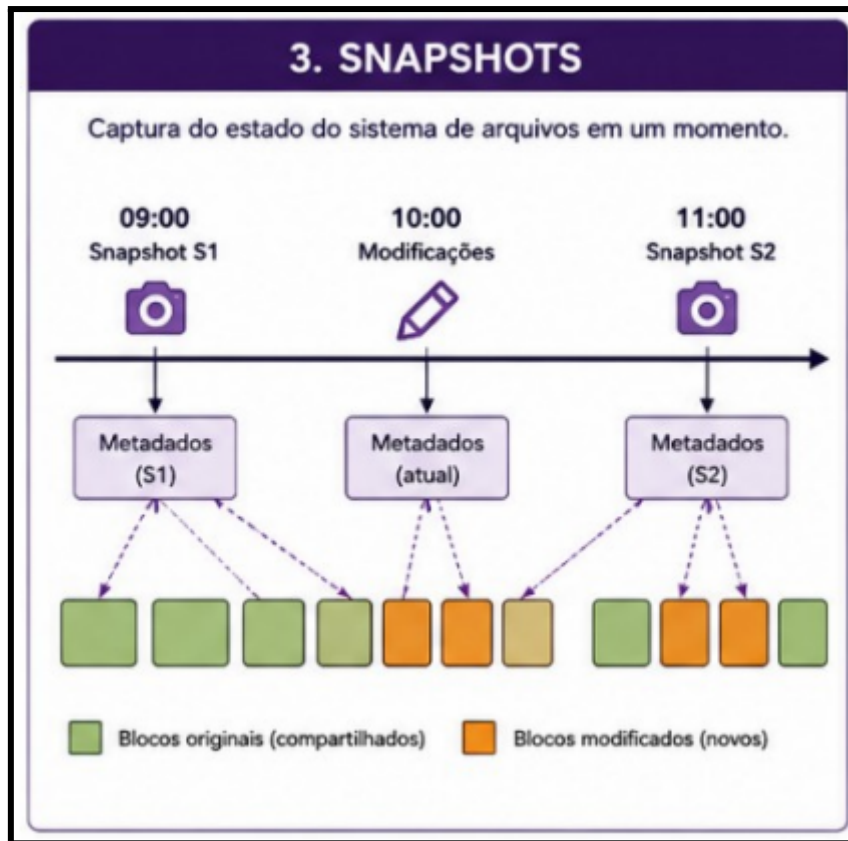
Copy-on-Write (CoW)

- **Blocos existentes nunca são alterados diretamente.**
- **Dados modificados são gravados em novos blocos.**
- **Maior integridade** (elimina corrupção por escritas parciais), viabiliza uso de **snapshots**.
- **Aumenta a sobrecarga de escrita e fragmentação.**
- Exemplos: **Btrfs, ZFS, APFS.**



Snapshots

- Registro do estado de um SA em um determinado momento.
- Compartilha blocos não modificados.
- Consome espaço somente para as diferenças após a captura.
- Snapshots x backups.
- Exemplos: ZFS, Btrfs, APFS.

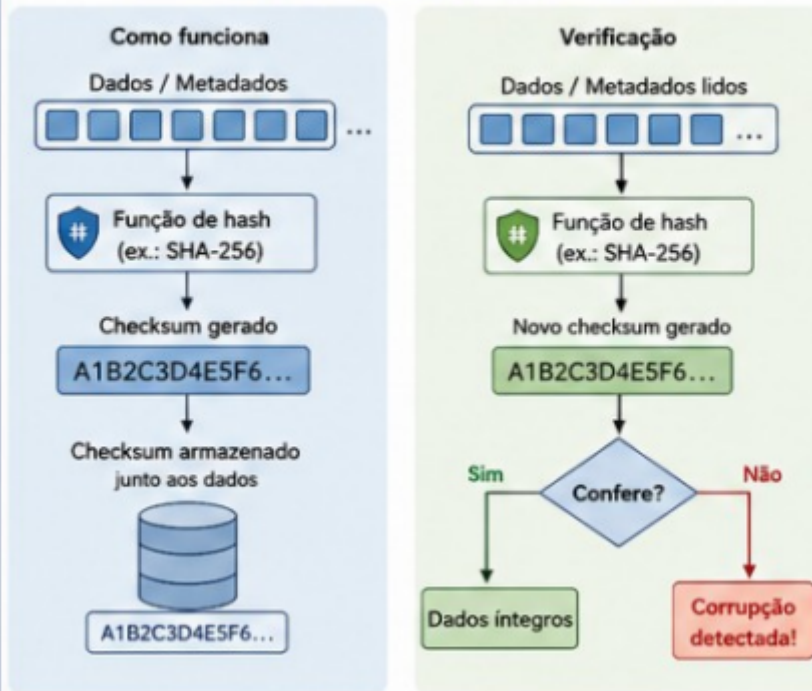


Checksums

- **Verificação de integridade dos metadados e dos dados.**
- **Usado para detectar alterações e corrupção** (p. ex. bit rot).
- **Aumenta confiabilidade**
- Exemplos: **ZFS, Btrfs.**

4. CHECKSUMS DE METADADOS E DADOS

Verificação de integridade para detectar corrupção.



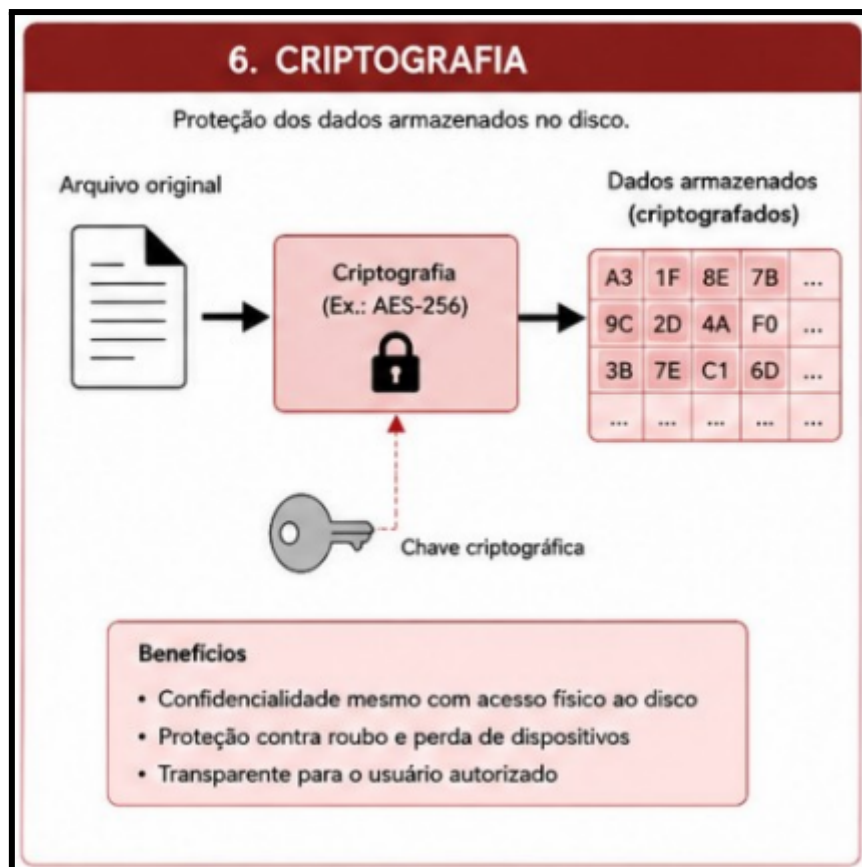
Compressão

- **Compressão automática dos dados antes de serem gravados.**
- **Economia de espaço e aumento de desempenho e vida útil de SSDs.**
- **Pequena sobrecarga para compactação.**
- Exemplos: **ZFS, Btrfs.**



Criptografia

- **Garante a confidencialidade dos dados gravados.**
- Pequena sobrecarga de processamento.
- Exemplos: **APFS, NTFS (BitLocker).**



Implementação

Introdução

- **Sistemas de Arquivos (SA)**
- **Arquitetura de Sistemas de Arquivos**
- **Implementação de arquivos e diretórios**
- **Gerenciamento de espaço**
- **Exemplos de Sistemas de Arquivos**

Arquitetura de Sistemas de Arquivos

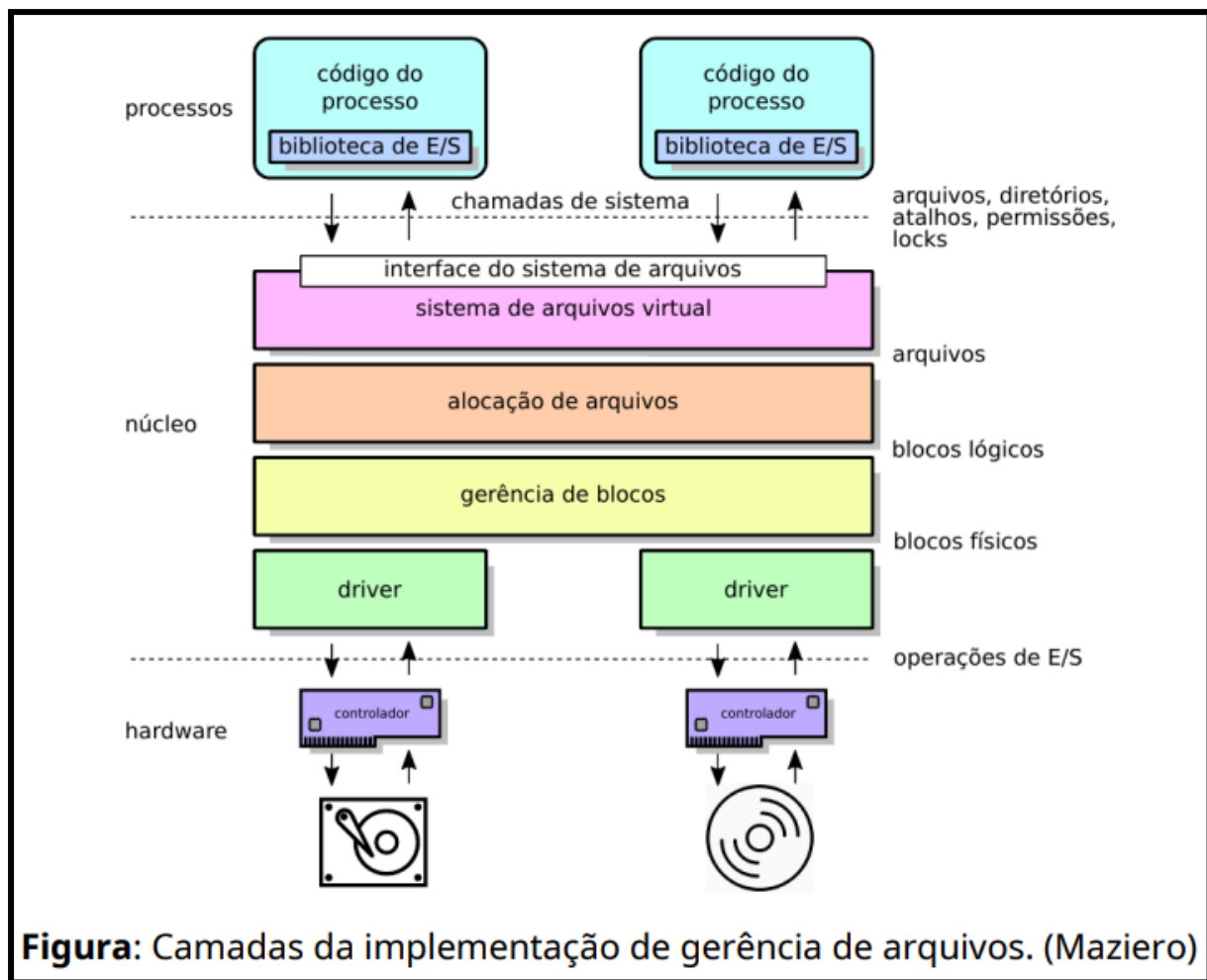
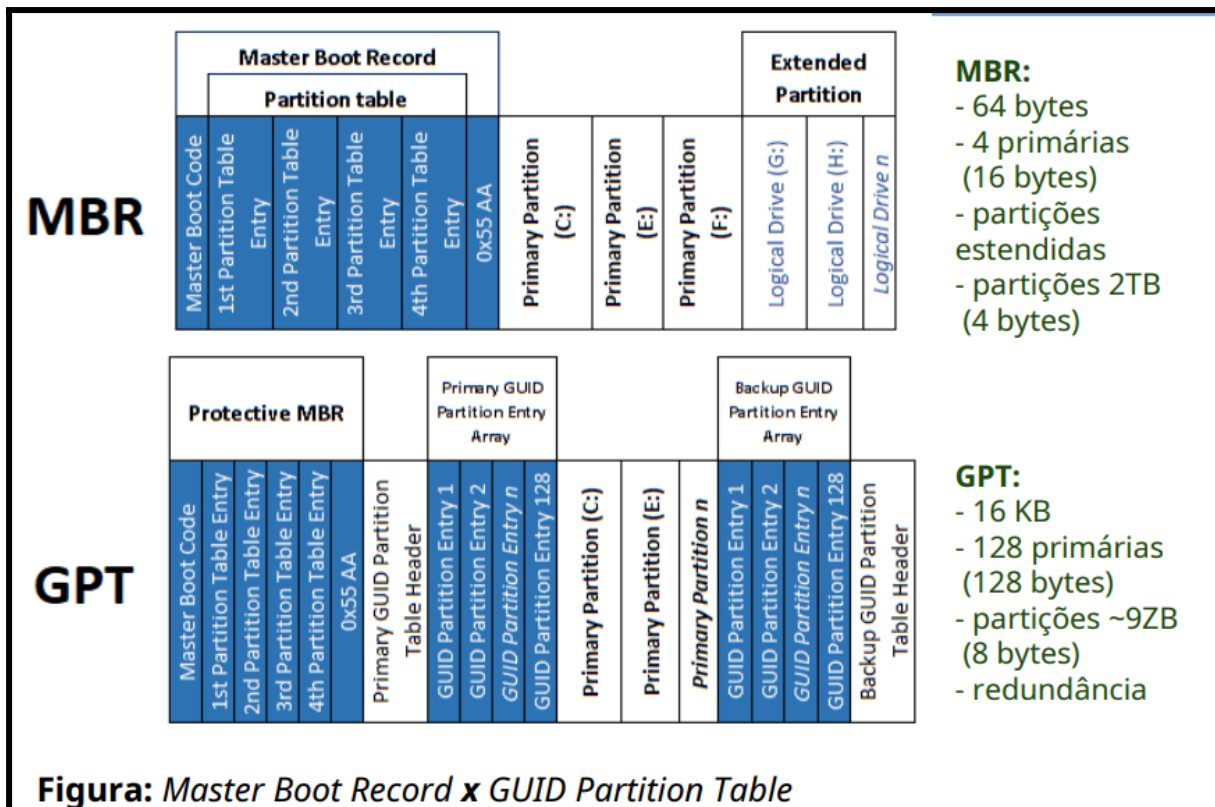
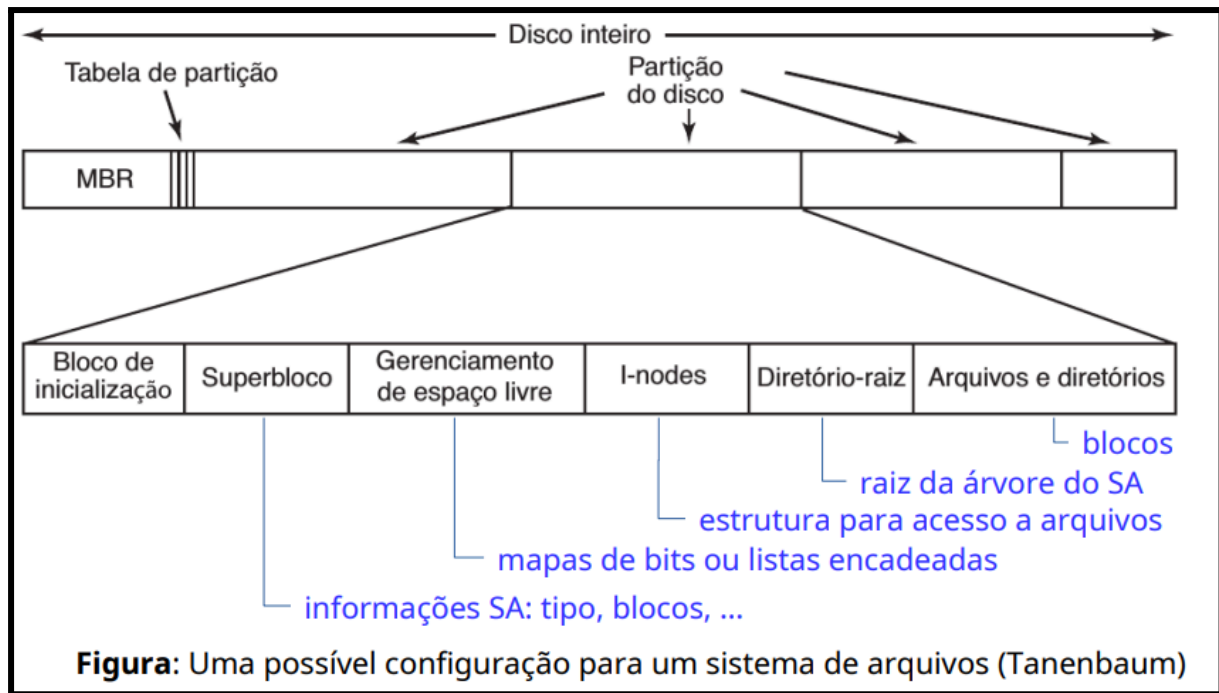


Figura: Camadas da implementação de gerência de arquivos. (Maziero)

• Discos e Partições

- **Acesso e armazenamento em blocos.**
- **Discos podem ser divididos em partições.**
- A tabela de partições armazena as informações de controle da partição.
- **A área inicial do disco armazena o código de inicialização do SO e a tabela de partições (MBR, GPT).**
- No início de cada partição há também blocos reservados para descrição da partição e inicialização do SO.





- **Gerência de Blocos**

- **Tipos de blocos**

- Blocos físicos (512 ou 4096 bytes)
- Blocos lógicos (4 KiB a 64 KiB)

- **Tamanho dos blocos**

- Pequenos (desempenho)
- Grandes (fragmentação interna)

- **Otimização de leitura**

- **Read-through**: consulta cache para localizar o bloco.
- **Read-ahead**: lê mais blocos que solicitado e armazena na cache.

- **Otimização de escrita**

- **Write-through**: escrita síncrona.
- **Write-back**: dados são armazenados em cache para futura escrita.

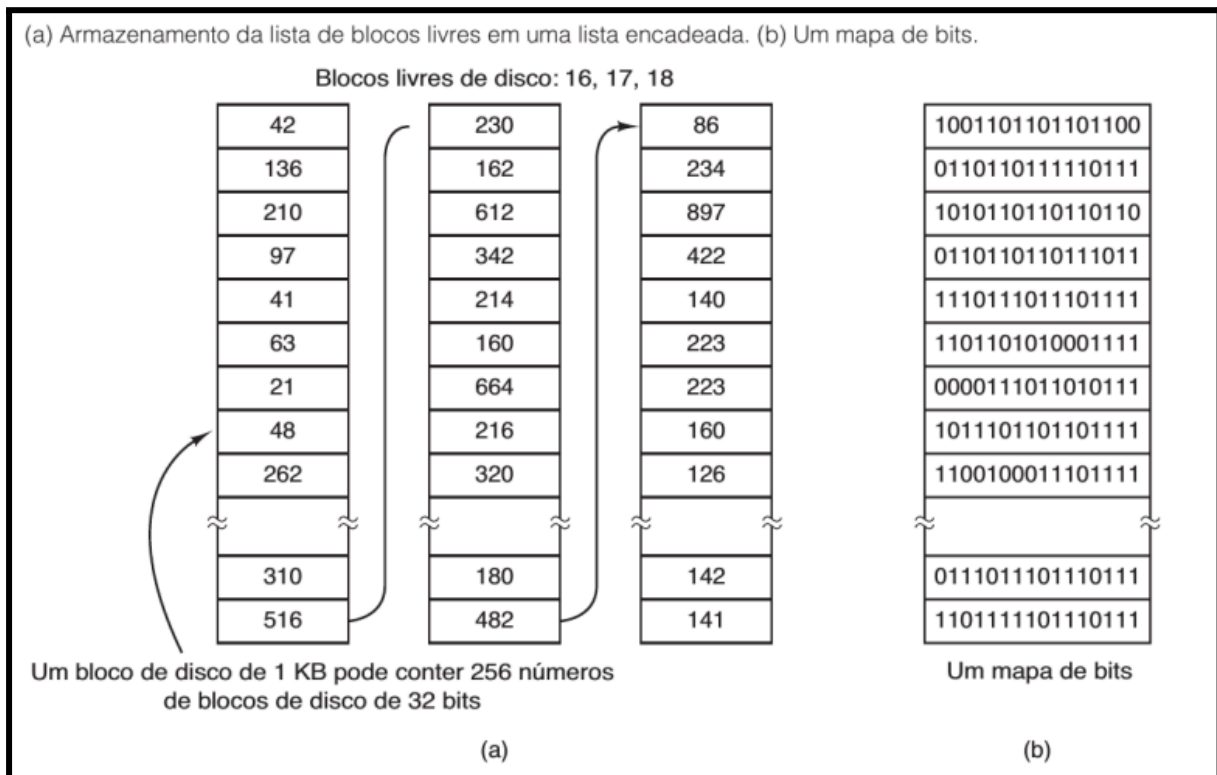
- **Gerência de Espaço Livre**

- **Mapa de bits**

- Manter um mapa de bits em uma área reservada do volume.

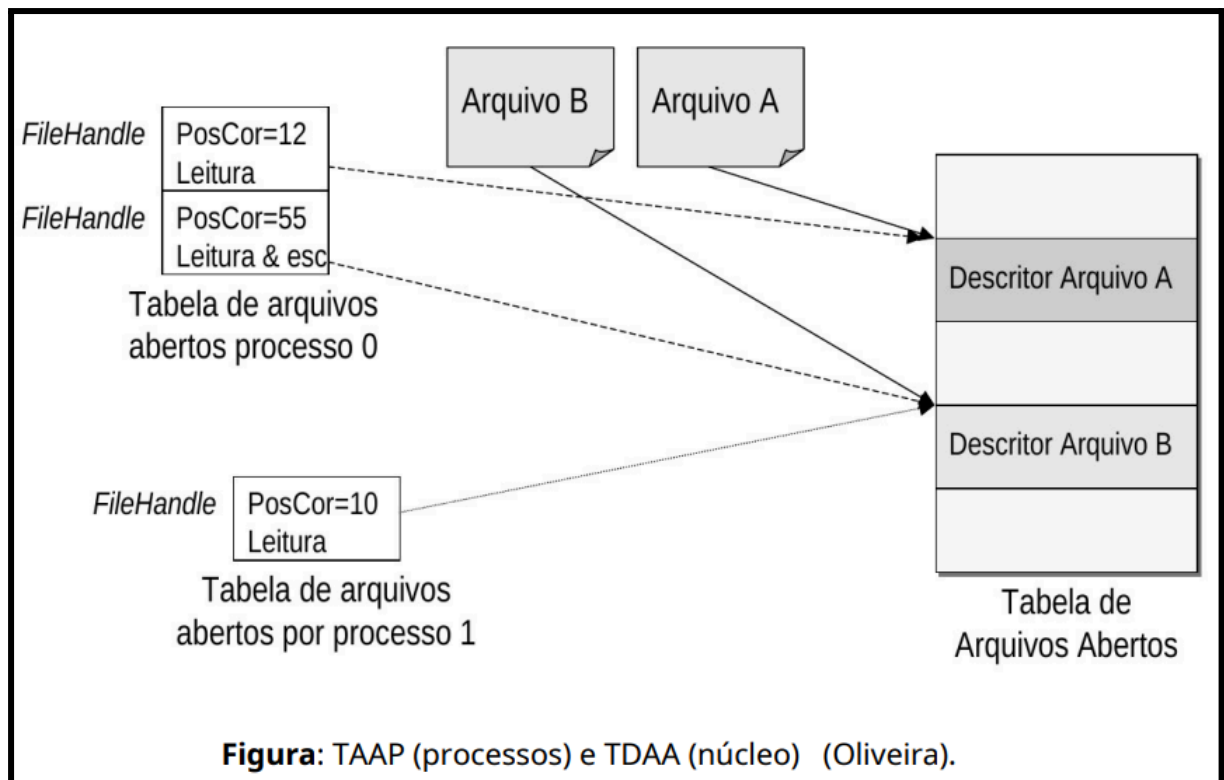
- **Cada bit representa um bloco livre ou ocupado.**

- **Simple de implementar e compacto.**
 - Disco 1TiB, blocos 4KiB, mapa de bits = 32MiB
- **Lista encadeada**
 - **Armazenar ponteiros para blocos livres.**
 - O último ponteiro apontaria para outro bloco de ponteiros.
 - Obter um grande número de blocos livres a cada acesso ao disco.
 - Blocos 1KiB, número blocos 32 bits, 1 bloco contém 255 blocos livres.
 - Uma alternativa seria uma **tabela de grupos de blocos livres com entradas [bloco, tamanho]**.



Implementação de Arquivos

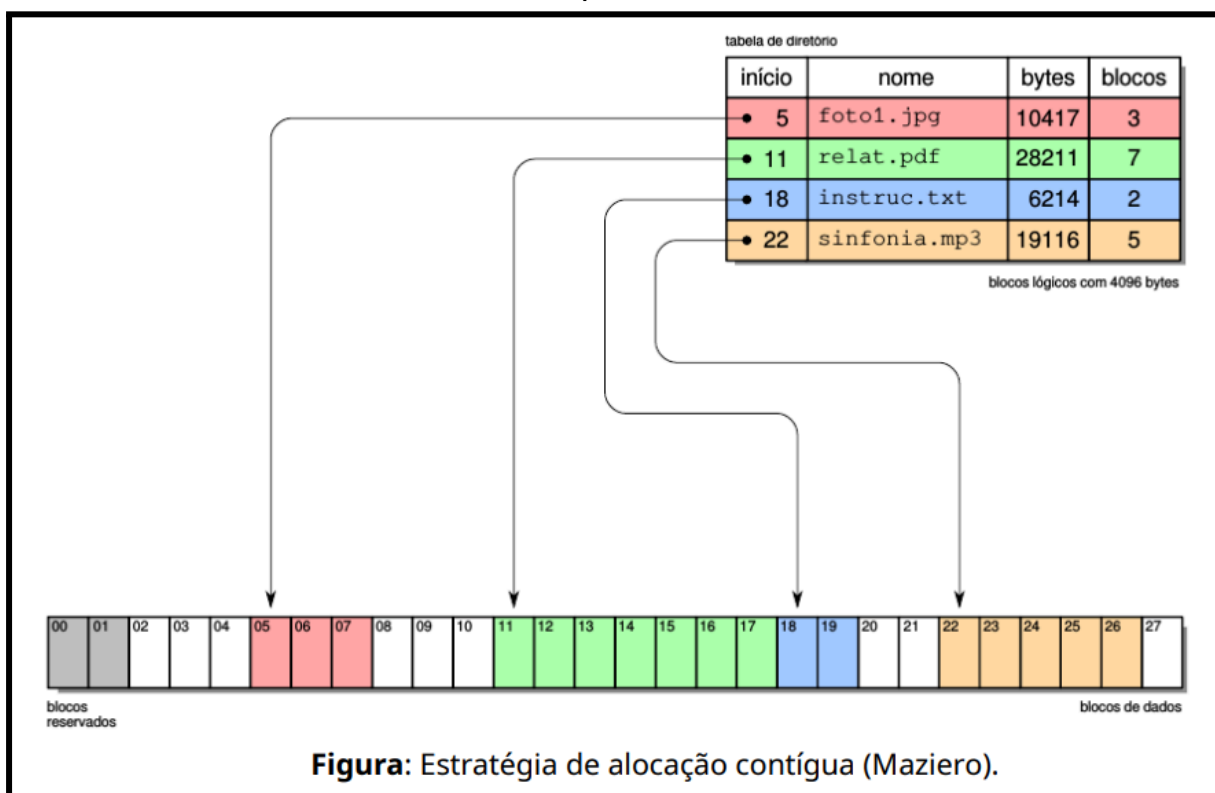
- Para manipular arquivos, precisa-se de estruturas de dados.
- O **Descritor de Arquivos** mantém informações de um arquivo.
- Duas estruturas são usadas para manter informações dos descritores de arquivos abertos:
 - **Tabela de Descritores de Arquivos Abertos (TDAA)** – Global File Table
 - **Tabela de Arquivos Abertos por Processo (TAAP)** – File Table



Alocação de Arquivos

- Espaço de armazenamento dividido em blocos lógicos fixos.
- **Alocação de Arquivos:** alocar os metadados e conteúdos dos arquivos dentro dos blocos lógicos.
- Desempenho é essencial:
 - **Rapidez:** acessos sequenciais e aleatórios.
 - **Robustez:** quanto a erros, blocos defeituosos, dados corrompidos.
 - **Flexibilidade:** criação, alteração e exclusão de arquivos.
- **Bloco de Controle de Arquivo**
 - File Control Block (FCB)
 - **Estrutura para armazenar os metadados e referências para os blocos do arquivo.**
 - Implementação
 - Entrada no diretório.
 - Estrutura independente:
 - **Master File Table (MFT)**
 - **i-nodes**
- **Alocação Contígua**

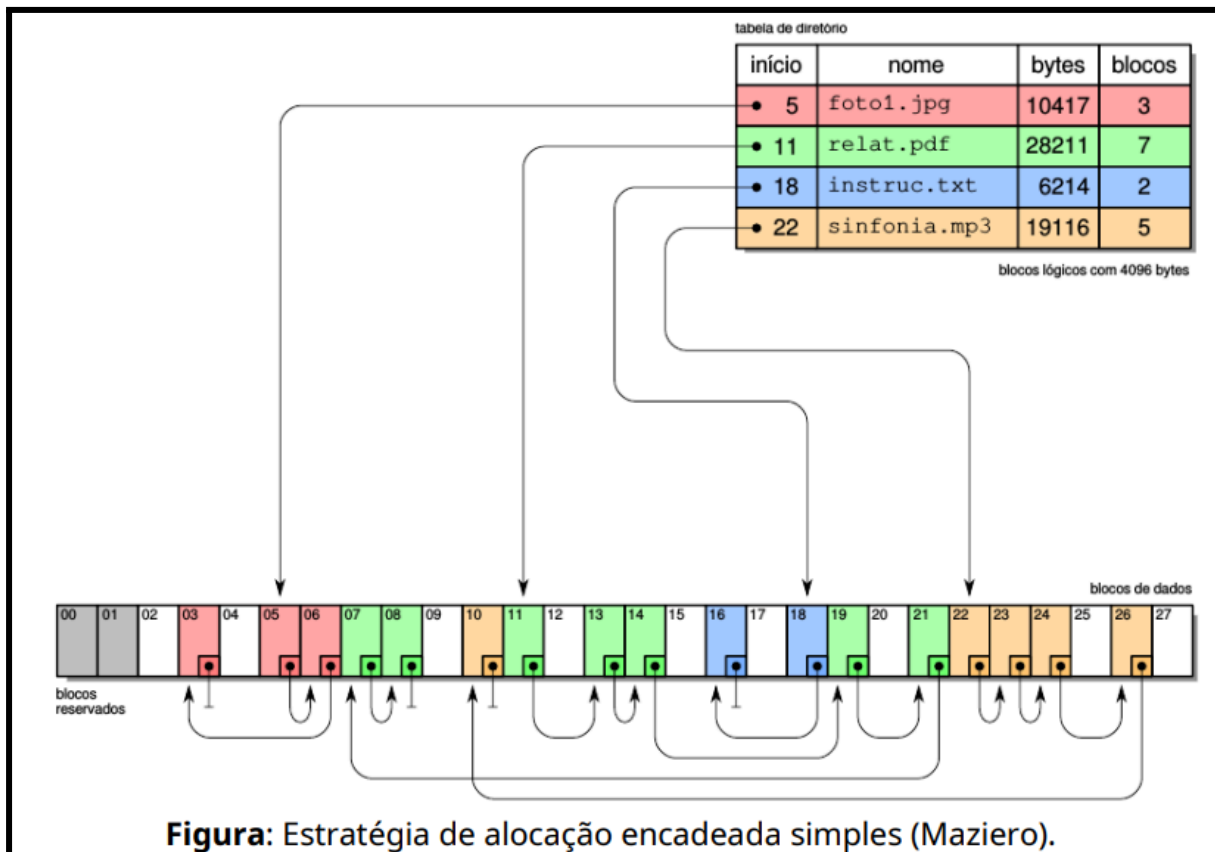
- Arquivos armazenados em um conjunto de blocos contíguos no disco.
- Vantagens:
 - **simples de implementar** [bloco inicial e tamanho];
 - **alto desempenho para leitura.**
- Desvantagens:
 - **escrita;**
 - **fragmentação externa.**
- Exemplo:
 - Sistema ISO 9660 para CD-ROM.



● **Alocação Encadeada Simples**

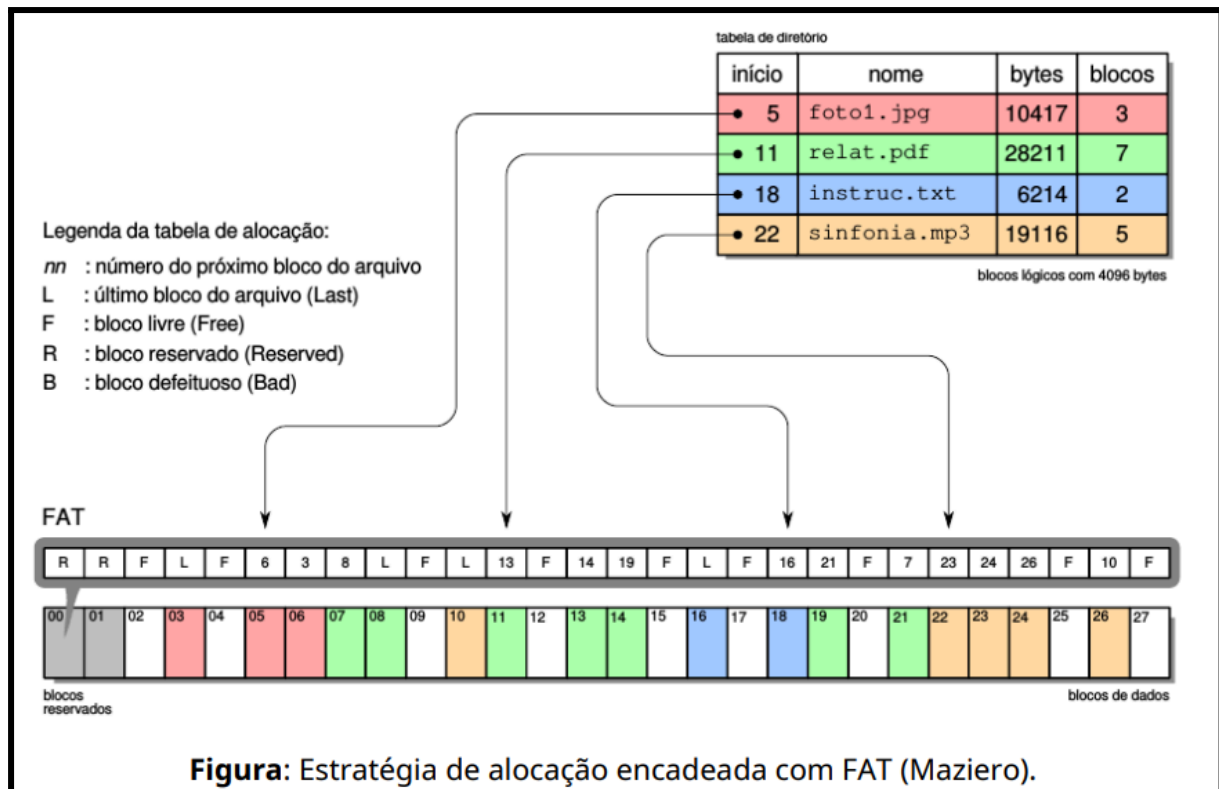
- Bloco do arquivo contém dados e ponteiro para o próximo bloco.
- Vantagens
 - **Elimina fragmentação externa;**
 - **não conhecimento prévio do tamanho do arquivo.**
- Desvantagens
 - **Acesso aleatório.**
 - **Problema de robustez: bloco defeituoso.**

■ Tamanho de dados dos blocos diminuído (ponteiro).



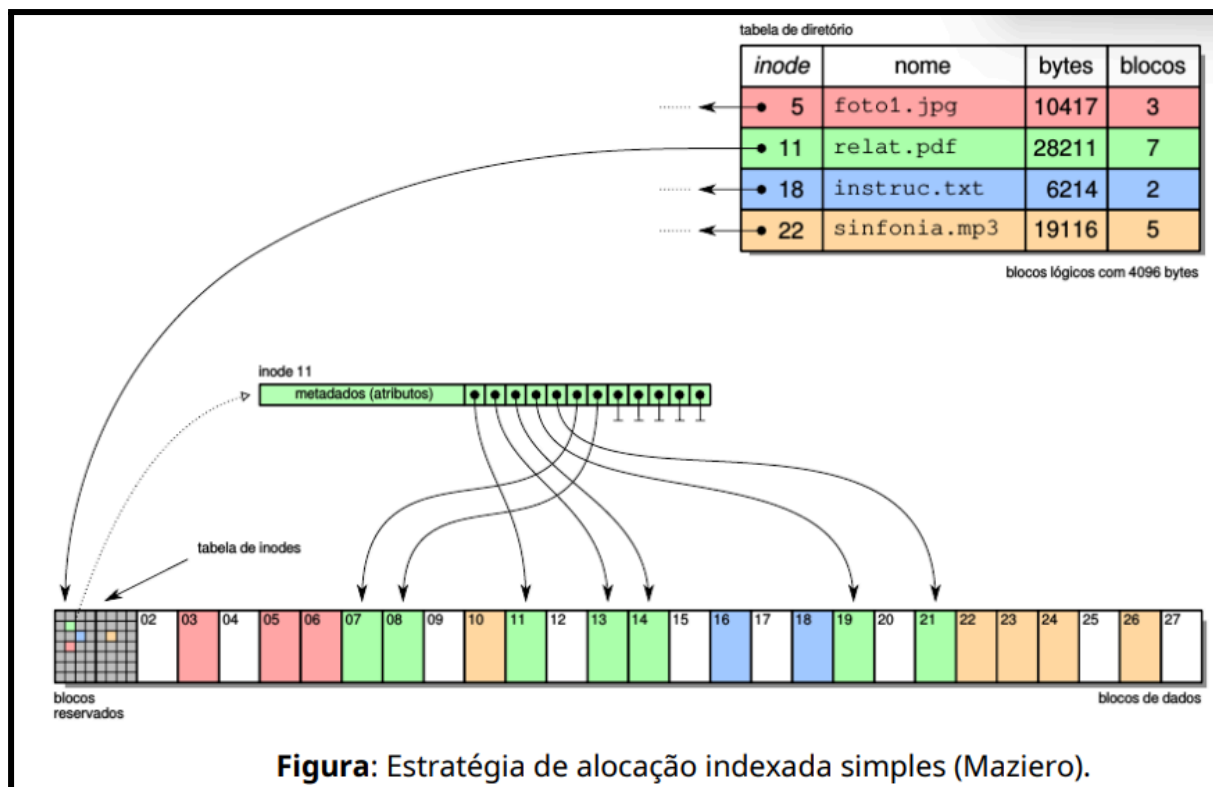
● Alocação Encadeada FAT

- Os ponteiros para os blocos de arquivos são mantidos em uma única tabela denominada **Tabela de Alocação de Arquivos – File Allocation Table (FAT)**
- A tabela é armazenada em blocos reservados na partição e carregada inteira na memória principal.
- Cada entrada corresponde a um bloco lógico e contém ponteiro para a próximo bloco do arquivo.** Pode conter tags para blocos livres, defeituosos e reservados.
- FAT geralmente possui uma cópia de segurança.
- Usada no MS-DOS, Windows e em sistema de arquivos para dispositivos de armazenamento portáteis.



• Alocação Indexada Simples

- Associar a cada arquivo uma **estrutura de dados que armazena os atributos e os índices de blocos de dados do arquivo**.
- A estrutura é denominada de **i-node (index-node)**.
- Uma **tabela de i-nodes** em uma área reservada armazena **todos os i-nodes**.
- **Desvantagens**
 - **Tamanho fixo dos i-nodes e da tabela: limitam tamanho de um arquivo e número máximo de arquivos.**



- **Alocação Indexada Multinível**

- Usar entradas dos i-nodes como ponteiros indiretos, isto é, para novos blocos de índices.
- No ext2/3/4 (usado no Linux) temos:
 - 12 ponteiros diretos (dados)
 - 1 ponteiro indireto
 - 1 ponteiro duplamente indireto
 - 1 ponteiro triplamente indireto

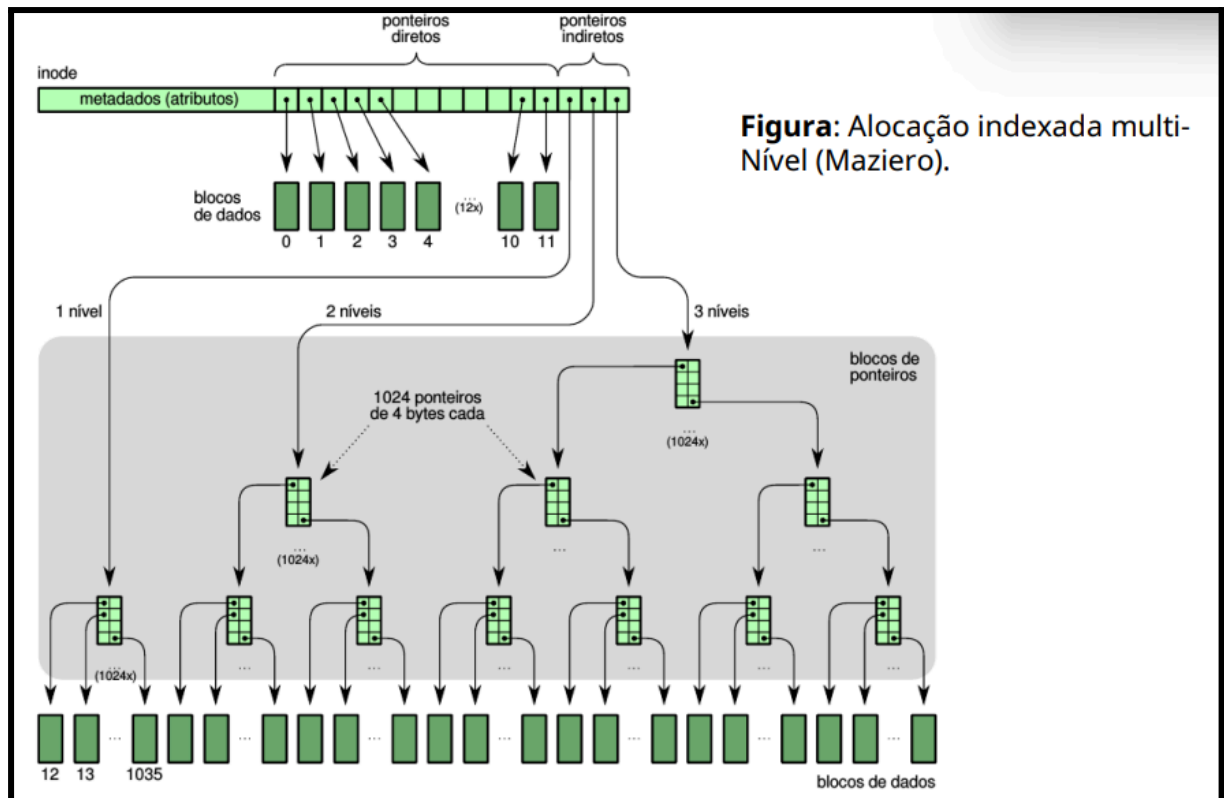


Figura: Alocação indexada multi-Nível (Maziero).

Implementação de Diretórios

- Implementado como um arquivo que o **conteúdo é uma relação de entradas (arquivos, diretórios, atalhos, entre outros)**.
- Cada entrada deve possuir ao menos: **nome do arquivo, tipo, localização física (i-node, bloco inicial)**.
- Os atributos podem estar na entrada ou na estrutura (i-nodes).
- A lista ou índice do diretório pode ser implementada como uma **lista simples** ou com estruturas de dados mais complexas (hash, árvores).

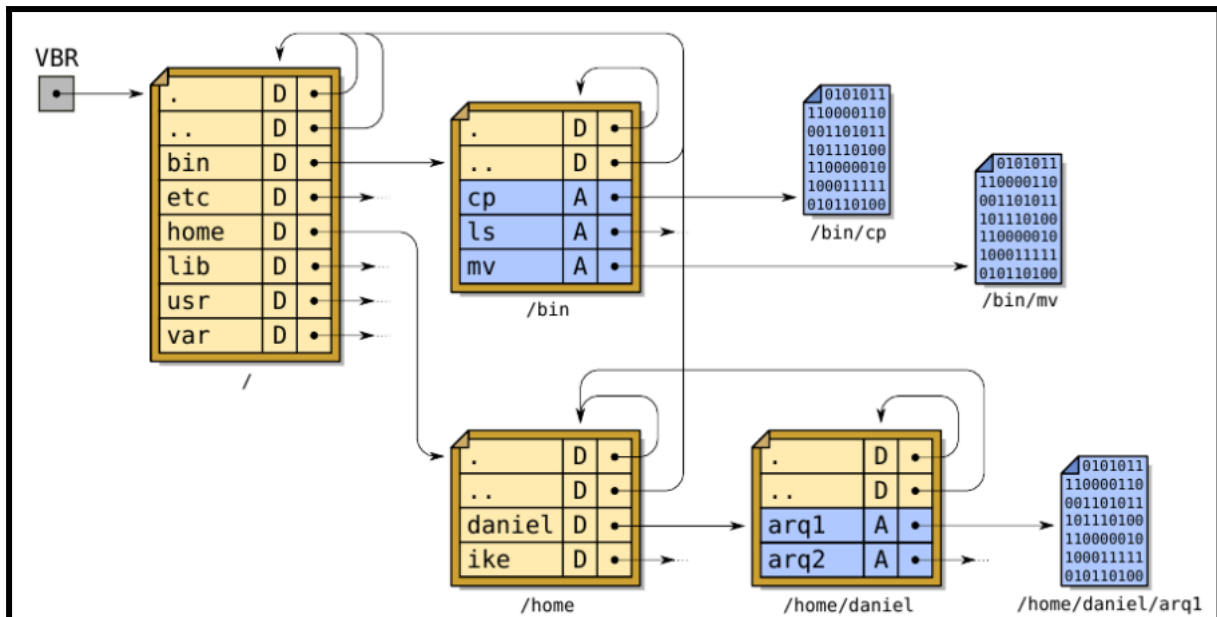


Figura: Implementação de uma estrutura de diretórios (Maziero).

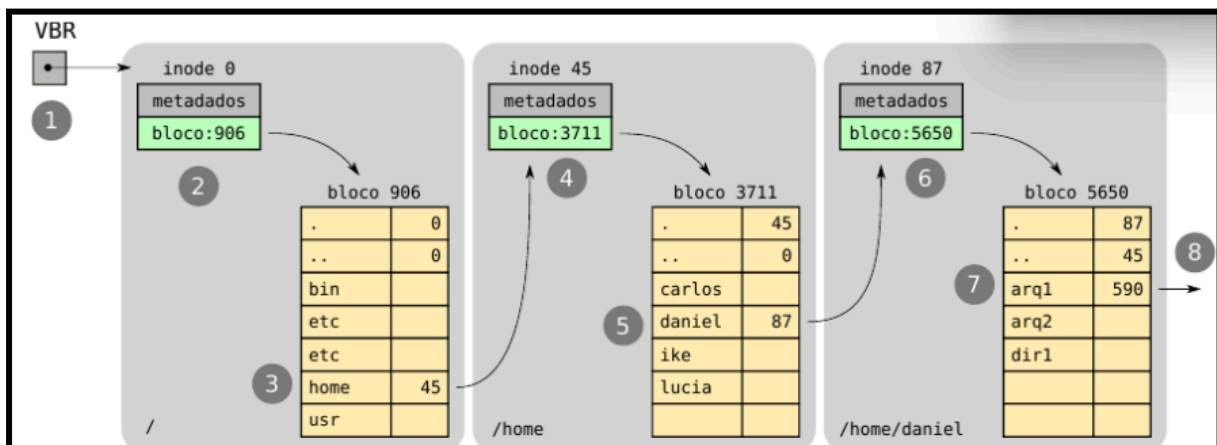


Figura: Resolução para o arquivo /home/daniel/arq1 (Maziero).

1. localizar i-node no dir. raiz
2. ler i-node "/" (permissão e localização)
3. ler conteúdo i-node "/" para localizar "/home"
4. ler i-node "/home" (permissão e localização)
5. ler i-node "/home" para localizar "/home/daniel"
6. ler i-node "/home/daniel" (permissão e localização)
7. ler i-node "/home/daniel" para localizar "/home/daniel/arq1".
8. Devolver o i-node do arquivo.

Exemplos

FAT

- FAT (1977), FAT12 (1980), FAT16 (1984), FAT32 (1996), exFAT (2006)

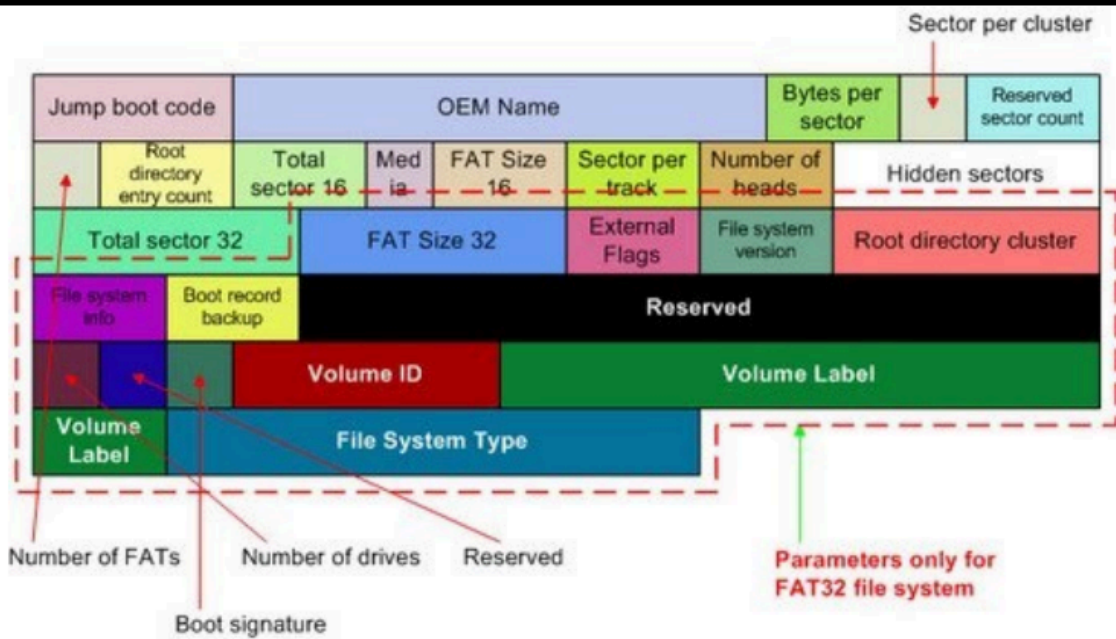
FAT16

Boot Record	FAT #1	FAT #2	Diretório Raiz	Arquivos e Diretórios
-------------	--------	--------	----------------	-----------------------

FAT32

Boot Record	FAT #1	FAT #2		Arquivos e Diretórios
-------------	--------	--------	--	-----------------------

Diretório raiz

**Figura: FAT32 - Estrutura do *Boot Record*.**Fonte: <https://sites.google.com/site/iprinceps/Home/embedded-system-and-operating-systems/understanding-file-system>

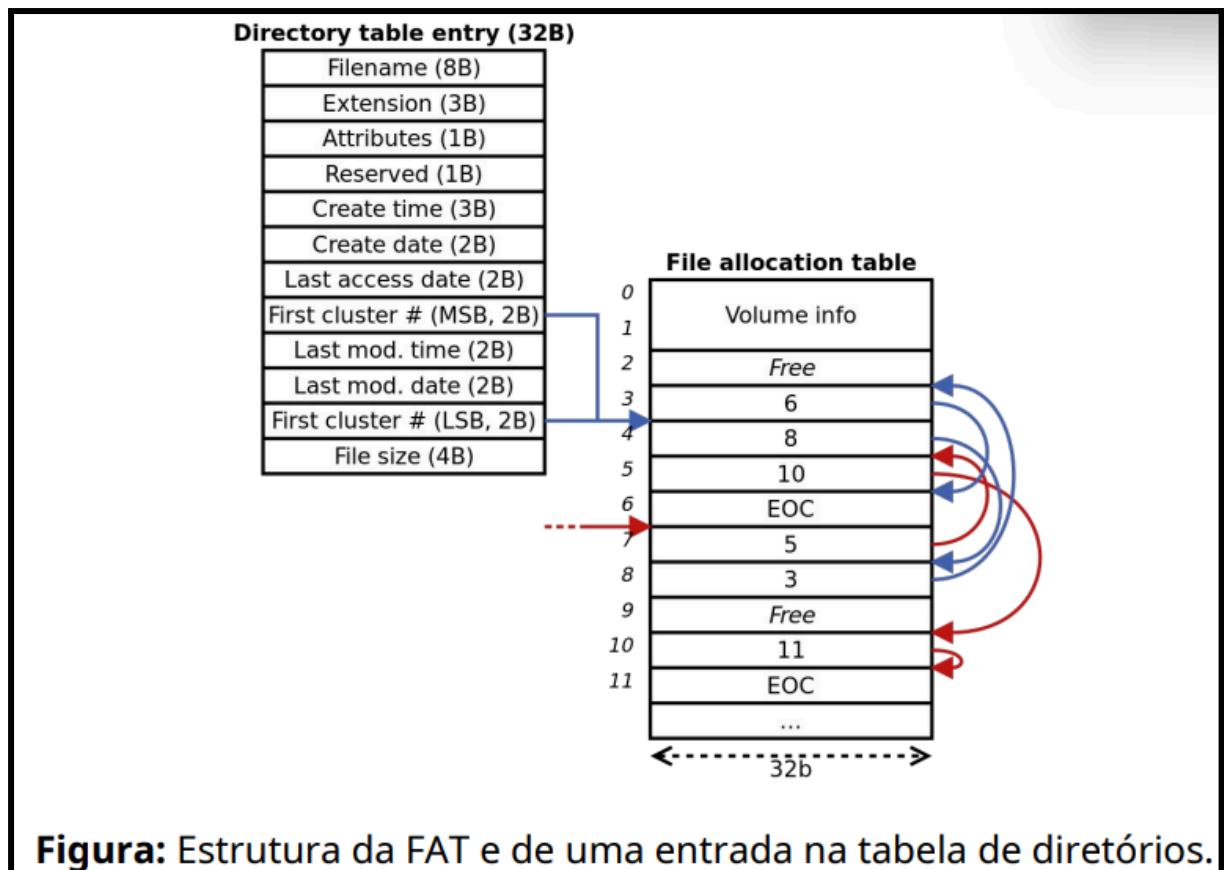


Figura: Estrutura da FAT e de uma entrada na tabela de diretórios.

NTFS

- **New Technology File System**
- Julho de 1993, Windows NT 3.1
- **Diretórios, arquivos e metadados de arquivos são armazenados como metadados na MFT.**
- **Provê escalabilidade, sistema de journaling, cotas, encriptação, compressão, ...**
- **Arquivos pequenos podem ser armazenados diretamente na MFT.**

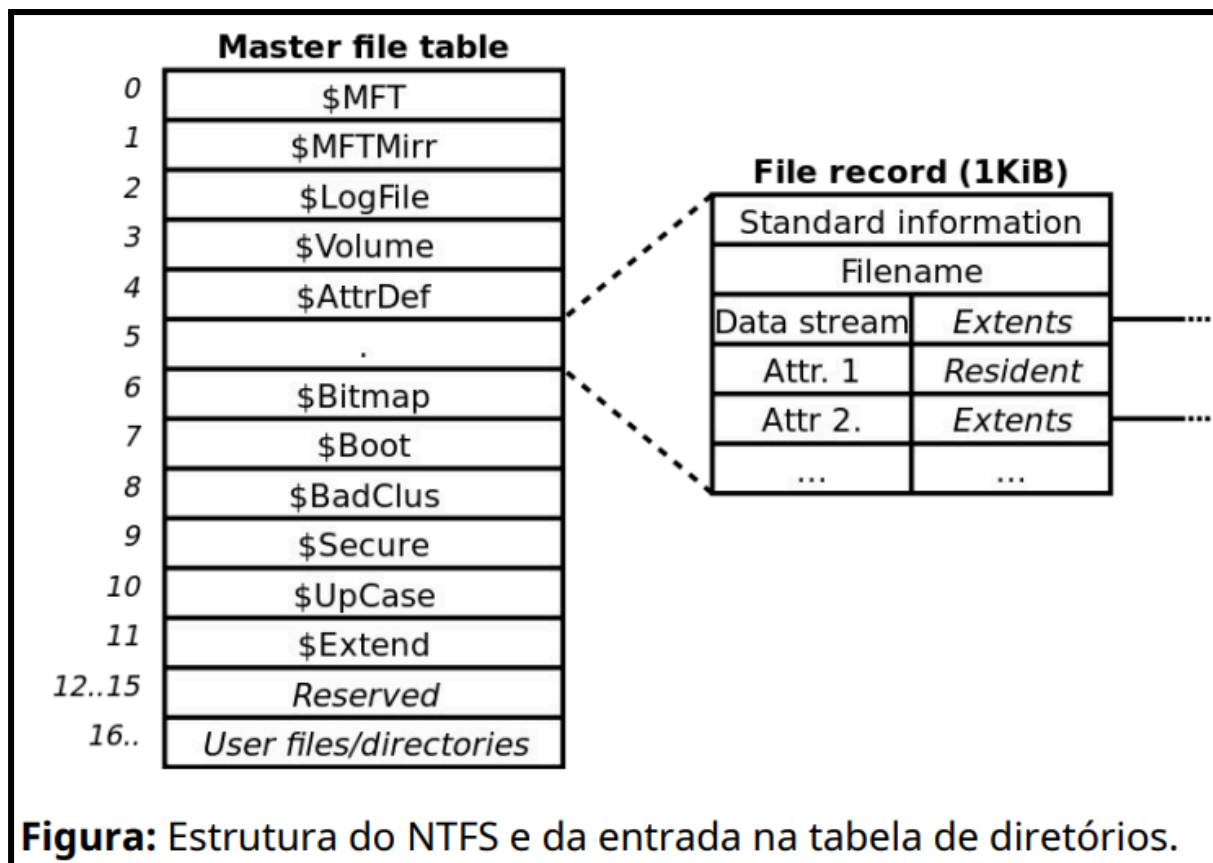
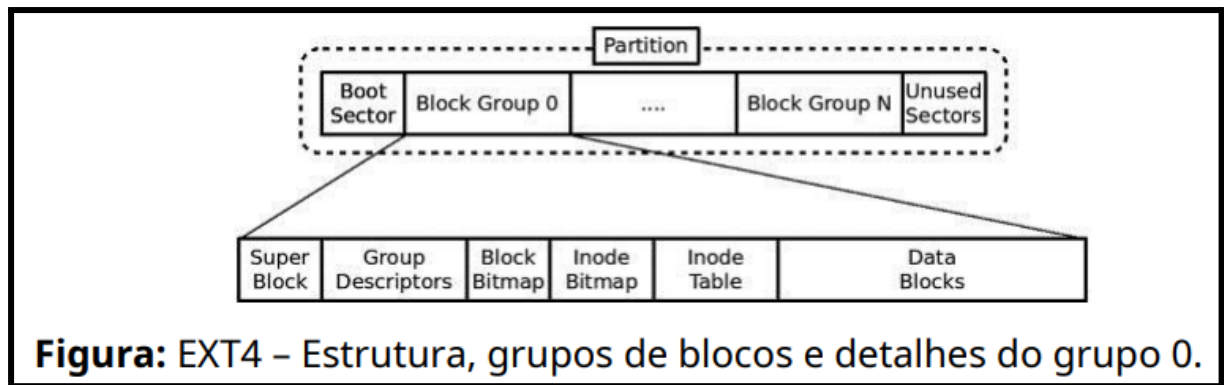


Figura: Estrutura do NTFS e da entrada na tabela de diretórios.

EXT4

- **Extended File System que faz uso da estrutura inode.**
- Versões: EXT (1992), EXT2 (1993), EXT3 (2001) e EXT4 (2006).
- Algumas características:
 - **Suporte para arquivos grandes** (aumento de desempenho e diminuição da fragmentação).
 - **Sistema com journaling e com checksum para o journal.**
 - **Checagem rápida do SA.**
 - **Uso de extents (sequências de blocos).**
- **Dividido em grupos de blocos**
 - **Blocos:** 4KiB
 - **Grupos de blocos:** 32K blocos
 - **Tamanho do grupos:** 128 MiB



Ferramentas Linux

- **Informações de sistema de arquivos**

- tune2fs -l /dev/sda1
- fdisk -l
- lsblk
- parted -l

- **Listar inodes de arquivos**

- ls -li

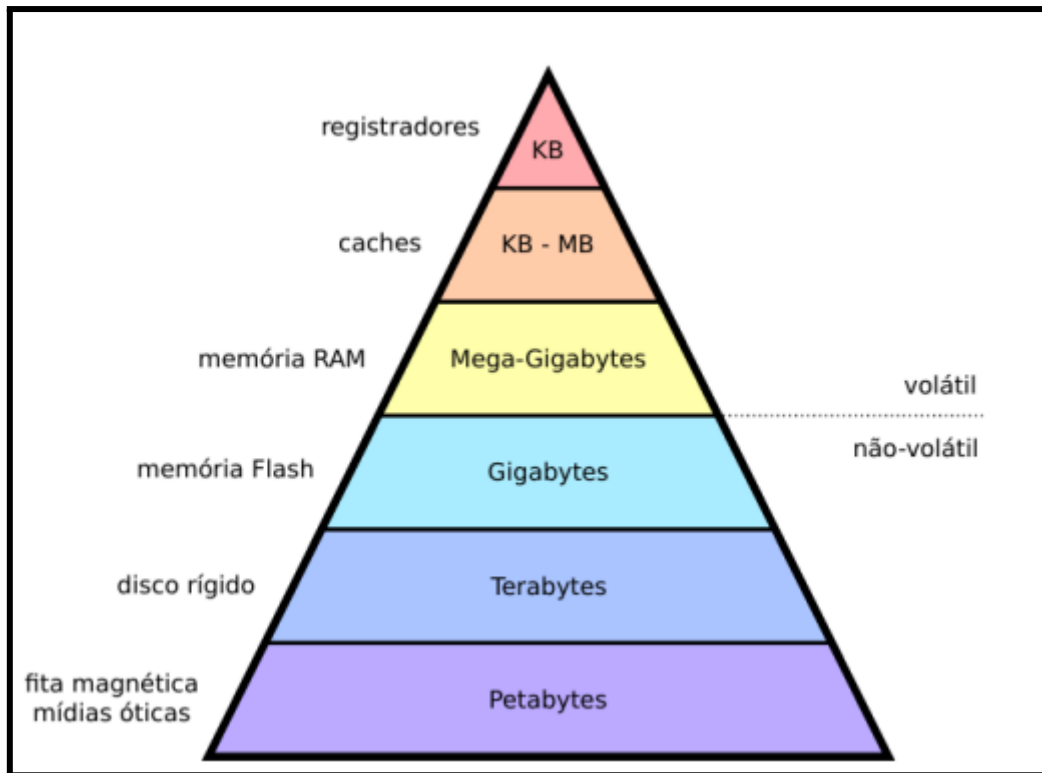
- **Informações de arquivo**

- stat filename.abc

Gerência de Memória

Introdução

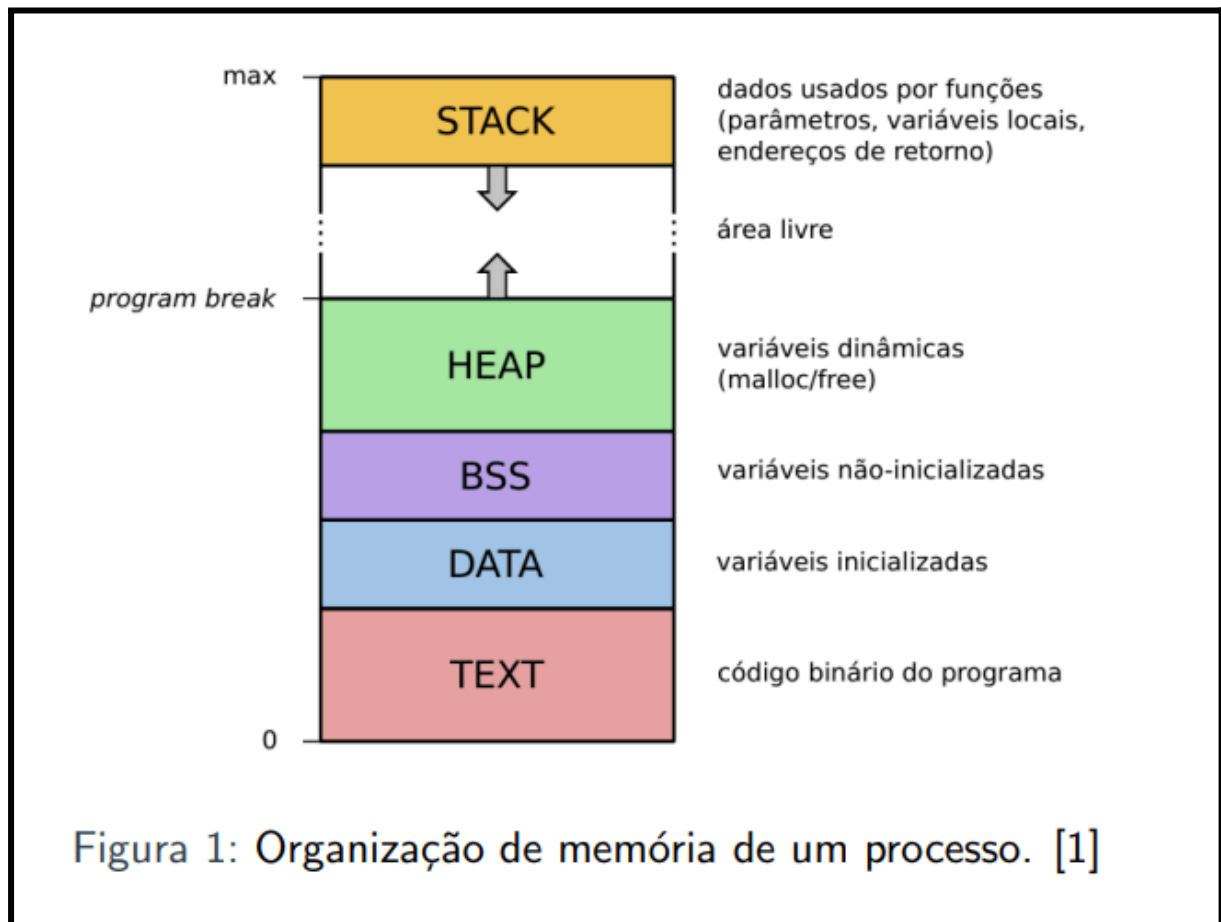
- Dois principais tipos de memória
 - **Primária: volátil (RAM).**
 - **Secundária: persistente (discos).**
- Os **programas são armazenados na memória secundária e carregados na memória primária** (processos) para a execução.
- A gerência de memória é responsável pelo **gerenciamento da memória primária** e depende das facilidades providas pelo hardware.



Endereço Lógico x Físico

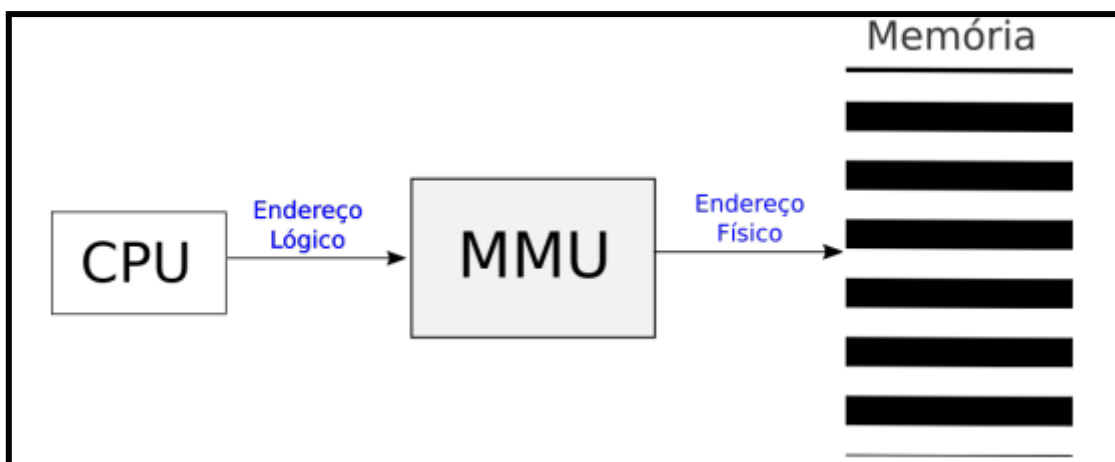
- Endereços referem-se a uma posição de memória.
- Endereços lógicos (virtuais) são gerados pela CPU durante a execução de um programa.
- Endereços físicos (reais) indicam localizações na unidade de memória.
- Endereços lógicos são traduzidos para endereços físicos.
- Espaço de endereçamento lógico (processo) x espaço de endereçamento físico (hardware).

Memória de um Processo



MMU

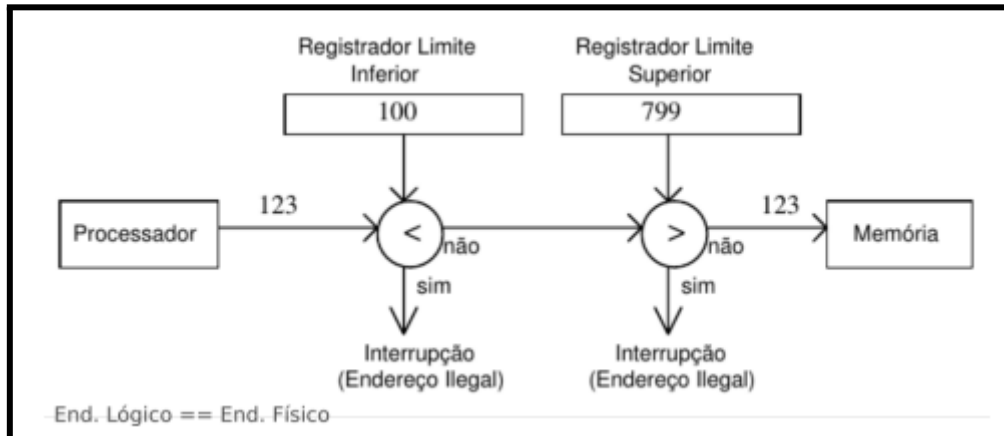
- **Memory Management Unit (MMU).**
- **Hardware responsável pelo mapeamento entre endereço lógico e endereço físico.**



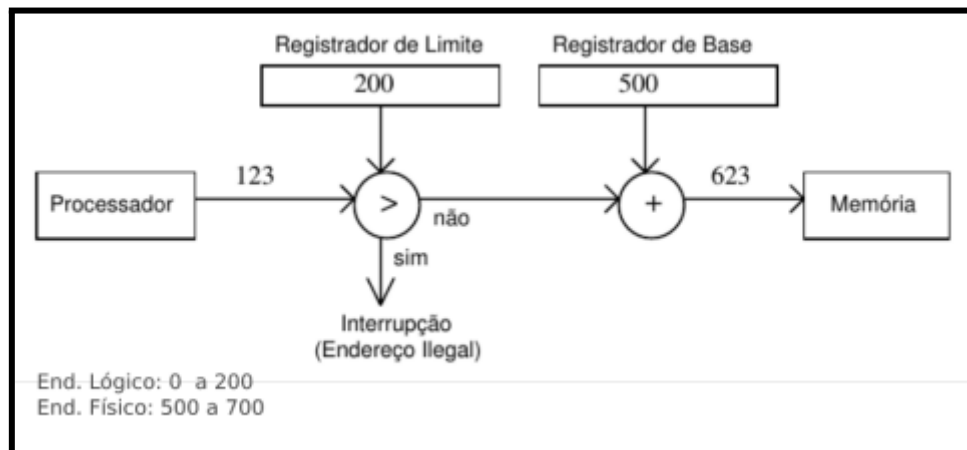
- Algumas das funcionalidades são: **tradução de endereços, proteção de memória, controle de cache.**

Proteção de Memória

- Proteção de memória usando **registradores de limite**.



- Proteção de memória usando **registradores de base e limite**.



Carregador Realocador x Absoluta

- **Carregador relocador**

- Realiza a **correção das referências de memória segundo a posição de carga do programa.**
- Usado quando o esquema de proteção é **registradores de limite.**
- **Correção via software.**

- **Carregador absoluto**

- Realiza a **tradução das referências de memória durante a execução.**
- Usado quando o esquema de proteção é **registradores base e limite.**
- **Correção via hardware.**

Gerência de Memória Livre

• Mapa de bits x Lista encadeada

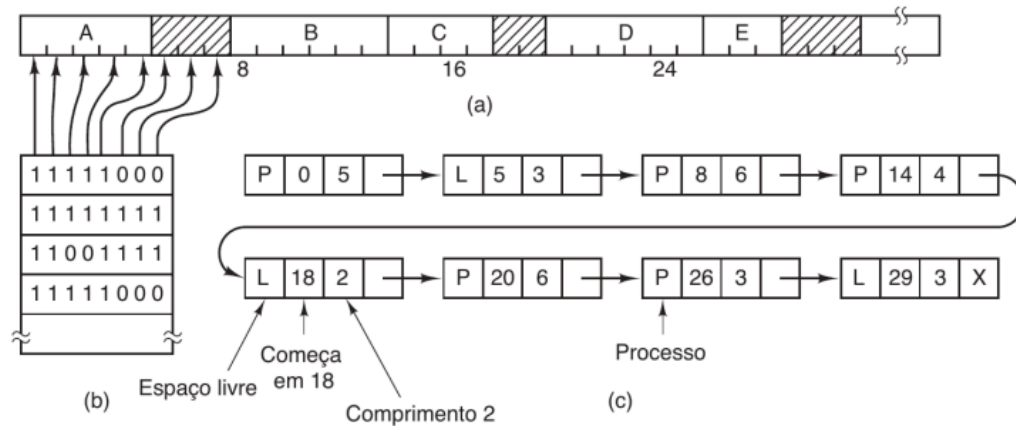


Figura 2: (a) memória com 5 processos e três lacunas livres. (b) mapa de bits. (c) lista encadeada. [3]

Alocação de Memória

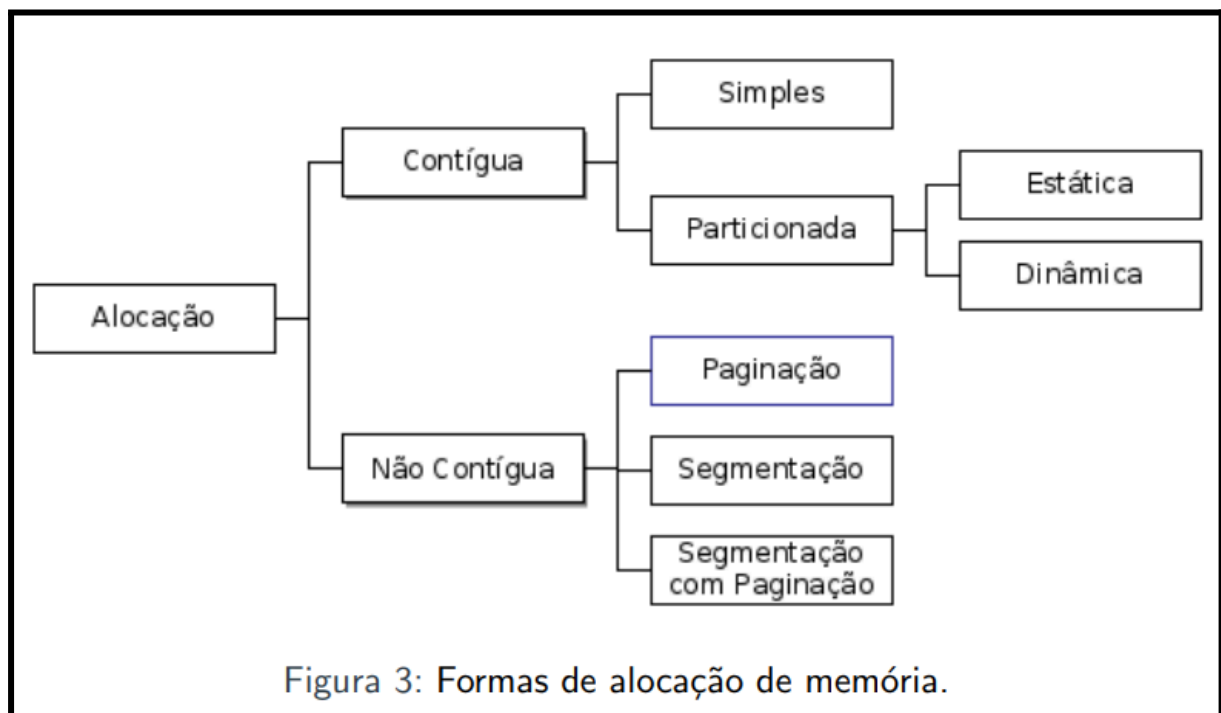


Figura 3: Formas de alocação de memória.

Alocação Contígua

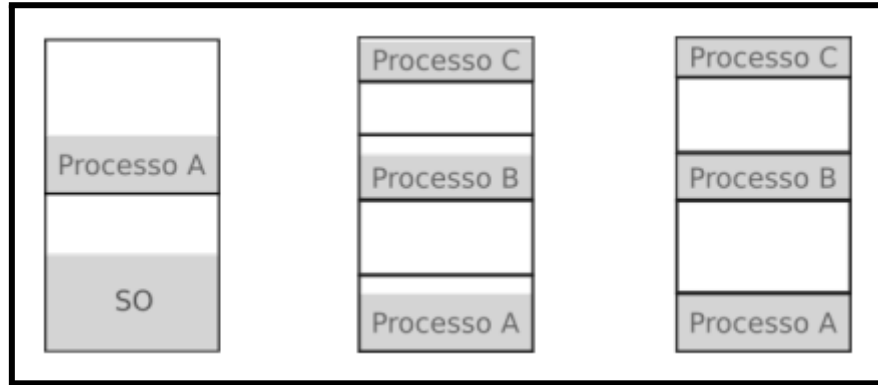
- **Simples**
 - Divide em duas partes: **SO e usuário**.

- **Particionada Estática**

- Partições de tamanho variável e estáticas.

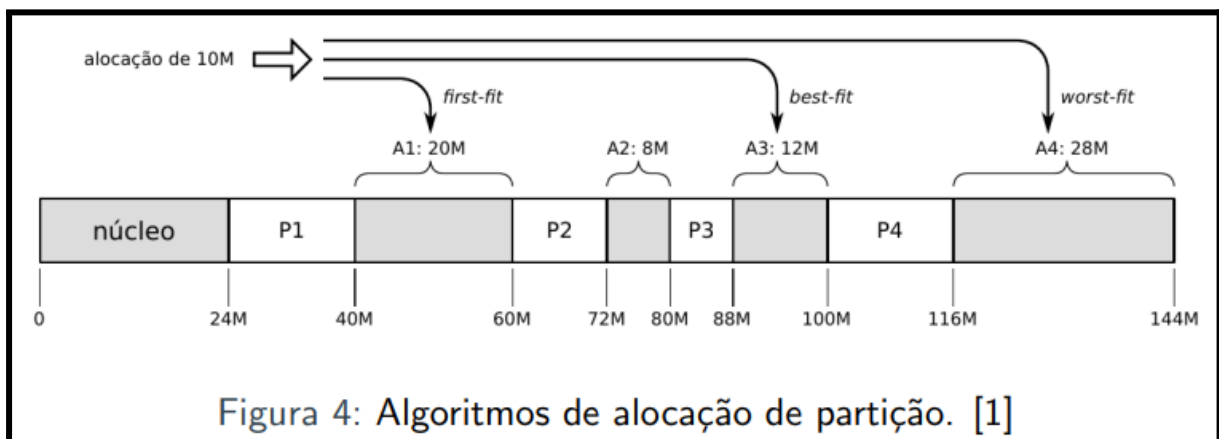
- **Particionada Dinâmica**

- Partições de tamanho variável e dinâmicas.
- Adaptam-se ao tamanho do processo.



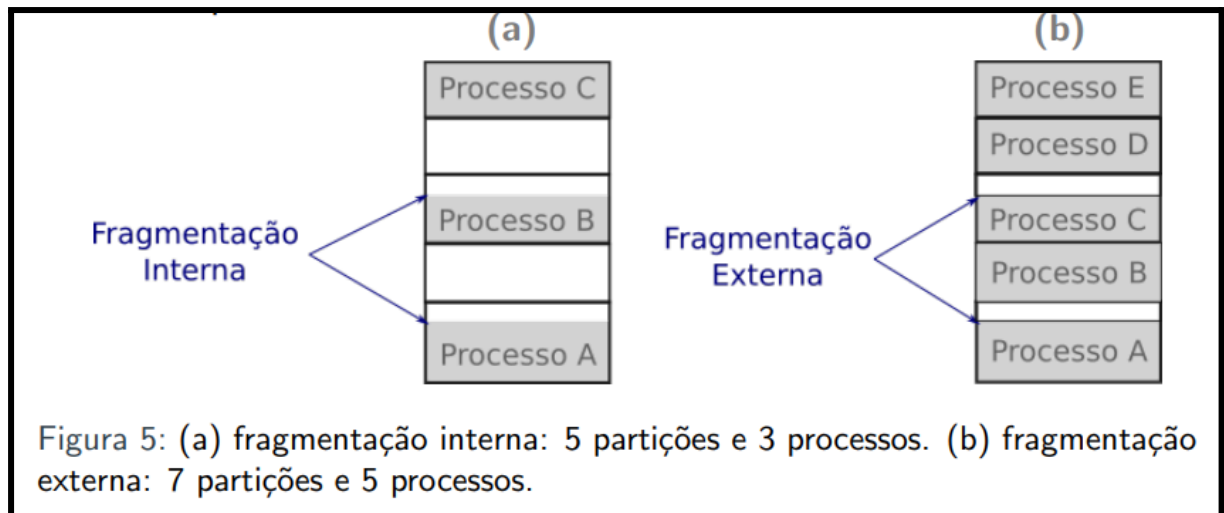
Algoritmos para Alocação

- **First-fit**: aloca a primeira lacuna.
- **Best-fit**: aloca lacuna com menor sobra.
- **Worst-fit**: aloca lacuna com maior sobra.
- **Circular-fit ou Next-fit**: aloca próxima lacuna a partir da última alocação.
- **Quick-fit**: possui listas de espaços livres para os tamanhos mais utilizados.



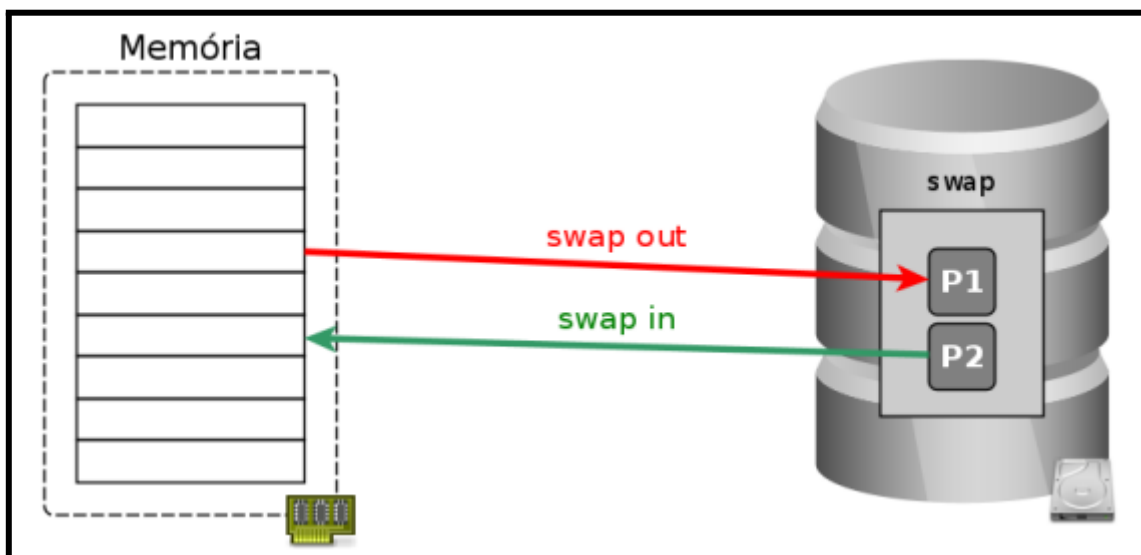
Fragmentação Interna x Externa

- **Fragmentação interna**: espaço interno à partição não usado.
- **Fragmentação externa**: espaços disponíveis, mas não contíguos para caber processos.



Swapping

- O **swapping** consiste em **mover processos entre a memória principal e a secundária.**



- **swap out** move para a **memória secundária.**
- **swap in** move para a **memória primária.**

Paginação

- **Consiste em dividir a memória física (sistema) e a memória lógica (processo) em blocos de tamanho fixo e idênticos.**
 - **memória física dividida em blocos fixos denominados quadros (frames).**
 - **memória lógica dividida em blocos fixos denominados páginas (pages).**

- Cada **processo** possui uma **Tabela de Páginas** com as suas **páginas**.
- Toda **página** está mapeada para um **quadro**.
- O tamanho da **página** é igual ao tamanho do **quadro**.

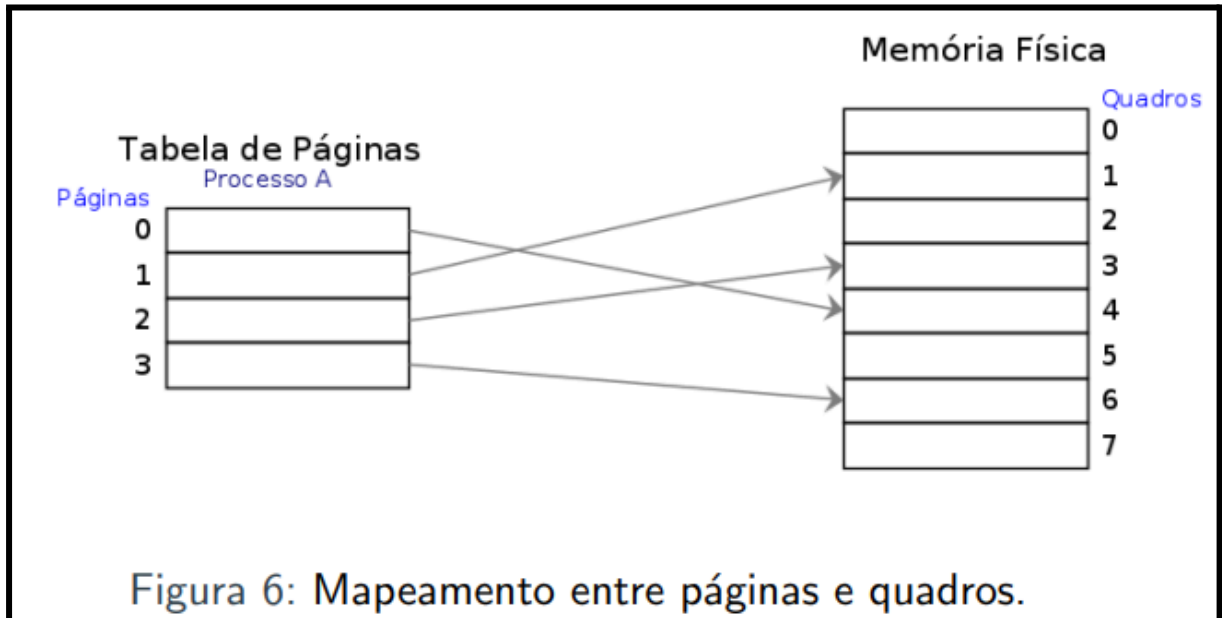
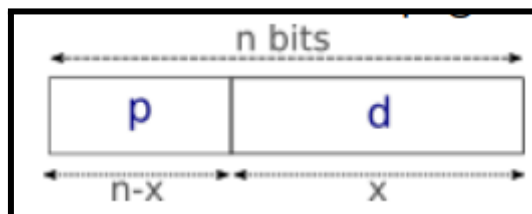
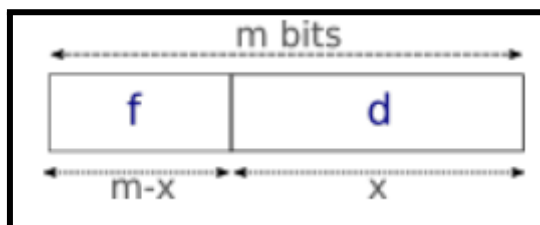


Figura 6: Mapeamento entre páginas e quadros.

- O **endereço lógico** é dividido em duas partes:
 - **número da página (p).**
 - **deslocamento na página (d).**



- O **endereço físico** é dividido em duas partes:
 - **número do quadro (f).**
 - **deslocamento no quadro (d).**



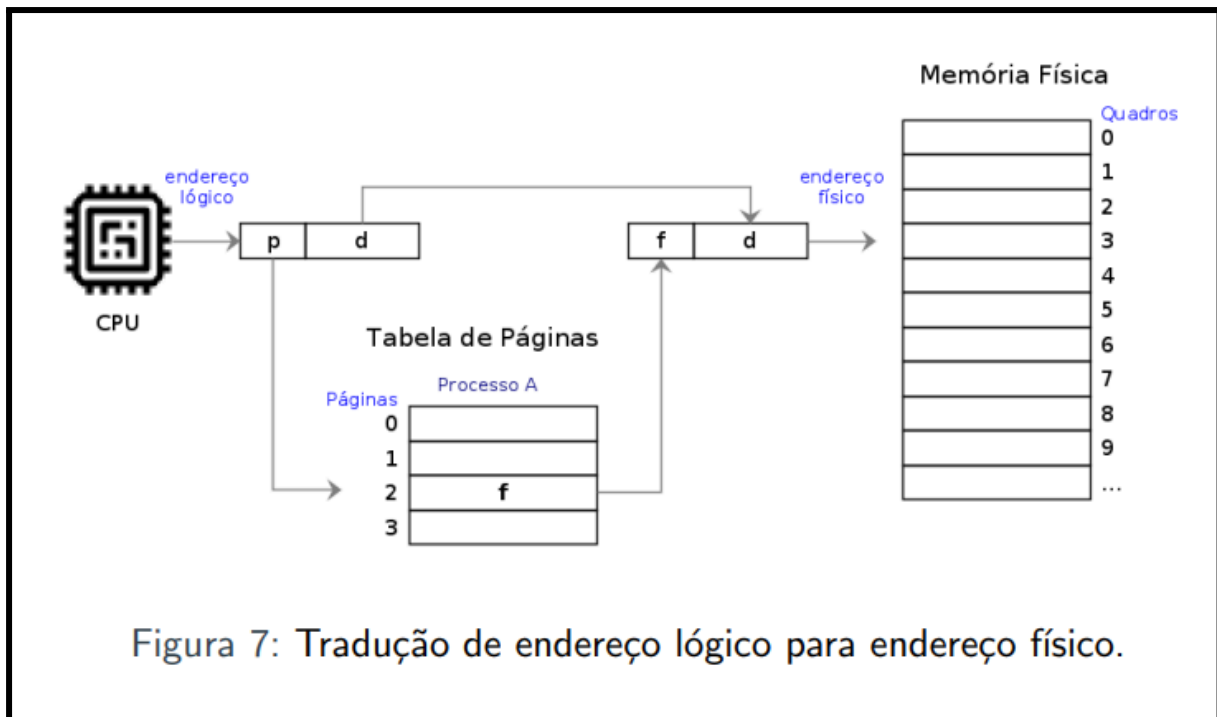


Figura 7: Tradução de endereço lógico para endereço físico.

- **Exemplo:**

- Considere o sistema:

- **memória física:** 64 KiB (16 bits).
- **tamanho máximo do processo:** 32 KiB (15 bits).
- **tamanho de páginas:** 8 KiB (13 bits).

Uma regra prática muito usada em Sistemas Operacionais é:

Se a memória possui 2^n bytes, então são necessários **n bits de endereço** para acessá-la.

Por exemplo:

Memória	Quantidade de bytes	Bits de endereço
1 KiB	2^{10}	10 bits
8 KiB	2^{13}	13 bits
32 KiB	2^{15}	15 bits
64 KiB	2^{16}	16 bits
1 MiB	2^{20}	20 bits

- Paginação:
 - **Número de quadros:** $64 \text{ KiB} / 8 \text{ KiB} = 8$ (3 bits) - quadros de 0 a 7.
 - **Número de páginas:** $32 \text{ KiB} / 8 \text{ KiB} = 4$ (2 bits) - páginas de 0 a 3.
 - **Deslocamento:** 8 KiB (13 bits).

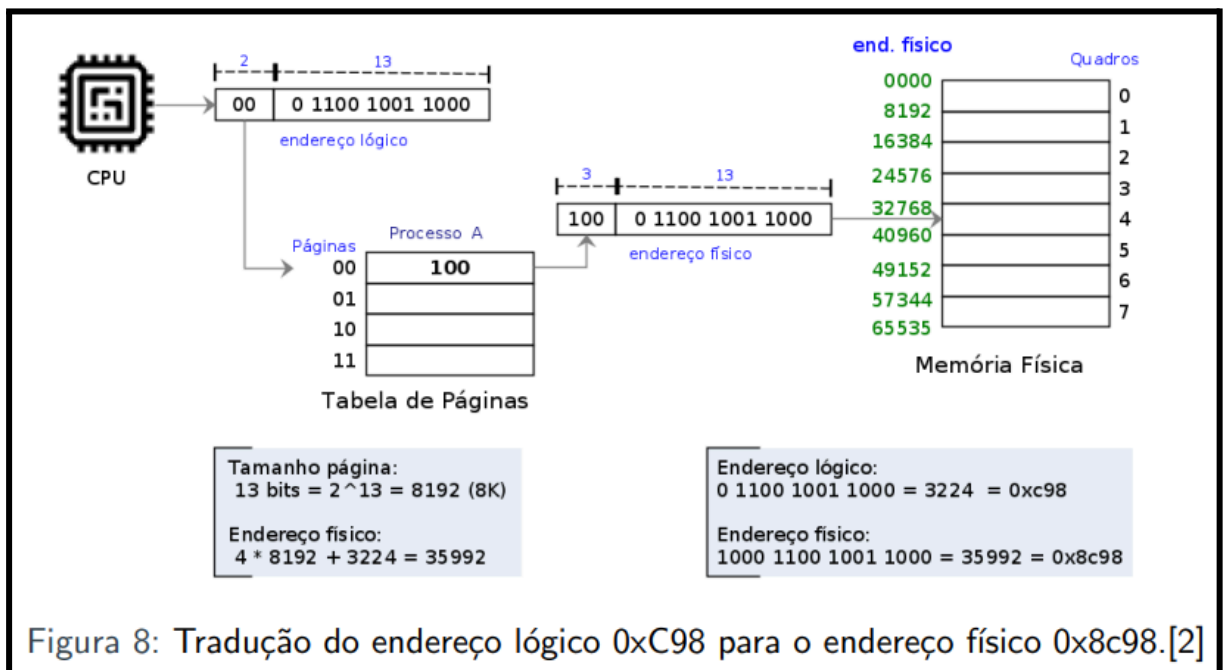
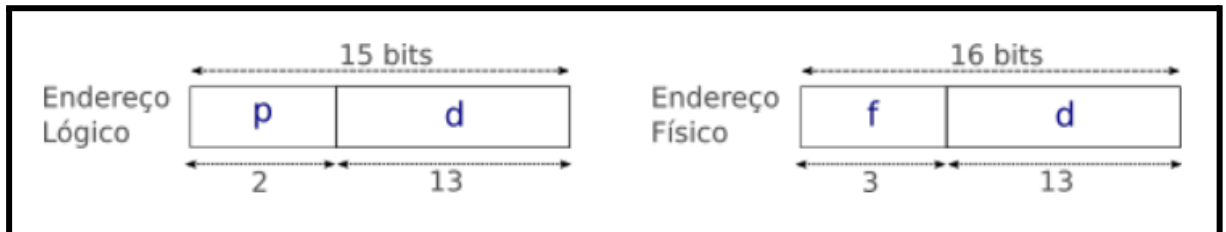


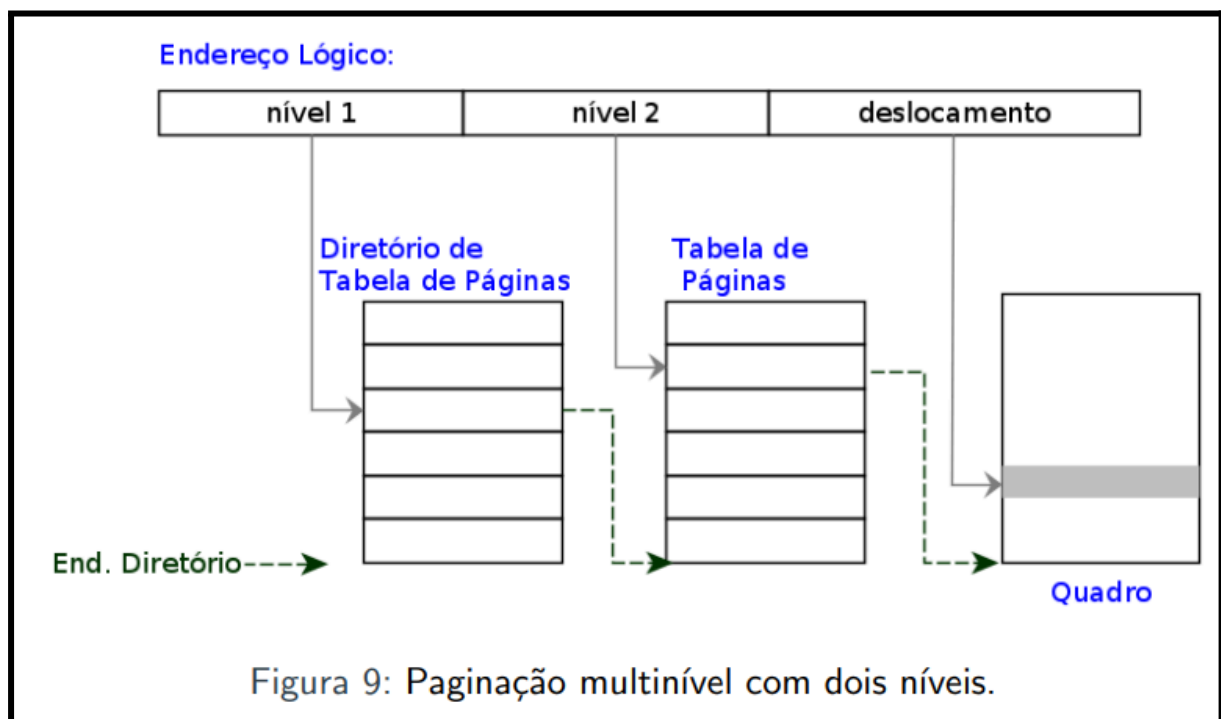
Figura 8: Tradução do endereço lógico 0xC98 para o endereço físico 0x8c98.[2]

- A **paginação elimina a fragmentação externa e diminui a fragmentação interna.**
- Como **processos são de diferentes tamanhos**, logo o **número usado de páginas por processos varia** e, por consequência, o **tamanho das tabelas de páginas também.**
- Questões:
 - Como dimensionar o tamanho da tabela de páginas: fixo ou variável?
 - Como armazenar a tabela de páginas: contíguo em memória (fragmentação externa) ou fazer a paginação na própria tabela?

- Solução: **Paginação Multinível.**

Paginação Multinível

- A **paginação multinível** divide a tabela em uma estrutura de níveis:
 - **Diretórios (níveis).**
 - **Tabela de páginas**
- As entradas dos diretórios apontam para outros diretórios ou para tabela de páginas.
- **Cada nível do diretório e cada tabela de páginas** deveriam **ocupar no máximo um quadro na memória física.**
- Os processos devem manter o endereço do diretório global (nível 1).



- **Exemplo:**
 - Considere o sistema com as seguintes características (80x86):
 - Endereço lógico: 4 GiB (32 bits).
 - Tamanho de páginas: 4 KiB (12 bits).
 - Quantas entradas possuiria uma tabela de páginas?

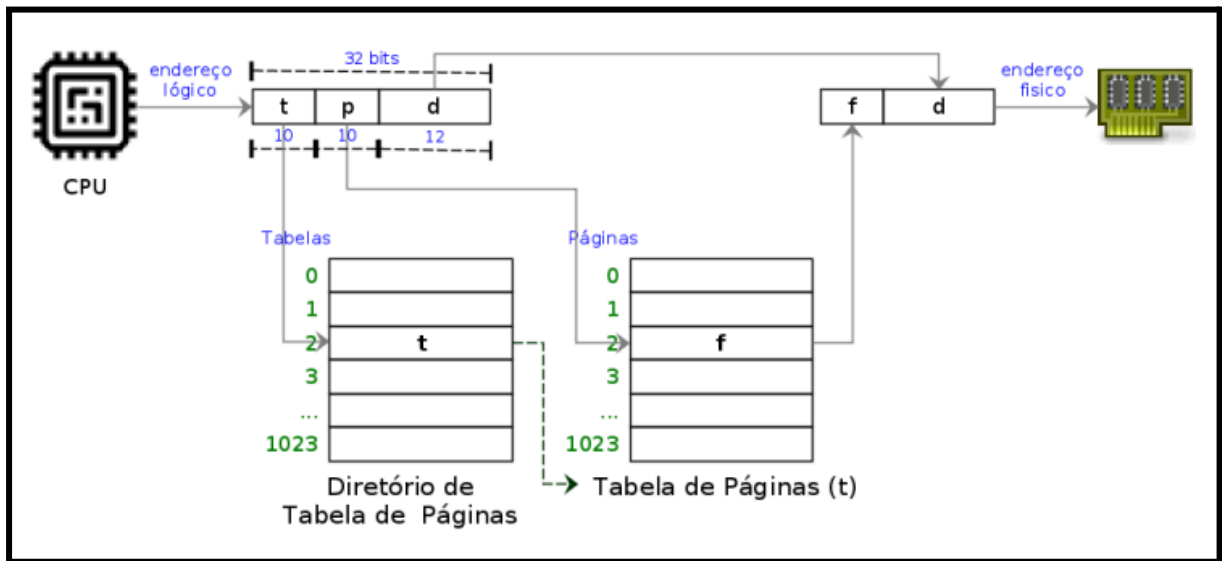
$$\frac{4GiB}{4KiB} = \frac{4 \times 2^{30} B}{4 \times 2^{10} B} = 2^{20} = 1.048.576$$

- Qual o tamanho da tabela de páginas por processo?

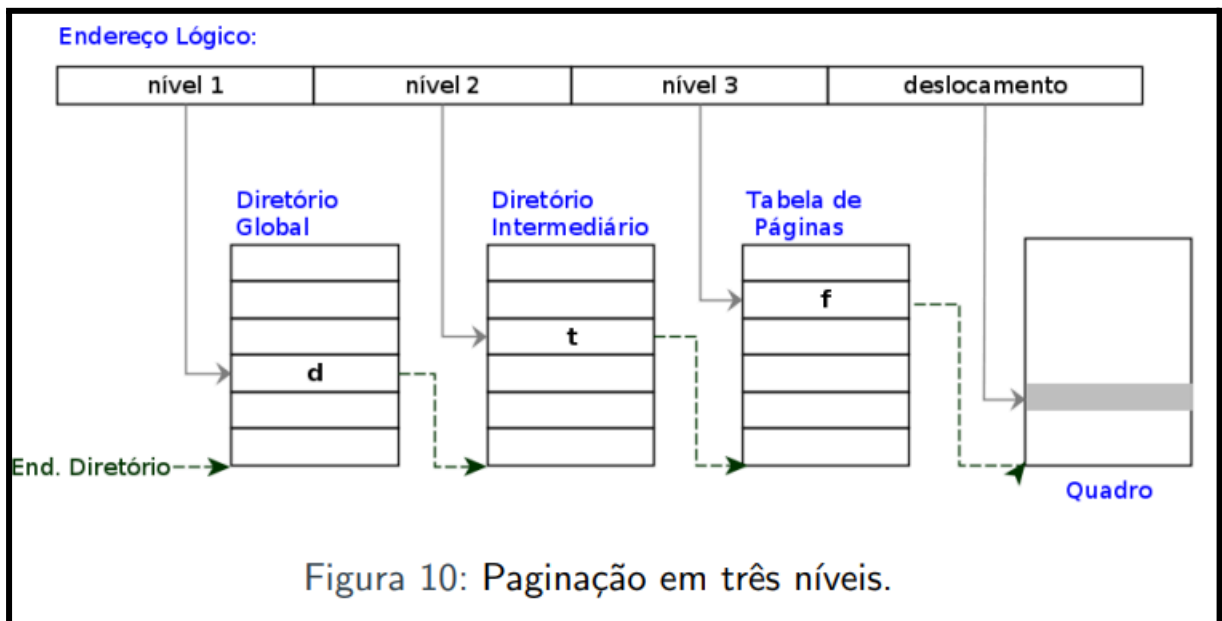
considerando cada entrada: 4 bytes (32 bits)
 número de entradas: $2^{20} = 1M$
 $2^{20} \times 4B = 1M * 4B = 4MiB$

- Usando paginação em dois níveis:

- Endereço lógico: 32 bits
- Tamanho página: 4 KiB
- Tabela de páginas: 1024 entradas
- Diretório de tabela de páginas: 1024 entradas (4 bytes)



- Paginação 3 níveis (arquiteturas 64 bits)



Proteção

- **Proteção de acesso**

- processos acessam somente suas páginas.
- endereços inválidos somente na última página (fragmentação interna).

- **Uso de bits de controle e validade na tabela de páginas.**

- Indicação se a página é leitura, escrita, código executável.
- A página pertence ao espaço de endereçamento do processo.

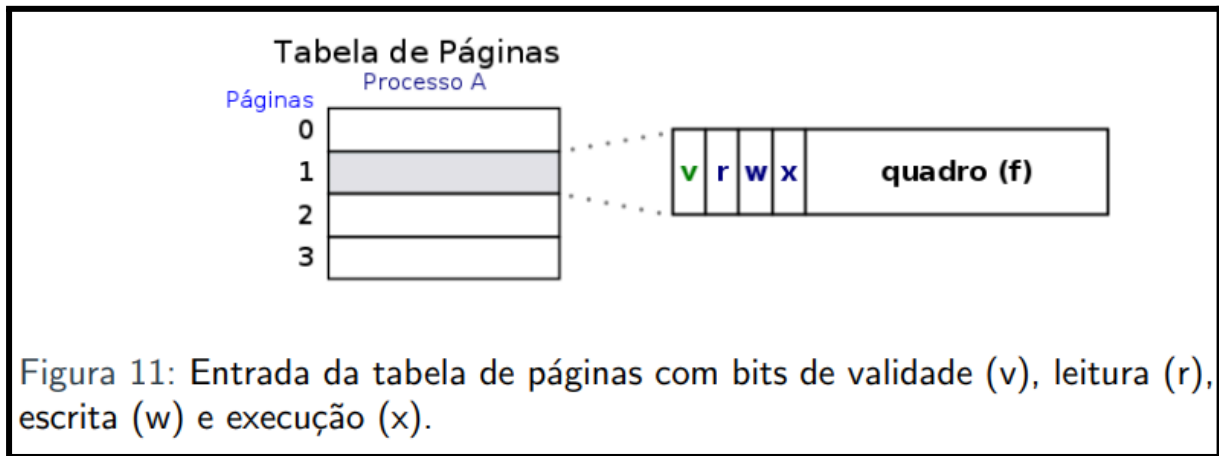


Figura 11: Entrada da tabela de páginas com bits de validade (v), leitura (r), escrita (w) e execução (x).

Implementação

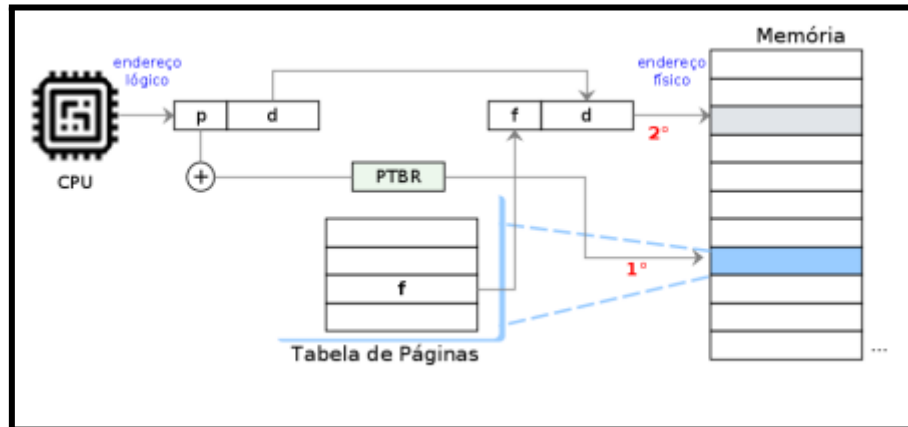
- **Forma 1: Registradores**

- A tabela de páginas é implementada por um conjunto de registradores.
- Cada página é referenciada por um registrador.
- BCP deve possuir cópias dos registradores para troca de contexto.
- Vantagens: **acesso rápido para tradução.**
- Desvantagens: **número de registradores.**

- **Forma 2: Memória**

- A tabela de páginas é **implementada em memória.**
- **Page Table Base Register (PTBR):** início da tabela de páginas.
- **Page Table Length Register (PTLR):** número de entradas na tabela de páginas.
- BCP deve manter os dois registradores para troca de contexto.

- **Vantagens:** suporta grande número de entradas para tradução.
- **Desvantagens:** para cada acesso, são necessários no mínimo dois acessos à memória.



- **Forma 3: Translation Lookaside Buffer (TLB)**

- A tabela de páginas é implementada em memória e faz uso de cache (TLB). A cache é composta por um banco de registradores (memória associativa).
- Consiste em manter a tabela de páginas em memória com uma parte na cache.
- A cache é consultada primeiro para a tradução.
- Página na TLB (hit) e página não está na TLB (miss).
- **Vantagens:** melhora o desempenho de acesso a tabela de páginas.
- **Desvantagens:** cache de tamanho limitado, memória cara, a TLB é compartilhada entre os processos.

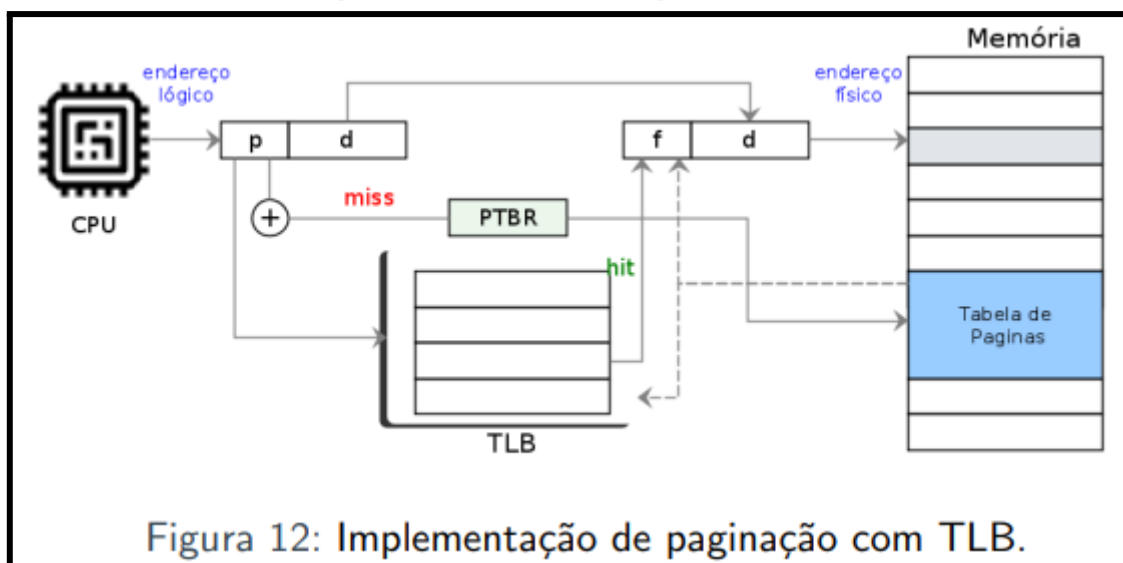


Figura 12: Implementação de paginação com TLB.

- **Tempo de Acesso**

- **Taxa de acerto (hit ratio - h):** endereço está na TLB.
- **Taxa de erro (miss ratio):** endereço não está na TLB.
- Tempo médio de acesso: $h(t_{atlb} + t_{amem}) + (1 - h)(t_{atlb} + 2 t_{amem})$
- Considere o exemplo:
- $t_{acesso_tlb} = 20ns$
- $t_{acesso_mem} = 100ns$
- $t_{acesso(hit)} = 20 + 100 = 120ns$
- $t_{acesso(miss)} = 20 + 100 + 100 = 220ns$
- Para um hit ratio 80%: $t_{medio} = 0.8 \times 120 + 0.2 \times 220 = 140ns$
- Para um hit ratio 98%: $t_{medio} = 0.98 \times 120 + 0.02 \times 220 = 122ns$

Tabela de Páginas Invertida

- Uma tabela para todo o sistema (não por processo).
- Possui uma entrada para cada quadro.
- O endereço lógico é composto por PID, página e deslocamento.
- Cada entrada possui PID e página.
- O índice da tabela refere-se a um quadro.
- Uma consulta à tabela de páginas devolve o índice associado à entrada.

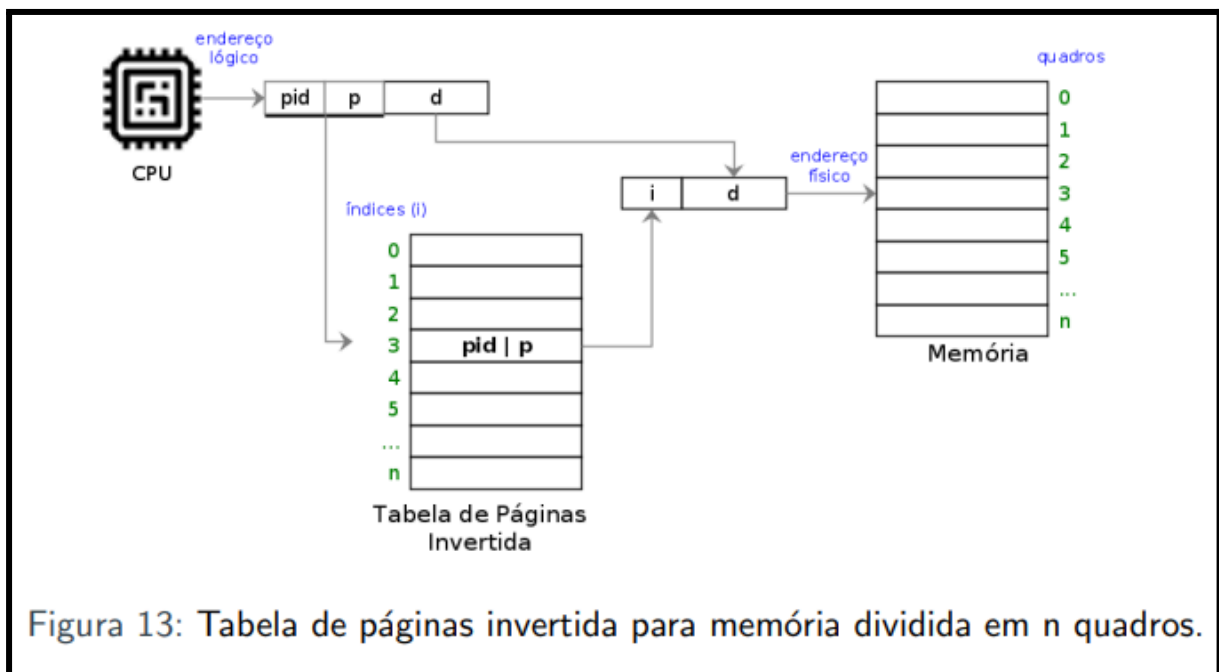
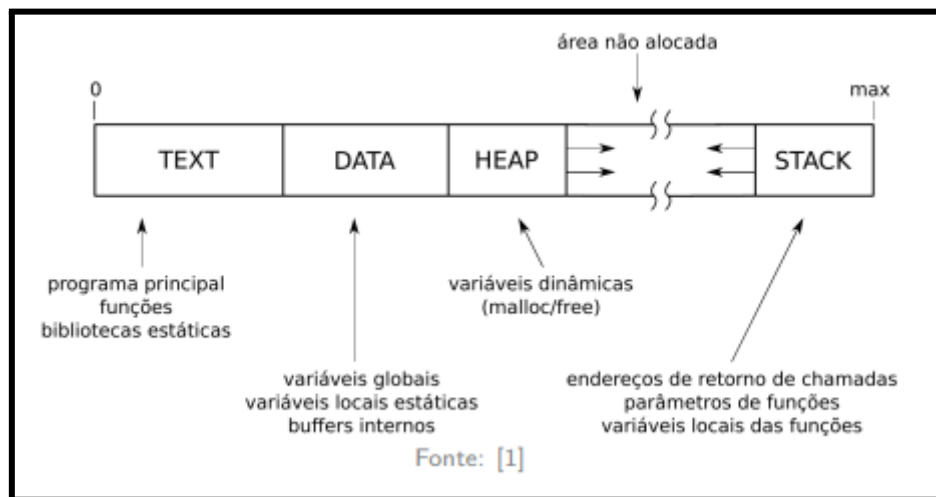


Figura 13: Tabela de páginas invertida para memória dividida em n quadros.

Segmentação

- **Espaço de endereço de um processo** é dividido em várias regiões (**segmentos**).
 - **Facilita o gerenciamento da memória**, pois cada região possui uma finalidade específica.
 - **Facilita o compartilhamento de áreas de memória.**
- A Gerência de Memória suporta o conceito de Segmentos.



- **Endereço lógico** composto por duas partes:
 - **Número do segmento.**
 - **Deslocamento no segmento.**
- **Segmentos podem ser de tamanhos diferentes** (há um limite máximo).
- Tradução similar a paginação por meio de uma **Tabela de Segmentos**.
- **Cada entrada possui:**
 - **base**: início do segmento (endereço físico).
 - **limite**: tamanho do segmento.
- Deve-se verificar a cada acesso a validade (via hardware).

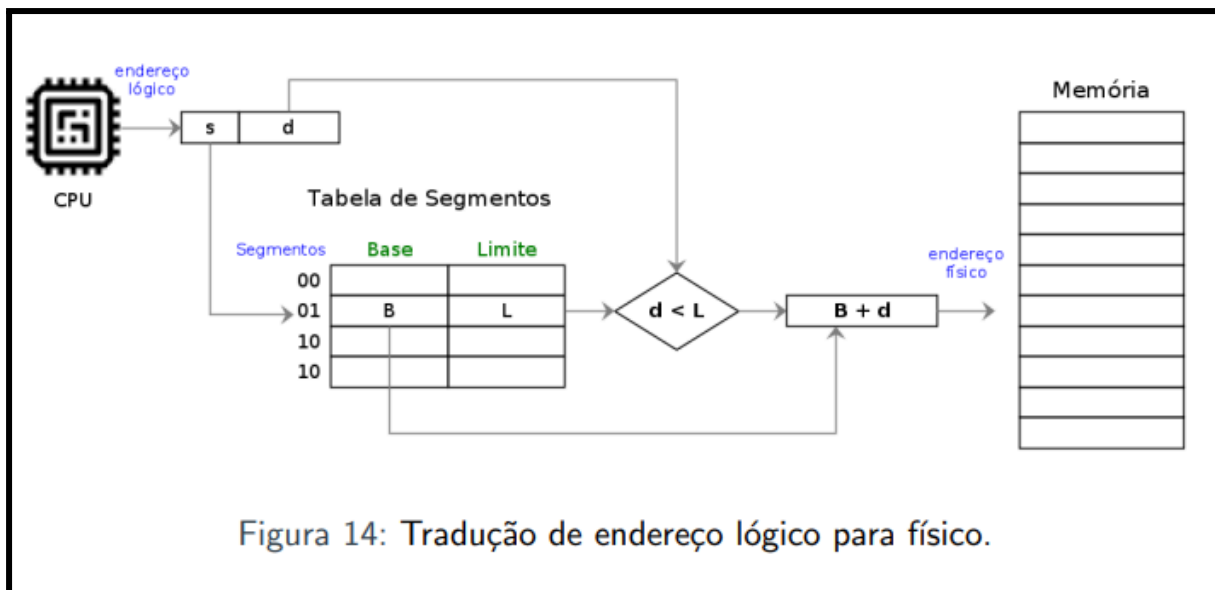


Figura 14: Tradução de endereço lógico para físico.

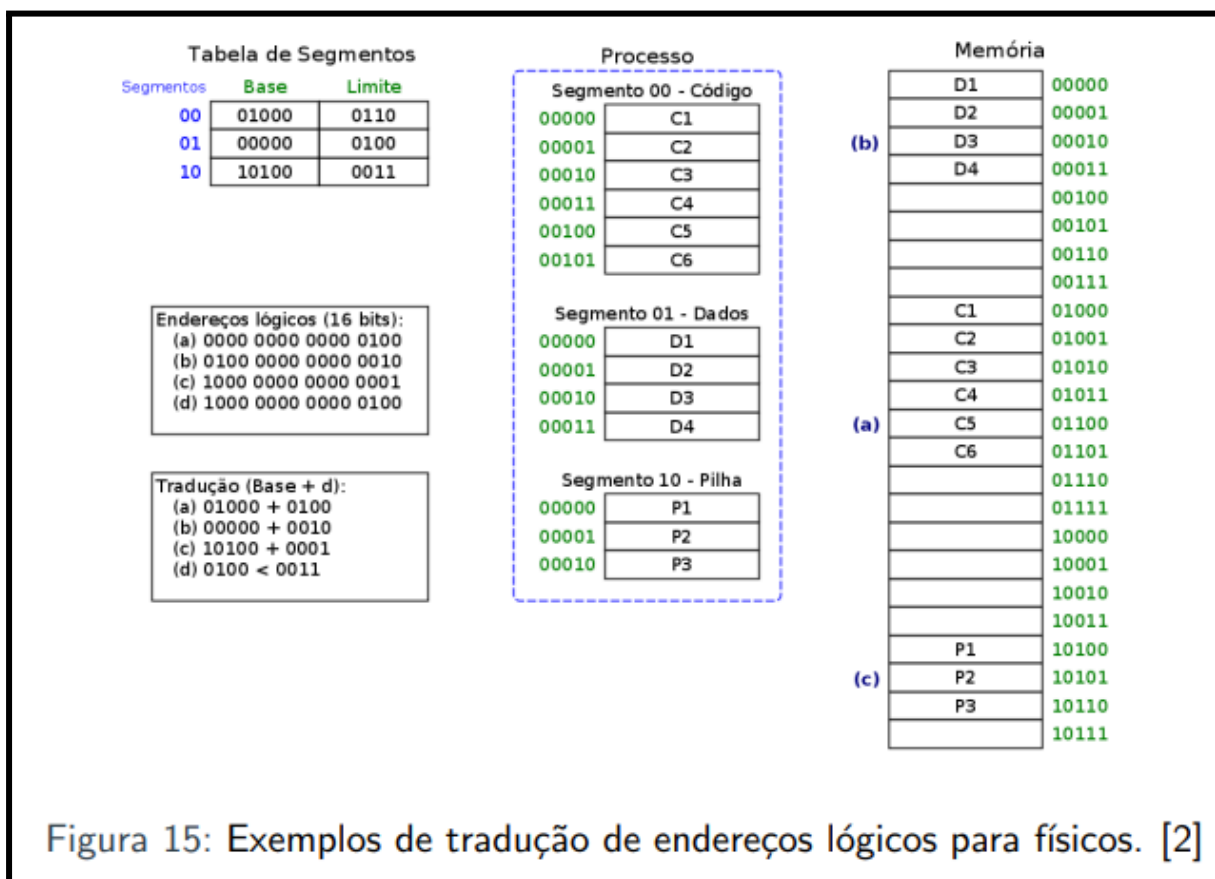
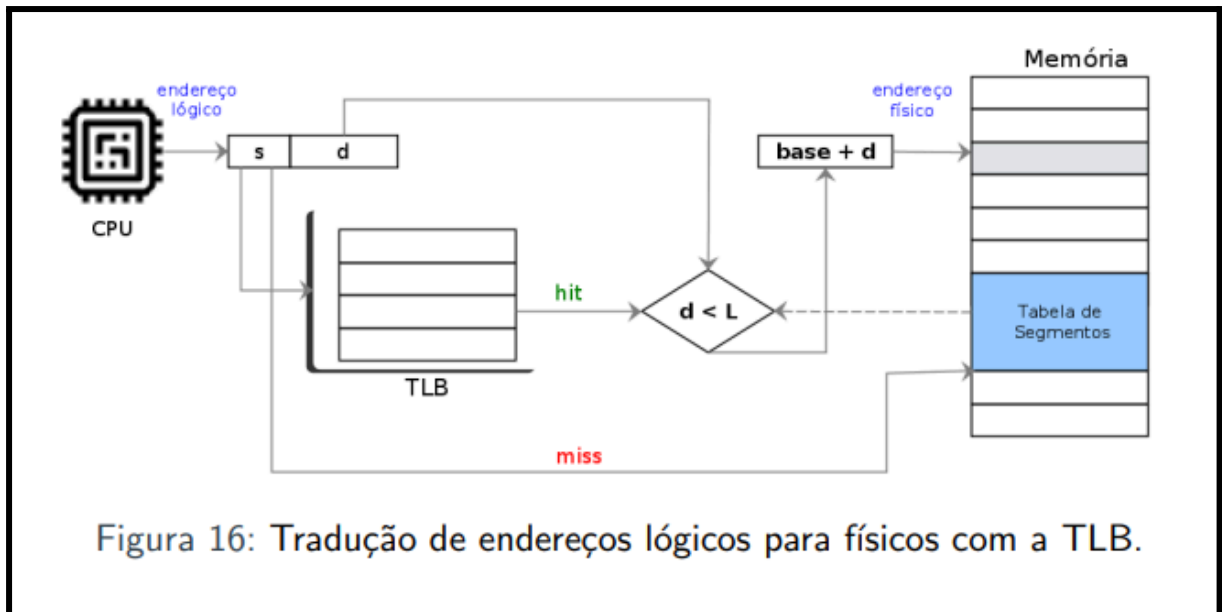
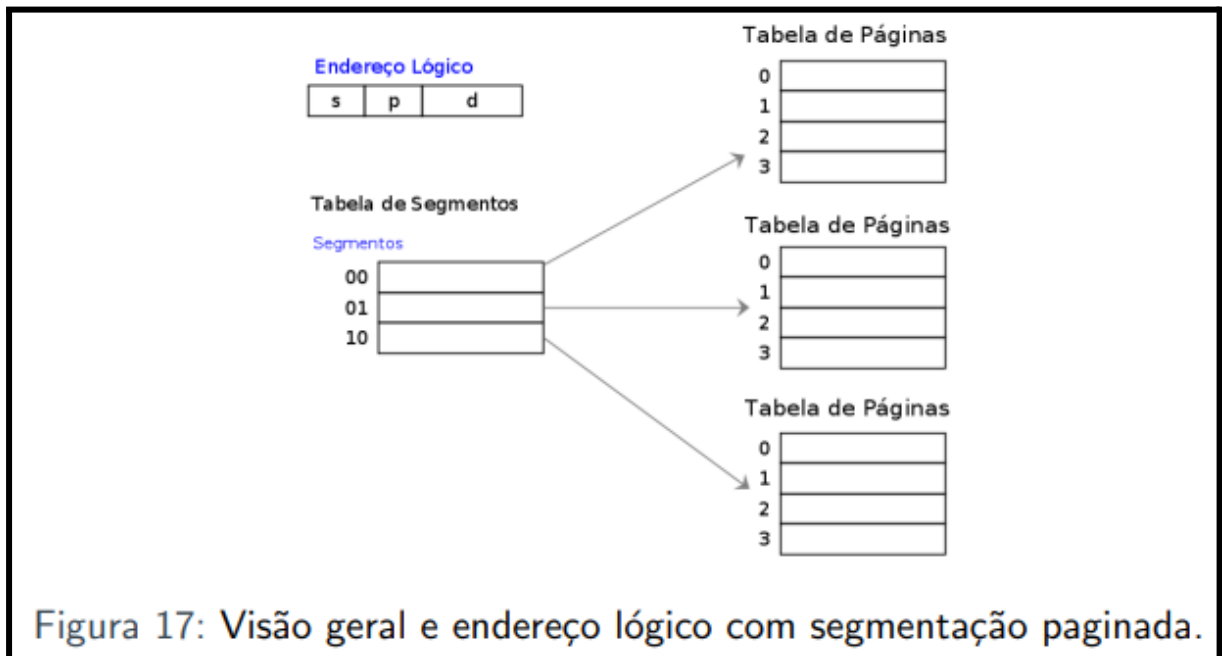


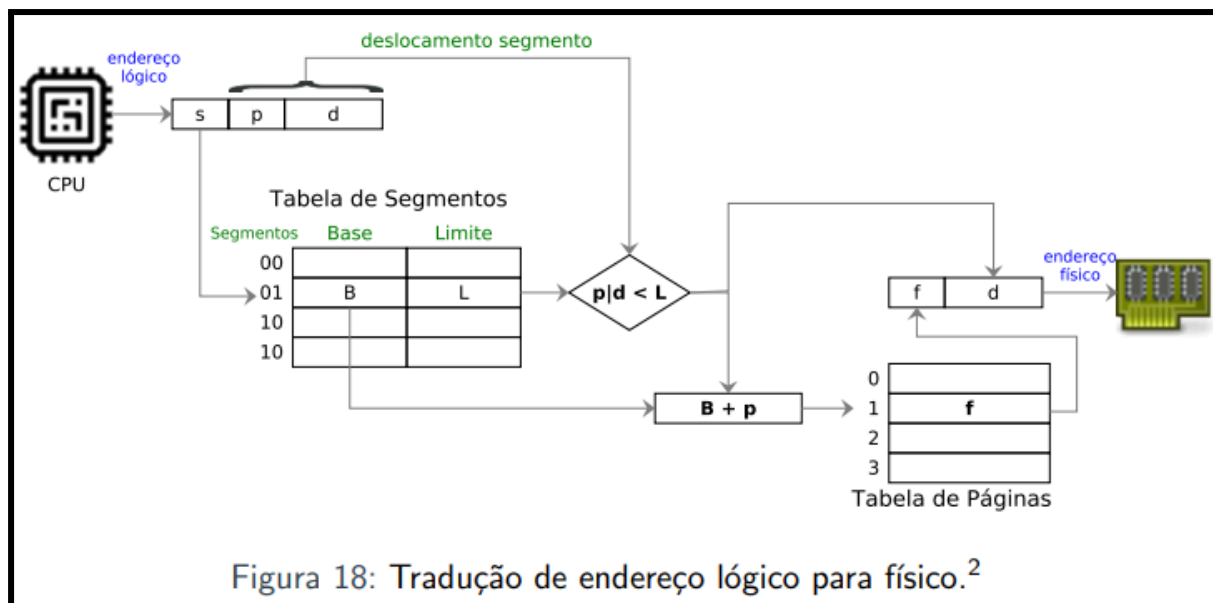
Figura 15: Exemplos de tradução de endereços lógicos para físicos. [2]



Segmentação Paginada

- Cada segmento possui uma tabela de páginas.
- O endereço lógico é composto por:
 - **segmento (s)**: segmento da tabela.
 - **página (p)**: número da página.
 - **deslocamento (d)**: deslocamento na página.





- **Vantagens:**
 - reduz o uso da memória.
 - tabela de páginas limitada pelo tamanho do segmento.
 - não há fragmentação externa.
- **Desvantagens:**
 - ainda há fragmentação interna.
 - tabelas de páginas devem estar contíguas na memória.

Memória Virtual

- **Memória virtual** possibilita a execução de um processo sem que esteja totalmente na memória.
- **Princípios básicos:**
 - carregar sob demanda páginas/segmentos.
 - manter somente páginas/segmentos necessários na memória.
- **Vantagens:**
 - executar processos maiores que a memória.
 - aumentar o grau de multiprogramação.
 - diminuir o número de operações E/S e swap.
- **Princípio da Localidade de Referência**
 - Processos executam acessos a instruções e dados limitados a sequências e trechos.
 - sequência de instruções (i, i+1, i+2, ..., n).

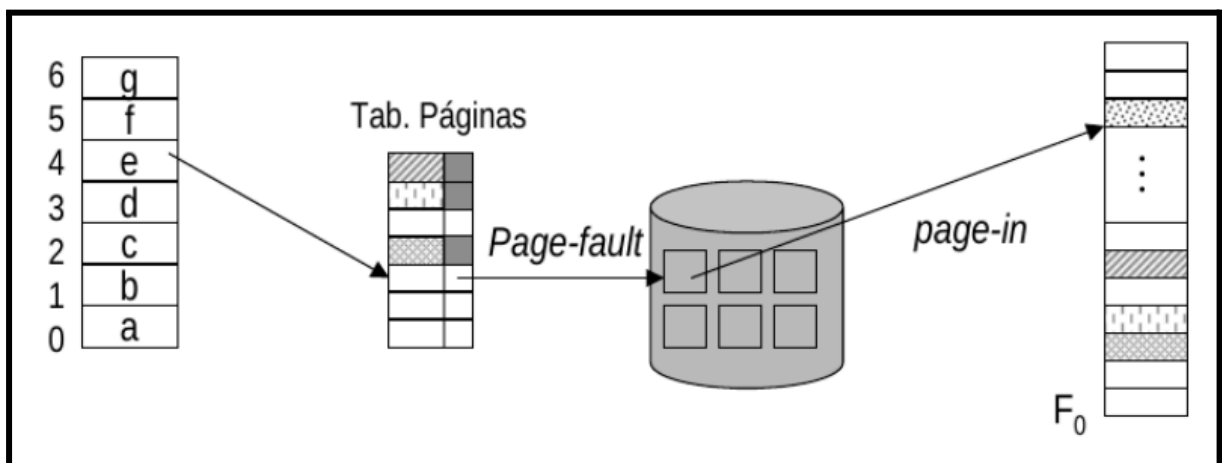
- laços de repetição (localização temporal).
- acessos a elementos de vetores (localização espacial).

○ Implicações:

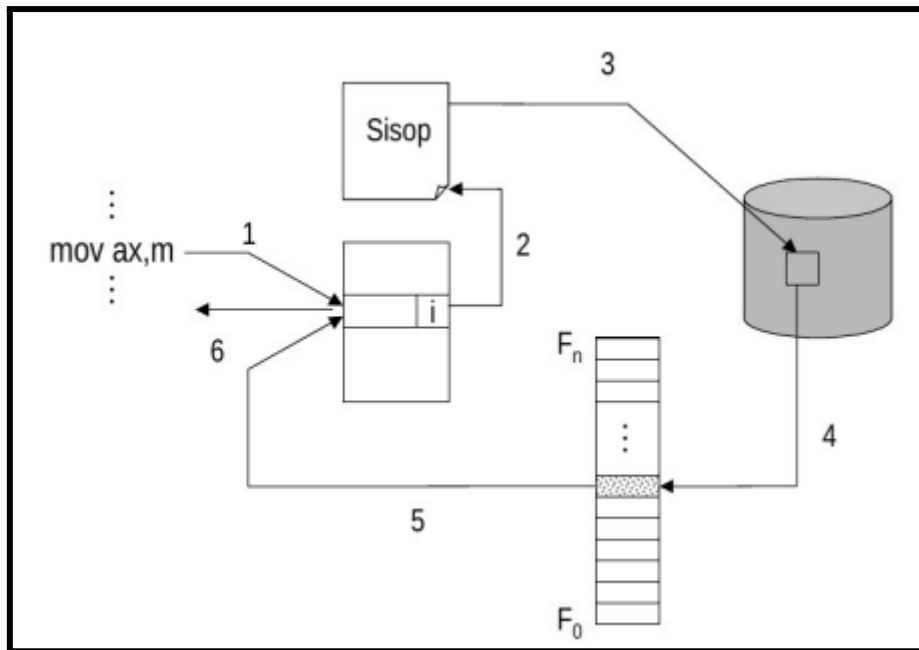
- Trechos de código e dados precisam estar na memória somente quando necessários.
- Previsibilidade dos próximos trechos.

● **Paginação por demanda**

- A página é carregada para a memória somente quando é necessária.
- Associar uma referência a cada página de presente ou não na memória primária.
 - uso de um bit de controle em cada entrada da tabela de páginas.
- Uma exceção de **page-fault** é gerada se a página não está na memória primária.



1. Acesso a um endereço de memória.
2. Ocorre uma exceção de page-fault.
3. Verifica se referência é válida e determina a localização da página no disco.
4. Solicita a leitura da página do disco para o quadro.
 - a. Bloqueia o processo e escalona outro.
 - b. Interrupção de término de leitura.
5. Atualiza a tabela de páginas.
 - a. Processo transita de bloqueado para pronto.
6. Processo executa novamente o acesso.



Implementação

- **Requisitos**

- Política de **carga de página.**
- Política de **localização de página.**
- Política de **substituição de página.**
- **Partição de swap.**
 - área específica do disco para armazenar páginas.
 - organizada para otimizar o acesso.

- **Política de carga de página**

- **Carrega uma página para um quadro.**
- **Situações:**
 - **quadro livre:** carregar página no quadro.
 - **não há quadro livre:**
 - liberar espaço transferindo da memória para o disco (**page-out**).
 - selecionar a página vítima usando a **política de substituição**.
- **Otimizações:**
 - carregar mais de uma página na memória.
 - páginas não modificadas e somente leitura não precisam ser armazenadas no disco.

- **Política de localização de página.**

- **Localizar as páginas de um processo na memória primária.**
- Realizada pela MMU e transparente para o sistema operacional.
- Política de **substituição de página.**
 - Não há quadros disponíveis para carregar novas páginas.
 - **Liberar um quadro para atender à falta de página.**
 - **Algoritmos de substituição de páginas** são responsáveis por selecionar uma **página vítima**.
 - Critérios:
 - **idade da página.**
 - **frequência de acessos à página.**
 - **marca de tempo do último acesso.**
 - **prioridade do processo (p. ex. tempo real).**
 - **conteúdo da página (p. ex. executável).**
 - **páginas especiais (p. ex. chaves criptográficas)**
 - Substituição de página: **Bits Auxiliares**
 - **Facilitam e otimizam a implementação do mecanismo de substituição de páginas.**
 - **bit de referência** (reference bit)
 - indica se a página foi acessada em um intervalo de tempo.
 - **bit de sujeira** (dirty bit) ou modificação
 - indica se a página foi modificada durante a execução.
 - **bit de trava** (lock bit)
 - indica que uma página não pode ser vítima.

Algoritmos de Substituição de Páginas

Algoritmo Ótimo (OPT)

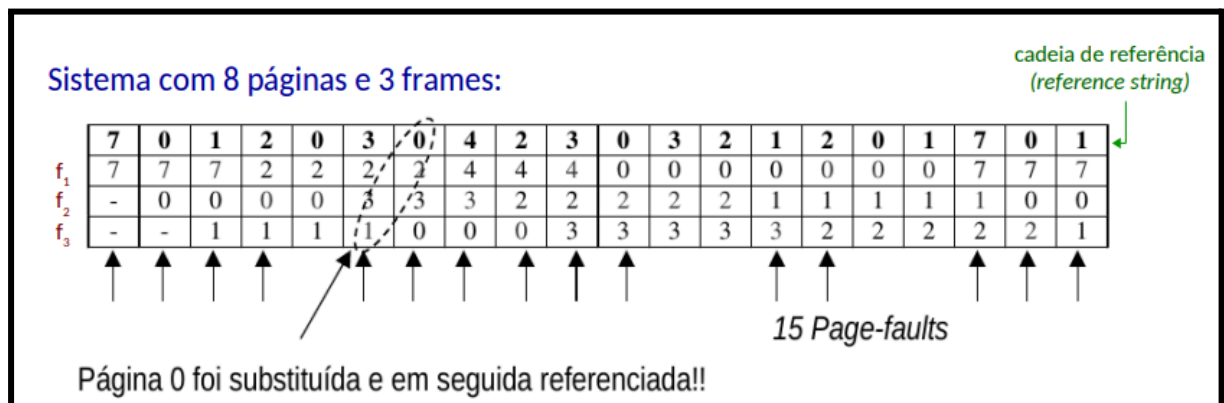
- **Retira da memória as páginas que demorarão mais para ser referenciadas.**
- **Problema:** prever as páginas que serão referenciadas primeiro.
- Interessante para comparar com outros algoritmos de paginação.

NRU (Not Recently Used)

- Substitui as páginas não usadas recentemente.
- Utiliza os bits R e M para selecionar as páginas vítimas
 - R: página referenciada
 - M: página modificada
- O bit R é limpo periodicamente (p. ex. interrupção de relógio)
- Faz uso de 4 classes:
 - Classe 0: não referenciada, não modificada.
 - Classe 1: não referenciada, modificada.
 - Classe 2: referenciada, não modificada.
 - Classe 3: referenciada, modificada.
- Substitui página aleatória de classe mais baixa.
- Vantagens:
 - fácil de compreender
 - moderadamente eficiente de implementar
 - proporciona um desempenho satisfatório.

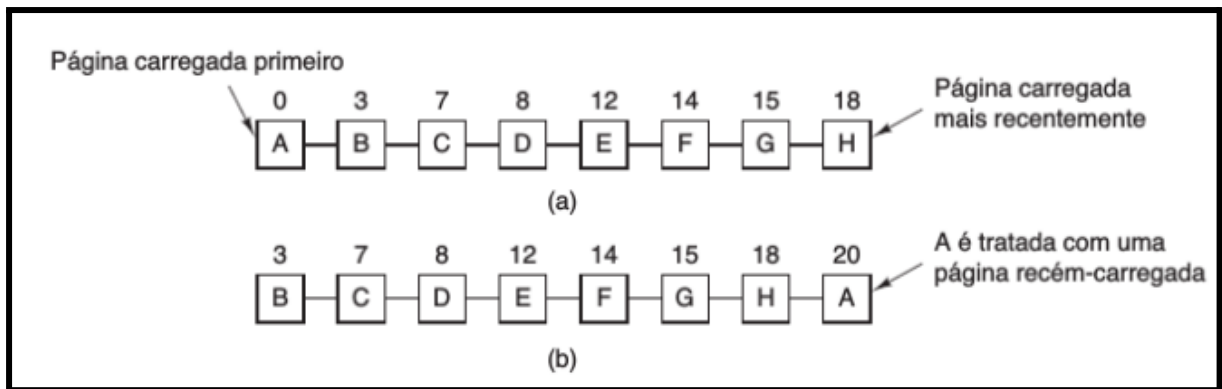
FIFO (First in, First out)

- Substitui a página mais antiga (primeira na fila)
- Problema: a página mais antiga pode ainda estar sendo usada.
- Raramente é usado esse algoritmo.



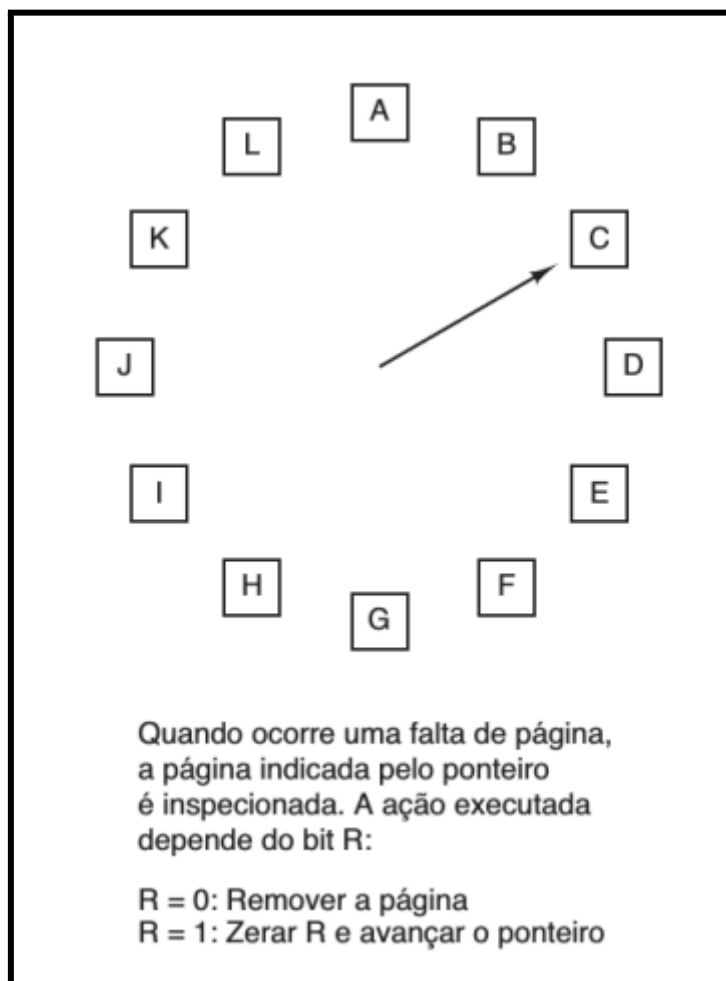
Segunda Chance

- FIFO + bit R
- Substitui a página mais velha não referenciada
- Implementado como uma lista encadeada de páginas na memória:
 - Se $R=0$, então remove página;
 - Senão, $R \leftarrow 0$, então coloca no final da lista.



Relógio

- Otimização do Segunda Chance
- Implementado como uma lista circular de páginas na memória.
- Ponteiro aponta para a página mais antiga.
 - Se $R == 0$ troca de página e desloca ponteiro;
 - Senão $R \leftarrow 0$, desloca ponteiro, continua a busca.



LRU (Least Recently Used)

- Substitui as páginas menos usadas recentemente.

- Implementado como uma lista de páginas ordenadas pelo uso recente.
- **Problema:** Custo para atualizar a lista a cada acesso à memória.
- **Solução:** Hardware especial, por exemplo, contador C que é incrementado e armazenado para cada entrada. Percorrer a tabela de páginas ativas e localizar o menor valor de C.
- LRU pode ser implementado em **software**
 - NFU – Not Frequently Used
 - Aging - Envelhecimento

NFU - Not Frequently Used

- **Substitui a página menos frequentemente usada**
- Contador de software associado a cada página
- Percorrer todas as páginas a cada interrupção do relógio.
- Então, **somar o valor do bit R (0 ou 1) ao contador.**
- **Durante a falta de página, seleciona o menor contador.**
- **Problema:** memória de elefante

Aging - Envelhecimento

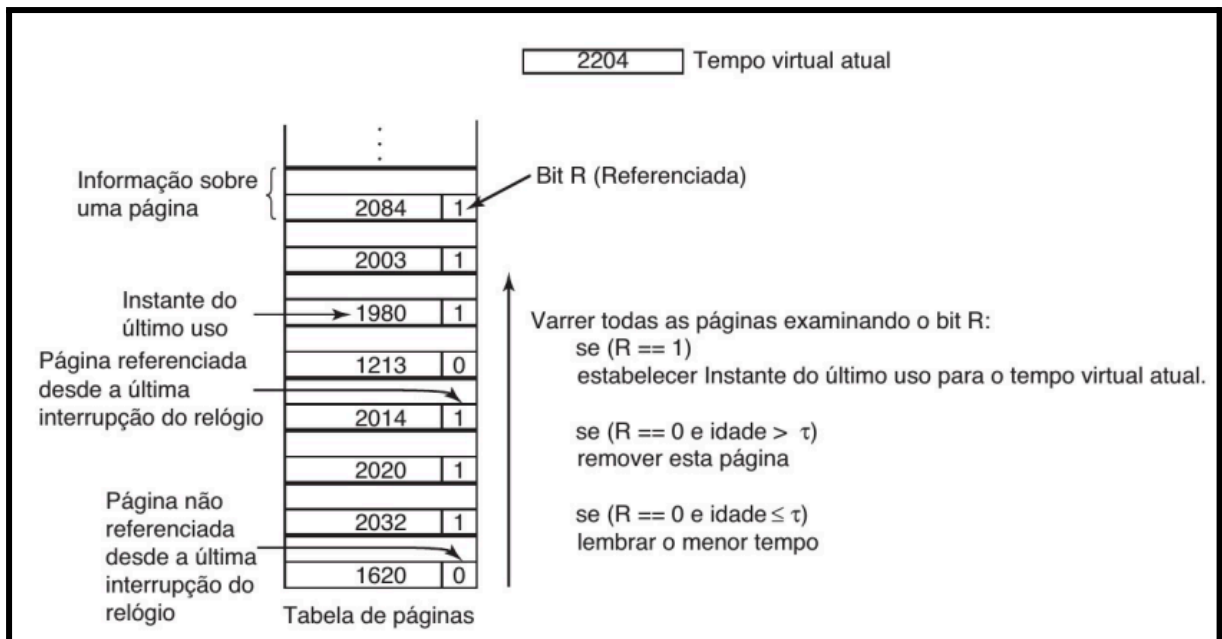
- Modificação do NFU
- A cada interrupção do relógio:
 - Contadores são deslocados 1 bit à direita
 - Acrescenta-se R ao bit mais à esquerda
- Duas principais diferenças ao LRU:
 - **Limite de passado (8 bits)**
 - **Entre as interrupções, não é possível determinar a ordem de acesso.**

As cinco interrupções de relógio são representadas por (a) a (e).

	Bits R para as páginas 0–5, interrupção de relógio 0	Bits R para as páginas 0–5, interrupção de relógio 1	Bits R para as páginas 0–5, interrupção de relógio 2	Bits R para as páginas 0–5, interrupção de relógio 3	Bits R para as páginas 0–5, interrupção de relógio 4
	0 1 2 3 4 5 1 0 1 0 1 1	0 1 2 3 4 5 1 1 0 0 1 0	0 1 2 3 4 5 1 1 0 1 0 1	0 1 2 3 4 5 1 0 0 0 1 0	0 1 2 3 4 5 0 1 1 0 0 0
Página					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00010000	10001000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000
	(a)	(b)	(c)	(d)	(e)

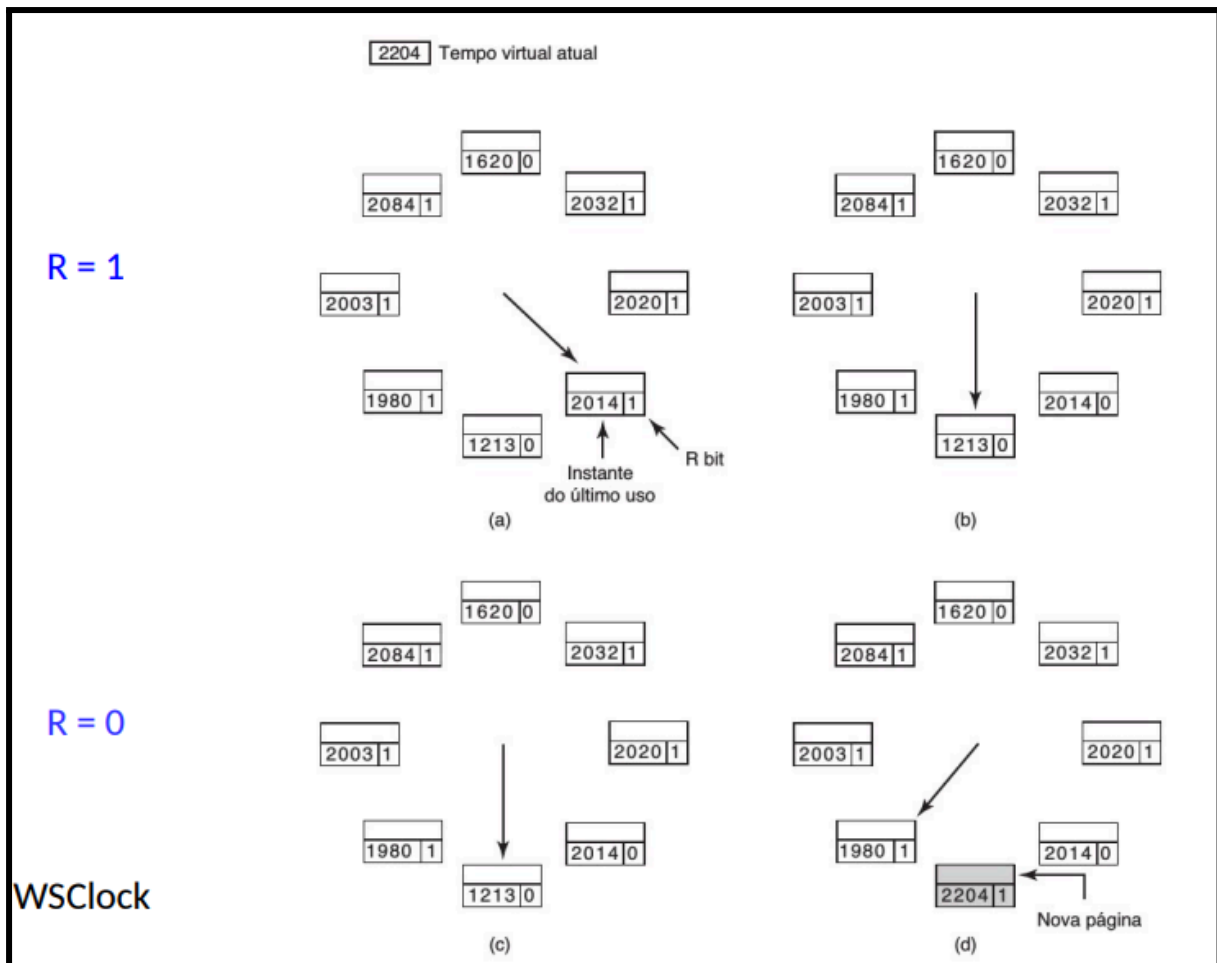
Algoritmo de Conjunto de Trabalho

- **Conjunto de trabalho** é o **conjunto de páginas que um processo está usando.**
- **Thrashing** é quando ocorre a todo instante falta de páginas.
- Tentar manter o conjunto de trabalho para não causar tantas faltas de páginas.
- O **algoritmo deve procurar remover páginas que não estão no conjunto de trabalho.**
- Uma solução para **determinar o conjunto de trabalho** é **usar o tempo de execução.**
- Por exemplo, definir o conjunto de trabalho como as **páginas acessadas nos últimos X ms de execução.**
- **Para cada processo, apenas seu tempo de execução conta.**



Algoritmo WSClock

- Combinação do algoritmo do Conjunto de Trabalho e Relógio.
- Simples de implementar e bom desempenho.
- **Se R==1:** R ← 0 e avança.
- **Se R==0 e idade > e página limpa:** substitui.
- **Se R==0 e idade > e página suja:** escalona para escrita e continue a procurar.



Resumo

Algoritmo	Comentário
Ótimo	Não implementável, mas útil como um padrão de desempenho
NRU (não usado recentemente)	Aproximação muito rudimentar do LRU
FIFO (primeiro a entrar, primeiro a sair)	Pode descartar páginas importantes
Segunda chance	Algoritmo FIFO bastante melhorado
Relógio	Realista
LRU (usada menos recentemente)	Excelente algoritmo, porém difícil de ser implementado de maneira exata
NFU (não frequentemente usado)	Aproximação bastante rudimentar do LRU
Envelhecimento (<i>aging</i>)	Algoritmo eficiente que aproxima bem o LRU
Conjunto de trabalho	Implementação um tanto cara
WSClock	Algoritmo bom e eficiente